Код проєкту: studentflow

Згенеровано: 2026-03-19 06:00:56
Директорія: `/home/vokov/projects/studentflow`
Формат: PDF Document

Структура проєкту

```
studentflow/
├── requirements.txt
├── alembic.ini
├── README.md
├── pyproject.toml
├── app/
│   ├── main.py
│   ├── __init__.py
│   ├── bootstrap.py
│   ├── cli.py
│   ├── errors.py
│   ├── config.py
│   ├── security/
│   │   ├── vault.py
│   │   ├── __init__.py
│   │   ├── masking.py
│   │   ├── crypto.py
│   ├── services/
│   │   ├── account_service.py
│   │   ├── template_service.py
│   │   ├── sync_service.py
│   │   ├── credential_service.py
│   │   ├── handoff_service.py
│   │   ├── __init__.py
│   │   ├── prompt_service.py
│   │   ├── repository_service.py
│   │   ├── audit_service.py
│   ├── jobs/
│   │   ├── scheduler.py
│   │   ├── __init__.py
│   │   ├── workers.py
│   ├── db/
│   │   ├── base.py
│   │   ├── __init__.py
│   │   ├── enums.py
│   │   ├── seeds.py
│   │   ├── models.py
│   │   ├── migrations/
│   │   │   ├── README
│   │   │   ├── script.py.mako
│   │   │   ├── .gitkeep
│   │   │   ├── env.py
│   │   │   ├── versions/
│   │   │   │   ├── 7ca526e3ce7e_initial_schema.py
│   │   │   │   ├── b1a2c3d4e5f6_add_prompt_and_student_number.py
│   ├── git/
│   │   ├── client.py
│   │   ├── __init__.py
│   │   ├── validators.py
│   │   ├── policies.py
│   ├── ui/
│   │   ├── __init__.py
│   │   ├── dialogs/
│   │   │   ├── __init__.py
```

```
│   │   │   ├── repository_dialog.py
│   │   │   ├── credential_dialog.py
│   │   │   ├── student_dialog.py
│   │   │   ├── account_dialog.py
│   │   ├── windows/
│   │   │   ├── repositories_screen.py
│   │   │   ├── dashboard_screen.py
│   │   │   ├── credentials_screen.py
│   │   │   ├── main_window.py
│   │   │   ├── prompts_screen.py
│   │   │   ├── __init__.py
│   │   │   ├── students_screen.py
│   │   │   ├── accounts_screen.py
│   │   │   ├── settings_screen.py
│   │   │   ├── logs_screen.py
│   │   ├── viewmodels/
│   │   │   ├── settings_viewmodel.py
│   │   │   ├── prompts_viewmodel.py
│   │   │   ├── __init__.py
│   │   │   ├── students_viewmodel.py
│   │   │   ├── dashboard_viewmodel.py
│   │   │   ├── repos_viewmodel.py
│   │   │   ├── logs_viewmodel.py
│   │   │   ├── credentials_viewmodel.py
│   │   │   ├── accounts_viewmodel.py
│   │   ├── widgets/
│   │   │   ├── __init__.py
│   ├── schemas/
│   │   ├── __init__.py
│   │   ├── account.py
│   │   ├── repository.py
│   │   ├── student.py
├── docs/
│   ├── audit-and-logging.md
│   ├── security.md
│   ├── handoff-protocol.md
│   ├── data-model.md
│   ├── troubleshooting.md
│   ├── operator-runbook.md
│   ├── ui-flows.md
│   ├── architecture.md
│   ├── setup.md
│   ├── sync-strategy.md
│   ├── uk/
│   │   ├── README.md
│   │   ├── rozgortannya.md
│   │   ├── metodyka-zastosuvannya.md
│   ├── plans/
│   │   ├── 2026-03-17-canonical-repo-ui.md
│   │   ├── 2026-03-17-prompt-delivery.md
├── codex-tasks/
│   ├── T13-tests.md
│   ├── T03-credential-service.md
│   ├── T09-audit-service.md
│   ├── T10-ui-skeleton.md
│   ├── T11-ui-screens.md
│   ├── T06-git-client.md
│   ├── T01-scaffold.md
│   ├── T14-seed-data.md
│   ├── T15-table-density-canonical-auth.md
│   ├── T07-sync-service.md
│   ├── T02-models.md
│   ├── T05-repository-service.md
│   ├── T12-validation-error-handling.md
│   ├── T08-handoff-service.md
│   ├── T04-template-service.md
├── tests/
```

```
│   ├── test_validators.py
│   ├── test_repos_viewmodel.py
│   ├── test_repository_service.py
│   ├── test_template_service.py
│   ├── test_prompt_service.py
│   ├── test_ui_skeleton.py
│   ├── test_credential_service.py
│   ├── test_sync_service.py
│   ├── test_account_service.py
│   ├── __init__.py
│   ├── test_settings_viewmodel.py
│   ├── test_schemas.py
│   ├── conftest.py
│   ├── test_models.py
│   ├── test_audit_service.py
│   ├── test_dashboard_viewmodel.py
│   ├── test_git_client.py
│   ├── test_prompts_viewmodel.py
│   ├── test_handoff_service.py```
---
### requirements.txt
**Розмір:** 169 байт

PySide6==6.7.0
SQLAlchemy==2.0.30
alembic==1.13.1
GitPython==3.1.43
keyring==25.2.1
structlog==24.2.0
APScheduler==3.10.4
pydantic==2.7.1
pytest==8.2.0
pytest-qt==4.4.0
---
### README.md
**Розмір:** 14,232 байт

# StudentFlow

StudentFlow is a local desktop operator tool for managing approximately 20 student accounts that sequentially work on a shared educational project through separate Git repositories. The operator controls which student is currently active, manages their credentials securely using the OS keyring, synchronizes code between student repositories and a central canonical repository, and handles handoffs between students as they progress through the project.

The tool is designed for a single trusted operator running locally. It is not a server application and has no network-facing interface. All state is stored in a local SQLite database and the OS credential store. Each student has their own Git remote (GitHub, GitLab, Bitbucket, etc.) and a local working clone managed by StudentFlow. The operator never needs to touch Git directly — StudentFlow handles all repository operations, checkpoint tagging, and handoff orchestration.

# Quick Start

1. **Clone and enter the project:**
   git clone &lt;this-repo&gt; studentflow
   cd studentflow

2. **Install dependencies in a virtual environment:**
   python3.11 -m venv .venv
   source .venv/bin/activate  # Windows: .venv\Scripts\activate
   pip install -r requirements.txt
```

3. **Initialize the database:**
   python -m app.bootstrap init-db

4. **(Optional) Load demo seed data:**
   python -m app.db.seeds

5. **Run the application:**
   QT_QPA_PLATFORM=xcb python -m app.main

6. **First-time manual setup (no wizard):**
The app opens directly to the main screen. Add students via **Students → Add Student**, then add accounts via **Accounts → Add Account**, then store credentials via **Credentials → Add Credential**. The canonical repo path defaults to `~/.studentflow/canonical/` and can be changed in `~/.studentflow/config.json`.

Architecture Overview

StudentFlow is a three-tier desktop application: the PySide6 UI layer handles all operator interaction, the Python service layer (AccountService, SyncService, HandoffService, etc.) contains all business logic, and the data layer consists of a local SQLite database (via SQLAlchemy) plus Git repositories on the filesystem. All secrets are stored exclusively in the OS keyring — the database never holds credential values. Service calls currently run synchronously from the UI thread (async workers are Phase 2).

Key Concepts

Concept	Description
Student	A person with a queue position, status, and associated Git account. Has one active account at a time.
Account	A Git provider account (GitHub, GitLab, etc.) belonging to a student. Holds provider/username/auth type.
Canonical Repo	The single bare Git repository at `~/.studentflow/canonical/` that is the source of truth for all project state. Every student syncs into and out of this repo.
Handoff	The process of completing one student's work, tagging a checkpoint on canonical, and activating the next student in the queue.
Sync	Pulling new commits from a student's remote into their local clone, merging into canonical. Each sync creates a SyncJob record.

Directory Structure

```
studentflow/
├── app/
│   ├── main.py              # Entry point (GUI or --cli mode)
│   ├── bootstrap.py           # DB init, logging init, dir creation
│   ├── config.py            # Config dataclass, ~/.studentflow/config.json
│   ├── db/
│   │   ├── base.py           # SQLAlchemy engine, SessionLocal, Base
│   │   ├── models.py          # All ORM models
│   │   ├── enums.py           # All enum classes
│   │   ├── seeds.py          # Demo seed data
│   │   └── migrations/        # Alembic migrations
│   ├── services/
│   │   ├── account_service.py
│   │   ├── repository_service.py
│   │   ├── credential_service.py
```

```
│   │   ├── sync_service.py
│   │   ├── handoff_service.py
│   │   ├── template_service.py
│   │   ├── prompt_service.py
│   │   └── audit_service.py
│   ├── git/
│   │   ├── client.py          # All Git operations (GitPython + subprocess)
│   │   ├── policies.py         # Merge/sync policy logic
│   │   └── validators.py       # Remote/local path validation
│   ├── security/
│   │   ├── vault.py           # OS keyring interface (SecretVault)
│   │   ├── masking.py          # mask_secret() for display
│   │   └── crypto.py           # Placeholder for future encryption
│   ├── jobs/
│   │   ├── scheduler.py        # APScheduler setup
│   │   └── workers.py          # QThread workers for async Git ops
│   ├── schemas/
│   │   ├── student.py
│   │   ├── account.py
│   │   └── repository.py
│   └── ui/
│       ├── windows/           # QMainWindow + 8 screen widgets
│       ├── dialogs/           # Add/Edit/Rotate dialogs
│       ├── widgets/           # Reusable custom widgets
│       └── viewmodels/         # ViewModels (data holders for screens)
├── tests/
│   ├── conftest.py            # Shared pytest fixtures (db_session, mock_keyring, mock_git_client)
│   ├── test_models.py
│   ├── test_account_service.py
│   ├── test_sync_service.py
│   ├── test_credential_service.py
│   ├── test_handoff_service.py
│   ├── test_audit_service.py
│   ├── test_repository_service.py
│   ├── test_template_service.py
│   ├── test_git_client.py
│   ├── test_validators.py
│   ├── test_schemas.py
│   └── test_ui_skeleton.py
├── docs/
│   ├── architecture.md
│   ├── data-model.md
│   ├── sync-strategy.md
│   ├── security.md
│   ├── ui-flows.md
│   ├── handoff-protocol.md
│   ├── audit-and-logging.md
│   ├── setup.md
│   ├── operator-runbook.md
│   └── troubleshooting.md
├── codex-tasks/               # Codex implementation task specs
├── requirements.txt
├── pyproject.toml
└── .gitignore
```

Known Limitations (v0.1.0-mvp)

Limitation	Status	Notes
Credential validation	Functional	`Validate` checks keyring presence; when a repository is configured, also verifies `git ls-remote` connectivity.
Dashboard screen	Functional	Shows active student, student queue, recent sync activity, and canonical repo status. Auto-sync trigger and System Health drill-down are Phase 2.

Settings screen	Functional	Canonical local path, canonical remote URL (optional), default_branch, sync_timeout_seconds, max_sync_retries.
Auto-sync scheduler	Stubbed	`SchedulerService` is empty. Manual sync only in Phase 1.
UI blocking	Known	Long git operations block the UI thread. QThread workers are Phase 2.
`resume_handoff()`	Guarded	Raises `HandoffError` if called. Full resume logic is Phase 2.
Prompt push blocking	Known	Git push during prompt delivery blocks the UI thread. QThread workers are Phase 2.
Add Repository dialog	MVP	Local path and remote URL accepted but not immediately cloned. Clone is a manual step.

Running Tests

```
QT_QPA_PLATFORM=offscreen .venv/bin/pytest -v --tb=short
# Expected: 155 passed (as of v0.1.9)
```

Release Notes

v0.1.9 (2026-03-17)

Network credential validation:
- `AccountService.validate_account()` now calls `GitClient.validate_remote()` (via `git ls-remote`) when a repository is configured for the account
- If no repository is configured, behaviour is unchanged: keyring presence confirmed, network noted as unverified
- If remote authentication fails (expired/revoked token), `status=error` is returned with a clear operator message before a sync or handoff reveals it the hard way
- `git_client` dependency injected into `AccountService.__init__` (defaults to `GitClient()`)
- 2 new tests: network success path, network failure path

---

v0.1.8 (2026-03-17)

Canonical remote push:
- When `canonical_remote_url` is configured in Settings, every successful `sync_student()` now pushes the local canonical bare repo to that remote
- Non-fatal on failure: sync job still succeeds; warning appended to SyncJob summary (visible in Recent Sync Activity and Sync Now dialog)
- No credential management required; operator configures system-level git auth (SSH agent, `~/.netrc`)
- `canonical_remote_url` empty → zero behavior change (backward compatible)
- `GitClient.push_to_url()` — new method for pushing to arbitrary remote URL without a named remote
- 3 new tests: unconfigured (no push), configured (push called), failure is non-fatal

---

v0.1.4 (2026-03-17)

Sync Now + Hand Off actions on Dashboard:
- `[Sync Now]` button in Active Student card — triggers `SyncService.sync_student()` for the active student, shows success/failure dialog, refreshes Dashboard
- `[Hand Off to Next Student]` button — confirmation dialog, triggers `HandoffService.handoff_to_next()`, shows checkpoint tag on success
- Both buttons disabled when no active student is set
- `DashboardViewModel.sync_active_student()` and `handoff_to_next()` — thin wrappers composing real services
- 3 new tests for viewmodel action methods

---

v0.1.3 (2026-03-17)

Students screen — queue reorder + archive safety:
- `[↑]` and `[↓]` inline buttons to reorder students in the queue (proper swap, boundary-safe)
- Archive button now shows a confirmation dialog before acting (default: No)
- `AccountService.move_student_up()` / `move_student_down()` with archived-gap handling
- 6 new tests for move operations

---

v0.1.2 (2026-03-17)

Dashboard enhancement:
- Active Student card: shows #NN name, queue position, prompts-pushed count
- Student Queue panel: ordered list of active/paused students with status colours
- Recent Sync Activity panel: last 5 sync jobs with timestamp, student, type, colour-coded status
- 12 new tests for DashboardViewModel (get_active_student_info, get_student_queue, get_recent_sync_activity)
- student_number auto-assigned in AccountService.create_student() (backport from hotfix)

---

v0.1.1 (2026-03-17)

Prompt Delivery feature:
- PromptService: write/commit/push Markdown prompt files to active student repo
- PromptsViewModel + PromptsScreen: compose and track prompts from UI
- Prompt data model with migration (b1a2c3d4e5f6)
- `student_number` field added to Student
- Navigation: Prompts screen added at sidebar index 5
- 25 new tests (19 service + 6 viewmodel)

Canonical Repo UI (same release):
- Dashboard: shows canonical repo status, path validity, branch, commit hash
- Settings: editable form for canonical_repo_path, default_branch, sync_timeout_seconds, max_sync_retries
- Repositories screen: canonical badge (★), Set as Canonical button
- Global Qt stylesheet: Catppuccin Mocha dark theme, visible status colors

---

v0.1.0-mvp (2026-03-16)

Initial MVP release. All T01–T14 tasks complete.

Included:
- Full data model: Student, Account, Repository, Credential, SyncJob, HandoffEvent, ActivityLog, AppSetting
- 7 services: AccountService, CredentialService, RepositoryService, SyncService, HandoffService, TemplateService, AuditService
- GitClient (subprocess + GitPython), GitValidator (filesystem/regex)
- Pydantic input schemas and centralized exception hierarchy
- 8 functional UI screens: Dashboard, Students, Accounts, Repositories, Credentials, Prompts, Logs, Settings
- 4 dialogs: Student, Account, Repository, Credential
- Demo seed data (5 students, idempotent)
- 83 tests, 0 failures

Post-MVP hardening (same tag):
- `resume_handoff()` converted from `NotImplementedError` to catchable `HandoffError`
- `validate_account()` now honestly checks keyring presence and reports result to operator
- `RepositoriesScreen` now has Add Repository dialog

Security Note

StudentFlow uses the OS keyring (`keyring` library) as the exclusive store for all credentials: SSH key passphrases, personal access tokens, app passwords, and account passwords. The SQLite database stores only a key reference string (e.g., `studentflow:github:johndoe:abc123de`) — never the credential value itself. On Linux this requires `python3-secretstorage` or a compatible SecretService implementation (e.g., GNOME Keyring or KWallet). On macOS the system Keychain is used automatically. On Windows the Credential Manager is used. See `docs/security.md` for

the full threat model and `docs/setup.md` for platform-specific keyring troubleshooting.

Documentation in Ukrainian / Документація українською

- [Огляд і навігація](docs/uk/README.md) — Що таке StudentFlow, для кого, навігація
- [Розгортання](docs/uk/rozgortannya.md) — Встановлення, запуск, keyring, оновлення
- [Методика застосування](docs/uk/metodyka-zastosuvannya.md) — Операційна методика для ~20 студентів

Documentation

- [Architecture](docs/architecture.md) — System design, component map, data flow
- [Data Model](docs/data-model.md) — All entities, fields, relationships, indexes
- [Sync Strategy](docs/sync-strategy.md) — How Git sync works, merge policy, divergence handling
- [Security](docs/security.md) — Threat model, credential storage, audit trail
- [UI Flows](docs/ui-flows.md) — Screen-by-screen UI specification
- [Handoff Protocol](docs/handoff-protocol.md) — Step-by-step handoff workflow
- [Audit and Logging](docs/audit-and-logging.md) — Audit log spec, structlog config
- [Setup](docs/setup.md) — Installation, first-run, platform notes
- [Operator Runbook](docs/operator-runbook.md) — Day-to-day operations
- [Troubleshooting](docs/troubleshooting.md) — Common problems and recovery

---
app/main.py
Розмір: 1,317 байт

```
"""Entry point for StudentFlow. Supports GUI mode (default) and --cli mode."""
import argparse
import sys

def main() -> None:
parser = argparse.ArgumentParser(
prog="studentflow",
description="StudentFlow — Student workflow management desktop app"
)
parser.add_argument(
"--cli",
action="store_true",
help="Run in headless CLI mode (no GUI)"
)
parser.add_argument(
"--debug",
action="store_true",
help="Enable debug logging"
)
args = parser.parse_args()

# Bootstrap: init DB, logging, config, directories
from app.bootstrap import init_db, init_logging, create_app_dirs, get_config
create_app_dirs()
config = get_config()
_ = config
init_logging(debug=args.debug)
init_db()

if args.cli:
from app.cli import run_cli
run_cli(args)
else:
from PySide6.QtWidgets import QApplication
app = QApplication(sys.argv)
app.setApplicationName("StudentFlow")
app.setApplicationVersion("0.1.0")
```

```
# MainWindow import deferred to avoid importing PySide6 in CLI mode
from app.ui.windows.main_window import MainWindow
window = MainWindow()
window.show()
sys.exit(app.exec())

if __name__ == "__main__":
main()
```
---
app/__init__.py
Розмір: 63 байт

```
"""StudentFlow — Student workflow management desktop app."""
```
---
app/bootstrap.py
Розмір: 4,467 байт

```
"""Application bootstrap: initialize DB, logging, config, and filesystem directories."""
import sys
from contextlib import contextmanager
from pathlib import Path

APP_DIR = Path.home() / ".studentflow"
LOG_DIR = APP_DIR / "logs"
CANONICAL_DIR = APP_DIR / "canonical"
DB_PATH = APP_DIR / "studentflow.db"
CONFIG_PATH = APP_DIR / "config.json"

def create_app_dirs() -> None:
"""Create ~/.studentflow/ directory structure if it doesn't exist."""
APP_DIR.mkdir(exist_ok=True)
LOG_DIR.mkdir(exist_ok=True)
CANONICAL_DIR.mkdir(exist_ok=True)

def get_config():
"""Load Config from ~/.studentflow/config.json (creates with defaults if missing)."""
from app.config import Config

return Config.load(CONFIG_PATH)

def init_logging(debug: bool = False) -> None:
"""Configure structlog for JSON output to ~/.studentflow/logs/app.log."""
import logging
import logging.handlers
import structlog

level = logging.DEBUG if debug else logging.INFO

handler = logging.handlers.RotatingFileHandler(
filename=LOG_DIR / "app.log",
maxBytes=10 * 1024 * 1024,
backupCount=5,
encoding="utf-8",
)
handler.setLevel(level)

logging.basicConfig(
format="%(message)s",
handlers=[handler, logging.StreamHandler(sys.stdout)],
level=level,
)

structlog.configure(
processors=[
structlog.stdlib.add_log_level,
structlog.stdlib.add_logger_name,
structlog.processors.TimeStamper(fmt="iso"),
structlog.processors.JSONRenderer(),
```

```
],
context_class=dict,
logger_factory=structlog.stdlib.LoggerFactory(),
wrapper_class=structlog.stdlib.BoundLogger,
cache_logger_on_first_use=True,
)

def init_db() -> None:
"""Run Alembic migrations and seed default AppSettings if missing."""
import app.db.models # noqa: F401 — must import before create_all to register models with
Base
from app.db.base import Base, engine

# Create all tables (used before Alembic is configured, or as fallback)
Base.metadata.create_all(bind=engine)
_seed_default_settings()

def _seed_default_settings() -> None:
"""Insert default AppSetting records if they don't already exist."""
from app.db.base import SessionLocal

defaults = {
"canonical_repo_path": str(CANONICAL_DIR),
"default_branch": "main",
"template_dir": str(Path.home() / "studentflow-templates"),
"sync_timeout_seconds": "60",
"max_sync_retries": "2",
"enable_auto_sync": "false",
"auto_sync_interval_minutes": "30",
}
descriptions = {
"canonical_repo_path": "Absolute path to the canonical bare git repository",
"default_branch": "Default git branch name for new repositories",
"template_dir": "Default directory to search for project templates",
"sync_timeout_seconds": "Seconds before a git operation is killed",
"max_sync_retries": "How many times to retry a failed sync",
"enable_auto_sync": "Whether SchedulerService runs periodic sync (true/false)",
"auto_sync_interval_minutes": "Interval for auto sync if enabled",
}
with SessionLocal() as session:
from app.db.models import AppSetting
from datetime import datetime

for key, value in defaults.items():
existing = session.get(AppSetting, key)
if existing is None:
session.add(AppSetting(
key=key,
value=value,
description=descriptions.get(key),
updated_at=datetime.utcnow(),
))
session.commit()

@contextmanager
def get_db_session():
"""Context manager that yields a SQLAlchemy session and handles commit/rollback."""
from app.db.base import SessionLocal

with SessionLocal() as session:
try:
yield session
session.commit()
except Exception:
session.rollback()
raise
```

Allow running as: python -m app.bootstrap init-db

if __name__ == "__main__":
import sys as _sys

if len(_sys.argv) > 1 and _sys.argv[1] == "init-db":
create_app_dirs()
config = get_config()
_ = config
init_logging()
init_db()
print(f"Database initialized at {DB_PATH}")
else:
print("Usage: python -m app.bootstrap init-db")
---
app/cli.py
Розмір: 135 байт

"""Headless CLI entry points for StudentFlow."""

def run_cli(args) -> None:
"""Run StudentFlow in headless mode."""
_ = args
---
app/errors.py
Розмір: 1,350 байт

"""Central exception hierarchy for StudentFlow."""

class StudentFlowError(Exception):
"""Base class for all StudentFlow exceptions."""

Account / Student

class StudentNotFoundError(StudentFlowError): ...
class AccountNotFoundError(StudentFlowError): ...
class DuplicateStudentError(StudentFlowError): ...
class StudentInWrongStateError(StudentFlowError): ...

Repository

class RepositoryNotFoundError(StudentFlowError): ...
class LocalPathNotFoundError(StudentFlowError): ...
class LocalPathNotDirectoryError(StudentFlowError): ...
class RemoteURLInvalidError(StudentFlowError): ...
class RepositoryAlreadyExistsError(StudentFlowError): ...

Credential

class CredentialNotFoundError(StudentFlowError): ...
class CredentialValidationError(StudentFlowError): ...
class SecretStorageError(StudentFlowError): ...

Git / Sync

class GitOperationError(StudentFlowError): ...
class SyncConflictError(StudentFlowError): ...
class SyncTimeoutError(StudentFlowError): ...
class DivergedRepositoryError(StudentFlowError): ...

Handoff

class HandoffError(StudentFlowError): ...
class NoActiveStudentError(HandoffError): ...
class NoNextStudentError(HandoffError): ...

Template

class TemplateNotFoundError(StudentFlowError): ...
class TemplateCopyError(StudentFlowError): ...

Config / Settings

class ConfigError(StudentFlowError): ...
---
app/config.py
Розмір: 1,224 байт

"""Application configuration loaded from ~/.studentflow/config.json."""
from __future__ import annotations

import json
from dataclasses import asdict, dataclass
from pathlib import Path

@dataclass
class Config:
canonical_repo_path: str = str(Path.home() / ".studentflow" / "canonical")
template_dir: str = str(Path.home() / "studentflow-templates")
default_branch: str = "main"
sync_timeout_seconds: int = 60
max_sync_retries: int = 2
enable_auto_sync: bool = False
auto_sync_interval_minutes: int = 30
log_level: str = "INFO"

@classmethod
def load(cls, path: Path) -> "Config":
"""Load config from JSON file, creating with defaults if missing."""
if not path.exists():
config = cls()
config.save(path)
return config
with open(path, "r", encoding="utf-8") as f:
data = json.load(f)
return cls(**{k: v for k, v in data.items() if k in cls.__dataclass_fields__})

def save(self, path: Path) -> None:
"""Write config to JSON file."""
path.parent.mkdir(parents=True, exist_ok=True)
with open(path, "w", encoding="utf-8") as f:
json.dump(asdict(self), f, indent=2)
---
app/security/vault.py
Розмір: 1,473 байт

```
"""SecretVault: thin wrapper around the OS keyring via the keyring library."""
import keyring
import keyring.errors
from typing import Optional

KEYRING_SERVICE = "studentflow"

class SecretVault:
"""Single interface for all OS keyring operations.

Key format: "studentflow:{provider}:{username}:{account_id[:8]}"
Example: "studentflow:github:alice-dev:a1b2c3d4"
"""

def set(self, secret_ref: str, value: str) -> None:
"""Store or overwrite a secret in the OS keyring."""
keyring.set_password(KEYRING_SERVICE, secret_ref, value)

def get(self, secret_ref: str) -> Optional[str]:
"""Retrieve a secret from the OS keyring. Returns None if not found."""
return keyring.get_password(KEYRING_SERVICE, secret_ref)

def delete(self, secret_ref: str) -> None:
"""Delete a secret. Raises keyring.errors.PasswordDeleteError if not found."""
keyring.delete_password(KEYRING_SERVICE, secret_ref)

def exists(self, secret_ref: str) -> bool:
"""Check whether a key exists in the keyring (does not validate the value)."""
return keyring.get_password(KEYRING_SERVICE, secret_ref) is not None

@staticmethod
def make_key(provider: str, username: str, account_id: str) -> str:
"""Construct the standardized secret_ref key name.

Returns: e.g., "studentflow:github:alice-dev:a1b2c3d4"
"""
return f"studentflow:{provider}:{username}:{account_id[:8]}"
---
### app/security/__init__.py
**Розмір:** 40 байт

"""Security helpers for StudentFlow."""
---
### app/security/masking.py
**Розмір:** 882 байт

"""Credential masking utilities for safe display in UI and logs."""
import re

def mask_secret(value: str) -> str:
"""Return a masked version of a secret, showing only the last 4 characters.

Used in UI display fields and log output. Never log the unmasked value.

Examples:
mask_secret("ghp_abc123xyz789") -> "••••••••z789"
mask_secret("abcd") -> "••••"
mask_secret("") -> "••••"
"""
if not value or len(value) <= 4:
return "••••"
return "••••••••" + value[-4:]

def mask_url(url: str) -> str:
"""Remove any embedded credentials from a git remote URL.

Converts: https://token:x-oauth-basic@github.com/user/repo.git
To: https://github.com/user/repo.git
"""
return re.sub(r'https?://[^@]+@', lambda m: m.group(0).split('//')[0] + '//', url)
```

---
app/security/crypto.py
Розмір: 240 байт

```
"""Placeholder for future encryption utilities."""
```

Reserved for future use.

Current architecture stores secrets in the OS keyring (see vault.py).

This module may be used for encrypting local SQLite data at rest

in a future release.

---
app/services/account_service.py
Розмір: 12,712 байт

```
"""AccountService: CRUD for Student and Account entities, queue management."""
from __future__ import annotations

import uuid
from typing import Optional

from app.db.base import SessionLocal
from app.db.enums import StudentStatus, AccountStatus
from app.db.models import Student, Account
from app.errors import DuplicateStudentError

class AccountService:
def __init__(self, session_factory=None, git_client=None):
self._session_factory = session_factory or SessionLocal
from app.git.client import GitClient
self._git = git_client or GitClient()

# ---- Student CRUD ----

def list_students(self, include_archived: bool = False) -> list[Student]:
with self._session_factory() as session:
q = session.query(Student)
if not include_archived:
q = q.filter(Student.status != StudentStatus.archived.value)
students = q.order_by(Student.queue_position.asc().nullslast(), Student.full_name).all()
session.expunge_all()
return students

def get_student(self, student_id: str) -> Optional[Student]:
with self._session_factory() as session:
s = session.query(Student).filter(Student.id == student_id).first()
if s:
session.expunge(s)
return s

def create_student(
self,
full_name: str,
status: str = StudentStatus.active.value,
```

```
queue_position: Optional[int] = None,
notes: Optional[str] = None,
) -> Student:
from sqlalchemy import func
with self._session_factory() as session:
existing = (
session.query(Student)
.filter(Student.full_name == full_name, Student.status == status)
.first()
)
if existing:
raise DuplicateStudentError(
f"Student '{full_name}' already exists with status '{status}'"
)
max_num = session.query(func.max(Student.student_number)).scalar()
student_number = (max_num or 0) + 1
student = Student(
id=str(uuid.uuid4()),
full_name=full_name,
status=status,
queue_position=queue_position,
notes=notes,
student_number=student_number,
)
session.add(student)
session.commit()
session.refresh(student)
session.expunge(student)
return student

def update_student(
self,
student_id: str,
full_name: Optional[str] = None,
status: Optional[str] = None,
queue_position: Optional[int] = None,
notes: Optional[str] = None,
) -> Student:
with self._session_factory() as session:
student = session.query(Student).filter(Student.id == student_id).one()
if full_name is not None:
student.full_name = full_name
if status is not None:
student.status = status
if queue_position is not None:
student.queue_position = queue_position
if notes is not None:
student.notes = notes
session.commit()
session.refresh(student)
session.expunge(student)
return student

def archive_student(self, student_id: str) -> None:
with self._session_factory() as session:
student = session.query(Student).filter(Student.id == student_id).one()
student.status = StudentStatus.archived.value
session.commit()

def move_student_up(self, student_id: str) -> None:
"""Swap queue_position with the student immediately above in queue order."""
with self._session_factory() as session:
students = (
session.query(Student)
.filter(
Student.queue_position.isnot(None),
Student.status != StudentStatus.archived.value,
)
```

```
.order_by(Student.queue_position)
.all()
)
idx = next((i for i, s in enumerate(students) if s.id == student_id), None)
if idx is None or idx == 0:
return
above = students[idx - 1]
target = students[idx]
target.queue_position, above.queue_position = above.queue_position, target.queue_position
session.commit()

def move_student_down(self, student_id: str) -> None:
"""Swap queue_position with the student immediately below in queue order."""
with self._session_factory() as session:
students = (
session.query(Student)
.filter(
Student.queue_position.isnot(None),
Student.status != StudentStatus.archived.value,
)
.order_by(Student.queue_position)
.all()
)
idx = next((i for i, s in enumerate(students) if s.id == student_id), None)
if idx is None or idx >= len(students) - 1:
return
below = students[idx + 1]
target = students[idx]
target.queue_position, below.queue_position = below.queue_position, target.queue_position
session.commit()

def set_queue_position(self, student_id: str, new_position: int) -> None:
with self._session_factory() as session:
student = session.query(Student).filter(Student.id == student_id).one()
student.queue_position = new_position
session.commit()

# ---- Account CRUD ----

def list_accounts(self) -> list[Account]:
with self._session_factory() as session:
accounts = session.query(Account).order_by(Account.username).all()
session.expunge_all()
return accounts

def create_account(
self,
student_id: str,
provider: str,
username: str,
auth_type: str,
) -> Account:
with self._session_factory() as session:
account = Account(
id=str(uuid.uuid4()),
student_id=student_id,
provider=provider,
username=username,
auth_type=auth_type,
)
session.add(account)
session.commit()
session.refresh(account)
session.expunge(account)
return account

def update_account(
self,
account_id: str,
```

```
student_id: Optional[str] = None,
provider: Optional[str] = None,
username: Optional[str] = None,
auth_type: Optional[str] = None,
) -> Account:
with self._session_factory() as session:
account = session.query(Account).filter(Account.id == account_id).one()
if student_id is not None:
account.student_id = student_id
if provider is not None:
account.provider = provider
if username is not None:
account.username = username
if auth_type is not None:
account.auth_type = auth_type
session.commit()
session.refresh(account)
session.expunge(account)
return account

def deactivate_account(self, account_id: str) -> None:
"""Set account status to inactive."""
with self._session_factory() as session:
account = session.query(Account).filter(Account.id == account_id).one()
account.status = AccountStatus.inactive.value
session.commit()

def validate_account(self, account_id: str) -> dict:
"""Check credential in keyring and verify remote access.

Returns a result dict with keys:
status (str): "active" | "unvalidated" | "error"
message (str): operator-facing explanation
validated (bool): True only if credential is confirmed (keyring + network when repo exists)

When no repository is configured for the account, only keyring presence
is checked and network access is noted as unverified.
"""
from datetime import datetime, timezone
import keyring as _keyring
from app.db.models import Credential as _Credential, Repository as _Repo

with self._session_factory() as session:
account = session.query(Account).filter(Account.id == account_id).one()

credential = (
session.query(_Credential)
.filter(_Credential.account_id == account_id)
.first()
)

if credential is None:
account.status = AccountStatus.error.value
session.commit()
return {
"status": "error",
"message": (
f"No credential stored for {account.username}. "
"Add a credential on the Credentials screen first."
),
"validated": False,
}

secret_ref = credential.secret_ref
try:
secret_value = _keyring.get_password("studentflow", secret_ref)
except Exception:
secret_value = None

if not secret_value:
account.status = AccountStatus.unvalidated.value
```

```
session.commit()
return {
"status": "unvalidated",
"message": (
f"Credential reference found ({secret_ref}) but the "
"secret is not present in the OS keyring. "
"Rotate the credential to re-store it."
),
"validated": False,
}

# Keyring check passed — attempt network validation via repo remote
repo = (
session.query(_Repo)
.filter_by(account_id=account_id, is_canonical=False)
.first()
)

if repo is None:
account.status = AccountStatus.active.value
account.last_validated_at = datetime.now(timezone.utc)
session.commit()
return {
"status": "active",
"message": (
f"Credential present in keyring for {account.username} "
f"({account.provider}). "
"No repository configured — network access not verified."
),
"validated": True,
}

remote_url = repo.remote_url
auth_type = account.auth_type
username = account.username
provider = account.provider

# Network call outside the session context
try:
reachable = self._git.validate_remote(remote_url, secret_value, auth_type)
except Exception:
reachable = False

with self._session_factory() as session:
account = session.query(Account).filter(Account.id == account_id).one()
if not reachable:
account.status = AccountStatus.error.value
session.commit()
return {
"status": "error",
"message": (
f"Credential found in keyring but remote authentication failed "
f"for {username} ({provider}). "
"The token may be expired or revoked."
),
"validated": False,
}

account.status = AccountStatus.active.value
account.last_validated_at = datetime.now(timezone.utc)
session.commit()
return {
"status": "active",
"message": (
f"Remote access verified: {username} ({provider}) "
f"authenticated to {remote_url}."
),
"validated": True,
}
```

```
def get_next_in_queue(self) -> Optional[Student]:
"""Return the active student with the lowest queue_position."""
with self._session_factory() as session:
student = (
session.query(Student)
.filter(Student.status == StudentStatus.active.value)
.order_by(Student.queue_position.asc().nullslast())
.first()
)
if student:
session.expunge(student)
return student
```
---
app/services/template_service.py
Розмір: 4,151 байт

```
"""TemplateService: project template discovery and provisioning."""
from __future__ import annotations

import hashlib
from datetime import datetime
from pathlib import Path

from app.db.base import SessionLocal
from app.db.models import TemplateSource


class TemplateService:
"""Manage local project templates stored as TemplateSource records."""

def __init__(self, session_factory=None):
self._session_factory = session_factory or SessionLocal

def import_from_path(self, path: str, name: str, description: str = "") -> TemplateSource:
"""Register a local directory as a template source."""
template_path = Path(path).resolve()

if not template_path.exists():
raise ValueError(f"Path does not exist: {path}")
if not template_path.is_dir():
raise ValueError(f"Path is not a directory: {path}")

with self._session_factory() as session:
existing = session.query(TemplateSource).filter_by(local_path=str(template_path)).first()
if existing is not None:
raise ValueError(f"Template already registered for path: {path}")

template = TemplateSource(
name=name,
local_path=str(template_path),
is_active=False,
checksum=self.compute_checksum(str(template_path)),
description=description or None,
created_at=datetime.utcnow(),
updated_at=datetime.utcnow(),
)
session.add(template)
session.commit()
session.refresh(template)
return template

def validate_template(self, template_source: TemplateSource) -> bool:
"""Check that the template path exists and matches the stored checksum."""
template_path = Path(template_source.local_path)
if not template_path.exists() or not template_path.is_dir():
return False

if template_source.checksum is None:
return True

return self.compute_checksum(template_source.local_path) == template_source.checksum

def list_templates(self, active_only: bool = False) -> list[TemplateSource]:
```

```
"""Return all template records ordered by newest first."""
with self._session_factory() as session:
query = session.query(TemplateSource)
if active_only:
query = query.filter_by(is_active=True)
return query.order_by(TemplateSource.created_at.desc()).all()

def set_active(self, template_id: str) -> TemplateSource:
"""Activate one template and deactivate all others."""
with self._session_factory() as session:
template = session.get(TemplateSource, template_id)
if template is None:
raise ValueError(f"TemplateSource {template_id} not found")

session.query(TemplateSource).update(
{TemplateSource.is_active: False, TemplateSource.updated_at: datetime.utcnow()}
)
template.is_active = True
template.updated_at = datetime.utcnow()
session.commit()
session.refresh(template)
return template

def compute_checksum(self, path: str) -> str:
"""Compute a deterministic SHA256 checksum of a directory tree."""
root = Path(path)
if not root.is_dir():
raise ValueError(f"Not a directory: {path}")

file_hashes: list[str] = []
for file_path in sorted(root.rglob("*")):
if file_path.is_file():
rel_path = file_path.relative_to(root).as_posix()
content_hash = hashlib.sha256(file_path.read_bytes()).hexdigest()
file_hashes.append(f"{rel_path}:{content_hash}")

combined = "\n".join(file_hashes)
return hashlib.sha256(combined.encode("utf-8")).hexdigest()

def get_active_template(self) -> TemplateSource | None:
"""Return the active template if one exists."""
with self._session_factory() as session:
return session.query(TemplateSource).filter_by(is_active=True).first()
---
### app/services/sync_service.py
**Розмір:** 19,473 байт

"""SyncService: orchestrates repository synchronization operations."""
from __future__ import annotations

from datetime import datetime
from pathlib import Path

import structlog

from app.db.base import SessionLocal
from app.db.enums import StudentStatus, SyncJobStatus, SyncJobType, SyncStatus
from app.db.models import Account, AppSetting, Credential, Repository, Student, SyncJob
from app.git.client import GitMergeError

logger = structlog.get_logger(__name__)

class SyncService:
"""Per-student sync pipeline and sync-all batch."""

def __init__(
self,
git_client=None,
credential_service=None,
audit_service=None,
session_factory=None,
):
```

```
from app.git.client import GitClient
from app.services.audit_service import AuditService
from app.services.credential_service import CredentialService

self._session_factory = session_factory or SessionLocal
self._git = git_client or GitClient()
self._creds = credential_service or CredentialService(session_factory=self._session_factory)
self._audit = audit_service or AuditService()

def sync_student(self, student_id: str) -> SyncJob:
"""Run the full sync pipeline for one student."""
with self._session_factory() as session:
student = session.get(Student, student_id)
if student is None:
raise ValueError(f"Student {student_id} not found")

account = session.query(Account).filter_by(student_id=student_id, is_current=True).first()
if account is None:
account = session.query(Account).filter_by(student_id=student_id).first()
if account is None:
raise ValueError(f"No account found for student {student_id}")

repo = session.query(Repository).filter_by(account_id=account.id, is_canonical=False).first()
if repo is None:
raise ValueError(f"No repository found for account {account.id}")

credential = session.query(Credential).filter_by(account_id=account.id).first()
if credential is None:
raise ValueError(f"No credential for account {account.id}")

commit_hash_before = repo.last_commit_hash
sync_job = SyncJob(
repo_id=repo.id,
student_id=student_id,
job_type=SyncJobType.sync_current.value,
status=SyncJobStatus.pending.value,
commit_hash_before=commit_hash_before,
created_at=datetime.utcnow(),
)
session.add(sync_job)
session.flush()
job_id = sync_job.id

sync_job.status = SyncJobStatus.running.value
sync_job.started_at = datetime.utcnow()
session.commit()

repo_id = repo.id
local_path = repo.local_path
integration_branch = repo.integration_branch
secret_ref = credential.secret_ref
username = account.username

cred_value = None
try:
cred_value = self._creds.retrieve(secret_ref)
except KeyError as exc:
return self._fail_job(job_id, repo_id, str(exc), SyncStatus.error)

try:
self._git.fetch(local_path, "student")
except Exception as exc:
logger.error("sync_fetch_failed", student_id=student_id, error=str(exc)[:200])
return self._fail_job(
job_id,
repo_id,
f"fetch failed: {exc}",
SyncStatus.error,
sensitive_values=[cred_value],
)

try:
```

```
ahead, behind = self._git.check_divergence(local_path, "student/main")
except Exception as exc:
return self._fail_job(
job_id,
repo_id,
f"divergence check failed: {exc}",
SyncStatus.error,
sensitive_values=[cred_value],
)

if ahead > 0 and behind > 0:
logger.warning("sync_diverged", student_id=student_id, ahead=ahead, behind=behind)
return self._fail_job(
job_id,
repo_id,
f"Diverged: student ahead={ahead}, local behind={behind}. Manual resolution required.",
SyncStatus.diverged,
)

if ahead == 0:
return self._complete_job(
job_id,
skipped=True,
summary=f"Already up to date ({username})",
commit_hash_after=commit_hash_before,
)

try:
self._git.merge_ff_only(local_path, "student/main")
except GitMergeError as exc:
return self._fail_job(
job_id,
repo_id,
f"merge failed: {exc}",
SyncStatus.diverged,
sensitive_values=[cred_value],
)
except Exception as exc:
return self._fail_job(
job_id,
repo_id,
f"merge failed: {exc}",
SyncStatus.diverged,
sensitive_values=[cred_value],
)

try:
self._git.push(local_path, "origin", integration_branch, credential=None)
except Exception as exc:
return self._fail_job(
job_id,
repo_id,
f"push to canonical failed: {exc}",
SyncStatus.error,
sensitive_values=[cred_value],
)

try:
new_hash = self._git.get_commit_hash(local_path)
except Exception:
new_hash = None

canonical_push_warning = self._try_push_canonical_remote(integration_branch)

with self._session_factory() as session:
repo = session.get(Repository, repo_id)
repo.last_commit_hash = new_hash
repo.sync_status = SyncStatus.clean.value
repo.updated_at = datetime.utcnow()
```

```
job = session.get(SyncJob, job_id)
job.status = SyncJobStatus.success.value
job.finished_at = datetime.utcnow()
job.commit_hash_after = new_hash
base_summary = f"Synced {ahead} commits from {username}"
job.summary = (
base_summary + f"; {canonical_push_warning}"
if canonical_push_warning else base_summary
)
session.commit()
session.refresh(job)
session.expunge(job)

self._audit.log(
"operator",
"repository",
repo_id,
"sync",
{
"job_id": job_id,
"student_id": student_id,
"commits_synced": ahead,
"commit_before": commit_hash_before,
"commit_after": new_hash,
},
)
logger.info("sync_complete", student_id=student_id, commits_synced=ahead)
return job

def sync_all(self) -> list[SyncJob]:
"""Run sync_student for all active students in queue order."""
with self._session_factory() as session:
students = (
session.query(Student)
.filter(
Student.status == StudentStatus.active.value,
Student.queue_position.isnot(None),
)
.order_by(Student.queue_position.asc())
.all()
)
student_ids = [student.id for student in students]

results: list[SyncJob] = []
for student_id in student_ids:
try:
job = self.sync_student(student_id)
results.append(job)
except ValueError as exc:
logger.warning("sync_all_skip", student_id=student_id, reason=str(exc))
except Exception as exc:
logger.error("sync_all_student_failed", student_id=student_id, error=str(exc)[:200])
with self._session_factory() as session:
repo = (
session.query(Repository)
.join(Account, Repository.account_id == Account.id)
.filter(Account.student_id == student_id, Repository.is_canonical.is_(False))
.first()
)
if repo is None:
continue

error_job = SyncJob(
repo_id=repo.id,
student_id=student_id,
job_type=SyncJobType.sync_all.value,
status=SyncJobStatus.failed.value,
error_message=self._sanitize_error_message(str(exc)),
created_at=datetime.utcnow(),
```

```
started_at=datetime.utcnow(),
finished_at=datetime.utcnow(),
)
session.add(error_job)
session.commit()
session.refresh(error_job)
session.expunge(error_job)
results.append(error_job)

return results

def detect_divergence(self, repo_id: str) -> bool:
"""Check if a repository is diverged."""
with self._session_factory() as session:
repo = session.get(Repository, repo_id)
if repo is None:
raise ValueError(f"Repository {repo_id} not found")
local_path = repo.local_path

ahead, behind = self._git.check_divergence(local_path, "student/main")
return ahead > 0 and behind > 0

def force_overwrite_from_canonical(self, repo_id: str) -> SyncJob:
"""Reset a student repo to canonical HEAD and force-push to the student remote."""
with self._session_factory() as session:
repo = session.get(Repository, repo_id)
if repo is None:
raise ValueError(f"Repository {repo_id} not found")

account = session.get(Account, repo.account_id)
if account is None:
raise ValueError(f"Account {repo.account_id} not found")

credential = session.query(Credential).filter_by(account_id=account.id).first()
if credential is None:
raise ValueError(f"No credential for account {account.id}")

setting = session.get(AppSetting, "canonical_repo_path")
canonical_path = setting.value if setting is not None else str(Path.home() / ".studentflow" /
"canonical")

sync_job = SyncJob(
repo_id=repo.id,
student_id=account.student_id,
job_type=SyncJobType.sync_current.value,
status=SyncJobStatus.pending.value,
commit_hash_before=repo.last_commit_hash,
created_at=datetime.utcnow(),
)
session.add(sync_job)
session.flush()
job_id = sync_job.id

sync_job.status = SyncJobStatus.running.value
sync_job.started_at = datetime.utcnow()
session.commit()

local_path = repo.local_path
branch = repo.integration_branch
secret_ref = credential.secret_ref
student_id = account.student_id

cred_value = self._creds.retrieve(secret_ref)

try:
self._git.fetch(local_path, "origin")
self._git.reset_hard(local_path, "origin/main")
self._git.push(local_path, "student", branch, cred_value, auth_type=account.auth_type,
force=True)
new_hash = self._git.get_commit_hash(local_path)
except Exception as exc:
return self._fail_job(
```

```
job_id,
repo_id,
f"force overwrite failed: {exc}",
SyncStatus.error,
sensitive_values=[cred_value],
)

with self._session_factory() as session:
repo = session.get(Repository, repo_id)
repo.sync_status = SyncStatus.clean.value
repo.last_commit_hash = new_hash
repo.updated_at = datetime.utcnow()

job = session.get(SyncJob, job_id)
job.status = SyncJobStatus.success.value
job.finished_at = datetime.utcnow()
job.commit_hash_after = new_hash
job.summary = f"Force-overwrote student remote from canonical {canonical_path}"
session.commit()
session.refresh(job)
session.expunge(job)

self._audit.log(
"operator",
"repository",
repo_id,
"force_overwrite",
{"job_id": job_id, "student_id": student_id, "commit_after": new_hash},
)
return job

def get_recent_jobs(
self,
limit: int = 50,
student_id: str | None = None,
status: str | None = None,
job_type: str | None = None,
) -> list[SyncJob]:
"""Query recent SyncJob records with optional filters."""
with self._session_factory() as session:
query = session.query(SyncJob)
if student_id is not None:
query = query.filter(SyncJob.student_id == student_id)
if status is not None:
query = query.filter(SyncJob.status == status)
if job_type is not None:
query = query.filter(SyncJob.job_type == job_type)

jobs = query.order_by(SyncJob.created_at.desc()).limit(limit).all()
for job in jobs:
session.expunge(job)
return jobs

def _try_push_canonical_remote(self, branch: str) -> str | None:
"""Push the local canonical bare repo to canonical_remote_url if configured.

Returns None when:
- canonical_remote_url is not configured (empty or missing AppSetting)
- the push succeeds

Returns a warning string when the push fails for any reason.
Failure is NON-FATAL.

Authentication: if canonical_remote_account_id is set in Settings, the bound
account's stored credential is used for the push. Otherwise falls back to
system-level git authentication (SSH agent, ~/.netrc, credential manager).
"""
with self._session_factory() as session:
url_setting = session.get(AppSetting, "canonical_remote_url")
canonical_remote_url = (
url_setting.value.strip()
```

```
if url_setting and url_setting.value and url_setting.value.strip()
else None
)
if not canonical_remote_url:
return None

path_setting = session.get(AppSetting, "canonical_repo_path")
canonical_path = (
path_setting.value.strip()
if path_setting and path_setting.value and path_setting.value.strip()
else None
)

account_id_setting = session.get(AppSetting, "canonical_remote_account_id")
canonical_account_id = (
account_id_setting.value.strip()
if account_id_setting and account_id_setting.value and account_id_setting.value.strip()
else None
)

if not canonical_path:
return "canonical_remote_url is set but canonical_repo_path is not configured"

# Resolve bound account credential (if configured)
credential_value: str | None = None
auth_type: str | None = None

if canonical_account_id:
with self._session_factory() as session:
account = session.get(Account, canonical_account_id)
if account is None:
return (
f"canonical remote account (id={canonical_account_id!r}) not found — "
"update Settings to fix the binding"
)
credential = session.query(Credential).filter_by(account_id=account.id).first()
if credential is None:
return (
f"canonical remote account '{account.username}' has no stored credential — "
"add a credential in the Credentials screen"
)
bound_secret_ref = credential.secret_ref
bound_auth_type = account.auth_type

try:
credential_value = self._creds.retrieve(bound_secret_ref)
auth_type = bound_auth_type
except KeyError as exc:
return f"canonical remote credential retrieval failed: {exc}"

try:
self._git.push_to_url(
canonical_path,
canonical_remote_url,
branch,
credential=credential_value,
auth_type=auth_type,
)
logger.info(
"canonical_remote_push_succeeded",
remote=canonical_remote_url,
auth="account_binding" if canonical_account_id else "system_auth",
)
return None
except Exception as exc:
warning = f"canonical remote push failed: {str(exc)[:150]}"
logger.warning(
"canonical_remote_push_failed",
remote=canonical_remote_url,
error=str(exc)[:200],
```

```
)
return warning

def _fail_job(
self,
job_id: str,
repo_id: str,
error_msg: str,
repo_status: SyncStatus,
sensitive_values: list[str | None] | None = None,
) -> SyncJob:
"""Mark a SyncJob as failed and update repo status."""
with self._session_factory() as session:
job = session.get(SyncJob, job_id)
job.status = SyncJobStatus.failed.value
job.finished_at = datetime.utcnow()
job.error_message = self._sanitize_error_message(error_msg, sensitive_values)

repo = session.get(Repository, repo_id)
if repo is not None:
repo.sync_status = repo_status.value if isinstance(repo_status, SyncStatus) else repo_status
repo.updated_at = datetime.utcnow()

session.commit()
session.refresh(job)
session.expunge(job)
return job

def _complete_job(
self,
job_id: str,
*,
skipped: bool,
summary: str,
commit_hash_after: str | None,
) -> SyncJob:
"""Mark a SyncJob as success or skipped."""
with self._session_factory() as session:
job = session.get(SyncJob, job_id)
job.status = SyncJobStatus.skipped.value if skipped else SyncJobStatus.success.value
job.finished_at = datetime.utcnow()
job.summary = summary
job.commit_hash_after = commit_hash_after
session.commit()
session.refresh(job)
session.expunge(job)
return job

def _sanitize_error_message(
self,
message: str,
sensitive_values: list[str | None] | None = None,
) -> str:
clean_message = str(message)
for value in sensitive_values or []:
if value:
clean_message = clean_message.replace(value, "[redacted]")
return clean_message[:500]
---
### app/services/credential_service.py
**Розмір:** 5,622 байт

"""CredentialService: manages keyring-based credentials and Credential DB records.

This is the only service that interacts with the OS keyring directly.
All other services that need credentials must call this service.
"""
from __future__ import annotations
from datetime import datetime
```

```
from typing import Optional

import structlog

from app.db.base import SessionLocal
from app.db.models import Credential, Account
from app.db.enums import SecretKind
from app.security.vault import SecretVault

logger = structlog.get_logger(__name__)


class CredentialService:
"""Manages OS keyring secrets and their corresponding Credential DB records.

Key naming: studentflow:{provider}:{username}:{account_id[:8]}
DB stores: only the key name (secret_ref) — never the secret value.
"""

def __init__(self, vault: Optional[SecretVault] = None, session_factory=None):
self._vault = vault or SecretVault()
self._session_factory = session_factory or SessionLocal

def store(self, account_id: str, secret_kind: SecretKind, value: str) -> Credential:
"""Store a new credential in keyring and create a Credential DB record.

Args:
account_id: UUID of the Account this credential belongs to
secret_kind: Type of secret (pat, ssh_key, app_password, password)
value: The actual secret — stored ONLY in keyring, never in DB

Returns:
The created Credential record (contains secret_ref, not value)

Raises:
ValueError: if Account does not exist
ValueError: if Credential already exists for this account
"""
with self._session_factory() as session:
account = session.get(Account, account_id)
if account is None:
raise ValueError(f"Account {account_id} not found")

existing = session.query(Credential).filter_by(account_id=account_id).first()
if existing is not None:
raise ValueError(
f"Credential already exists for account {account_id}. Use rotate() to update."
)

secret_ref = SecretVault.make_key(
provider=account.provider,
username=account.username,
account_id=account_id,
)

# Store in keyring FIRST — if this fails, no DB record is created
self._vault.set(secret_ref, value)

credential = Credential(
account_id=account_id,
secret_ref=secret_ref,
secret_kind=secret_kind.value if isinstance(secret_kind, SecretKind) else secret_kind,
is_encrypted=True,
created_at=datetime.utcnow(),
updated_at=datetime.utcnow(),
)
session.add(credential)
session.commit()
session.refresh(credential)

logger.info("credential_stored", account_id=account_id, secret_kind=str(secret_kind))
return credential

def retrieve(self, secret_ref: str) -> str:
```

"""Retrieve a credential value from the OS keyring.

Raises:
KeyError: if no value found at secret_ref in keyring
"""
value = self._vault.get(secret_ref)
if value is None:
logger.warning("credential_not_found", secret_ref=secret_ref)
raise KeyError(f"No credential found for key: {secret_ref}")
return value

def rotate(self, credential_id: str, new_value: str) -> None:
"""Replace an existing credential value in the keyring.

The secret_ref (key name) does not change. Only the stored value changes.

Raises:
ValueError: if no Credential with credential_id exists
"""
with self._session_factory() as session:
credential = session.get(Credential, credential_id)
if credential is None:
raise ValueError(f"Credential {credential_id} not found")

self._vault.set(credential.secret_ref, new_value)

credential.last_validated_at = datetime.utcnow()
credential.updated_at = datetime.utcnow()
session.commit()

logger.info(
"credential_rotated",
credential_id=credential_id,
account_id=credential.account_id,
)

def delete(self, secret_ref: str) -> None:
"""Remove a credential from keyring and delete the Credential DB record."""
with self._session_factory() as session:
credential = session.query(Credential).filter_by(secret_ref=secret_ref).first()

try:
self._vault.delete(secret_ref)
except Exception as e:
logger.warning("credential_delete_keyring_error", secret_ref=secret_ref, error=str(e))

if credential is not None:
session.delete(credential)
session.commit()
logger.info("credential_deleted", account_id=credential.account_id)

def validate_exists(self, secret_ref: str) -> bool:
"""Check that a keyring entry exists for the given secret_ref.

Does NOT test the credential against the provider — only checks keyring.
"""
return self._vault.exists(secret_ref)

def get_by_account(self, account_id: str) -> Optional[Credential]:
"""Retrieve the Credential DB record for an account (not the value)."""
with self._session_factory() as session:
return session.query(Credential).filter_by(account_id=account_id).first()
---
app/services/handoff_service.py
Розмір: 11,533 байт

"""HandoffService: exports and handoff workflow coordination."""
from __future__ import annotations

import uuid
from datetime import date, datetime

from app.db.base import SessionLocal

```
from app.db.enums import HandoffStatus, StudentStatus, SyncStatus
from app.db.models import Account, AppSetting, HandoffEvent, Repository, Student

from app.errors import HandoffError, NoActiveStudentError, NoNextStudentError # noqa: F401

def _build_checkpoint_tag(username: str, commit_hash: str) -> str:
return f"checkpoint/{username}/{date.today().isoformat()}/{commit_hash[:8]}"

class HandoffService:
"""Coordinate handoff from the current student to the next queued student."""

def __init__(
self,
git_client,
sync_service,
credential_service,
audit_service,
session_factory=None,
):
self._session_factory = session_factory or SessionLocal
self._git = git_client
self._sync_service = sync_service
self._credential_service = credential_service
self._audit_service = audit_service

def get_active_student(self) -> Student | None:
"""Return the currently active student, or None."""
with self._session_factory() as session:
student = self._get_active_student(session)
if student is None:
return None
session.expunge(student)
return student

def get_next_student(self) -> Student | None:
"""Return the next eligible student by queue position, or None."""
with self._session_factory() as session:
current = self._get_active_student(session)
current_id = current.id if current is not None else None
student = self._get_next_student(session, current_id)
if student is None:
return None
session.expunge(student)
return student

def handoff_to_next(self, operator: str = "operator") -> HandoffEvent:
"""Run the full handoff pipeline."""
with self._session_factory() as session:
current = self._get_active_student(session)
if current is None:
raise NoActiveStudentError("No active student found")

next_student = self._get_next_student(session, current.id)
if next_student is None:
raise NoNextStudentError("No next student found in queue")

current_id = current.id
next_id = next_student.id

self._sync_service.sync_student(current_id)

event_payload: dict[str, str | None] | None = None
try:
with self._session_factory() as session:
current = session.get(Student, current_id)
next_student = session.get(Student, next_id)
if current is None or next_student is None:
raise HandoffError("Student record disappeared during handoff")

current_account = self._get_student_account(session, current.id)
next_account = self._get_student_account(session, next_student.id)
```

```
current_repo = self._get_student_repository(session, current_account.id)
next_repo = self._get_student_repository(session, next_account.id)

canonical_path = self._get_canonical_path(session, current_repo)
canonical_repo_id = self._get_canonical_repo_id(session, current_repo)
commit_hash = self._git.get_commit_hash(canonical_path)
tag_name = _build_checkpoint_tag(current_account.username, commit_hash)
self._git.create_tag(canonical_path, tag_name)

event_id = str(uuid.uuid4())
handoff_event = HandoffEvent(
id=event_id,
from_student_id=current.id,
to_student_id=next_student.id,
repo_id=canonical_repo_id,
commit_hash=commit_hash,
checkpoint_tag=tag_name,
status=HandoffStatus.pending.value,
)
session.add(handoff_event)
session.flush()

event_payload = {
"id": event_id,
"from_student_id": current.id,
"to_student_id": next_student.id,
"repo_id": canonical_repo_id,
"commit_hash": commit_hash,
"checkpoint_tag": tag_name,
}

self._bootstrap_or_update_next(next_repo, canonical_path)

credential_value = self._credential_service.retrieve(next_account.credential.secret_ref)
self._git.push(
next_repo.local_path,
"student",
next_repo.default_branch,
credential_value,
auth_type=next_account.auth_type,
)

current_account.is_current = False
next_account.is_current = True
current.status = StudentStatus.exhausted.value
next_student.status = StudentStatus.active.value
next_repo.last_commit_hash = commit_hash
next_repo.sync_status = SyncStatus.clean.value
handoff_event.status = HandoffStatus.completed.value
handoff_event.completed_at = datetime.utcnow()

session.commit()
session.refresh(handoff_event)
session.expunge(handoff_event)

self._log_audit(
operator,
"handoff_complete",
event_payload["id"],
{
"from_student_id": current_id,
"to_student_id": next_id,
"commit_hash": event_payload["commit_hash"],
"checkpoint_tag": event_payload["checkpoint_tag"],
},
)
return handoff_event
except (NoActiveStudentError, NoNextStudentError):
raise
except Exception as exc:
```

```
failed_event = self._record_failed_handoff(event_payload)
self._log_audit(
operator,
"handoff_failed",
failed_event.id if failed_event is not None else None,
{
"from_student_id": current_id,
"to_student_id": next_id,
"error": str(exc),
},
)
raise HandoffError(f"Handoff failed: {exc}") from exc

def resume_handoff(self, handoff_event_id: str) -> HandoffEvent:
"""Retry a failed handoff event. (Phase 2 — not yet implemented.)"""
with self._session_factory() as session:
handoff_event = session.get(HandoffEvent, handoff_event_id)
if handoff_event is None:
raise ValueError(f"Handoff event {handoff_event_id} not found")
if handoff_event.status != HandoffStatus.failed.value:
raise ValueError("Only failed handoff events can be resumed")
raise HandoffError(
"resume_handoff is not implemented in Phase 1. "
"Manually retry the handoff or restore from checkpoint tag."
)

def _bootstrap_or_update_next(self, repo: Repository, canonical_path: str) -> None:
if repo.sync_status == SyncStatus.uninitialized.value:
self._git.clone(canonical_path, repo.local_path, credential=None)
self._git.add_remote(repo.local_path, "student", repo.remote_url)
return

self._git.fetch(repo.local_path, "origin")
self._git.merge_ff_only(repo.local_path, f"origin/{repo.default_branch}")

def _record_failed_handoff(self, event_payload: dict[str, str | None] | None) -> HandoffEvent |
None:
if event_payload is None:
return None

with self._session_factory() as session:
handoff_event = session.get(HandoffEvent, event_payload["id"])
if handoff_event is None:
handoff_event = HandoffEvent(
id=event_payload["id"],
from_student_id=event_payload["from_student_id"],
to_student_id=event_payload["to_student_id"],
repo_id=event_payload["repo_id"],
commit_hash=event_payload["commit_hash"],
checkpoint_tag=event_payload["checkpoint_tag"],
status=HandoffStatus.failed.value,
)
session.add(handoff_event)
else:
handoff_event.status = HandoffStatus.failed.value

session.commit()
session.refresh(handoff_event)
session.expunge(handoff_event)
return handoff_event

def _log_audit(self, actor: str, action: str, entity_id: str | None, details: dict) -> None:
log = getattr(self._audit_service, "log", None)
if log is None:
return
try:
log(actor, "handoff", entity_id, action, details)
except Exception:
return
```

```
def _get_active_student(self, session) -> Student | None:
return (
session.query(Student)
.filter(Student.status == StudentStatus.active.value)
.order_by(Student.queue_position.asc(), Student.created_at.asc())
.first()
)

def _get_next_student(self, session, current_student_id: str | None) -> Student | None:
query = session.query(Student).filter(
Student.status.in_([StudentStatus.active.value, StudentStatus.paused.value]),
)
if current_student_id is not None:
query = query.filter(Student.id != current_student_id)

return query.order_by(Student.queue_position.asc(), Student.created_at.asc()).first()

def _get_student_account(self, session, student_id: str) -> Account:
account = session.query(Account).filter_by(student_id=student_id, is_current=True).first()
if account is None:
account = session.query(Account).filter_by(student_id=student_id).order_by(Account.created_at.
asc()).first()
if account is None:
raise HandoffError(f"No account found for student {student_id}")
return account

def _get_student_repository(self, session, account_id: str) -> Repository:
repo = session.query(Repository).filter_by(account_id=account_id, is_canonical=False).first()
if repo is None:
raise HandoffError(f"No repository found for account {account_id}")
return repo

def _get_canonical_path(self, session, fallback_repo: Repository) -> str:
setting = session.get(AppSetting, "canonical_repo_path")
if setting is not None and setting.value:
return setting.value

canonical_repo = session.query(Repository).filter_by(is_canonical=True).first()
if canonical_repo is not None:
return canonical_repo.local_path

return fallback_repo.local_path

def _get_canonical_repo_id(self, session, fallback_repo: Repository) -> str:
canonical_repo = session.query(Repository).filter_by(is_canonical=True).first()
if canonical_repo is not None:
return canonical_repo.id
return fallback_repo.id

__all__ = [
"HandoffError",
"HandoffService",
"NoActiveStudentError",
"NoNextStudentError",
"_build_checkpoint_tag",
]
---
### app/services/__init__.py
**Розмір:** 45 байт

"""Service layer package for StudentFlow."""
---
### app/services/prompt_service.py
**Розмір:** 12,219 байт

"""PromptService: create, commit, and push prompt files for the active student."""
from __future__ import annotations

import re
from datetime import datetime
from pathlib import Path
```

```
from app.db.base import SessionLocal
from app.db.enums import PromptStatus, StudentStatus
from app.db.models import Account, Credential, Prompt, Repository, Student

── helpers ──────────────
```

```
def __init__(
self,
git_client=None,
credential_service=None,
audit_service=None,
session_factory=None,
):
from app.git.client import GitClient
from app.services.audit_service import AuditService
from app.services.credential_service import CredentialService

self._git = git_client or GitClient()
self._creds = credential_service or CredentialService(session_factory=session_factory)
self._audit = audit_service or AuditService(session_factory=session_factory)
self._session_factory = session_factory or SessionLocal

# ── internal helpers ──────────────────────────────────────────────────────

def _get_active_student(self, session) -> Student:
student = (
session.query(Student)
.filter(Student.status == StudentStatus.active.value)
.order_by(Student.queue_position.asc(), Student.created_at.asc())
.first()
)
if student is None:
raise ValueError("No active student found")
return student

def _get_student_repo(self, session, student_id: str) -> tuple[Repository, Credential]:
account = (
session.query(Account).filter_by(student_id=student_id, is_current=True).first()
or session.query(Account).filter_by(student_id=student_id).first()
)
if account is None:
raise ValueError(f"No account for student {student_id}")
repo = session.query(Repository).filter_by(
account_id=account.id, is_canonical=False
).first()
if repo is None:
raise ValueError(f"No repository for student {student_id}")
cred = session.query(Credential).filter_by(account_id=account.id).first()
if cred is None:
raise ValueError(f"No credential for account {account.id}")
return repo, cred

def _next_seq(self, session, student_id: str) -> int:
from sqlalchemy import func
result = (
session.query(func.max(Prompt.seq_number))
.filter(Prompt.student_id == student_id)
.scalar()
)
return (result or 0) + 1

# ── public API ────────────────────────────────────────────────────────────

def create_and_push_prompt(
self,
title: str,
content: str,
created_by: str = "operator",
) -> Prompt:
"""Write prompt to student repo, commit, push. Returns persisted Prompt."""
with self._session_factory() as session:
student = self._get_active_student(session)
repo, cred = self._get_student_repo(session, student.id)

student_number = student.student_number
if student_number is None:
```

```
raise ValueError(
f"Student {student.full_name!r} (id={student.id}) has no student_number. "
"Open the Students screen, edit this student, and ensure a student number is assigned."
)
seq_number = self._next_seq(session, student.id)
slug = _make_slug(title)
file_path = _make_file_path(student_number, seq_number, slug)
branch = repo.default_branch or "main"
created_at = datetime.utcnow()

file_content = _make_file_content(
title=title,
student_number=student_number,
seq_number=seq_number,
created_by=created_by,
repo_name=repo.repo_name,
branch=branch,
prompt_text=content,
created_at=created_at,
)

prompt = Prompt(
student_id=student.id,
repository_id=repo.id,
student_number=student_number,
seq_number=seq_number,
title=title.strip(),
slug=slug,
content=content,
file_path=file_path,
git_branch=branch,
status=PromptStatus.draft.value,
created_by=created_by,
created_at=created_at,
)
session.add(prompt)
session.commit()
session.refresh(prompt)
prompt_id = prompt.id

local_path = repo.local_path
secret_ref = cred.secret_ref
auth_type = cred.secret_kind

# Write file
abs_file = Path(local_path) / file_path
abs_file.parent.mkdir(parents=True, exist_ok=True)
abs_file.write_text(file_content, encoding="utf-8")

with self._session_factory() as session:
p = session.get(Prompt, prompt_id)
p.status = PromptStatus.written.value
session.commit()

# Git add
try:
self._git.add_file(local_path, file_path)
except Exception as exc:
with self._session_factory() as session:
p = session.get(Prompt, prompt_id)
p.status = PromptStatus.failed.value
p.push_error = _sanitize_error(str(exc))
session.commit()
session.refresh(p)
session.expunge(p)
return p

# Git commit
commit_hash: str | None = None
try:
```

```
commit_msg = f"prompt: {Path(file_path).stem}"
commit_hash = self._git.commit(local_path, commit_msg)
except Exception as exc:
with self._session_factory() as session:
p = session.get(Prompt, prompt_id)
p.status = PromptStatus.failed.value
p.push_error = _sanitize_error(str(exc))
session.commit()
session.refresh(p)
session.expunge(p)
return p

with self._session_factory() as session:
p = session.get(Prompt, prompt_id)
p.status = PromptStatus.committed.value
p.git_commit_hash = commit_hash
session.commit()

self._audit.log(
"prompt_committed", "prompt", prompt_id,
details={"file_path": file_path, "commit_hash": commit_hash},
)

# Git push
try:
credential_value = self._creds.retrieve(secret_ref)
self._git.push(local_path, "student", branch,
credential=credential_value, auth_type=auth_type)
except Exception as exc:
with self._session_factory() as session:
p = session.get(Prompt, prompt_id)
p.status = PromptStatus.failed.value
p.push_error = _sanitize_error(str(exc))
session.commit()
session.expunge(p)
self._audit.log(
"prompt_push_failed", "prompt", prompt_id,
details={"file_path": file_path, "error": _sanitize_error(str(exc))},
)
return p

with self._session_factory() as session:
p = session.get(Prompt, prompt_id)
p.status = PromptStatus.pushed.value
p.pushed_at = datetime.utcnow()
session.commit()
session.refresh(p)
session.expunge(p)

self._audit.log(
"prompt_pushed", "prompt", prompt_id,
details={"file_path": file_path, "student_number": student_number},
)
return p

def retry_push(self, prompt_id: str) -> Prompt:
"""Re-attempt push for a failed/committed prompt."""
with self._session_factory() as session:
prompt = session.get(Prompt, prompt_id)
if prompt is None:
raise ValueError(f"Prompt {prompt_id} not found")
if prompt.status not in (PromptStatus.failed.value, PromptStatus.committed.value):
raise ValueError(f"Prompt {prompt_id} is not in a retryable state")
repo, cred = self._get_student_repo(session, prompt.student_id)
local_path = repo.local_path
branch = prompt.git_branch
secret_ref = cred.secret_ref
auth_type = cred.secret_kind
file_path = prompt.file_path
```

```
try:
credential_value = self._creds.retrieve(secret_ref)
self._git.push(local_path, "student", branch,
credential=credential_value, auth_type=auth_type)
except Exception as exc:
with self._session_factory() as session:
p = session.get(Prompt, prompt_id)
p.status = PromptStatus.failed.value
p.push_error = _sanitize_error(str(exc))
session.commit()
session.expunge(p)
return p

with self._session_factory() as session:
p = session.get(Prompt, prompt_id)
p.status = PromptStatus.pushed.value
p.pushed_at = datetime.utcnow()
p.push_error = None
session.commit()
session.expunge(p)

self._audit.log("prompt_pushed", "prompt", prompt_id, details={"retry": True})
return p

def list_prompts(self, student_id: str | None = None) -> list[Prompt]:
"""Return prompts ordered by created_at desc."""
with self._session_factory() as session:
q = session.query(Prompt).order_by(Prompt.created_at.desc())
if student_id is not None:
q = q.filter(Prompt.student_id == student_id)
prompts = q.all()
session.expunge_all()
return prompts
```

---

app/services/repository_service.py

Розмір: 8,972 байт

```
"""RepositoryService: repository lifecycle and local workspace management."""
from __future__ import annotations

from datetime import datetime
from pathlib import Path

from app.db.base import SessionLocal
from app.db.enums import SyncStatus
from app.db.models import Account, AppSetting, Credential, Repository, TemplateSource

class RepositoryService:
"""Manage repository bootstrap, validation, and sync status queries."""

def __init__(
self,
git_client=None,
credential_service=None,
template_service=None,
session_factory=None,
):
from app.git.client import GitClient
from app.services.credential_service import CredentialService
from app.services.template_service import TemplateService

self._git = git_client or GitClient()
self._creds = credential_service or CredentialService(session_factory=session_factory)
self._templates = template_service or TemplateService(session_factory=session_factory)
self._session_factory = session_factory or SessionLocal

def _get_canonical_path(self, session) -> str:
"""Read canonical_repo_path from AppSetting."""
setting = session.get(AppSetting, "canonical_repo_path")
if setting is None:
```

```
return str(Path.home() / ".studentflow" / "canonical")
return setting.value

def _get_default_branch(self, session) -> str:
setting = session.get(AppSetting, "default_branch")
return setting.value if setting is not None else "main"

def bootstrap_from_template(
self,
student_id: str,
account_id: str,
template_source_id: str,
local_path: str,
remote_url: str,
) -> Repository:
"""Initialize a student repository from a local template source."""
repo_path = Path(local_path)
if repo_path.exists():
raise ValueError(f"Local path already exists: {local_path}")

with self._session_factory() as session:
account = session.get(Account, account_id)
if account is None or account.student_id != student_id:
raise ValueError(f"Account {account_id} not found or not owned by student {student_id}")

credential = session.query(Credential).filter_by(account_id=account_id).first()
if credential is None:
raise ValueError(f"No credential configured for account {account_id}")

template = session.get(TemplateSource, template_source_id)
if template is None:
raise ValueError(f"TemplateSource {template_source_id} not found")
if not self._templates.validate_template(template):
raise ValueError(f"TemplateSource {template_source_id} is invalid")

canonical_path = self._get_canonical_path(session)
branch = self._get_default_branch(session)

cred_value = self._creds.retrieve(credential.secret_ref)
self._git.init_from_dir(template.local_path, local_path)
self._git.add_remote(local_path, "student", remote_url)
self._git.push(local_path, "student", branch, cred_value)
self._git.add_remote(local_path, "origin", canonical_path)
self._git.push(local_path, "origin", branch, None)
commit_hash = self._git.get_commit_hash(local_path)

with self._session_factory() as session:
repo = Repository(
account_id=account_id,
repo_name=f"{account.username}-project",
remote_url=remote_url,
local_path=local_path,
default_branch=branch,
integration_branch=branch,
sync_status=SyncStatus.clean.value,
last_commit_hash=commit_hash,
template_source_id=template_source_id,
is_canonical=False,
created_at=datetime.utcnow(),
updated_at=datetime.utcnow(),
)
session.add(repo)
session.commit()
session.refresh(repo)
return repo

def bootstrap_from_canonical(
self,
student_id: str,
account_id: str,
local_path: str,
```

```
remote_url: str,
) -> Repository:
"""Initialize a student repository by cloning from canonical."""
with self._session_factory() as session:
account = session.get(Account, account_id)
if account is None or account.student_id != student_id:
raise ValueError(f"Account {account_id} not found or not owned by student {student_id}")

credential = session.query(Credential).filter_by(account_id=account_id).first()
if credential is None:
raise ValueError(f"No credential configured for account {account_id}")

canonical_path = self._get_canonical_path(session)
if not self.validate_local_path(canonical_path):
raise ValueError(f"Canonical repo not found at {canonical_path}")

branch = self._get_default_branch(session)
username = account.username
secret_ref = credential.secret_ref

cred_value = self._creds.retrieve(secret_ref)
self._git.clone(canonical_path, local_path, credential=None)
self._git.add_remote(local_path, "student", remote_url)
self._git.push(local_path, "student", branch, cred_value)
commit_hash = self._git.get_commit_hash(local_path)

with self._session_factory() as session:
repo = Repository(
account_id=account_id,
repo_name=f"{username}-project",
remote_url=remote_url,
local_path=local_path,
default_branch=branch,
integration_branch=branch,
sync_status=SyncStatus.clean.value,
last_commit_hash=commit_hash,
template_source_id=None,
is_canonical=False,
created_at=datetime.utcnow(),
updated_at=datetime.utcnow(),
)
session.add(repo)
session.commit()
session.refresh(repo)
return repo

def validate_local_path(self, path: str) -> bool:
"""Return True when path is an accessible git repo, otherwise False."""
repo_path = Path(path)
if not repo_path.exists() or not repo_path.is_dir():
return False
return (repo_path / ".git").exists() or (repo_path / "HEAD").exists()

def validate_remote(self, repo_id: str) -> bool:
"""Check student remote connectivity with the current credential."""
with self._session_factory() as session:
repo = session.get(Repository, repo_id)
if repo is None:
return False

credential = session.query(Credential).filter_by(account_id=repo.account_id).first()
if credential is None:
return False

try:
cred_value = self._creds.retrieve(credential.secret_ref)
return bool(self._git.validate_remote(repo.remote_url, cred_value))
except Exception:
return False

def get_repo_status(self, repo_id: str) -> SyncStatus:
```

```
"""Return the cached sync status for a repository."""
with self._session_factory() as session:
repo = session.get(Repository, repo_id)
if repo is None:
raise ValueError(f"Repository {repo_id} not found")
return SyncStatus(repo.sync_status)

def get_canonical_repo(self) -> Repository | None:
"""Return the canonical Repository record, if one exists."""
with self._session_factory() as session:
return session.query(Repository).filter_by(is_canonical=True).first()

def list_repos_for_student(self, student_id: str) -> list[Repository]:
"""Return all repositories for a given student's accounts."""
with self._session_factory() as session:
return (
session.query(Repository)
.join(Account, Repository.account_id == Account.id)
.filter(Account.student_id == student_id)
.all()
)

def update_sync_status(
self,
repo_id: str,
status: SyncStatus,
commit_hash: str | None = None,
) -> None:
"""Update the cached sync status and optional commit hash."""
with self._session_factory() as session:
repo = session.get(Repository, repo_id)
if repo is None:
raise ValueError(f"Repository {repo_id} not found")

repo.sync_status = status.value if isinstance(status, SyncStatus) else status
if commit_hash is not None:
repo.last_commit_hash = commit_hash
repo.updated_at = datetime.utcnow()
session.commit()
---
### app/services/audit_service.py
**Розмір:** 3,936 байт

"""AuditService: structured audit trail and event recording."""
from __future__ import annotations

import json
import re
import sys
from datetime import datetime, timezone
from typing import Any

import structlog

from app.db.base import SessionLocal
from app.db.models import ActivityLog

log = structlog.get_logger(__name__)

_SECRET_PATTERNS = [
r"ghp_[A-Za-z0-9]{36}",
r"glpat-[A-Za-z0-9\-_]{20}",
r"github_pat_[A-Za-z0-9_]{82}",
]

_VALID_ENTITY_TYPES = {
"student",
"account",
"repository",
"credential",
"sync_job",
```

```
"handoff_event",
"app_setting",
"prompt",
"system",
}

class AuditService:
"""Persist audit records and emit matching structlog events."""

def __init__(self, session_factory=None):
self._session_factory = session_factory or SessionLocal

def log(
self,
action: str,
entity_type: str,
entity_id: str | None = None,
actor: str = "operator",
details: dict[str, Any] | None = None,
) -> None:
"""Write one ActivityLog record and one structlog event, never raising."""
try:
if entity_type not in _VALID_ENTITY_TYPES:
raise ValueError(f"Unsupported entity_type: {entity_type}")

sanitised_details = self._sanitise_details(details or {})
timestamp =
datetime.now(timezone.utc).replace(microsecond=0).isoformat().replace("+00:00", "Z")

try:
with self._session_factory() as session:
record = ActivityLog(
actor=actor,
entity_type=entity_type,
entity_id=entity_id,
action=action,
details=json.dumps(sanitised_details),
)
session.add(record)
session.commit()
except Exception as exc:
self._stderr(f"AuditService.log DB insert failed: {exc}")

log.info(
"audit",
action=action,
entity_type=entity_type,
entity_id=entity_id,
actor=actor,
details=sanitised_details,
timestamp=timestamp,
)
except Exception as exc:
self._stderr(f"AuditService.log failed: {exc}")

def get_logs(
self,
entity_type: str | None = None,
entity_id: str | None = None,
action: str | None = None,
limit: int = 200,
offset: int = 0,
) -> list[ActivityLog]:
"""Query ActivityLog with optional filters. Returns list, never raises."""
try:
with self._session_factory() as session:
query = session.query(ActivityLog)

if entity_type is not None:
query = query.filter(ActivityLog.entity_type == entity_type)
```

```
if entity_id is not None:
query = query.filter(ActivityLog.entity_id == entity_id)
if action is not None:
query = query.filter(ActivityLog.action == action)

return (
query.order_by(ActivityLog.created_at.desc())
.offset(max(offset, 0))
.limit(max(limit, 0))
.all()
)
except Exception as exc:
self._stderr(f"AuditService.get_logs failed: {exc}")
return []

def _sanitise_details(self, details: dict[str, Any]) -> dict[str, Any]:
"""Strip secret patterns from JSON-serialised values without mutating input."""
payload = json.dumps(details)
for pattern in _SECRET_PATTERNS:
payload = re.sub(pattern, "[REDACTED]", payload)
return json.loads(payload)

@staticmethod
def _stderr(message: str) -> None:
print(message, file=sys.stderr)
---
### app/jobs/scheduler.py
**Розмір:** 104 байт

"""SchedulerService: APScheduler-based background job management."""

class SchedulerService:
pass
---
### app/jobs/__init__.py
**Розмір:** 47 байт

"""Background jobs package for StudentFlow."""
---
### app/jobs/workers.py
**Розмір:** 221 байт

"""QThread workers for async Git and sync operations."""
try:
from PySide6.QtCore import QThread
except ImportError:
QThread = object # Allow import in non-GUI/test contexts

class SyncWorker(QThread):
pass
---
### app/db/base.py
**Розмір:** 707 байт

"""SQLAlchemy engine, session factory, and declarative base."""
```

Convention: all PKs are UUID strings generated as str(uuid.uuid4())

Future models should use default=lambda: str(uuid.uuid4()) for ID columns.

```
import uuid
from pathlib import Path

from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

_ = uuid
_DB_PATH = Path.home() / ".studentflow" / "studentflow.db"
DATABASE_URL = f"sqlite:///{_DB_PATH}"

engine = create_engine(
DATABASE_URL,
connect_args={"check_same_thread": False},
echo=False,
)

SessionLocal = sessionmaker(
bind=engine,
autocommit=False,
autoflush=False,
expire_on_commit=False,
)

Base = declarative_base()
```
---
app/db/__init__.py
Розмір: 40 байт

```
"""Database package for StudentFlow."""
```
---
app/db/enums.py
Розмір: 1,826 байт

```
"""All enum types for StudentFlow database models."""
import enum

class StudentStatus(str, enum.Enum):
active = "active"
paused = "paused"
exhausted = "exhausted"
archived = "archived"
error = "error"

class AccountProvider(str, enum.Enum):
github = "github"
gitlab = "gitlab"
bitbucket = "bitbucket"
gitea = "gitea"
other = "other"

class AuthType(str, enum.Enum):
ssh = "ssh"
pat = "pat"
app_password = "app_password"
password = "password"

class AccountStatus(str, enum.Enum):
active = "active"
inactive = "inactive"
error = "error"
unvalidated = "unvalidated"

class SyncStatus(str, enum.Enum):
clean = "clean"
dirty = "dirty"
diverged = "diverged"
error = "error"
uninitialized = "uninitialized"
```

```
class SyncJobType(str, enum.Enum):
sync_current = "sync_current"
sync_all = "sync_all"
bootstrap = "bootstrap"
handoff_push = "handoff_push"
handoff_pull = "handoff_pull"

class SyncJobStatus(str, enum.Enum):
pending = "pending"
running = "running"
success = "success"
failed = "failed"
skipped = "skipped"

class HandoffStatus(str, enum.Enum):
pending = "pending"
completed = "completed"
failed = "failed"
rolled_back = "rolled_back"

class EntityType(str, enum.Enum):
student = "student"
account = "account"
repository = "repository"
credential = "credential"
sync_job = "sync_job"
handoff = "handoff"
template = "template"
system = "system"

class SecretKind(str, enum.Enum):
ssh_key = "ssh_key"
pat = "pat"
app_password = "app_password"
password = "password"

class PromptStatus(str, enum.Enum):
draft = "draft"
written = "written"
committed = "committed"
pushed = "pushed"
failed = "failed"
```
---
app/db/seeds.py
Розмір: 6,166 байт

```
"""Demo seed data for StudentFlow. Idempotent — safe to run multiple times."""
import uuid
from datetime import datetime, timezone

from app.db.base import SessionLocal
from app.db.enums import (
StudentStatus, AccountStatus, AuthType, SyncStatus, SecretKind,
)

DEMO_STUDENTS = [
{"full_name": "Alice Kovalenko", "queue_position": 1, "status": StudentStatus.exhausted},
{"full_name": "Bohdan Marchenko", "queue_position": 2, "status": StudentStatus.active},
{"full_name": "Daria Lysenko", "queue_position": 3, "status": StudentStatus.paused},
{"full_name": "Evhen Petrenko", "queue_position": 4, "status": StudentStatus.paused},
{"full_name": "Fedir Bondarenko", "queue_position": 5, "status": StudentStatus.paused},
]

DEMO_ACCOUNTS = [
{"provider": "github", "username": "alice-kovalenko", "auth_type": AuthType.pat, "secret_kind":
SecretKind.pat},
```

```
{"provider": "github", "username": "b-marchenko", "auth_type": AuthType.pat, "secret_kind":
SecretKind.pat},
{"provider": "gitlab", "username": "daria.lysenko", "auth_type": AuthType.ssh, "secret_kind":
SecretKind.ssh_key},
{"provider": "github", "username": "evhen-petrenko", "auth_type": AuthType.pat, "secret_kind":
SecretKind.pat},
{"provider": "bitbucket", "username": "fbondarenko", "auth_type": AuthType.app_password,
"secret_kind": SecretKind.app_password},
]

def seed_demo_data() -> None:
"""Insert demo students, accounts, repos, credentials, and audit logs.

Safe to call multiple times — existing records are not duplicated.
"""
import app.db.models # noqa: F401 — ensure models are registered
from app.db.models import Student, Account, Repository, Credential, ActivityLog

with SessionLocal() as session:
existing_names = {s.full_name for s in session.query(Student).all()}

student_objects = []
for i, s_data in enumerate(DEMO_STUDENTS):
if s_data["full_name"] in existing_names:
student = session.query(Student).filter_by(full_name=s_data["full_name"]).first()
else:
student = Student(
full_name=s_data["full_name"],
status=s_data["status"].value,
queue_position=s_data["queue_position"],
notes=f"Demo student #{i + 1}",
student_number=i + 1,
)
session.add(student)
student_objects.append(student)

session.flush()

for i, (student, acc_data) in enumerate(zip(student_objects, DEMO_ACCOUNTS)):
existing_acc = session.query(Account).filter_by(
student_id=student.id, username=acc_data["username"]
).first()

if existing_acc:
account = existing_acc
else:
account = Account(
student_id=student.id,
provider=acc_data["provider"],
username=acc_data["username"],
auth_type=acc_data["auth_type"].value,
status=AccountStatus.active.value,
)
session.add(account)
session.flush()

repo = Repository(
account_id=account.id,
repo_name=f"student-project-{i + 1}",
remote_url=f"https://{acc_data['provider']}.com/{acc_data['username']}/student-project.git",
local_path=f"/tmp/studentflow-demo/repos/student-{i + 1}",
default_branch="main",
integration_branch="integration",
sync_status=SyncStatus.uninitialized.value,
)
session.add(repo)

# Credential: no actual secret — demo placeholder ref only
secret_ref = f"demo-token-ref-{i + 1}"
credential = Credential(
```

```
account_id=account.id,
secret_ref=secret_ref,
secret_kind=acc_data["secret_kind"].value,
is_encrypted=True,
)
session.add(credential)

session.flush()

audit_count = session.query(ActivityLog).count()
if audit_count < 5:
actions = [
("student_created", "student", None, {"message": "Alice added to the system"}),
("sync_completed", "sync_job", None, {"message": "Alice sync: 3 commits pushed to
canonical"}),
("handoff_completed", "handoff_event",None, {"message": "Handoff Alice → Bohdan, tag
checkpoint/alice-kovalenko/2025-01-15/abc12345"}),
("student_created", "student", None, {"message": "Bohdan added to the system"}),
("sync_started", "sync_job", None, {"message": "Bohdan initial sync started"}),
("credential_stored", "credential", None, {"message": "PAT stored for b-marchenko"}),
("credential_validated", "credential", None, {"message": "PAT validated OK for b-marchenko"}),
("repo_created", "repository", None, {"message": "student-project-2 created from template"}),
("sync_completed", "sync_job", None, {"message": "Bohdan sync: up to date"}),
("settings_updated", "app_setting", None, {"message": "canonical_repo_path updated"}),
]
for action, entity_type, entity_id, details in actions:
session.add(ActivityLog(
actor="operator",
entity_type=entity_type,
entity_id=entity_id,
action=action,
details=details,
))

session.commit()
print(f"Seed data: {len(DEMO_STUDENTS)} students, accounts, repos, credentials, audit entries
inserted.")

if __name__ == "__main__":
from app.bootstrap import create_app_dirs, init_db, init_logging
create_app_dirs()
init_logging()
init_db()
seed_demo_data()
---
### app/db/models.py
**Розмір:** 11,040 байт

"""SQLAlchemy ORM models for the StudentFlow database schema."""
import uuid
from datetime import datetime

from sqlalchemy import JSON, Boolean, Column, DateTime, ForeignKey, Index, Integer, String,
Text
from sqlalchemy.orm import relationship

from app.db.base import Base
from app.db.enums import (
AccountStatus,
AuthType,
HandoffStatus,
PromptStatus,
SecretKind,
StudentStatus,
SyncJobStatus,
SyncJobType,
SyncStatus,
)
```

```
class Student(Base):
__tablename__ = "student"

id = Column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
full_name = Column(String(255), nullable=False)
email = Column(String(255), nullable=True)
status = Column(String(20), nullable=False, default=StudentStatus.active.value)
queue_position = Column(Integer, nullable=True)
priority = Column(Integer, nullable=False, default=0)
notes = Column(Text, nullable=True)
student_number = Column(Integer, nullable=True, unique=True)
created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
updated_at = Column(DateTime, nullable=False, default=datetime.utcnow,
onupdate=datetime.utcnow)

accounts = relationship("Account", back_populates="student", cascade="all, delete-orphan")
sync_jobs = relationship("SyncJob", back_populates="student")
prompts = relationship("Prompt", back_populates="student")
handoffs_from = relationship(
"HandoffEvent",
foreign_keys="HandoffEvent.from_student_id",
back_populates="from_student",
)
handoffs_to = relationship(
"HandoffEvent",
foreign_keys="HandoffEvent.to_student_id",
back_populates="to_student",
)

__table_args__ = (
Index("idx_student_status", "status"),
Index("idx_student_queue_position", "queue_position"),
Index("idx_student_number", "student_number"),
)

class Account(Base):
__tablename__ = "account"

id = Column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
student_id = Column(String(36), ForeignKey("student.id"), nullable=False)
provider = Column(String(20), nullable=False)
username = Column(String(255), nullable=False)
auth_type = Column(String(20), nullable=False)
status = Column(String(20), nullable=False, default=AccountStatus.unvalidated.value)
last_validated_at = Column(DateTime, nullable=True)
is_current = Column(Boolean, nullable=False, default=False)
created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
updated_at = Column(DateTime, nullable=False, default=datetime.utcnow,
onupdate=datetime.utcnow)

student = relationship("Student", back_populates="accounts")
repositories = relationship("Repository", back_populates="account", cascade="all,
delete-orphan")
credential = relationship(
"Credential",
back_populates="account",
cascade="all, delete-orphan",
uselist=False,
)

__table_args__ = (
Index("idx_account_student_id", "student_id"),
Index("idx_account_is_current", "is_current"),
Index("idx_account_status", "status"),
)

class TemplateSource(Base):
__tablename__ = "template_source"

id = Column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
```

```
name = Column(String(255), nullable=False)
local_path = Column(String(255), nullable=False)
is_active = Column(Boolean, nullable=False, default=True)
checksum = Column(String(64), nullable=True)
description = Column(Text, nullable=True)
created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
updated_at = Column(DateTime, nullable=False, default=datetime.utcnow,
onupdate=datetime.utcnow)

repositories = relationship("Repository", back_populates="template_source")

class Repository(Base):
__tablename__ = "repository"

id = Column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
account_id = Column(String(36), ForeignKey("account.id"), nullable=False)
repo_name = Column(String(255), nullable=False)
remote_url = Column(String(255), nullable=False)
local_path = Column(String(255), nullable=False)
default_branch = Column(String(255), nullable=False, default="main")
integration_branch = Column(String(255), nullable=False, default="main")
sync_status = Column(String(20), nullable=False, default=SyncStatus.uninitialized.value)
last_commit_hash = Column(String(40), nullable=True)
template_source_id = Column(String(36), ForeignKey("template_source.id"), nullable=True)
is_canonical = Column(Boolean, nullable=False, default=False)
created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
updated_at = Column(DateTime, nullable=False, default=datetime.utcnow,
onupdate=datetime.utcnow)

account = relationship("Account", back_populates="repositories")
template_source = relationship("TemplateSource", back_populates="repositories")
sync_jobs = relationship("SyncJob", back_populates="repository")
handoff_events = relationship("HandoffEvent", back_populates="repository")
prompts = relationship("Prompt", back_populates="repository")

__table_args__ = (
Index("idx_repository_account_id", "account_id"),
Index("idx_repository_sync_status", "sync_status"),
Index("idx_repository_is_canonical", "is_canonical"),
)

class Credential(Base):
__tablename__ = "credential"

id = Column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
account_id = Column(String(36), ForeignKey("account.id"), nullable=False, unique=True)
secret_ref = Column(String(255), nullable=False)
secret_kind = Column(String(20), nullable=False)
is_encrypted = Column(Boolean, nullable=False, default=True)
last_validated_at = Column(DateTime, nullable=True)
created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
updated_at = Column(DateTime, nullable=False, default=datetime.utcnow,
onupdate=datetime.utcnow)

account = relationship("Account", back_populates="credential")

class SyncJob(Base):
__tablename__ = "sync_job"

id = Column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
repo_id = Column(String(36), ForeignKey("repository.id"), nullable=False)
student_id = Column(String(36), ForeignKey("student.id"), nullable=False)
job_type = Column(String(20), nullable=False)
status = Column(String(20), nullable=False, default=SyncJobStatus.pending.value)
started_at = Column(DateTime, nullable=True)
finished_at = Column(DateTime, nullable=True)
summary = Column(Text, nullable=True)
error_message = Column(Text, nullable=True)
commit_hash_before = Column(String(40), nullable=True)
commit_hash_after = Column(String(40), nullable=True)
```

```
created_at = Column(DateTime, nullable=False, default=datetime.utcnow)

repository = relationship("Repository", back_populates="sync_jobs")
student = relationship("Student", back_populates="sync_jobs")

__table_args__ = (
Index("idx_syncjob_repo_id", "repo_id"),
Index("idx_syncjob_student_id", "student_id"),
Index("idx_syncjob_status", "status"),
Index("idx_syncjob_job_type", "job_type"),
)

class HandoffEvent(Base):
__tablename__ = "handoff_event"

id = Column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
from_student_id = Column(String(36), ForeignKey("student.id"), nullable=True)
to_student_id = Column(String(36), ForeignKey("student.id"), nullable=False)
repo_id = Column(String(36), ForeignKey("repository.id"), nullable=False)
commit_hash = Column(String(40), nullable=False)
checkpoint_tag = Column(String(255), nullable=False)
status = Column(String(20), nullable=False, default=HandoffStatus.pending.value)
notes = Column(Text, nullable=True)
created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
completed_at = Column(DateTime, nullable=True)

from_student = relationship("Student", foreign_keys=[from_student_id],
back_populates="handoffs_from")
to_student = relationship("Student", foreign_keys=[to_student_id],
back_populates="handoffs_to")
repository = relationship("Repository", back_populates="handoff_events")

class ActivityLog(Base):
__tablename__ = "activity_log"

id = Column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
actor = Column(String(100), nullable=False, default="operator")
entity_type = Column(String(50), nullable=False)
entity_id = Column(String(36), nullable=True)
action = Column(String(100), nullable=False)
details = Column(JSON, nullable=True)
created_at = Column(DateTime, nullable=False, default=datetime.utcnow)

__table_args__ = (
Index("idx_activitylog_entity_type", "entity_type"),
Index("idx_activitylog_entity_id", "entity_id"),
Index("idx_activitylog_created_at", "created_at"),
)

class AppSetting(Base):
__tablename__ = "app_setting"

key = Column(String(100), primary_key=True)
value = Column(Text, nullable=False)
description = Column(Text, nullable=True)
updated_at = Column(DateTime, nullable=False, default=datetime.utcnow,
onupdate=datetime.utcnow)

class Prompt(Base):
__tablename__ = "prompt"

id = Column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
student_id = Column(String(36), ForeignKey("student.id"), nullable=False)
repository_id = Column(String(36), ForeignKey("repository.id"), nullable=False)
student_number = Column(Integer, nullable=False)
seq_number = Column(Integer, nullable=False)
title = Column(String(255), nullable=False, default="")
slug = Column(String(255), nullable=False)
content = Column(Text, nullable=False)
file_path = Column(String(255), nullable=False)
```

```
git_branch = Column(String(255), nullable=False)
git_commit_hash = Column(String(40), nullable=True)
status = Column(String(20), nullable=False, default=PromptStatus.draft.value)
push_error = Column(Text, nullable=True)
created_by = Column(String(100), nullable=False, default="operator")
created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
pushed_at = Column(DateTime, nullable=True)

student = relationship("Student", back_populates="prompts")
repository = relationship("Repository", back_populates="prompts")

__table_args__ = (
Index("idx_prompt_student_id", "student_id"),
Index("idx_prompt_status", "status"),
Index("idx_prompt_student_seq", "student_id", "seq_number"),
)

__all__ = [
"Student",
"Account",
"Repository",
"Credential",
"SyncJob",
"HandoffEvent",
"TemplateSource",
"ActivityLog",
"AppSetting",
"Prompt",
"StudentStatus",
"PromptStatus",
"AccountStatus",
"AuthType",
"SyncStatus",
"SyncJobType",
"SyncJobStatus",
"HandoffStatus",
"SecretKind",
]
---
### app/db/migrations/README
**Розмір:** 38 байт

Generic single-database configuration.
---
### app/db/migrations/.gitkeep
**Розмір:** 1 байт | **Прихований файл**


---
### app/db/migrations/env.py
**Розмір:** 2,162 байт

from logging.config import fileConfig

from sqlalchemy import engine_from_config
from sqlalchemy import pool

from alembic import context

# this is the Alembic Config object, which provides


# access to the values within the .ini file in use.


config = context.config
```

Interpret the config file for Python logging.

This line sets up loggers basically.

```
if config.config_file_name is not None:
fileConfig(config.config_file_name)
```

add your model's MetaData object here

for 'autogenerate' support

```
import app.db.models # noqa: F401 — registers all models with Base before autogenerate
from app.db.base import Base

target_metadata = Base.metadata
```

other values from the config, defined by the needs of env.py,

can be acquired:

my_important_option = config.get_main_option("my_important_option")

... etc.

```
def run_migrations_offline() -> None:
"""Run migrations in 'offline' mode.

This configures the context with just a URL
and not an Engine, though an Engine is acceptable
here as well. By skipping the Engine creation
we don't even need a DBAPI to be available.

Calls to context.execute() here emit the given string to the
script output.

"""
url = config.get_main_option("sqlalchemy.url")
context.configure(
url=url,
target_metadata=target_metadata,
literal_binds=True,
dialect_opts={"paramstyle": "named"},
)

with context.begin_transaction():
context.run_migrations()

def run_migrations_online() -> None:
```

"""Run migrations in 'online' mode.

In this scenario we need to create an Engine
and associate a connection with the context.

"""
connectable = engine_from_config(
config.get_section(config.config_ini_section, {}),
prefix="sqlalchemy.",
poolclass=pool.NullPool,
)

with connectable.connect() as connection:
context.configure(
connection=connection, target_metadata=target_metadata
)

with context.begin_transaction():
context.run_migrations()

if context.is_offline_mode():
run_migrations_offline()
else:
run_migrations_online()
---
app/db/migrations/versions/7ca526e3ce7e_initial_schema.py
Розмір: 9,699 байт

"""initial_schema

Revision ID: 7ca526e3ce7e
Revises:
Create Date: 2026-03-16 01:04:22.694148

"""
from typing import Sequence, Union

from alembic import op
import sqlalchemy as sa

revision identifiers, used by Alembic.

revision: str = '7ca526e3ce7e'
down_revision: Union[str, None] = None
branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None

def upgrade() -> None:
# ### commands auto generated by Alembic - please adjust! ###
op.create_table('activity_log',
sa.Column('id', sa.String(length=36), nullable=False),
sa.Column('actor', sa.String(length=100), nullable=False),
sa.Column('entity_type', sa.String(length=50), nullable=False),
sa.Column('entity_id', sa.String(length=36), nullable=True),
sa.Column('action', sa.String(length=100), nullable=False),
sa.Column('details', sa.JSON(), nullable=True),
sa.Column('created_at', sa.DateTime(), nullable=False),
sa.PrimaryKeyConstraint('id')
)
op.create_index('idx_activitylog_created_at', 'activity_log', ['created_at'], unique=False)
op.create_index('idx_activitylog_entity_id', 'activity_log', ['entity_id'], unique=False)
op.create_index('idx_activitylog_entity_type', 'activity_log', ['entity_type'], unique=False)
op.create_table('app_setting',
sa.Column('key', sa.String(length=100), nullable=False),
sa.Column('value', sa.Text(), nullable=False),
sa.Column('description', sa.Text(), nullable=True),
sa.Column('updated_at', sa.DateTime(), nullable=False),

```
sa.PrimaryKeyConstraint('key')
)
op.create_table('student',
sa.Column('id', sa.String(length=36), nullable=False),
sa.Column('full_name', sa.String(length=255), nullable=False),
sa.Column('email', sa.String(length=255), nullable=True),
sa.Column('status', sa.String(length=20), nullable=False),
sa.Column('queue_position', sa.Integer(), nullable=True),
sa.Column('priority', sa.Integer(), nullable=False),
sa.Column('notes', sa.Text(), nullable=True),
sa.Column('created_at', sa.DateTime(), nullable=False),
sa.Column('updated_at', sa.DateTime(), nullable=False),
sa.PrimaryKeyConstraint('id')
)
op.create_index('idx_student_queue_position', 'student', ['queue_position'], unique=False)
op.create_index('idx_student_status', 'student', ['status'], unique=False)
op.create_table('template_source',
sa.Column('id', sa.String(length=36), nullable=False),
sa.Column('name', sa.String(length=255), nullable=False),
sa.Column('local_path', sa.String(length=255), nullable=False),
sa.Column('is_active', sa.Boolean(), nullable=False),
sa.Column('checksum', sa.String(length=64), nullable=True),
sa.Column('description', sa.Text(), nullable=True),
sa.Column('created_at', sa.DateTime(), nullable=False),
sa.Column('updated_at', sa.DateTime(), nullable=False),
sa.PrimaryKeyConstraint('id')
)
op.create_table('account',
sa.Column('id', sa.String(length=36), nullable=False),
sa.Column('student_id', sa.String(length=36), nullable=False),
sa.Column('provider', sa.String(length=20), nullable=False),
sa.Column('username', sa.String(length=255), nullable=False),
sa.Column('auth_type', sa.String(length=20), nullable=False),
sa.Column('status', sa.String(length=20), nullable=False),
sa.Column('last_validated_at', sa.DateTime(), nullable=True),
sa.Column('is_current', sa.Boolean(), nullable=False),
sa.Column('created_at', sa.DateTime(), nullable=False),
sa.Column('updated_at', sa.DateTime(), nullable=False),
sa.ForeignKeyConstraint(['student_id'], ['student.id'], ),
sa.PrimaryKeyConstraint('id')
)
op.create_index('idx_account_is_current', 'account', ['is_current'], unique=False)
op.create_index('idx_account_status', 'account', ['status'], unique=False)
op.create_index('idx_account_student_id', 'account', ['student_id'], unique=False)
op.create_table('credential',
sa.Column('id', sa.String(length=36), nullable=False),
sa.Column('account_id', sa.String(length=36), nullable=False),
sa.Column('secret_ref', sa.String(length=255), nullable=False),
sa.Column('secret_kind', sa.String(length=20), nullable=False),
sa.Column('is_encrypted', sa.Boolean(), nullable=False),
sa.Column('last_validated_at', sa.DateTime(), nullable=True),
sa.Column('created_at', sa.DateTime(), nullable=False),
sa.Column('updated_at', sa.DateTime(), nullable=False),
sa.ForeignKeyConstraint(['account_id'], ['account.id'], ),
sa.PrimaryKeyConstraint('id'),
sa.UniqueConstraint('account_id')
)
op.create_table('repository',
sa.Column('id', sa.String(length=36), nullable=False),
sa.Column('account_id', sa.String(length=36), nullable=False),
sa.Column('repo_name', sa.String(length=255), nullable=False),
sa.Column('remote_url', sa.String(length=255), nullable=False),
sa.Column('local_path', sa.String(length=255), nullable=False),
sa.Column('default_branch', sa.String(length=255), nullable=False),
sa.Column('integration_branch', sa.String(length=255), nullable=False),
sa.Column('sync_status', sa.String(length=20), nullable=False),
sa.Column('last_commit_hash', sa.String(length=40), nullable=True),
```

```
sa.Column('template_source_id', sa.String(length=36), nullable=True),
sa.Column('is_canonical', sa.Boolean(), nullable=False),
sa.Column('created_at', sa.DateTime(), nullable=False),
sa.Column('updated_at', sa.DateTime(), nullable=False),
sa.ForeignKeyConstraint(['account_id'], ['account.id'], ),
sa.ForeignKeyConstraint(['template_source_id'], ['template_source.id'], ),
sa.PrimaryKeyConstraint('id')
)
op.create_index('idx_repository_account_id', 'repository', ['account_id'], unique=False)
op.create_index('idx_repository_is_canonical', 'repository', ['is_canonical'], unique=False)
op.create_index('idx_repository_sync_status', 'repository', ['sync_status'], unique=False)
op.create_table('handoff_event',
sa.Column('id', sa.String(length=36), nullable=False),
sa.Column('from_student_id', sa.String(length=36), nullable=True),
sa.Column('to_student_id', sa.String(length=36), nullable=False),
sa.Column('repo_id', sa.String(length=36), nullable=False),
sa.Column('commit_hash', sa.String(length=40), nullable=False),
sa.Column('checkpoint_tag', sa.String(length=255), nullable=False),
sa.Column('status', sa.String(length=20), nullable=False),
sa.Column('notes', sa.Text(), nullable=True),
sa.Column('created_at', sa.DateTime(), nullable=False),
sa.Column('completed_at', sa.DateTime(), nullable=True),
sa.ForeignKeyConstraint(['from_student_id'], ['student.id'], ),
sa.ForeignKeyConstraint(['repo_id'], ['repository.id'], ),
sa.ForeignKeyConstraint(['to_student_id'], ['student.id'], ),
sa.PrimaryKeyConstraint('id')
)
op.create_table('sync_job',
sa.Column('id', sa.String(length=36), nullable=False),
sa.Column('repo_id', sa.String(length=36), nullable=False),
sa.Column('student_id', sa.String(length=36), nullable=False),
sa.Column('job_type', sa.String(length=20), nullable=False),
sa.Column('status', sa.String(length=20), nullable=False),
sa.Column('started_at', sa.DateTime(), nullable=True),
sa.Column('finished_at', sa.DateTime(), nullable=True),
sa.Column('summary', sa.Text(), nullable=True),
sa.Column('error_message', sa.Text(), nullable=True),
sa.Column('commit_hash_before', sa.String(length=40), nullable=True),
sa.Column('commit_hash_after', sa.String(length=40), nullable=True),
sa.Column('created_at', sa.DateTime(), nullable=False),
sa.ForeignKeyConstraint(['repo_id'], ['repository.id'], ),
sa.ForeignKeyConstraint(['student_id'], ['student.id'], ),
sa.PrimaryKeyConstraint('id')
)
op.create_index('idx_syncjob_job_type', 'sync_job', ['job_type'], unique=False)
op.create_index('idx_syncjob_repo_id', 'sync_job', ['repo_id'], unique=False)
op.create_index('idx_syncjob_status', 'sync_job', ['status'], unique=False)
op.create_index('idx_syncjob_student_id', 'sync_job', ['student_id'], unique=False)
# ### end Alembic commands ###

def downgrade() -> None:
# ### commands auto generated by Alembic - please adjust! ###
op.drop_index('idx_syncjob_student_id', table_name='sync_job')
op.drop_index('idx_syncjob_status', table_name='sync_job')
op.drop_index('idx_syncjob_repo_id', table_name='sync_job')
op.drop_index('idx_syncjob_job_type', table_name='sync_job')
op.drop_table('sync_job')
op.drop_table('handoff_event')
op.drop_index('idx_repository_sync_status', table_name='repository')
op.drop_index('idx_repository_is_canonical', table_name='repository')
op.drop_index('idx_repository_account_id', table_name='repository')
op.drop_table('repository')
op.drop_table('credential')
op.drop_index('idx_account_student_id', table_name='account')
op.drop_index('idx_account_status', table_name='account')
op.drop_index('idx_account_is_current', table_name='account')
op.drop_table('account')
```

```
op.drop_table('template_source')
op.drop_index('idx_student_status', table_name='student')
op.drop_index('idx_student_queue_position', table_name='student')
op.drop_table('student')
op.drop_table('app_setting')
op.drop_index('idx_activitylog_entity_type', table_name='activity_log')
op.drop_index('idx_activitylog_entity_id', table_name='activity_log')
op.drop_index('idx_activitylog_created_at', table_name='activity_log')
op.drop_table('activity_log')
# ### end Alembic commands ###
```
---
app/db/migrations/versions/b1a2c3d4e5f6_add_prompt_and_student_number.py
Розмір: 2,731 байт

```
"""add_prompt_and_student_number

Revision ID: b1a2c3d4e5f6
Revises: 7ca526e3ce7e
Create Date: 2026-03-17 12:00:00.000000

"""
from typing import Sequence, Union

from alembic import op
import sqlalchemy as sa

revision: str = 'b1a2c3d4e5f6'
down_revision: Union[str, None] = '7ca526e3ce7e'
branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None

def upgrade() -> None:
op.add_column('student', sa.Column('student_number', sa.Integer(), nullable=True))
op.create_index('idx_student_number', 'student', ['student_number'], unique=True)

# Assign sequential student_numbers to existing students ordered by created_at
op.execute("""
UPDATE student SET student_number = (
SELECT COUNT(*) FROM student s2
WHERE s2.created_at < student.created_at
OR (s2.created_at = student.created_at AND s2.id <= student.id)
)
""")

op.create_table(
'prompt',
sa.Column('id', sa.String(36), nullable=False),
sa.Column('student_id', sa.String(36), nullable=False),
sa.Column('repository_id', sa.String(36), nullable=False),
sa.Column('student_number', sa.Integer(), nullable=False),
sa.Column('seq_number', sa.Integer(), nullable=False),
sa.Column('title', sa.String(255), nullable=False),
sa.Column('slug', sa.String(255), nullable=False),
sa.Column('content', sa.Text(), nullable=False),
sa.Column('file_path', sa.String(255), nullable=False),
sa.Column('git_branch', sa.String(255), nullable=False),
sa.Column('git_commit_hash', sa.String(40), nullable=True),
sa.Column('status', sa.String(20), nullable=False),
sa.Column('push_error', sa.Text(), nullable=True),
sa.Column('created_by', sa.String(100), nullable=False),
sa.Column('created_at', sa.DateTime(), nullable=False),
sa.Column('pushed_at', sa.DateTime(), nullable=True),
sa.ForeignKeyConstraint(['student_id'], ['student.id']),
sa.ForeignKeyConstraint(['repository_id'], ['repository.id']),
sa.PrimaryKeyConstraint('id'),
)
op.create_index('idx_prompt_student_id', 'prompt', ['student_id'])
op.create_index('idx_prompt_status', 'prompt', ['status'])
op.create_index('idx_prompt_student_seq', 'prompt', ['student_id', 'seq_number'])
```

```
def downgrade() -> None:
op.drop_index('idx_prompt_student_seq', table_name='prompt')
op.drop_index('idx_prompt_status', table_name='prompt')
op.drop_index('idx_prompt_student_id', table_name='prompt')
op.drop_table('prompt')
op.drop_index('idx_student_number', table_name='student')
op.drop_column('student', 'student_number')
```
---
app/git/client.py
Розмір: 14,518 байт

```
"""GitClient: all Git operations via GitPython and subprocess fallback."""

from __future__ import annotations

import os
import shutil
import stat
import subprocess
import tempfile
from pathlib import Path
from urllib.parse import urlparse

from git import InvalidGitRepositoryError, NoSuchPathError, Repo
from git.exc import BadName, GitCommandError

class GitOperationError(Exception):
"""Raised when a git operation fails."""

class GitMergeError(GitOperationError):
"""Raised when a fast-forward merge fails."""

class GitMergeConflictError(GitOperationError):
"""Raised when a merge produces conflicts."""

class GitAuthError(GitOperationError):
"""Raised when a git operation fails due to authentication."""

class GitClient:
"""All Git operations for StudentFlow.

Uses GitPython for high-level operations and subprocess.run() as fallback.
Authentication is always passed via environment variables (GIT_SSH_COMMAND
for SSH, temp netrc file for HTTPS/PAT). Credentials are never passed
as URL components or command-line arguments.
"""

def __init__(self, timeout: int = 60):
"""Initialize the client with a default subprocess timeout."""
self._timeout = timeout

def clone(
self,
remote_url: str,
local_path: str,
credential: str | None = None,
auth_type: str | None = None,
ssh_key_path: str | None = None,
) -> None:
"""Clone a repository to local_path."""
cmd = ["git", "clone", remote_url, local_path]
try:
if credential and auth_type:
self._run_with_credential(
cmd,
remote_url=remote_url,
credential=credential,
auth_type=auth_type,
```

```
ssh_key_path=ssh_key_path,
)
else:
self._run_command(cmd)
except GitOperationError:
raise
except Exception as exc:
raise GitOperationError(f"clone failed for {remote_url}") from exc

def fetch(self, repo_path: str, remote_name: str = "student") -> None:
"""Fetch from a remote without merging."""
self._run_command(["git", "-C", repo_path, "fetch", remote_name])

def merge_ff_only(self, repo_path: str, branch: str) -> None:
"""Attempt a fast-forward-only merge."""
try:
self._run_command(["git", "-C", repo_path, "merge", "--ff-only", branch])
except GitOperationError as exc:
raise GitMergeError(f"ff-only merge failed for {branch}") from exc

def merge_no_ff(self, repo_path: str, branch: str, message: str) -> None:
"""Merge with an explicit merge commit (--no-ff)."""
try:
result = subprocess.run(
["git", "-C", repo_path, "merge", "--no-ff", branch, "-m", message],
capture_output=True,
text=True,
timeout=self._timeout,
)
except subprocess.TimeoutExpired as exc:
raise GitMergeConflictError("merge timed out") from exc

if result.returncode == 0:
return

stderr = (result.stderr or "").lower()
stdout = (result.stdout or "").lower()
if "conflict" in stderr or "conflict" in stdout:
raise GitMergeConflictError(f"merge conflict while merging {branch}")
raise GitOperationError(f"merge failed for {branch}: {self._clean_error_text(result.stderr)}")

def push(
self,
repo_path: str,
remote_name: str,
branch: str,
credential: str | None = None,
auth_type: str | None = None,
ssh_key_path: str | None = None,
force: bool = False,
) -> None:
"""Push a branch to a remote."""
cmd = ["git", "-C", repo_path, "push"]
if force:
cmd.append("--force")
cmd.extend([remote_name, branch])

if credential and auth_type:
remote_url = self._get_remote_url(repo_path, remote_name)
self._run_with_credential(
cmd,
remote_url=remote_url,
credential=credential,
auth_type=auth_type,
ssh_key_path=ssh_key_path,
)
return

self._run_command(cmd)
```

```
def push_to_url(
self,
repo_path: str,
remote_url: str,
branch: str,
credential: str | None = None,
auth_type: str | None = None,
force: bool = False,
) -> None:
"""Push a branch to an arbitrary remote URL (no named remote required).

Uses 'git push <url> <branch>' which git supports natively.
If credential and auth_type are provided, environment-based auth is used.
Otherwise relies on the operator's system git credentials (SSH agent,
~/.netrc, credential manager, etc.).

Raises GitOperationError on failure so callers can catch and decide
whether to treat the failure as fatal.
"""
cmd = ["git", "-C", repo_path, "push"]
if force:
cmd.append("--force")
cmd.extend([remote_url, branch])

if credential and auth_type:
self._run_with_credential(
cmd,
remote_url=remote_url,
credential=credential,
auth_type=auth_type,
)
return

self._run_command(cmd)

def add_remote(self, repo_path: str, remote_name: str, remote_url: str) -> None:
"""Add a named remote to a repository."""
repo = self._open_repo(repo_path)
try:
repo.create_remote(remote_name, remote_url)
except GitCommandError as exc:
raise GitOperationError(f"add remote failed for {remote_name}") from exc

def create_tag(self, repo_path: str, tag_name: str, message: str | None = None) -> None:
"""Create a lightweight or annotated tag at HEAD."""
repo = self._open_repo(repo_path)
try:
if message:
repo.create_tag(tag_name, message=message)
else:
repo.create_tag(tag_name)
except GitCommandError as exc:
raise GitOperationError(f"create tag failed for {tag_name}") from exc

def get_commit_hash(self, repo_path: str, ref: str = "HEAD") -> str:
"""Return the full 40-char SHA1 of a git ref."""
repo = self._open_repo(repo_path, value_error=True)
try:
return repo.commit(ref).hexsha
except (BadName, ValueError, GitCommandError) as exc:
raise ValueError(f"Unknown git ref: {ref}") from exc

def check_divergence(self, repo_path: str, remote_branch: str) -> tuple[int, int]:
"""Count commits ahead and behind between HEAD and a remote branch."""
ahead_result = subprocess.run(
["git", "-C", repo_path, "rev-list", "--count", f"HEAD..{remote_branch}"],
capture_output=True,
text=True,
timeout=self._timeout,
)
```

```
behind_result = subprocess.run(
["git", "-C", repo_path, "rev-list", "--count", f"{remote_branch}..HEAD"],
capture_output=True,
text=True,
timeout=self._timeout,
)
if ahead_result.returncode != 0 or behind_result.returncode != 0:
error_text = ahead_result.stderr or behind_result.stderr or "unknown git error"
raise GitOperationError(f"check_divergence failed: {self._clean_error_text(error_text)}")

ahead = int(ahead_result.stdout.strip())
behind = int(behind_result.stdout.strip())
return ahead, behind

def init_from_dir(self, src_path: str, dest_path: str) -> None:
"""Create a new git repo at dest_path initialized with files from src_path."""
shutil.copytree(src_path, dest_path)
self._run_command(["git", "init", dest_path])
self._run_command(["git", "-C", dest_path, "config", "user.email",
"studentflow@example.invalid"])
self._run_command(["git", "-C", dest_path, "config", "user.name", "StudentFlow"])
self._run_command(["git", "-C", dest_path, "add", "."])
self._run_command(["git", "-C", dest_path, "commit", "-m", "Initial project template"])

def validate_remote(
self,
remote_url: str,
credential: str | None = None,
auth_type: str | None = None,
ssh_key_path: str | None = None,
) -> bool:
"""Test connectivity to a remote URL."""
try:
cmd = ["git", "ls-remote", remote_url]
if credential and auth_type:
self._run_with_credential(
cmd,
remote_url=remote_url,
credential=credential,
auth_type=auth_type,
ssh_key_path=ssh_key_path,
)
else:
self._run_command(cmd)
return True
except GitOperationError:
return False

def reset_hard(self, repo_path: str, ref: str = "HEAD") -> None:
"""Hard reset the working tree to a given ref."""
self._run_command(["git", "-C", repo_path, "reset", "--hard", ref])

def get_log(self, repo_path: str, limit: int = 10) -> list[dict]:
"""Return recent commit log."""
repo = self._open_repo(repo_path)
commits = []
for commit in repo.iter_commits(max_count=limit):
commits.append(
{
"hash": commit.hexsha,
"message": commit.message.strip(),
"author": str(commit.author),
"date": commit.committed_datetime.isoformat(),
}
)
return commits

def add_file(self, repo_path: str, file_path: str) -> None:
"""Stage a single file by relative path. Raises GitOperationError if not found."""
```

```
self._run_command(["git", "-C", repo_path, "add", "--", file_path])

def commit(self, repo_path: str, message: str) -> str:
"""Commit staged changes. Returns the 40-char commit hash."""
self._run_command(["git", "-C", repo_path, "commit", "-m", message])
result = self._run_command(["git", "-C", repo_path, "rev-parse", "HEAD"])
return result.stdout.strip()

def _make_netrc_env(self, remote_url: str, credential: str, username: str = "git") -> tuple[dict,
str]:
"""Create a temp netrc file and return (env_dict, temp_file_path)."""
host = urlparse(remote_url).hostname or "github.com"
netrc_content = f"machine {host}\nlogin {username}\npassword {credential}\n"

with tempfile.NamedTemporaryFile(mode="w", suffix=".netrc", delete=False, encoding="utf-8")
as handle:
handle.write(netrc_content)
temp_path = handle.name

os.chmod(temp_path, stat.S_IRUSR | stat.S_IWUSR)
env = {
**os.environ,
"GIT_CONFIG_COUNT": "1",
"GIT_CONFIG_KEY_0": "credential.helper",
"GIT_CONFIG_VALUE_0": f"store --file {temp_path}",
"HOME": str(Path.home()),
}
return env, temp_path

def _run_with_credential(
self,
cmd: list[str],
remote_url: str,
credential: str,
auth_type: str,
ssh_key_path: str | None = None,
) -> subprocess.CompletedProcess[str]:
"""Run a git command with environment-based authentication."""
if auth_type == "ssh":
env = {
**os.environ,
"GIT_SSH_COMMAND": (
f"ssh -i {ssh_key_path} -o StrictHostKeyChecking=no -o BatchMode=yes"
if ssh_key_path
else "ssh -o StrictHostKeyChecking=no -o BatchMode=yes"
),
}
return self._run_command(cmd, env=env)

if auth_type in {"pat", "app_password", "password"}:
env, temp_path = self._make_netrc_env(remote_url, credential)
try:
return self._run_command(cmd, env=env)
finally:
try:
os.unlink(temp_path)
except OSError:
pass

return self._run_command(cmd)

def _run_command(
self,
cmd: list[str],
*,
env: dict[str, str] | None = None,
) -> subprocess.CompletedProcess[str]:
"""Run a git subprocess with consistent timeout and error handling."""
try:
return subprocess.run(
```

```
cmd,
capture_output=True,
text=True,
timeout=self._timeout,
check=True,
env=env,
)
except subprocess.TimeoutExpired as exc:
raise GitOperationError(f"git command timed out after {self._timeout}s") from exc
except subprocess.CalledProcessError as exc:
raise self._classify_process_error(exc) from exc

def _classify_process_error(self, exc: subprocess.CalledProcessError) -> GitOperationError:
"""Map subprocess failures to git-specific exceptions."""
message = self._clean_error_text(exc.stderr or exc.stdout or "git command failed")
lowered = message.lower()
if any(token in lowered for token in ("authentication failed", "permission denied", "could not
read", "access denied")):
return GitAuthError(message)
return GitOperationError(message)

def _get_remote_url(self, repo_path: str, remote_name: str) -> str:
"""Resolve a configured remote URL from a local repository."""
result = self._run_command(["git", "-C", repo_path, "remote", "get-url", remote_name])
return result.stdout.strip()

def _open_repo(self, repo_path: str, *, value_error: bool = False) -> Repo:
"""Open a repository and normalize invalid-repo errors."""
try:
return Repo(repo_path)
except (InvalidGitRepositoryError, NoSuchPathError) as exc:
if value_error:
raise ValueError(f"Not a git repository: {repo_path}") from exc
raise GitOperationError(f"Not a git repository: {repo_path}") from exc

@staticmethod
def _clean_error_text(text: str | None) -> str:
"""Normalize git stderr for exceptions without exposing secrets."""
return (text or "").strip() or "git command failed"
---
### app/git/__init__.py
**Розмір:** 47 байт

"""Git integration package for StudentFlow."""
---
### app/git/validators.py
**Розмір:** 2,057 байт

"""GitValidator: filesystem and URL validation. No git operations."""
import re
from pathlib import Path

from app.errors import (
LocalPathNotDirectoryError,
LocalPathNotFoundError,
RemoteURLInvalidError,
)

_SSH_PATTERN = re.compile(r'^git@[\w\.\-]+:[\w\.\-/]+\.git$')
_HTTPS_PATTERN = re.compile(r'^https?://[\w\.\-]+(:\d+)?/[\w\.\-/]+\.git$')
_HTTPS_NO_SUFFIX = re.compile(r'^https?://[\w\.\-]+(:\d+)?/[\w\.\-/]+$')

_BAD_BRANCH = re.compile(r'\.\.| |@\{|[\\:?*\[\]^~]')

class GitValidator:
@staticmethod
def validate_local_path(path: str) -> Path:
"""Check that path exists and is a directory. Returns resolved Path."""
p = Path(path)
if not p.exists():
```

```
raise LocalPathNotFoundError(f"Path does not exist: {path}")
if not p.is_dir():
raise LocalPathNotDirectoryError(f"Path is not a directory: {path}")
return p.resolve()

@staticmethod
def validate_remote_url(url: str) -> str:
"""Check that url matches SSH or HTTPS git remote patterns. Returns url unchanged."""
if (
_SSH_PATTERN.match(url)
or _HTTPS_PATTERN.match(url)
or _HTTPS_NO_SUFFIX.match(url)
):
return url
raise RemoteURLInvalidError(f"Invalid remote URL format: {url}")

@staticmethod
def is_git_repo(path: str | Path) -> bool:
"""Return True if path contains a .git directory or is a bare repo."""
p = Path(path)
return (p / ".git").exists() or (p / "HEAD").exists()

@staticmethod
def validate_branch_name(branch: str) -> str:
"""Check branch name is valid. Raises ValueError if invalid."""
if not branch:
raise ValueError("Branch name must not be empty")
if branch.startswith("/") or branch.endswith("/"):
raise ValueError(f"Branch name must not start or end with '/': {branch}")
if _BAD_BRANCH.search(branch):
raise ValueError(f"Branch name contains invalid characters: {branch}")
return branch
---
### app/git/policies.py
**Розмір:** 90 байт

"""SyncPolicy: merge/rebase policy logic for StudentFlow."""

class SyncPolicy:
pass
---
### app/ui/__init__.py
**Розмір:** 34 байт

"""UI package for StudentFlow."""
---
### app/ui/dialogs/__init__.py
**Розмір:** 38 байт

"""Dialog classes for StudentFlow."""
---
### app/ui/dialogs/repository_dialog.py
**Розмір:** 3,631 байт

"""Add Repository dialog."""
from __future__ import annotations

from pathlib import Path

from PySide6.QtWidgets import (
QComboBox,
QDialog,
QDialogButtonBox,
QFormLayout,
QLabel,
QLineEdit,
QVBoxLayout,
)
```

```
class RepositoryDialog(QDialog):
def __init__(self, parent=None, accounts=None):
super().__init__(parent)
self._accounts = accounts or []
self.setWindowTitle("Add Repository")
self.setMinimumWidth(460)
self._build_ui()

def _build_ui(self) -> None:
layout = QVBoxLayout(self)
form = QFormLayout()

self._account = QComboBox(self)
for a in self._accounts:
self._account.addItem(f"{a.username} ({a.provider})", a.id)
form.addRow("Account *", self._account)

self._name = QLineEdit(self)
self._name.setPlaceholderText("e.g. student-project")
form.addRow("Repo Name *", self._name)

self._url = QLineEdit(self)
self._url.setPlaceholderText("https://github.com/user/repo.git or git@…")
self._url.textChanged.connect(self._validate_url_live)
form.addRow("Remote URL *", self._url)

self._path = QLineEdit(self)
self._path.setPlaceholderText("C:\\repos\\student-1 or /absolute/path/to/clone")
form.addRow("Local Path *", self._path)

self._branch = QLineEdit(self)
self._branch.setText("main")
form.addRow("Default Branch", self._branch)

layout.addLayout(form)

self._error = QLabel("", self)
self._error.setStyleSheet("color: red;")
self._error.setWordWrap(True)
layout.addWidget(self._error)

buttons = QDialogButtonBox(
QDialogButtonBox.StandardButton.Save | QDialogButtonBox.StandardButton.Cancel,
self,
)
buttons.accepted.connect(self._on_save)
buttons.rejected.connect(self.reject)
layout.addWidget(buttons)

def _validate_url_live(self, text: str) -> None:
if not text:
self._error.setText("")
return
try:
from app.git.validators import GitValidator
GitValidator.validate_remote_url(text)
self._error.setText("")
except Exception:
self._error.setText("Remote URL format invalid (use HTTPS or SSH git URL).")

def _on_save(self) -> None:
if self._account.count() == 0:
self._error.setText("No accounts available. Add an account first.")
return
if not self._name.text().strip():
self._error.setText("Repo Name is required.")
return
url = self._url.text().strip()
if not url:
self._error.setText("Remote URL is required.")
return
```

```
try:
from app.git.validators import GitValidator
GitValidator.validate_remote_url(url)
except Exception as e:
self._error.setText(str(e))
return
local = self._path.text().strip()
if not local:
self._error.setText("Local Path is required.")
return
if not Path(local).is_absolute():
self._error.setText("Local Path must be an absolute path.")
return
self.accept()

def get_data(self) -> dict:
return {
"account_id": self._account.currentData(),
"repo_name": self._name.text().strip(),
"remote_url": self._url.text().strip(),
"local_path": self._path.text().strip(),
"default_branch": self._branch.text().strip() or "main",
}
```
---
app/ui/dialogs/credential_dialog.py
Розмір: 3,626 байт

```
"""Add/Rotate Credential dialog."""
from __future__ import annotations

from PySide6.QtWidgets import (
QComboBox,
QDialog,
QDialogButtonBox,
QFormLayout,
QLabel,
QLineEdit,
QVBoxLayout,
)

from app.db.enums import SecretKind

class CredentialDialog(QDialog):
def __init__(self, parent=None, auth_type: str = "", account_id: str = "", accounts=None):
super().__init__(parent)
self._account_id = account_id
self._auth_type = auth_type
self._accounts = accounts or []
self.setWindowTitle("Add / Rotate Credential")
self.setMinimumWidth(400)
self._build_ui()
self._update_save_state()

def _build_ui(self) -> None:
layout = QVBoxLayout(self)
form = QFormLayout()

self._account_combo = QComboBox(self)
for a in self._accounts:
self._account_combo.addItem(f"{a.username} ({a.provider})", a.id)
if self._account_id:
idx = self._account_combo.findData(self._account_id)
if idx >= 0:
self._account_combo.setCurrentIndex(idx)
self._account_combo.setEnabled(not bool(self._account_id))
form.addRow("Account", self._account_combo)

self._auth_combo = QComboBox(self)
for k in SecretKind:
```

```
self._auth_combo.addItem(k.value, k.value)
if self._auth_type:
idx = self._auth_combo.findData(self._auth_type)
if idx >= 0:
self._auth_combo.setCurrentIndex(idx)
self._auth_combo.setEnabled(False)
form.addRow("Auth Type", self._auth_combo)

self._secret = QLineEdit(self)
self._secret.setEchoMode(QLineEdit.EchoMode.Password)
self._secret.setPlaceholderText("New secret")
self._secret.textChanged.connect(self._update_save_state)
form.addRow("New Secret *", self._secret)

self._confirm = QLineEdit(self)
self._confirm.setEchoMode(QLineEdit.EchoMode.Password)
self._confirm.setPlaceholderText("Confirm secret")
self._confirm.textChanged.connect(self._update_save_state)
form.addRow("Confirm Secret *", self._confirm)

layout.addLayout(form)

self._error = QLabel("", self)
self._error.setStyleSheet("color: red;")
layout.addWidget(self._error)

self._buttons = QDialogButtonBox(
QDialogButtonBox.StandardButton.Save | QDialogButtonBox.StandardButton.Cancel,
self,
)
self._save_btn = self._buttons.button(QDialogButtonBox.StandardButton.Save)
self._buttons.accepted.connect(self._on_save)
self._buttons.rejected.connect(self.reject)
layout.addWidget(self._buttons)

def _update_save_state(self) -> None:
s = self._secret.text()
c = self._confirm.text()
if s and c and s != c:
self._error.setText("Secrets do not match.")
self._save_btn.setEnabled(False)
else:
self._error.setText("")
self._save_btn.setEnabled(bool(s))

def _on_save(self) -> None:
if self._secret.text() != self._confirm.text():
self._error.setText("Secrets do not match.")
return
if not self._secret.text():
self._error.setText("Secret is required.")
return
self.accept()

def get_account_id(self) -> str:
return self._account_combo.currentData() or self._account_id

def get_auth_type(self) -> str:
return self._auth_combo.currentData()

def get_secret(self) -> str:
return self._secret.text()
---
### app/ui/dialogs/student_dialog.py
**Розмір:** 2,788 байт

"""Add/Edit Student dialog."""
from __future__ import annotations

from PySide6.QtWidgets import (
QComboBox,
QDialog,
```

```
QDialogButtonBox,
QFormLayout,
QLabel,
QLineEdit,
QSpinBox,
QTextEdit,
QVBoxLayout,
)

from app.db.enums import StudentStatus


class StudentDialog(QDialog):
def __init__(self, parent=None, student=None):
super().__init__(parent)
self._student = student
self.setWindowTitle("Edit Student" if student else "Add Student")
self.setMinimumWidth(400)
self._build_ui()
if student:
self._populate(student)

def _build_ui(self) -> None:
layout = QVBoxLayout(self)
form = QFormLayout()

self._name = QLineEdit(self)
form.addRow("Full Name *", self._name)

self._status = QComboBox(self)
for s in StudentStatus:
self._status.addItem(s.value, s.value)
form.addRow("Status", self._status)

self._queue = QSpinBox(self)
self._queue.setRange(0, 100)
self._queue.setSpecialValueText("(none)")
form.addRow("Queue Position", self._queue)

self._notes = QTextEdit(self)
self._notes.setMaximumHeight(100)
self._notes.setPlaceholderText("Optional notes (max 1000 chars)")
form.addRow("Notes", self._notes)

layout.addLayout(form)

self._error = QLabel("", self)
self._error.setStyleSheet("color: red;")
layout.addWidget(self._error)

buttons = QDialogButtonBox(
QDialogButtonBox.StandardButton.Save | QDialogButtonBox.StandardButton.Cancel,
self,
)
buttons.accepted.connect(self._on_save)
buttons.rejected.connect(self.reject)
layout.addWidget(buttons)

def _populate(self, student) -> None:
self._name.setText(student.full_name or "")
idx = self._status.findData(student.status)
if idx >= 0:
self._status.setCurrentIndex(idx)
if student.queue_position:
self._queue.setValue(student.queue_position)
self._notes.setPlainText(student.notes or "")

def _on_save(self) -> None:
if not self._name.text().strip():
self._error.setText("Full Name is required.")
return
notes = self._notes.toPlainText()
if len(notes) > 1000:
```

```
self._error.setText("Notes exceed 1000 characters.")
return
self.accept()

def get_data(self) -> dict:
queue = self._queue.value()
return {
"full_name": self._name.text().strip(),
"status": self._status.currentData(),
"queue_position": queue if queue > 0 else None,
"notes": self._notes.toPlainText() or None,
}
```
---
app/ui/dialogs/account_dialog.py
Розмір: 2,892 байт

```
"""Add/Edit Account dialog."""
from __future__ import annotations

from PySide6.QtWidgets import (
QComboBox,
QDialog,
QDialogButtonBox,
QFormLayout,
QLabel,
QLineEdit,
QVBoxLayout,
)

from app.db.enums import AccountProvider, AuthType

class AccountDialog(QDialog):
def __init__(self, parent=None, account=None, students=None):
super().__init__(parent)
self._account = account
self._students = students or []
self.setWindowTitle("Edit Account" if account else "Add Account")
self.setMinimumWidth(400)
self._build_ui()
if account:
self._populate(account)

def _build_ui(self) -> None:
layout = QVBoxLayout(self)
form = QFormLayout()

self._student = QComboBox(self)
for s in self._students:
self._student.addItem(s.full_name, s.id)
form.addRow("Linked Student *", self._student)

self._provider = QComboBox(self)
for p in AccountProvider:
self._provider.addItem(p.value, p.value)
form.addRow("Provider", self._provider)

self._username = QLineEdit(self)
form.addRow("Username *", self._username)

self._auth_type = QComboBox(self)
for a in AuthType:
self._auth_type.addItem(a.value, a.value)
form.addRow("Auth Type", self._auth_type)

layout.addLayout(form)

self._error = QLabel("", self)
self._error.setStyleSheet("color: red;")
layout.addWidget(self._error)

buttons = QDialogButtonBox(
QDialogButtonBox.StandardButton.Save | QDialogButtonBox.StandardButton.Cancel,
```

```
self,
)
buttons.accepted.connect(self._on_save)
buttons.rejected.connect(self.reject)
layout.addWidget(buttons)

def _populate(self, account) -> None:
idx = self._student.findData(account.student_id)
if idx >= 0:
self._student.setCurrentIndex(idx)
idx = self._provider.findData(account.provider)
if idx >= 0:
self._provider.setCurrentIndex(idx)
self._username.setText(account.username or "")
idx = self._auth_type.findData(account.auth_type)
if idx >= 0:
self._auth_type.setCurrentIndex(idx)

def _on_save(self) -> None:
if not self._username.text().strip():
self._error.setText("Username is required.")
return
if self._student.count() == 0:
self._error.setText("No students available. Add a student first.")
return
self.accept()

def get_data(self) -> dict:
return {
"student_id": self._student.currentData(),
"provider": self._provider.currentData(),
"username": self._username.text().strip(),
"auth_type": self._auth_type.currentData(),
}
```

---

app/ui/windows/repositories_screen.py

Розмір: 6,730 байт

```
"""Repositories screen — data-driven list with validation actions."""
from __future__ import annotations

import os
import platform
import subprocess

from PySide6.QtGui import QColor
from PySide6.QtWidgets import (
QHBoxLayout,
QLabel,
QMessageBox,
QPushButton,
QTableWidget,
QTableWidgetItem,
QVBoxLayout,
QWidget,
)

from app.ui.viewmodels.repos_viewmodel import RepoViewModel

_SYNC_COLORS = {
"clean": "#a6e3a1",
"dirty": "#f9e2af",
"diverged": "#fab387",
"error": "#f38ba8",
"uninitialized": "#9399b2",
}

_COLUMNS = ["Repo Name", "Local Path", "Remote URL", "Branch", "Sync Status", "Last
Commit", "Actions"]
```

```
class RepositoriesScreen(QWidget):
def __init__(self, parent=None):
super().__init__(parent)
self._vm = RepoViewModel()
self._repos: list = []
self._build_ui()
self.refresh()

def _build_ui(self) -> None:
layout = QVBoxLayout(self)
layout.setContentsMargins(20, 20, 20, 20)
layout.setSpacing(12)

title = QLabel("Repositories", self)
title.setObjectName("screenTitle")
layout.addWidget(title)

toolbar = QWidget(self)
tb = QHBoxLayout(toolbar)
tb.setContentsMargins(0, 0, 0, 0)
add_btn = QPushButton("Add Repository", toolbar)
add_btn.setProperty("role", "primary")
add_btn.clicked.connect(self._on_add)
tb.addWidget(add_btn)
ref_btn = QPushButton("Refresh", toolbar)
ref_btn.clicked.connect(self.refresh)
tb.addWidget(ref_btn)
tb.addStretch(1)
layout.addWidget(toolbar)

self._table = QTableWidget(self)
self._table.setColumnCount(len(_COLUMNS))
self._table.setHorizontalHeaderLabels(_COLUMNS)
self._table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
self._table.setSelectionBehavior(QTableWidget.SelectionBehavior.SelectRows)
self._table.setAlternatingRowColors(True)
self._table.verticalHeader().setVisible(False)
self._table.verticalHeader().setDefaultSectionSize(44)
self._table.verticalHeader().setMinimumSectionSize(40)
self._table.horizontalHeader().setStretchLastSection(True)
layout.addWidget(self._table)

self._empty = QLabel("No repositories found.", self)
self._empty.setObjectName("emptyState")
self._empty.setVisible(False)
layout.addWidget(self._empty)

def refresh(self) -> None:
try:
self._repos = self._vm.list_repos()
except Exception as e:
QMessageBox.critical(self, "Error", str(e))
return
self._fill_table()

def _fill_table(self) -> None:
repos = self._repos
self._table.setVisible(bool(repos))
self._empty.setVisible(not bool(repos))
self._table.setRowCount(0)
for row, r in enumerate(repos):
self._table.insertRow(row)

name_text = f"★ {r.repo_name}" if getattr(r, "is_canonical", False) else r.repo_name
name_item = QTableWidgetItem(name_text)
if getattr(r, "is_canonical", False):
name_item.setForeground(QColor("#cba6f7"))
self._table.setItem(row, 0, name_item)

self._table.setItem(row, 1, QTableWidgetItem(r.local_path))
self._table.setItem(row, 2, QTableWidgetItem(r.remote_url))
```

```
self._table.setItem(row, 3, QTableWidgetItem(r.default_branch))

sync_item = QTableWidgetItem(r.sync_status)
sync_item.setForeground(QColor(_SYNC_COLORS.get(r.sync_status, "#cdd6f4")))
self._table.setItem(row, 4, sync_item)

self._table.setItem(row, 5, QTableWidgetItem((r.last_commit_hash or "")[:8]))

cell = QWidget()
cl = QHBoxLayout(cell)
cl.setContentsMargins(4, 6, 4, 6)
cl.setSpacing(4)

vl_btn = QPushButton("Validate Path")
vl_btn.clicked.connect(lambda _=False, path=r.local_path: self._validate_path(path))
cl.addWidget(vl_btn)

vr_btn = QPushButton("Validate Remote")
vr_btn.clicked.connect(lambda _=False, rid=r.id: self._validate_remote(rid))
cl.addWidget(vr_btn)

fm_btn = QPushButton("Open Folder")
fm_btn.clicked.connect(lambda _=False, path=r.local_path: self._open_folder(path))
cl.addWidget(fm_btn)

sc_btn = QPushButton("Set as Canonical")
sc_btn.setProperty("role", "primary")
sc_btn.setEnabled(not getattr(r, "is_canonical", False))
sc_btn.clicked.connect(lambda _=False, rid=r.id: self._set_canonical(rid))
cl.addWidget(sc_btn)

self._table.setCellWidget(row, 6, cell)

def _validate_path(self, path: str) -> None:
try:
ok = self._vm.validate_local_path(path)
msg = "Path is valid." if ok else "Path is invalid or not a git repo."
QMessageBox.information(self, "Validate Path", msg)
except Exception as e:
QMessageBox.critical(self, "Error", str(e))

def _validate_remote(self, repo_id: str) -> None:
try:
ok = self._vm.validate_remote(repo_id)
msg = "Remote is reachable." if ok else "Remote is not reachable."
QMessageBox.information(self, "Validate Remote", msg)
except Exception as e:
QMessageBox.critical(self, "Error", str(e))

def _on_add(self) -> None:
try:
accounts = self._vm.list_accounts()
except Exception as e:
QMessageBox.critical(self, "Error", str(e))
return
from app.ui.dialogs.repository_dialog import RepositoryDialog
dlg = RepositoryDialog(self, accounts=accounts)
if dlg.exec() == RepositoryDialog.DialogCode.Accepted:
try:
self._vm.create_repository(dlg.get_data())
self.refresh()
except Exception as e:
QMessageBox.critical(self, "Error", str(e))

def _set_canonical(self, repo_id: str) -> None:
try:
self._vm.set_canonical(repo_id)
self.refresh()
except Exception as e:
QMessageBox.critical(self, "Error", str(e))

def _open_folder(self, path: str) -> None:
try:
```

```
system = platform.system()
if system == "Windows":
os.startfile(path)
elif system == "Darwin":
subprocess.Popen(["open", path])
else:
subprocess.Popen(["xdg-open", path])
except Exception as e:
QMessageBox.critical(self, "Error", str(e))
```
---
app/ui/windows/dashboard_screen.py
Розмір: 11,073 байт

```
"""Dashboard screen -- active student, queue, canonical repo, recent sync activity."""
from __future__ import annotations

from PySide6.QtWidgets import (
QFormLayout,
QGroupBox,
QHBoxLayout,
QLabel,
QMessageBox,
QPushButton,
QVBoxLayout,
QWidget,
)

from app.ui.viewmodels.dashboard_viewmodel import DashboardViewModel

_STATUS_STYLE = {
"active": "color: #a6e3a1; font-weight: 600;",
"paused": "color: #f9e2af;",
"exhausted": "color: #f38ba8;",
"archived": "color: #6c7086;",
"error": "color: #f38ba8; font-weight: 600;",
}

_SYNC_STATUS_STYLE = {
"success": "color: #a6e3a1;",
"failed": "color: #f38ba8;",
"running": "color: #89dceb;",
"pending": "color: #cdd6f4;",
"skipped": "color: #6c7086;",
}

class DashboardScreen(QWidget):
def __init__(self, parent=None):
super().__init__(parent)
self._vm = DashboardViewModel()
self._build_ui()
self.refresh()

# -- layout ------------------------------------------------------------

def _build_ui(self) -> None:
layout = QVBoxLayout(self)
layout.setContentsMargins(24, 24, 24, 24)
layout.setSpacing(16)

title = QLabel("Dashboard", self)
title.setObjectName("screenTitle")
layout.addWidget(title)

# Active Student ----------------------------------------------------
active_group = QGroupBox("Active Student", self)
self._active_form = QFormLayout(active_group)

self._active_name_label = QLabel("--", self)
self._active_name_label.setStyleSheet("font-size: 15px; font-weight: bold;")
self._active_form.addRow("Name:", self._active_name_label)
```

```
self._active_queue_label = QLabel("--", self)
self._active_form.addRow("Queue position:", self._active_queue_label)

self._active_prompts_label = QLabel("--", self)
self._active_form.addRow("Prompts pushed:", self._active_prompts_label)

btn_row = QWidget()
btn_layout = QHBoxLayout(btn_row)
btn_layout.setContentsMargins(0, 4, 0, 0)
btn_layout.setSpacing(8)

self._sync_btn = QPushButton("Sync Now")
self._sync_btn.setProperty("role", "primary")
self._sync_btn.setEnabled(False)
self._sync_btn.clicked.connect(self._on_sync_now)
btn_layout.addWidget(self._sync_btn)

self._handoff_btn = QPushButton("Hand Off to Next Student")
self._handoff_btn.setProperty("role", "destructive")
self._handoff_btn.setEnabled(False)
self._handoff_btn.clicked.connect(self._on_handoff)
btn_layout.addWidget(self._handoff_btn)

btn_layout.addStretch(1)
self._active_form.addRow(btn_row)

layout.addWidget(active_group)

# Student Queue -------------------------------------------------------
queue_group = QGroupBox("Student Queue (active & paused)", self)
self._queue_layout = QVBoxLayout(queue_group)
self._queue_layout.setSpacing(3)
layout.addWidget(queue_group)

# Canonical Repository ------------------------------------------------
canon_group = QGroupBox("Canonical Repository", self)
self._form = QFormLayout(canon_group)

self._path_label = QLabel("--", self)
self._path_label.setWordWrap(True)
self._form.addRow("Path:", self._path_label)

self._valid_label = QLabel("--", self)
self._form.addRow("Path accessible:", self._valid_label)

self._repo_label = QLabel("--", self)
self._form.addRow("Registered as:", self._repo_label)

self._branch_label = QLabel("--", self)
self._form.addRow("Branch:", self._branch_label)

self._commit_label = QLabel("--", self)
self._form.addRow("Last commit:", self._commit_label)

layout.addWidget(canon_group)

# Recent Sync Activity ------------------------------------------------
activity_group = QGroupBox("Recent Sync Activity", self)
self._activity_layout = QVBoxLayout(activity_group)
self._activity_layout.setSpacing(3)
layout.addWidget(activity_group)

refresh_btn = QPushButton("Refresh", self)
refresh_btn.clicked.connect(self.refresh)
layout.addWidget(refresh_btn)

layout.addStretch(1)

# -- data refresh -------------------------------------------------------

def refresh(self) -> None:
self._refresh_active_student()
self._refresh_student_queue()
self._refresh_canonical()
self._refresh_sync_activity()
```

```
def _refresh_active_student(self) -> None:
try:
info = self._vm.get_active_student_info()
except Exception as exc:
self._active_name_label.setText(f"Error: {exc}")
self._sync_btn.setEnabled(False)
self._handoff_btn.setEnabled(False)
return

if info is None:
self._active_name_label.setText("(no active student)")
self._active_name_label.setStyleSheet("color: #f9e2af; font-size: 15px;")
self._active_queue_label.setText("--")
self._active_prompts_label.setText("--")
self._sync_btn.setEnabled(False)
self._handoff_btn.setEnabled(False)
return

num = info["student_number"]
num_str = f"#{num:02d}" if isinstance(num, int) else "#?"
self._active_name_label.setText(f"{num_str} {info['full_name']}")
self._active_name_label.setStyleSheet(
"font-size: 15px; font-weight: bold; color: #a6e3a1;"
)
self._active_queue_label.setText(str(info["queue_position"] or "--"))
self._active_prompts_label.setText(str(info["prompt_count"]))
self._sync_btn.setEnabled(True)
self._handoff_btn.setEnabled(True)

def _refresh_student_queue(self) -> None:
self._clear_layout(self._queue_layout)
try:
queue = self._vm.get_student_queue()
except Exception as exc:
self._queue_layout.addWidget(QLabel(f"Error: {exc}", self))
return

if not queue:
lbl = QLabel("(no active or paused students)", self)
lbl.setStyleSheet("color: #6c7086;")
self._queue_layout.addWidget(lbl)
return

for entry in queue:
num = entry["student_number"]
num_str = f"#{num:02d}" if isinstance(num, int) else "#?"
status = entry["status"]
marker = ">" if status == "active" else " "
text = f"{marker} {num_str} {entry['full_name']} (pos {entry['queue_position']})"
lbl = QLabel(text, self)
lbl.setStyleSheet(_STATUS_STYLE.get(status, ""))
self._queue_layout.addWidget(lbl)

def _refresh_canonical(self) -> None:
try:
summary = self._vm.get_canonical_summary()
except Exception as exc:
self._path_label.setText(f"Error: {exc}")
return

self._path_label.setText(summary.get("path") or "(not configured)")

if summary.get("path_valid", False):
self._valid_label.setText("Yes")
self._valid_label.setStyleSheet("color: #a6e3a1;")
else:
self._valid_label.setText("No -- path does not exist or is not a git repo")
self._valid_label.setStyleSheet("color: #f38ba8;")

repo = summary.get("canonical_repo")
if repo is not None:
```

```
self._repo_label.setText(repo.repo_name)
self._repo_label.setStyleSheet("")
self._branch_label.setText(repo.default_branch or "--")
self._commit_label.setText((repo.last_commit_hash or "")[:8] or "--")
else:
self._repo_label.setText("(no canonical repo record in database)")
self._repo_label.setStyleSheet("color: #f9e2af;")
self._branch_label.setText("--")
self._commit_label.setText("--")

def _refresh_sync_activity(self) -> None:
self._clear_layout(self._activity_layout)
try:
jobs = self._vm.get_recent_sync_activity()
except Exception as exc:
self._activity_layout.addWidget(QLabel(f"Error: {exc}", self))
return

if not jobs:
lbl = QLabel("(no sync jobs recorded)", self)
lbl.setStyleSheet("color: #6c7086;")
self._activity_layout.addWidget(lbl)
return

for job in jobs:
ts = str(job["started_at"])[:16] if job["started_at"] else " -- "
num = job["student_number"]
num_str = f"#{num:02d}" if isinstance(num, int) else "#?"
job_type = job["job_type"].replace("_", " ")
status = job["status"]
text = f"{ts} {num_str} {job['student_name']} {job_type} [{status}]"
lbl = QLabel(text, self)
lbl.setStyleSheet(_SYNC_STATUS_STYLE.get(status, ""))
self._activity_layout.addWidget(lbl)

# -- action handlers -------------------------------------------------------

def _on_sync_now(self) -> None:
try:
result = self._vm.sync_active_student()
except Exception as exc:
QMessageBox.critical(self, "Sync Failed", str(exc))
return
status = result["status"]
detail = result.get("summary") or result.get("error_message") or ""
if status == "success":
QMessageBox.information(
self, "Sync Complete",
(f"Sync succeeded for {result['student_name']}." + (f"\n{detail}" if detail else "")).strip(),
)
else:
QMessageBox.warning(
self, "Sync Result",
(f"Sync {status} for {result['student_name']}." + (f"\n{detail}" if detail else "")).strip(),
)
self.refresh()

def _on_handoff(self) -> None:
reply = QMessageBox.question(
self,
"Hand Off to Next Student",
"Hand off from the current active student to the next in queue?\n\n"
"This will run a final sync and move the project forward.",
QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No,
QMessageBox.StandardButton.No,
)
if reply != QMessageBox.StandardButton.Yes:
return
try:
```

```
result = self._vm.handoff_to_next()
except Exception as exc:
QMessageBox.critical(self, "Handoff Failed", str(exc))
return
QMessageBox.information(
self, "Handoff Complete",
"Handoff complete.\n"
f"From: {result['from_student']}\n"
f"To: {result['to_student']}\n"
f"Checkpoint: {result['checkpoint_tag']}",
)
self.refresh()

@staticmethod
def _clear_layout(layout) -> None:
while layout.count():
item = layout.takeAt(0)
if item.widget():
item.widget().deleteLater()
```
---
app/ui/windows/credentials_screen.py
Розмір: 5,285 байт

```
"""Credentials screen — shows masked secrets, Validate/Rotate actions."""
from __future__ import annotations

from PySide6.QtWidgets import (
QHBoxLayout,
QLabel,
QMessageBox,
QPushButton,
QTableWidget,
QTableWidgetItem,
QVBoxLayout,
QWidget,
)

from app.ui.viewmodels.credentials_viewmodel import CredentialViewModel

_COLUMNS = ["Account", "Auth Type", "Secret (masked)", "Last Validated", "Actions"]

class CredentialsScreen(QWidget):
def __init__(self, parent=None):
super().__init__(parent)
self._vm = CredentialViewModel()
self._rows: list[dict] = []
self._accounts: list = []
self._build_ui()
self.refresh()

def _build_ui(self) -> None:
layout = QVBoxLayout(self)
layout.setContentsMargins(20, 20, 20, 20)
layout.setSpacing(12)

title = QLabel("Credentials", self)
title.setObjectName("screenTitle")
layout.addWidget(title)

toolbar = QWidget(self)
tb = QHBoxLayout(toolbar)
tb.setContentsMargins(0, 0, 0, 0)
add_btn = QPushButton("Add Credential", toolbar)
add_btn.setProperty("role", "primary")
add_btn.clicked.connect(self._on_add)
tb.addWidget(add_btn)
ref_btn = QPushButton("Refresh", toolbar)
ref_btn.clicked.connect(self.refresh)
tb.addWidget(ref_btn)
tb.addStretch(1)
```

```
layout.addWidget(toolbar)

self._table = QTableWidget(self)
self._table.setColumnCount(len(_COLUMNS))
self._table.setHorizontalHeaderLabels(_COLUMNS)
self._table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
self._table.setSelectionBehavior(QTableWidget.SelectionBehavior.SelectRows)
self._table.setAlternatingRowColors(True)
self._table.verticalHeader().setVisible(False)
self._table.verticalHeader().setDefaultSectionSize(44)
self._table.verticalHeader().setMinimumSectionSize(40)
self._table.horizontalHeader().setStretchLastSection(True)
layout.addWidget(self._table)

self._empty = QLabel("No credentials stored yet.", self)
self._empty.setObjectName("emptyState")
self._empty.setVisible(False)
layout.addWidget(self._empty)

def refresh(self) -> None:
try:
self._rows = self._vm.list_credentials()
self._accounts = self._vm.list_accounts()
except Exception as e:
QMessageBox.critical(self, "Error", str(e))
return
self._fill_table()

def _fill_table(self) -> None:
rows = self._rows
self._table.setVisible(bool(rows))
self._empty.setVisible(not bool(rows))
self._table.setRowCount(0)
for row, r in enumerate(rows):
self._table.insertRow(row)
self._table.setItem(row, 0, QTableWidgetItem(r["username"]))
self._table.setItem(row, 1, QTableWidgetItem(r["auth_type"]))
masked = self._vm.masked_secret(r["secret_ref"])
self._table.setItem(row, 2, QTableWidgetItem(masked))
last_val = str(r["last_validated_at"])[:16] if r["last_validated_at"] else ""
self._table.setItem(row, 3, QTableWidgetItem(last_val))

cell = QWidget()
cl = QHBoxLayout(cell)
cl.setContentsMargins(4, 6, 4, 6)
cl.setSpacing(4)

val_btn = QPushButton("Validate")
val_btn.clicked.connect(lambda _=False, ref=r["secret_ref"]: self._on_validate(ref))
cl.addWidget(val_btn)

rot_btn = QPushButton("Rotate")
rot_btn.setProperty("role", "destructive")
rot_btn.clicked.connect(
lambda _=False, cid=r["id"], at=r["auth_type"], aid=r["account_id"]: self._on_rotate(cid, at, aid)
)
cl.addWidget(rot_btn)

self._table.setCellWidget(row, 4, cell)

def _on_add(self) -> None:
from app.ui.dialogs.credential_dialog import CredentialDialog
dlg = CredentialDialog(self, accounts=self._accounts)
if dlg.exec() == CredentialDialog.DialogCode.Accepted:
try:
self._vm.store(dlg.get_account_id(), dlg.get_auth_type(), dlg.get_secret())
self.refresh()
except Exception as e:
QMessageBox.critical(self, "Error", str(e))

def _on_validate(self, secret_ref: str) -> None:
```

```
try:
ok = self._vm.validate_exists(secret_ref)
msg = "Secret exists in keyring." if ok else "Secret NOT found in keyring."
QMessageBox.information(self, "Validate", msg)
except Exception as e:
QMessageBox.critical(self, "Error", str(e))

def _on_rotate(self, credential_id: str, auth_type: str, account_id: str) -> None:
from app.ui.dialogs.credential_dialog import CredentialDialog
dlg = CredentialDialog(self, auth_type=auth_type, account_id=account_id,
accounts=self._accounts)
if dlg.exec() == CredentialDialog.DialogCode.Accepted:
try:
self._vm.rotate(credential_id, dlg.get_secret())
self.refresh()
except Exception as e:
QMessageBox.critical(self, "Error", str(e))
```

---

app/ui/windows/main_window.py

Розмір: 16,780 байт

```
"""Main application window for StudentFlow."""

from __future__ import annotations

from PySide6.QtCore import QSize, Qt
from PySide6.QtGui import QColor, QPalette
from PySide6.QtWidgets import (
QFrame,
QHBoxLayout,
QLabel,
QMainWindow,
QPushButton,
QSizePolicy,
QStackedWidget,
QStatusBar,
QStyle,
QVBoxLayout,
QWidget,
)

_APP_QSS = """
/* ── Global ──────────────────────────────────────────────────────────── */
QWidget {
font-size: 13px;
color: #cdd6f4;
}

/* ── Buttons (base / secondary) ──────────────────────────────────────── */
QPushButton {
background-color: #313244;
color: #cdd6f4;
border: 1px solid #45475a;
border-radius: 6px;
padding: 6px 16px;
font-size: 13px;
font-weight: 500;
min-height: 28px;
}
QPushButton:hover {
background-color: #45475a;
border-color: #89b4fa;
color: #ffffff;
}
QPushButton:pressed {
background-color: #89b4fa;
color: #1e1e2e;
border-color: #89b4fa;
```

```
}
QPushButton:disabled {
background-color: #181825;
color: #45475a;
border-color: #313244;
}

/* ── Primary button ──────────────────────────────────────────────── */
QPushButton[role="primary"] {
background-color: #89b4fa;
color: #1e1e2e;
border-color: #89b4fa;
font-weight: 600;
}
QPushButton[role="primary"]:hover {
background-color: #b4d0ff;
border-color: #b4d0ff;
color: #1e1e2e;
}
QPushButton[role="primary"]:pressed {
background-color: #6e9de0;
border-color: #6e9de0;
color: #1e1e2e;
}
QPushButton[role="primary"]:disabled {
background-color: #2b3a52;
color: #4a5f7a;
border-color: #2b3a52;
}

/* ── Destructive button ──────────────────────────────────────────── */
QPushButton[role="destructive"] {
background-color: transparent;
color: #f38ba8;
border-color: #f38ba8;
font-weight: 500;
}
QPushButton[role="destructive"]:hover {
background-color: #3d1a24;
color: #f38ba8;
border-color: #f38ba8;
}
QPushButton[role="destructive"]:pressed {
background-color: #f38ba8;
color: #1e1e2e;
border-color: #f38ba8;
}
QPushButton[role="destructive"]:disabled {
background-color: transparent;
color: #5a3040;
border-color: #5a3040;
}

/* ── Line edits ──────────────────────────────────────────────────── */
QLineEdit {
background-color: #181825;
border: 1px solid #45475a;
border-radius: 5px;
padding: 5px 10px;
font-size: 13px;
color: #cdd6f4;
min-height: 26px;
selection-background-color: #89b4fa;
selection-color: #1e1e2e;
}
QLineEdit:focus {
border-color: #89b4fa;
background-color: #1e1e2e;
```

```
}
QLineEdit:disabled {
color: #45475a;
border-color: #313244;
}

/* ── Group boxes ──────────────────────────────────────────────────── */
QGroupBox {
border: 1px solid #45475a;
border-radius: 8px;
margin-top: 14px;
padding: 12px 12px 12px 12px;
font-size: 13px;
font-weight: 600;
color: #89b4fa;
}
QGroupBox::title {
subcontrol-origin: margin;
subcontrol-position: top left;
left: 12px;
padding: 0 6px;
color: #89b4fa;
}

/* ── Tables ───────────────────────────────────────────────────────── */
QTableWidget {
background-color: #181825;
border: 1px solid #45475a;
border-radius: 6px;
gridline-color: #313244;
font-size: 13px;
color: #cdd6f4;
alternate-background-color: #1e1e2e;
outline: none;
}
QTableWidget::item {
padding: 10px 12px;
border: none;
min-height: 24px;
}
QTableWidget::item:selected {
background-color: #313244;
color: #cdd6f4;
}
QHeaderView {
background-color: #1e1e2e;
}
QHeaderView::section {
background-color: #1e1e2e;
color: #89b4fa;
border: none;
border-right: 1px solid #313244;
border-bottom: 2px solid #45475a;
padding: 10px 12px;
font-size: 12px;
font-weight: 700;
letter-spacing: 0.5px;
}
QHeaderView::section:last {
border-right: none;
}

/* ── ComboBox ─────────────────────────────────────────────────────── */
QComboBox {
background-color: #181825;
border: 1px solid #45475a;
border-radius: 5px;
padding: 5px 10px;
```

```
color: #cdd6f4;
font-size: 13px;
min-height: 26px;
}
QComboBox:focus { border-color: #89b4fa; }
QComboBox::drop-down { border: none; padding-right: 10px; }
QComboBox QAbstractItemView {
background-color: #181825;
border: 1px solid #45475a;
selection-background-color: #313244;
color: #cdd6f4;
outline: none;
}

/* ── Scrollbars ──────────────────────────────────────────────── */
QScrollBar:vertical {
background: #1e1e2e;
width: 8px;
margin: 0;
border-radius: 4px;
}
QScrollBar::handle:vertical {
background: #45475a;
border-radius: 4px;
min-height: 30px;
}
QScrollBar::handle:vertical:hover { background: #6c7086; }
QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical { height: 0; }
QScrollBar:horizontal {
background: #1e1e2e;
height: 8px;
margin: 0;
border-radius: 4px;
}
QScrollBar::handle:horizontal {
background: #45475a;
border-radius: 4px;
min-width: 30px;
}
QScrollBar::handle:horizontal:hover { background: #6c7086; }
QScrollBar::add-line:horizontal, QScrollBar::sub-line:horizontal { width: 0; }

/* ── Status bar ──────────────────────────────────────────────── */
QStatusBar {
background-color: #181825;
color: #6c7086;
border-top: 1px solid #313244;
font-size: 12px;
padding: 2px 8px;
}

/* ── Dialog ──────────────────────────────────────────────────── */
QDialog {
background-color: #1e1e2e;
}
QDialogButtonBox QPushButton {
min-width: 88px;
}

/* ── Message box ─────────────────────────────────────────────── */
QMessageBox {
background-color: #1e1e2e;
}
QMessageBox QLabel {
color: #cdd6f4;
font-size: 13px;
}

/* ── Labels ──────────────────────────────────────────────────── */
```

```
QLabel {
color: #cdd6f4;
font-size: 13px;
}

/* ── Form layout labels ──────────────────────────────────────────── */
QFormLayout QLabel {
color: #a6adc8;
font-size: 13px;
}

/* ── Stack / content area ────────────────────────────────────────── */
QStackedWidget {
background-color: #1e1e2e;
}

/* ── Toolbar area (button rows) ──────────────────────────────────── */
QToolBar {
background-color: #1e1e2e;
border: none;
spacing: 4px;
}

/* ── TextEdit (prompt compose area) ──────────────────────────────── */
QTextEdit {
background-color: #181825;
border: 1px solid #45475a;
border-radius: 5px;
padding: 6px 10px;
font-size: 13px;
color: #cdd6f4;
selection-background-color: #89b4fa;
selection-color: #1e1e2e;
}
QTextEdit:focus {
border-color: #89b4fa;
}

/* ── SpinBox ─────────────────────────────────────────────────────── */
QSpinBox {
background-color: #181825;
border: 1px solid #45475a;
border-radius: 5px;
padding: 5px 10px;
font-size: 13px;
color: #cdd6f4;
min-height: 26px;
}
QSpinBox:focus { border-color: #89b4fa; }
QSpinBox::up-button, QSpinBox::down-button {
width: 18px;
border: none;
background: #313244;
}
QSpinBox::up-button:hover, QSpinBox::down-button:hover {
background: #45475a;
}

/* ── Splitter handle ─────────────────────────────────────────────── */
QSplitter::handle {
background-color: #313244;
height: 2px;
width: 2px;
}
QSplitter::handle:hover {
background-color: #89b4fa;
}

/* ── Table row hover ─────────────────────────────────────────────── */
QTableWidget::item:hover {
```

```
background-color: #2a2a3d;
}
QTableWidget::item:selected {
background-color: #45475a;
color: #ffffff;
}
QTableWidget::item:selected:active {
background-color: #3d3f5c;
border-left: 2px solid #89b4fa;
color: #ffffff;
}

/* ── Screen title labels ──────────────────────────────────────── */
QLabel#screenTitle {
font-size: 20px;
font-weight: 700;
color: #cdd6f4;
padding-bottom: 4px;
}

/* ── Sidebar brand header ─────────────────────────────────────── */
QLabel#sidebarBrand {
font-size: 15px;
font-weight: 800;
color: #89b4fa;
padding: 4px 4px 12px 4px;
letter-spacing: 1px;
}

/* ── Sidebar separator ────────────────────────────────────────── */
QFrame#sidebarSep {
border: none;
border-top: 1px solid #313244;
margin: 0 0 8px 0;
max-height: 1px;
}

/* ── Toolbar separator ────────────────────────────────────────── */
QFrame#toolbarSep {
border: none;
border-top: 1px solid #313244;
margin: 4px 0;
max-height: 1px;
}

/* ── Empty state label ────────────────────────────────────────── */
QLabel#emptyState {
color: #6c7086;
font-size: 14px;
font-style: italic;
padding: 32px 0;
qproperty-alignment: AlignCenter;
}
"""

NAV_ITEMS = [
(0, "Dashboard", QStyle.StandardPixmap.SP_DesktopIcon),
(1, "Students", QStyle.StandardPixmap.SP_FileDialogDetailedView),
(2, "Accounts", QStyle.StandardPixmap.SP_DirHomeIcon),
(3, "Repositories", QStyle.StandardPixmap.SP_DirIcon),
(4, "Credentials", QStyle.StandardPixmap.SP_DialogApplyButton),
(5, "Prompts", QStyle.StandardPixmap.SP_FileDialogNewFolder),
(6, "Logs", QStyle.StandardPixmap.SP_FileDialogInfoView),
(7, "Settings", QStyle.StandardPixmap.SP_FileDialogContentsView),
]

class MainWindow(QMainWindow):
"""Desktop shell with sidebar navigation and stacked screens."""

def __init__(self) -> None:
```

```
super().__init__()
self._nav_buttons: list[QPushButton] = []
self._stack = QStackedWidget()
self._sidebar: QWidget | None = None

self.setWindowTitle("StudentFlow")
self.setMinimumSize(1024, 700)
self._setup_palette()
self._build_ui()

status_bar = QStatusBar(self)
status_bar.showMessage("StudentFlow ready.")
self.setStatusBar(status_bar)

self._navigate_to(0)

def _setup_palette(self) -> None:
"""Apply a dark palette and full QSS for the desktop shell."""
palette = QPalette()
palette.setColor(QPalette.ColorRole.Window, QColor("#1e1e2e"))
palette.setColor(QPalette.ColorRole.WindowText, QColor("#cdd6f4"))
palette.setColor(QPalette.ColorRole.Base, QColor("#181825"))
palette.setColor(QPalette.ColorRole.AlternateBase, QColor("#313244"))
palette.setColor(QPalette.ColorRole.Button, QColor("#313244"))
palette.setColor(QPalette.ColorRole.ButtonText, QColor("#cdd6f4"))
palette.setColor(QPalette.ColorRole.Highlight, QColor("#89b4fa"))
palette.setColor(QPalette.ColorRole.HighlightedText, QColor("#1e1e2e"))
self.setPalette(palette)
self.setStyleSheet(_APP_QSS)

def _build_ui(self) -> None:
"""Construct the sidebar and content stack."""
central = QWidget(self)
root_layout = QHBoxLayout(central)
root_layout.setContentsMargins(0, 0, 0, 0)
root_layout.setSpacing(0)

self._sidebar = self._build_sidebar()
root_layout.addWidget(self._sidebar)

self._load_screens()
root_layout.addWidget(self._stack, 1)

self.setCentralWidget(central)

def _build_sidebar(self) -> QWidget:
"""Create the left navigation panel."""
sidebar = QWidget(self)
sidebar.setFixedWidth(220)
sidebar.setSizePolicy(QSizePolicy.Policy.Fixed, QSizePolicy.Policy.Expanding)
sidebar.setStyleSheet("QWidget { background-color: #181825; border-right: 1px solid #313244;
}")

layout = QVBoxLayout(sidebar)
layout.setContentsMargins(10, 16, 10, 16)
layout.setSpacing(4)

brand = QLabel("StudentFlow", sidebar)
brand.setObjectName("sidebarBrand")
brand.setAlignment(Qt.AlignmentFlag.AlignLeft)
layout.addWidget(brand)

sep = QFrame(sidebar)
sep.setObjectName("sidebarSep")
sep.setFrameShape(QFrame.Shape.HLine)
layout.addWidget(sep)

for index, label, icon_kind in NAV_ITEMS:
button = QPushButton(label, sidebar)
button.setIcon(self.style().standardIcon(icon_kind))
button.setIconSize(QSize(20, 20))
button.setCheckable(True)
button.setFlat(True)
```

```
button.setCursor(Qt.CursorShape.PointingHandCursor)
button.setSizePolicy(
QSizePolicy.Policy.Expanding,
QSizePolicy.Policy.Fixed,
)
button.setStyleSheet(
"QPushButton{"
" padding: 10px 14px;"
" text-align: left;"
" border: 0;"
" border-radius: 6px;"
" font-size: 14px;"
" font-weight: 500;"
" color: #a6adc8;"
"}"
"QPushButton:hover{"
" background: #313244;"
" color: #cdd6f4;"
"}"
"QPushButton:checked{"
" background: #89b4fa;"
" color: #1e1e2e;"
" font-weight: 600;"
"}"
)
button.clicked.connect(
lambda checked=False, target=index: self._navigate_to(target)
)
self._nav_buttons.append(button)
layout.addWidget(button)

layout.addStretch(1)
return sidebar

def _load_screens(self) -> None:
"""Lazily import and add the placeholder screen widgets."""
from app.ui.windows.accounts_screen import AccountsScreen
from app.ui.windows.credentials_screen import CredentialsScreen
from app.ui.windows.dashboard_screen import DashboardScreen
from app.ui.windows.logs_screen import LogsScreen
from app.ui.windows.prompts_screen import PromptsScreen
from app.ui.windows.repositories_screen import RepositoriesScreen
from app.ui.windows.settings_screen import SettingsScreen
from app.ui.windows.students_screen import StudentsScreen

screens = [
DashboardScreen,
StudentsScreen,
AccountsScreen,
RepositoriesScreen,
CredentialsScreen,
PromptsScreen,
LogsScreen,
SettingsScreen,
]
for screen_cls in screens:
self._stack.addWidget(screen_cls(self))

def _navigate_to(self, index: int) -> None:
"""Switch the active screen and update button highlight."""
self._stack.setCurrentIndex(index)
for button_index, button in enumerate(self._nav_buttons):
button.setChecked(button_index == index)
---
### app/ui/windows/prompts_screen.py
**Розмір:** 10,025 байт

"""Prompts screen — create and track prompt delivery to student repos."""
```

```
from __future__ import annotations

from PySide6.QtCore import Qt
from PySide6.QtGui import QColor
from PySide6.QtWidgets import (
QHBoxLayout,
QLabel,
QLineEdit,
QMessageBox,
QPushButton,
QSizePolicy,
QSplitter,
QTableWidget,
QTableWidgetItem,
QTextEdit,
QVBoxLayout,
QWidget,
)

from app.db.enums import PromptStatus
from app.ui.viewmodels.prompts_viewmodel import PromptsViewModel

_STATUS_COLORS = {
PromptStatus.pushed.value: "#a6e3a1", # green
PromptStatus.committed.value: "#89dceb", # teal
PromptStatus.written.value: "#fab387", # peach
PromptStatus.draft.value: "#a6adc8", # subtext
PromptStatus.failed.value: "#f38ba8", # red
}

_COLUMNS = ["#", "Title", "File Path", "Status", "Pushed At"]


class PromptsScreen(QWidget):
def __init__(self, parent=None):
super().__init__(parent)
self._vm = PromptsViewModel()
self._prompts: list = []
self._build_ui()
self.refresh()

# ── layout ──────────────────────────────────────────────────────────────────

def _build_ui(self) -> None:
root = QVBoxLayout(self)
root.setContentsMargins(20, 20, 20, 20)
root.setSpacing(12)

title = QLabel("Prompts", self)
title.setObjectName("screenTitle")
root.addWidget(title)

# Active student banner
self._student_label = QLabel("Active student: —", self)
self._student_label.setStyleSheet("font-weight: 600; font-size: 14px; color: #89b4fa;")
root.addWidget(self._student_label)

splitter = QSplitter(Qt.Orientation.Vertical, self)
splitter.setChildrenCollapsible(False)
root.addWidget(splitter, 1)

# ── top: compose form ───────────────────────────────────────────────────────
compose = QWidget(splitter)
form = QVBoxLayout(compose)
form.setContentsMargins(0, 0, 0, 0)
form.setSpacing(6)

title_row = QWidget(compose)
tr = QHBoxLayout(title_row)
tr.setContentsMargins(0, 0, 0, 0)
tr.addWidget(QLabel("Title:", title_row))
self._title_edit = QLineEdit(title_row)
self._title_edit.setPlaceholderText("Prompt title (optional)")
```

```
tr.addWidget(self._title_edit, 1)
form.addWidget(title_row)

form.addWidget(QLabel("Prompt text:", compose))
self._content_edit = QTextEdit(compose)
self._content_edit.setPlaceholderText("Enter the full prompt here…")
self._content_edit.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Expanding)
form.addWidget(self._content_edit, 1)

btn_row = QWidget(compose)
br = QHBoxLayout(btn_row)
br.setContentsMargins(0, 0, 0, 0)
self._send_btn = QPushButton("Commit & Push", btn_row)
self._send_btn.setProperty("role", "primary")
self._send_btn.clicked.connect(self._on_send)
br.addWidget(self._send_btn)
br.addStretch(1)
form.addWidget(btn_row)

splitter.addWidget(compose)

# ── bottom: history table ──────────────────────────────────────────────
history = QWidget(splitter)
hv = QVBoxLayout(history)
hv.setContentsMargins(0, 0, 0, 0)
hv.setSpacing(6)

hdr = QWidget(history)
hdr_layout = QHBoxLayout(hdr)
hdr_layout.setContentsMargins(0, 0, 0, 0)
hdr_layout.addWidget(QLabel("History", hdr))
ref_btn = QPushButton("Refresh", hdr)
ref_btn.clicked.connect(self.refresh)
hdr_layout.addWidget(ref_btn)
self._retry_btn = QPushButton("Retry Push", hdr)
self._retry_btn.setEnabled(False)
self._retry_btn.clicked.connect(self._on_retry)
hdr_layout.addWidget(self._retry_btn)
hdr_layout.addStretch(1)
hv.addWidget(hdr)

self._table = QTableWidget(history)
self._table.setColumnCount(len(_COLUMNS))
self._table.setHorizontalHeaderLabels(_COLUMNS)
self._table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
self._table.setSelectionBehavior(QTableWidget.SelectionBehavior.SelectRows)
self._table.setAlternatingRowColors(True)
self._table.verticalHeader().setVisible(False)
self._table.verticalHeader().setDefaultSectionSize(44)
self._table.verticalHeader().setMinimumSectionSize(40)
self._table.horizontalHeader().setStretchLastSection(True)
self._table.selectionModel().selectionChanged.connect(self._on_selection)
hv.addWidget(self._table, 1)

# path copy helper
self._path_label = QLabel("", history)
self._path_label.setTextInteractionFlags(Qt.TextInteractionFlag.TextSelectableByMouse)
self._path_label.setStyleSheet("color: #89b4fa; font-family: monospace;")
self._path_label.setWordWrap(True)
hv.addWidget(self._path_label)

splitter.addWidget(history)
splitter.setSizes([250, 350])

# ── data ────────────────────────────────────────────────────────────────────

def refresh(self) -> None:
info = self._vm.get_active_student()
if info:
num = info["student_number"] or "?"
self._student_label.setText(
```

```
f"Active student: #{num:02d} {info['full_name']}"
if isinstance(num, int) else
f"Active student: #{num} {info['full_name']}"
)
self._send_btn.setEnabled(True)
else:
self._student_label.setText("Active student: none")
self._send_btn.setEnabled(False)

self._prompts = self._vm.list_prompts()
self._fill_table()

def _fill_table(self) -> None:
self._table.setRowCount(0)
for row, p in enumerate(self._prompts):
self._table.insertRow(row)
num = f"{p.student_number:02d}-{p.seq_number:03d}" if p.student_number else
f"?-{p.seq_number:03d}"
self._table.setItem(row, 0, QTableWidgetItem(num))
self._table.setItem(row, 1, QTableWidgetItem(p.title or ""))
path_item = QTableWidgetItem(p.file_path or "")
path_item.setToolTip(p.file_path or "")
self._table.setItem(row, 2, path_item)

status_item = QTableWidgetItem(p.status or "")
status_item.setForeground(QColor(_STATUS_COLORS.get(p.status, "#cdd6f4")))
self._table.setItem(row, 3, status_item)

pushed = str(p.pushed_at)[:16] if p.pushed_at else ""
self._table.setItem(row, 4, QTableWidgetItem(pushed))

if p.push_error:
for col in range(len(_COLUMNS)):
item = self._table.item(row, col)
if item:
item.setToolTip(f"Error: {p.push_error}")

self._path_label.setText("")
self._retry_btn.setEnabled(False)

# ── actions ──────────────────────────────────────────────────────────────────

def _on_send(self) -> None:
title = self._title_edit.text().strip()
content = self._content_edit.toPlainText().strip()
if not content:
QMessageBox.warning(self, "Validation", "Prompt text cannot be empty.")
return

self._send_btn.setEnabled(False)
self._send_btn.setText("Pushing…")
try:
prompt = self._vm.create_prompt(title=title, content=content)
except Exception as exc:
QMessageBox.critical(self, "Error", str(exc))
self._send_btn.setEnabled(True)
self._send_btn.setText("Commit & Push")
return
finally:
self._send_btn.setText("Commit & Push")
self._send_btn.setEnabled(True)

if prompt.status == PromptStatus.pushed.value:
self._title_edit.clear()
self._content_edit.clear()
QMessageBox.information(
self, "Pushed",
f"Prompt pushed successfully.\n\nFile path:\n{prompt.file_path}"
)
else:
err = prompt.push_error or "unknown error"
```

```
QMessageBox.warning(
self, "Push Failed",
f"Prompt was committed but push failed:\n{err}\n\nFile path:\n{prompt.file_path}"
)

self.refresh()

def _on_retry(self) -> None:
rows = self._table.selectionModel().selectedRows()
if not rows:
return
prompt = self._prompts[rows[0].row()]
try:
result = self._vm.retry_push(prompt.id)
except Exception as exc:
QMessageBox.critical(self, "Error", str(exc))
return
if result.status == PromptStatus.pushed.value:
QMessageBox.information(self, "Pushed", "Retry succeeded.")
else:
QMessageBox.warning(self, "Failed", f"Retry failed:\n{result.push_error}")
self.refresh()

def _on_selection(self) -> None:
rows = self._table.selectionModel().selectedRows()
if not rows:
self._path_label.setText("")
self._retry_btn.setEnabled(False)
return
prompt = self._prompts[rows[0].row()]
self._path_label.setText(f"Path: {prompt.file_path}")
can_retry = prompt.status in (PromptStatus.failed.value, PromptStatus.committed.value)
self._retry_btn.setEnabled(can_retry)
---
### app/ui/windows/__init__.py
**Розмір:** 38 байт

"""Window classes for StudentFlow."""
---
### app/ui/windows/students_screen.py
**Розмір:** 7,256 байт

"""Students screen — data-driven list with Add/Edit/Archive/Reorder actions."""
from __future__ import annotations

from PySide6.QtGui import QColor
from PySide6.QtWidgets import (
QHBoxLayout,
QLabel,
QMessageBox,
QPushButton,
QTableWidget,
QTableWidgetItem,
QVBoxLayout,
QWidget,
)

from app.ui.viewmodels.students_viewmodel import StudentViewModel

_STATUS_COLORS = {
"active": "#a6e3a1",
"paused": "#f9e2af",
"exhausted": "#9399b2",
"error": "#f38ba8",
"archived": "#585b70",
}

_COLUMNS = ["#", "Full Name", "Status", "Queue Position", "Notes", "Actions"]

class StudentsScreen(QWidget):
```

```
def __init__(self, parent=None):
super().__init__(parent)
self._vm = StudentViewModel()
self._students: list = []
self._build_ui()
self.refresh()

def _build_ui(self) -> None:
layout = QVBoxLayout(self)
layout.setContentsMargins(20, 20, 20, 20)
layout.setSpacing(12)

title = QLabel("Students", self)
title.setObjectName("screenTitle")
layout.addWidget(title)

toolbar = QWidget(self)
tb = QHBoxLayout(toolbar)
tb.setContentsMargins(0, 0, 0, 0)
add_btn = QPushButton("Add Student", toolbar)
add_btn.setProperty("role", "primary")
add_btn.clicked.connect(self._on_add)
tb.addWidget(add_btn)
ref_btn = QPushButton("Refresh", toolbar)
ref_btn.clicked.connect(self.refresh)
tb.addWidget(ref_btn)
tb.addStretch(1)
layout.addWidget(toolbar)

self._table = QTableWidget(self)
self._table.setColumnCount(len(_COLUMNS))
self._table.setHorizontalHeaderLabels(_COLUMNS)
self._table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
self._table.setSelectionBehavior(QTableWidget.SelectionBehavior.SelectRows)
self._table.setAlternatingRowColors(True)
self._table.verticalHeader().setVisible(False)
self._table.verticalHeader().setDefaultSectionSize(44)
self._table.verticalHeader().setMinimumSectionSize(40)
self._table.horizontalHeader().setStretchLastSection(True)
self._table.doubleClicked.connect(self._on_double_click)
layout.addWidget(self._table)

self._empty = QLabel("No students yet. Click Add Student to begin.", self)
self._empty.setObjectName("emptyState")
self._empty.setVisible(False)
layout.addWidget(self._empty)

def refresh(self) -> None:
try:
self._students = self._vm.list_students()
except Exception as e:
QMessageBox.critical(self, "Error", str(e))
return
self._fill_table()

def _fill_table(self) -> None:
students = self._students
n = len(students)
self._table.setVisible(bool(students))
self._empty.setVisible(not bool(students))
self._table.setRowCount(0)
for row, s in enumerate(students):
self._table.insertRow(row)
num_val = s.student_number
num_text = f"#{num_val:02d}" if isinstance(num_val, int) else "?"
self._table.setItem(row, 0, QTableWidgetItem(num_text))
self._table.setItem(row, 1, QTableWidgetItem(s.full_name))

status_item = QTableWidgetItem(s.status)
status_item.setForeground(QColor(_STATUS_COLORS.get(s.status, "#cdd6f4")))
```

```
self._table.setItem(row, 2, status_item)

self._table.setItem(row, 3, QTableWidgetItem(str(s.queue_position or "")))
self._table.setItem(row, 4, QTableWidgetItem((s.notes or "")[:80]))

cell = QWidget()
cl = QHBoxLayout(cell)
cl.setContentsMargins(4, 6, 4, 6)
cl.setSpacing(4)

up_btn = QPushButton("↑")
up_btn.setFixedWidth(28)
up_btn.setToolTip("Move up in queue")
up_btn.setEnabled(row > 0)
up_btn.clicked.connect(lambda _=False, sid=s.id: self._on_move_up(sid))
cl.addWidget(up_btn)

dn_btn = QPushButton("↓")
dn_btn.setFixedWidth(28)
dn_btn.setToolTip("Move down in queue")
dn_btn.setEnabled(row < n - 1)
dn_btn.clicked.connect(lambda _=False, sid=s.id: self._on_move_down(sid))
cl.addWidget(dn_btn)

edit_btn = QPushButton("Edit")
edit_btn.clicked.connect(lambda _=False, sid=s.id: self._on_edit(sid))
cl.addWidget(edit_btn)

arch_btn = QPushButton("Archive")
arch_btn.setProperty("role", "destructive")
arch_btn.clicked.connect(lambda _=False, sid=s.id: self._on_archive(sid))
cl.addWidget(arch_btn)

self._table.setCellWidget(row, 5, cell)

def _on_add(self) -> None:
from app.ui.dialogs.student_dialog import StudentDialog
dlg = StudentDialog(self)
if dlg.exec() == StudentDialog.DialogCode.Accepted:
try:
self._vm.create_student(dlg.get_data())
self.refresh()
except Exception as e:
QMessageBox.critical(self, "Error", str(e))

def _on_edit(self, student_id: str) -> None:
student = next((s for s in self._students if s.id == student_id), None)
if not student:
return
from app.ui.dialogs.student_dialog import StudentDialog
dlg = StudentDialog(self, student=student)
if dlg.exec() == StudentDialog.DialogCode.Accepted:
try:
self._vm.update_student(student_id, dlg.get_data())
self.refresh()
except Exception as e:
QMessageBox.critical(self, "Error", str(e))

def _on_move_up(self, student_id: str) -> None:
try:
self._vm.move_up(student_id)
self.refresh()
except Exception as e:
QMessageBox.critical(self, "Error", str(e))

def _on_move_down(self, student_id: str) -> None:
try:
self._vm.move_down(student_id)
self.refresh()
except Exception as e:
QMessageBox.critical(self, "Error", str(e))
```

```
def _on_archive(self, student_id: str) -> None:
student = next((s for s in self._students if s.id == student_id), None)
name = student.full_name if student else "this student"
reply = QMessageBox.question(
self,
"Archive Student",
f"Archive {name!r}?\n\nThey will be removed from the active queue "
"and their status will be set to archived.\nThis action can only be "
"reversed by editing the student record directly.",
QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No,
QMessageBox.StandardButton.No,
)
if reply != QMessageBox.StandardButton.Yes:
return
try:
self._vm.archive_student(student_id)
self.refresh()
except Exception as e:
QMessageBox.critical(self, "Error", str(e))

def _on_double_click(self, index) -> None:
row = index.row()
if 0 <= row < len(self._students):
self._on_edit(self._students[row].id)
---
### app/ui/windows/accounts_screen.py
**Розмір:** 5,425 байт

"""Accounts screen — data-driven list with Add/Edit/Validate actions."""
from __future__ import annotations

from PySide6.QtWidgets import (
QHBoxLayout,
QLabel,
QMessageBox,
QPushButton,
QTableWidget,
QTableWidgetItem,
QVBoxLayout,
QWidget,
)

from app.ui.viewmodels.accounts_viewmodel import AccountViewModel

_COLUMNS = ["Username", "Provider", "Auth Type", "Status", "Linked Student", "Last Validated",
"Actions"]

class AccountsScreen(QWidget):
def __init__(self, parent=None):
super().__init__(parent)
self._vm = AccountViewModel()
self._accounts: list = []
self._build_ui()
self.refresh()

def _build_ui(self) -> None:
layout = QVBoxLayout(self)
layout.setContentsMargins(20, 20, 20, 20)
layout.setSpacing(12)

title = QLabel("Accounts", self)
title.setObjectName("screenTitle")
layout.addWidget(title)

toolbar = QWidget(self)
tb = QHBoxLayout(toolbar)
tb.setContentsMargins(0, 0, 0, 0)
add_btn = QPushButton("Add Account", toolbar)
add_btn.setProperty("role", "primary")
add_btn.clicked.connect(self._on_add)
```

```
tb.addWidget(add_btn)
ref_btn = QPushButton("Refresh", toolbar)
ref_btn.clicked.connect(self.refresh)
tb.addWidget(ref_btn)
tb.addStretch(1)
layout.addWidget(toolbar)

self._table = QTableWidget(self)
self._table.setColumnCount(len(_COLUMNS))
self._table.setHorizontalHeaderLabels(_COLUMNS)
self._table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
self._table.setSelectionBehavior(QTableWidget.SelectionBehavior.SelectRows)
self._table.setAlternatingRowColors(True)
self._table.verticalHeader().setVisible(False)
self._table.verticalHeader().setDefaultSectionSize(44)
self._table.verticalHeader().setMinimumSectionSize(40)
self._table.horizontalHeader().setStretchLastSection(True)
layout.addWidget(self._table)

self._empty = QLabel("No accounts yet. Click Add Account to begin.", self)
self._empty.setObjectName("emptyState")
self._empty.setVisible(False)
layout.addWidget(self._empty)

def refresh(self) -> None:
try:
self._accounts = self._vm.list_accounts()
self._students = self._vm.list_students()
except Exception as e:
QMessageBox.critical(self, "Error", str(e))
return
self._fill_table()

def _fill_table(self) -> None:
accounts = self._accounts
students_by_id = {s.id: s.full_name for s in self._students}
self._table.setVisible(bool(accounts))
self._empty.setVisible(not bool(accounts))
self._table.setRowCount(0)
for row, a in enumerate(accounts):
self._table.insertRow(row)
self._table.setItem(row, 0, QTableWidgetItem(a.username))
self._table.setItem(row, 1, QTableWidgetItem(a.provider))
self._table.setItem(row, 2, QTableWidgetItem(a.auth_type))
self._table.setItem(row, 3, QTableWidgetItem(a.status))
self._table.setItem(row, 4, QTableWidgetItem(students_by_id.get(a.student_id, a.student_id)))
last_val = str(a.last_validated_at)[:16] if a.last_validated_at else ""
self._table.setItem(row, 5, QTableWidgetItem(last_val))

cell = QWidget()
cl = QHBoxLayout(cell)
cl.setContentsMargins(4, 6, 4, 6)
cl.setSpacing(4)
edit_btn = QPushButton("Edit")
edit_btn.clicked.connect(lambda _=False, aid=a.id: self._on_edit(aid))
cl.addWidget(edit_btn)
val_btn = QPushButton("Validate")
val_btn.clicked.connect(lambda _=False, aid=a.id: self._on_validate(aid))
cl.addWidget(val_btn)
self._table.setCellWidget(row, 6, cell)

def _on_add(self) -> None:
from app.ui.dialogs.account_dialog import AccountDialog
dlg = AccountDialog(self, students=self._students)
if dlg.exec() == AccountDialog.DialogCode.Accepted:
try:
self._vm.create_account(dlg.get_data())
self.refresh()
except Exception as e:
```

```
QMessageBox.critical(self, "Error", str(e))

def _on_edit(self, account_id: str) -> None:
account = next((a for a in self._accounts if a.id == account_id), None)
if not account:
return
from app.ui.dialogs.account_dialog import AccountDialog
dlg = AccountDialog(self, account=account, students=self._students)
if dlg.exec() == AccountDialog.DialogCode.Accepted:
try:
self._vm.update_account(account_id, dlg.get_data())
self.refresh()
except Exception as e:
QMessageBox.critical(self, "Error", str(e))

def _on_validate(self, account_id: str) -> None:
try:
result = self._vm.validate_account(account_id)
self.refresh()
if result.get("validated"):
QMessageBox.information(self, "Validation Result", result["message"])
else:
QMessageBox.warning(self, "Validation Result", result["message"])
except Exception as e:
QMessageBox.critical(self, "Error", str(e))
---
### app/ui/windows/settings_screen.py
**Розмір:** 5,299 байт

"""Settings screen — operator-visible AppSetting configuration."""
from __future__ import annotations

from PySide6.QtWidgets import (
QComboBox,
QFormLayout,
QGroupBox,
QHBoxLayout,
QLabel,
QLineEdit,
QMessageBox,
QPushButton,
QVBoxLayout,
QWidget,
)

from app.ui.viewmodels.settings_viewmodel import SettingsViewModel

_LABELS = {
"canonical_repo_path": "Canonical local path",
"canonical_remote_url": "Canonical remote URL (optional)",
"default_branch": "Default branch",
"sync_timeout_seconds": "Sync timeout (seconds)",
"max_sync_retries": "Max sync retries",
}

_OPTIONAL_KEYS = {"canonical_remote_url"}
```

Keys rendered as QLineEdit (canonical_remote_account_id is a ComboBox)

```
_LINE_EDIT_KEYS = [
"canonical_repo_path",
"canonical_remote_url",
"default_branch",
"sync_timeout_seconds",
"max_sync_retries",
```

```
]

class SettingsScreen(QWidget):
def __init__(self, parent=None):
super().__init__(parent)
self._vm = SettingsViewModel()
self._fields: dict[str, QLineEdit] = {}
self._account_combo: QComboBox | None = None
self._build_ui()
self._load()

def _build_ui(self) -> None:
layout = QVBoxLayout(self)
layout.setContentsMargins(24, 24, 24, 24)
layout.setSpacing(16)

title = QLabel("Settings", self)
title.setObjectName("screenTitle")
layout.addWidget(title)

group = QGroupBox("Canonical Repository & Sync", self)
form = QFormLayout(group)
form.setLabelAlignment(form.labelAlignment())
form.setRowWrapPolicy(QFormLayout.RowWrapPolicy.WrapLongRows)

for key in _LINE_EDIT_KEYS:
field = QLineEdit(self)
field.setObjectName(key)
if key in _OPTIONAL_KEYS:
field.setPlaceholderText("e.g. https://github.com/org/canonical-project.git")
form.addRow(f"{_LABELS.get(key, key)}:", field)
self._fields[key] = field

# Insert the account ComboBox directly after canonical_remote_url
if key == "canonical_remote_url":
self._account_combo = QComboBox(self)
self._account_combo.setObjectName("canonical_remote_account_id")
self._account_combo.setToolTip(
"Account whose stored credential is used when pushing to the canonical remote.\n"
"Select 'None' to rely on system-level git authentication instead."
)
form.addRow("Canonical remote account:", self._account_combo)

layout.addWidget(group)

button_row = QWidget(self)
button_layout = QHBoxLayout(button_row)
button_layout.setContentsMargins(0, 0, 0, 0)

save_btn = QPushButton("Save", button_row)
save_btn.setProperty("role", "primary")
save_btn.clicked.connect(self._on_save)
button_layout.addWidget(save_btn)

reset_btn = QPushButton("Reset to saved", button_row)
reset_btn.clicked.connect(self._load)
button_layout.addWidget(reset_btn)

button_layout.addStretch(1)
layout.addWidget(button_row)
layout.addStretch(1)

def _load(self) -> None:
try:
rows = {row["key"]: row["value"] for row in self._vm.list_operator_settings()}
for key, field in self._fields.items():
field.setText(rows.get(key, ""))
self._reload_account_combo(rows.get("canonical_remote_account_id", ""))
except Exception as exc:
QMessageBox.critical(self, "Error", str(exc))

def _reload_account_combo(self, selected_id: str) -> None:
```

```
if self._account_combo is None:
return
self._account_combo.clear()
self._account_combo.addItem("— None (system auth) —", "")
try:
for acct in self._vm.list_accounts():
label = f"{acct['username']} ({acct['provider']} / {acct['auth_type']})"
self._account_combo.addItem(label, acct["id"])
except Exception:
pass # Accounts unavailable — combo stays with None only
# Select the previously saved account
for i in range(self._account_combo.count()):
if self._account_combo.itemData(i) == selected_id:
self._account_combo.setCurrentIndex(i)
break

def _on_save(self) -> None:
errors = []
for key, field in self._fields.items():
value = field.text().strip()
if not value and key not in _OPTIONAL_KEYS:
errors.append(f"'{_LABELS.get(key, key)}' cannot be empty")

if errors:
QMessageBox.warning(self, "Validation", "\n".join(errors))
return

try:
for key, field in self._fields.items():
self._vm.set_setting(key, field.text().strip())
if self._account_combo is not None:
account_id = self._account_combo.currentData() or ""
self._vm.set_setting("canonical_remote_account_id", account_id)
QMessageBox.information(self, "Saved", "Settings saved successfully.")
except Exception as exc:
QMessageBox.critical(self, "Error", str(exc))
```

---

app/ui/windows/logs_screen.py

Розмір: 4,185 байт

```
"""Logs screen — read-only audit trail with filter and CSV export."""
from __future__ import annotations

import json

from PySide6.QtWidgets import (
QComboBox,
QHBoxLayout,
QLabel,
QLineEdit,
QMessageBox,
QPushButton,
QTableWidget,
QTableWidgetItem,
QVBoxLayout,
QWidget,
)

from app.db.enums import EntityType
from app.ui.viewmodels.logs_viewmodel import LogViewModel

_COLUMNS = ["Timestamp", "Actor", "Action", "Entity Type", "Entity ID", "Details"]

class LogsScreen(QWidget):
def __init__(self, parent=None):
super().__init__(parent)
self._vm = LogViewModel()
self._logs: list = []
self._build_ui()
self.refresh()
```

```
def _build_ui(self) -> None:
layout = QVBoxLayout(self)
layout.setContentsMargins(20, 20, 20, 20)
layout.setSpacing(12)

title = QLabel("Logs", self)
title.setObjectName("screenTitle")
layout.addWidget(title)

toolbar = QWidget(self)
tb = QHBoxLayout(toolbar)
tb.setContentsMargins(0, 0, 0, 0)

tb.addWidget(QLabel("Entity Type:", toolbar))
self._type_filter = QComboBox(toolbar)
self._type_filter.addItem("(all)", "")
for et in EntityType:
self._type_filter.addItem(et.value, et.value)
self._type_filter.currentIndexChanged.connect(self.refresh)
tb.addWidget(self._type_filter)

tb.addWidget(QLabel("Action:", toolbar))
self._action_filter = QLineEdit(toolbar)
self._action_filter.setPlaceholderText("filter by action...")
self._action_filter.setMaximumWidth(180)
self._action_filter.returnPressed.connect(self.refresh)
tb.addWidget(self._action_filter)

ref_btn = QPushButton("Refresh", toolbar)
ref_btn.clicked.connect(self.refresh)
tb.addWidget(ref_btn)

exp_btn = QPushButton("Export CSV", toolbar)
exp_btn.clicked.connect(self._on_export)
tb.addWidget(exp_btn)
tb.addStretch(1)
layout.addWidget(toolbar)

self._table = QTableWidget(self)
self._table.setColumnCount(len(_COLUMNS))
self._table.setHorizontalHeaderLabels(_COLUMNS)
self._table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
self._table.setSelectionBehavior(QTableWidget.SelectionBehavior.SelectRows)
self._table.setAlternatingRowColors(True)
self._table.verticalHeader().setVisible(False)
self._table.verticalHeader().setDefaultSectionSize(44)
self._table.verticalHeader().setMinimumSectionSize(40)
self._table.horizontalHeader().setStretchLastSection(True)
layout.addWidget(self._table)

def refresh(self) -> None:
entity_type = self._type_filter.currentData() or None
action = self._action_filter.text().strip() or None
try:
self._logs = self._vm.get_logs(entity_type=entity_type, action=action)
except Exception as e:
QMessageBox.critical(self, "Error", str(e))
return
self._fill_table()

def _fill_table(self) -> None:
self._table.setRowCount(0)
for row, log in enumerate(self._logs):
self._table.insertRow(row)
self._table.setItem(row, 0, QTableWidgetItem(str(log.created_at)[:19]))
self._table.setItem(row, 1, QTableWidgetItem(log.actor or ""))
self._table.setItem(row, 2, QTableWidgetItem(log.action or ""))
self._table.setItem(row, 3, QTableWidgetItem(log.entity_type or ""))
self._table.setItem(row, 4, QTableWidgetItem(log.entity_id or ""))

details_raw = json.dumps(log.details) if log.details else ""
```

```
details_item = QTableWidgetItem(details_raw[:80])
if details_raw:
details_item.setToolTip(details_raw)
self._table.setItem(row, 5, details_item)

def _on_export(self) -> None:
try:
path = self._vm.export_csv(self._logs)
QMessageBox.information(self, "Export CSV", f"Exported to:\n{path}")
except Exception as e:
QMessageBox.critical(self, "Error", str(e))
```
---
app/ui/viewmodels/settings_viewmodel.py
Розмір: 2,370 байт

```
"""ViewModel for SettingsScreen — read/write AppSetting records."""
from __future__ import annotations

from app.db.base import SessionLocal
from app.db.models import AppSetting

OPERATOR_SETTINGS = [
"canonical_repo_path",
"canonical_remote_url",
"canonical_remote_account_id",
"default_branch",
"sync_timeout_seconds",
"max_sync_retries",
]
```

Settings that may be left empty (will be stored as empty string, not rejected)

```
OPTIONAL_SETTINGS = {"canonical_remote_url", "canonical_remote_account_id"}

class SettingsViewModel:
def __init__(self, session_factory=None):
self._factory = session_factory or SessionLocal

def get_setting(self, key: str) -> str | None:
with self._factory() as session:
row = session.get(AppSetting, key)
return row.value if row is not None else None

def set_setting(self, key: str, value: str) -> None:
if key not in OPTIONAL_SETTINGS and (not value or not value.strip()):
raise ValueError(f"Setting '{key}' cannot be empty")

with self._factory() as session:
row = session.get(AppSetting, key)
if row is None:
row = AppSetting(key=key, value=value.strip(), description="")
session.add(row)
else:
row.value = value.strip()
session.commit()

def list_accounts(self) -> list[dict]:
"""Return all accounts for the canonical remote account dropdown."""
from app.db.models import Account
with self._factory() as session:
accounts = session.query(Account).order_by(Account.username).all()
return [
{"id": a.id, "username": a.username, "provider": a.provider, "auth_type": a.auth_type}
for a in accounts
]
```

```
def list_operator_settings(self) -> list[dict]:
"""Return operator-visible settings as list of {key, value, description}."""
with self._factory() as session:
rows = (
session.query(AppSetting)
.filter(AppSetting.key.in_(OPERATOR_SETTINGS))
.order_by(AppSetting.key)
.all()
)
return [
{"key": row.key, "value": row.value, "description": row.description or ""}
for row in rows
]
```
---
app/ui/viewmodels/prompts_viewmodel.py
Розмір: 1,689 байт

```
"""ViewModel for PromptsScreen."""
from __future__ import annotations

from app.db.base import SessionLocal
from app.db.enums import StudentStatus
from app.db.models import Student
from app.services.prompt_service import PromptService

class PromptsViewModel:
def __init__(self, session_factory=None, prompt_service=None):
self._factory = session_factory or SessionLocal
self._service = prompt_service or PromptService(session_factory=self._factory)

def get_active_student(self) -> dict | None:
"""Return basic info about the current active student, or None."""
with self._factory() as session:
student = (
session.query(Student)
.filter(Student.status == StudentStatus.active.value)
.order_by(Student.queue_position.asc(), Student.created_at.asc())
.first()
)
if student is None:
return None
return {
"id": student.id,
"full_name": student.full_name,
"student_number": student.student_number,
}

def create_prompt(self, title: str, content: str, created_by: str = "operator"):
"""Delegate to PromptService. Returns the Prompt ORM object."""
return self._service.create_and_push_prompt(
title=title, content=content, created_by=created_by
)

def retry_push(self, prompt_id: str):
"""Re-attempt push for a failed prompt."""
return self._service.retry_push(prompt_id)

def list_prompts(self, student_id: str | None = None) -> list:
return self._service.list_prompts(student_id=student_id)
```
---
app/ui/viewmodels/__init__.py
Розмір: 42 байт

```
"""View-model package for StudentFlow."""
```
---
app/ui/viewmodels/students_viewmodel.py
Розмір: 1,146 байт

```
"""ViewModel for StudentsScreen."""
```

```
from __future__ import annotations

from app.db.base import SessionLocal
from app.services.account_service import AccountService

class StudentViewModel:
def __init__(self):
self._factory = SessionLocal
self._service = AccountService(session_factory=self._factory)

def list_students(self, include_archived: bool = False) -> list:
return self._service.list_students(include_archived=include_archived)

def create_student(self, data: dict) -> None:
self._service.create_student(**data)

def update_student(self, student_id: str, data: dict) -> None:
self._service.update_student(student_id, **data)

def archive_student(self, student_id: str) -> None:
self._service.archive_student(student_id)

def move_up(self, student_id: str) -> None:
self._service.move_student_up(student_id)

def move_down(self, student_id: str) -> None:
self._service.move_student_down(student_id)

def set_queue_position(self, student_id: str, new_position: int) -> None:
self._service.set_queue_position(student_id, new_position)
```
---
app/ui/viewmodels/dashboard_viewmodel.py
Розмір: 6,457 байт

```
"""ViewModel for DashboardScreen — canonical repo summary and operational state."""
from __future__ import annotations

from pathlib import Path

from app.db.base import SessionLocal
from app.db.enums import PromptStatus, StudentStatus
from app.db.models import AppSetting, Prompt, Repository, Student, SyncJob

class DashboardViewModel:
def __init__(self, session_factory=None):
self._factory = session_factory or SessionLocal

def get_canonical_summary(self) -> dict:
"""Return a dict with keys: path, path_valid, canonical_repo."""
with self._factory() as session:
setting = session.get(AppSetting, "canonical_repo_path")
path = setting.value if setting is not None else None

path_valid = False
if path:
repo_path = Path(path)
path_valid = repo_path.exists() and (
(repo_path / ".git").exists() or (repo_path / "HEAD").exists()
)

canonical_repo = session.query(Repository).filter_by(is_canonical=True).first()
if canonical_repo is not None:
session.expunge(canonical_repo)

return {
"path": path,
"path_valid": path_valid,
"canonical_repo": canonical_repo,
}

def get_active_student_info(self) -> dict | None:
"""Return active student info dict, or None if no active student.

Keys: id, student_number, full_name, queue_position, status, prompt_count
```

```
"""
with self._factory() as session:
student = (
session.query(Student)
.filter(Student.status == StudentStatus.active.value)
.order_by(Student.queue_position)
.first()
)
if student is None:
return None
prompt_count = (
session.query(Prompt)
.filter(
Prompt.student_id == student.id,
Prompt.status == PromptStatus.pushed.value,
)
.count()
)
return {
"id": student.id,
"student_number": student.student_number,
"full_name": student.full_name,
"queue_position": student.queue_position,
"status": student.status,
"prompt_count": prompt_count,
}

def get_student_queue(self, limit: int = 6) -> list[dict]:
"""Return active/paused students ordered by queue_position.

Keys: student_number, full_name, queue_position, status
"""
with self._factory() as session:
students = (
session.query(Student)
.filter(
Student.status.in_([
StudentStatus.active.value,
StudentStatus.paused.value,
])
)
.order_by(Student.queue_position)
.limit(limit)
.all()
)
return [
{
"student_number": s.student_number,
"full_name": s.full_name,
"queue_position": s.queue_position,
"status": s.status,
}
for s in students
]

def get_recent_sync_activity(self, limit: int = 5) -> list[dict]:
"""Return most recent sync jobs ordered by created_at desc.

Keys: student_name, student_number, job_type, status, started_at, summary
"""
with self._factory() as session:
rows = (
session.query(SyncJob, Student)
.join(Student, SyncJob.student_id == Student.id)
.order_by(SyncJob.created_at.desc())
.limit(limit)
.all()
)
return [
```

```
{
"student_name": student.full_name,
"student_number": student.student_number,
"job_type": job.job_type,
"status": job.status,
"started_at": job.started_at,
"summary": job.summary,
}
for job, student in rows
]

def sync_active_student(self) -> dict:
"""Sync the active student repo. Returns result dict or raises ValueError."""
from app.services.sync_service import SyncService
info = self.get_active_student_info()
if info is None:
raise ValueError("No active student to sync.")
service = SyncService(session_factory=self._factory)
job = service.sync_student(info["id"])
return {
"status": job.status,
"summary": job.summary,
"error_message": job.error_message,
"student_name": info["full_name"],
}

def handoff_to_next(self, operator: str = "operator") -> dict:
"""Hand off from active student to next in queue. Returns result dict or raises."""
from app.git.client import GitClient
from app.services.sync_service import SyncService
from app.services.credential_service import CredentialService
from app.services.audit_service import AuditService
from app.services.handoff_service import HandoffService
git_client = GitClient()
credential_service = CredentialService(session_factory=self._factory)
audit_service = AuditService(session_factory=self._factory)
sync_service = SyncService(
git_client=git_client,
credential_service=credential_service,
audit_service=audit_service,
session_factory=self._factory,
)
handoff_service = HandoffService(
git_client=git_client,
sync_service=sync_service,
credential_service=credential_service,
audit_service=audit_service,
session_factory=self._factory,
)
active = handoff_service.get_active_student()
next_s = handoff_service.get_next_student()
event = handoff_service.handoff_to_next(operator=operator)
return {
"from_student": active.full_name if active else "?",
"to_student": next_s.full_name if next_s else "?",
"checkpoint_tag": event.checkpoint_tag,
}
---
### app/ui/viewmodels/repos_viewmodel.py
**Розмір:** 2,546 байт

"""ViewModel for RepositoriesScreen."""
from __future__ import annotations

import uuid

from app.db.base import SessionLocal
from app.db.enums import SyncStatus
```

```
from app.db.models import Repository
from app.services.repository_service import RepositoryService

class RepoViewModel:
def __init__(self, session_factory=None):
self._factory = session_factory or SessionLocal
self._service = RepositoryService(session_factory=self._factory)

def list_repos(self) -> list[Repository]:
with self._factory() as session:
repos = session.query(Repository).order_by(Repository.repo_name).all()
session.expunge_all()
return repos

def validate_local_path(self, path: str) -> bool:
return self._service.validate_local_path(path)

def validate_remote(self, repo_id: str) -> bool:
return self._service.validate_remote(repo_id)

def get_repo_status(self, repo_id: str) -> str:
return self._service.get_repo_status(repo_id).value

def create_repository(self, data: dict) -> Repository:
with self._factory() as session:
repo = Repository(
id=str(uuid.uuid4()),
account_id=data["account_id"],
repo_name=data["repo_name"],
remote_url=data["remote_url"],
local_path=data["local_path"],
default_branch=data.get("default_branch", "main"),
integration_branch=data.get("integration_branch", "main"),
sync_status=SyncStatus.uninitialized.value,
)
session.add(repo)
session.commit()
session.refresh(repo)
session.expunge(repo)
return repo

def set_canonical(self, repo_id: str) -> None:
"""Mark repo_id as canonical, clearing any previous canonical flag."""
with self._factory() as session:
repo = session.get(Repository, repo_id)
if repo is None:
raise ValueError(f"Repository {repo_id} not found")
session.query(Repository).filter(
Repository.is_canonical == True # noqa: E712
).update({"is_canonical": False})
repo.is_canonical = True
session.commit()

def list_accounts(self) -> list:
from app.db.models import Account
with self._factory() as session:
accounts = session.query(Account).order_by(Account.username).all()
session.expunge_all()
return accounts
---
### app/ui/viewmodels/logs_viewmodel.py
**Розмір:** 1,562 байт

"""ViewModel for LogsScreen."""
from __future__ import annotations

import csv
import json
from datetime import datetime
from pathlib import Path
```

```
from app.db.base import SessionLocal
from app.db.models import ActivityLog
from app.services.audit_service import AuditService

class LogViewModel:
def __init__(self):
self._factory = SessionLocal
self._service = AuditService(session_factory=self._factory)

def get_logs(
self,
entity_type: str | None = None,
action: str | None = None,
limit: int = 200,
) -> list[ActivityLog]:
return self._service.get_logs(
entity_type=entity_type or None,
action=action or None,
limit=limit,
)

def export_csv(self, logs: list[ActivityLog]) -> Path:
export_dir = Path.home() / ".studentflow" / "logs"
export_dir.mkdir(parents=True, exist_ok=True)
ts = datetime.now().strftime("%Y%m%d_%H%M%S")
path = export_dir / f"export-{ts}.csv"
with path.open("w", newline="", encoding="utf-8") as f:
writer = csv.writer(f)
writer.writerow(["Timestamp", "Actor", "Action", "Entity Type", "Entity ID", "Details"])
for log in logs:
writer.writerow([
log.created_at,
log.actor,
log.action,
log.entity_type,
log.entity_id or "",
json.dumps(log.details) if log.details else "",
])
return path
---
### app/ui/viewmodels/credentials_viewmodel.py
**Розмір:** 2,142 байт

"""ViewModel for CredentialsScreen."""
from __future__ import annotations

from app.db.base import SessionLocal
from app.db.models import Account, Credential
from app.security.masking import mask_secret
from app.services.credential_service import CredentialService

class CredentialViewModel:
def __init__(self):
self._factory = SessionLocal
self._service = CredentialService(session_factory=self._factory)

def list_credentials(self) -> list[dict]:
"""Return credential rows with masked secret values."""
with self._factory() as session:
rows = (
session.query(Credential, Account)
.join(Account, Credential.account_id == Account.id)
.all()
)
result = []
for cred, account in rows:
result.append({
"id": cred.id,
```

```
"account_id": account.id,
"username": account.username,
"provider": account.provider,
"auth_type": cred.secret_kind,
"secret_ref": cred.secret_ref,
"last_validated_at": cred.last_validated_at,
})
return result

def masked_secret(self, secret_ref: str) -> str:
try:
value = self._service.retrieve(secret_ref)
return mask_secret(value)
except Exception:
return "••••••••[error]"

def rotate(self, credential_id: str, new_value: str) -> None:
self._service.rotate(credential_id, new_value)

def validate_exists(self, secret_ref: str) -> bool:
return self._service.validate_exists(secret_ref)

def list_accounts(self) -> list[Account]:
with self._factory() as session:
accounts = session.query(Account).order_by(Account.username).all()
session.expunge_all()
return accounts

def store(self, account_id: str, secret_kind: str, value: str) -> None:
from app.db.enums import SecretKind
self._service.store(account_id, SecretKind(secret_kind), value)
---
### app/ui/viewmodels/accounts_viewmodel.py
**Розмір:** 847 байт

"""ViewModel for AccountsScreen."""
from __future__ import annotations

from app.db.base import SessionLocal
from app.services.account_service import AccountService

class AccountViewModel:
def __init__(self):
self._factory = SessionLocal
self._service = AccountService(session_factory=self._factory)

def list_accounts(self) -> list:
return self._service.list_accounts()

def list_students(self) -> list:
return self._service.list_students(include_archived=False)

def create_account(self, data: dict) -> None:
self._service.create_account(**data)

def update_account(self, account_id: str, data: dict) -> None:
self._service.update_account(account_id, **data)

def validate_account(self, account_id: str) -> dict:
return self._service.validate_account(account_id)
---
### app/ui/widgets/__init__.py
**Розмір:** 38 байт

"""Widget classes for StudentFlow."""
---
### app/schemas/__init__.py
**Розмір:** 38 байт

"""Schema package for StudentFlow."""
---
```

app/schemas/account.py
Розмір: 680 байт

```
"""Pydantic input schemas for Account operations."""
from typing import Optional

from pydantic import BaseModel, Field

from app.db.enums import AccountStatus, AuthType

class AccountCreate(BaseModel):
student_id: str = Field(min_length=1)
provider: str = Field(pattern=r"^(github|gitlab|bitbucket|gitea|other)$")
username: str = Field(min_length=1, max_length=100, pattern=r"^[\w\.\-]+$")
auth_type: AuthType
status: AccountStatus = AccountStatus.unvalidated

class AccountUpdate(BaseModel):
username: Optional[str] = Field(default=None, min_length=1, max_length=100)
auth_type: Optional[AuthType] = None
status: Optional[AccountStatus] = None
```

---
app/schemas/repository.py
Розмір: 959 байт

```
"""Pydantic input schemas for Repository operations."""
from pathlib import Path
from typing import Optional

from pydantic import BaseModel, Field, field_validator

class RepositoryCreate(BaseModel):
account_id: str = Field(min_length=1)
repo_name: str = Field(min_length=1, max_length=200, pattern=r"^[\w\.\-]+$")
remote_url: str = Field(min_length=1)
local_path: str = Field(min_length=1)
default_branch: str = Field(default="main", min_length=1, max_length=100)
template_source_id: Optional[str] = None

@field_validator("remote_url")
@classmethod
def validate_remote(cls, v: str) -> str:
from app.git.validators import GitValidator
return GitValidator.validate_remote_url(v)

@field_validator("local_path")
@classmethod
def validate_path_format(cls, v: str) -> str:
if not Path(v).is_absolute():
raise ValueError("local_path must be an absolute path")
return v
```

---
app/schemas/student.py
Розмір: 968 байт

```
"""Pydantic input schemas for Student operations."""
from typing import Optional

from pydantic import BaseModel, Field, field_validator

from app.db.enums import StudentStatus

class StudentCreate(BaseModel):
full_name: str = Field(min_length=1, max_length=200)
status: StudentStatus = StudentStatus.active
queue_position: int = Field(ge=1, le=999, default=999)
notes: Optional[str] = Field(default=None, max_length=1000)

@field_validator("full_name")
@classmethod
```

```
def strip_whitespace(cls, v: str) -> str:
stripped = v.strip()
if not stripped:
raise ValueError("full_name must not be blank")
return stripped

class StudentUpdate(BaseModel):
full_name: Optional[str] = Field(default=None, min_length=1, max_length=200)
status: Optional[StudentStatus] = None
queue_position: Optional[int] = Field(default=None, ge=1, le=999)
notes: Optional[str] = Field(default=None, max_length=1000)
```
---
docs/audit-and-logging.md
Розмір: 11,284 байт

Audit and Logging

StudentFlow uses two complementary logging systems: a structured audit log in the database (`ActivityLog` table) for operator-visible history, and structured JSON file logs via `structlog` for developer/debugging visibility. Both systems filter secrets before any write.

Audit Log (ActivityLog table)

The `ActivityLog` table is the definitive record of every significant operator action. It is append-only — once written, records are never modified or deleted.

Writing Rules

1. **Always write through `AuditService.log()`** — no other code path should INSERT into `ActivityLog`.
2. **`AuditService.log()` never raises.** It wraps all DB and serialization operations in a try/except. If the DB write fails, the error is logged to structlog instead. This ensures that a logging failure never interrupts a real operation.
3. **Write AFTER the primary operation succeeds.** Audit log entries confirm what happened, not what was attempted. (Exception: for failed operations, write with the failure details.)
4. **Filter secrets before write.** The `_filter_details()` function removes any key whose name matches the secret pattern before the JSON is stored.

Secret Filtering

```
import re

SECRET_KEY_PATTERN = re.compile(
    r&apos;password|token|secret|key|credential|pat|passphrase&apos;,
    re.IGNORECASE
)

def _filter_details(details: dict | None) -&gt; dict | None:
    if not details:
        return details
    return {
        k: v
        for k, v in details.items()
        if not SECRET_KEY_PATTERN.search(str(k))
    }
```

This filter removes top-level keys only. Nested dicts are not recursively filtered — instead, callers must not pass nested dicts with secret values. If a nested structure is needed, flatten it and filter at construction time.

AuditService.log() Signature

```
def log(
    actor: str,           # &quot;operator&quot; in all current cases
    entity_type: str,     # EntityType enum value
    entity_id: str | None,  # UUID of the affected record (stringified)
    action: str,          # verb, e.g. &quot;create&quot;, &quot;sync&quot;, &quot;handoff&quot;
    details: dict | None = None  # structured context, will be filtered
) -&gt; ActivityLog | None:
    # Returns the created ActivityLog record, or None if write failed
```

Complete Event Reference

entity_type	action	Required details keys	Notes
student	create	student_id, full_name	
student	archive	student_id, full_name, reason	reason = operator-supplied or "no reason given"
student	update	student_id, changed_fields	changed_fields = list of field names changed
student	status_change	student_id, old_status, new_status	
account	create	account_id, student_id, username, provider	
account	deactivate	account_id, username	
account	validate	account_id, username, provider, result	result = "success" or "failed"
credential	store	account_id, username, secret_kind	NO value
credential	rotate	account_id, username, secret_kind	NO value
credential	delete	account_id, username	
credential	validate	account_id, username, result	result = "success" or "failed"
repository	create	repo_id, student_id, repo_name, remote_url	
repository	bootstrap	repo_id, student_id, template_source_id	
repository	sync	repo_id, student_id, job_id, commit_before, commit_after, duration_ms	
repository	sync_failed	repo_id, student_id, error_summary	error_summary truncated to 500 chars
repository	force_overwrite	repo_id, student_id, canonical_commit	destructive — always log
sync_job	start	job_id, job_type, student_id, repo_id	
sync_job	complete	job_id, status, commits_synced	
sync_job	failed	job_id, error_summary	
handoff	start	handoff_id, from_student, to_student	from_student may be None
handoff	complete	handoff_id, from_student, to_student, commit_hash, tag	
handoff	failed	handoff_id, step, error	step = step number (1-9) where failure occurred
handoff	rolled_back	handoff_id, reason	
template	import	template_id, name, local_path, checksum	
template	validate	template_id, result	result = "ok" or "checksum_mismatch"
system	startup	version, db_path, canonical_path	
system	settings_change	changed_keys	list of AppSetting keys changed
system	error	module, error_type, error_summary	for unhandled exceptions

Query Interface

```
def query(
    entity_type: str | None = None,
    entity_id: str | None = None,
    action: str | None = None,
    actor: str | None = None,
    since: datetime | None = None,
    until: datetime | None = None,
    limit: int = 100,
    offset: int = 0
) -&gt; list[ActivityLog]:
```

All filters are optional and combined with AND. Results are ordered by `created_at DESC`.

Structured Logs (structlog)

Structlog writes JSON Lines to `~/.studentflow/logs/app.log`. These logs are for the operator and developer — not surfaced in the main UI (only the Logs screen's "Errors" tab shows structured

log errors).

Configuration

```
import structlog
import logging
import logging.handlers

def init_logging(log_dir: Path, debug: bool = False) -&gt; None:
    # Rotating file handler: 10MB max, 5 backups
    handler = logging.handlers.RotatingFileHandler(
        filename=log_dir / &quot;app.log&quot;,
        maxBytes=10 * 1024 * 1024,  # 10MB
        backupCount=5,
        encoding=&quot;utf-8&quot;
    )

    structlog.configure(
        processors=[
            structlog.stdlib.add_log_level,
            structlog.stdlib.add_logger_name,
            structlog.processors.TimeStamper(fmt=&quot;iso&quot;),
            LogSecretFilter(),          # Custom processor — strips secret fields
            structlog.processors.JSONRenderer()
        ],
        context_class=dict,
        logger_factory=structlog.stdlib.LoggerFactory(),
        wrapper_class=structlog.stdlib.BoundLogger,
        cache_logger_on_first_use=True,
    )
```

LogSecretFilter Processor

```
class LogSecretFilter:
    &quot;&quot;&quot;Structlog processor that removes secret fields from log events.&quot;&quot;&quot;

    SECRET_PATTERN = re.compile(
        r&apos;password|token|secret|key|credential|pat|passphrase&apos;,
        re.IGNORECASE
    )

    def __call__(self, logger, method, event_dict: dict) -&gt; dict:
        return {
            k: v
            for k, v in event_dict.items()
            if not self.SECRET_PATTERN.search(str(k))
        }
```

This processor runs before `JSONRenderer` — secrets are never serialized to the log file.

Log Entry Structure

Every log entry emitted by structlog includes these fields:

```
{
  &quot;timestamp&quot;: &quot;2024-01-15T14:30:22.145678Z&quot;,
  &quot;level&quot;: &quot;info&quot;,
  &quot;logger&quot;: &quot;sync_service&quot;,
  &quot;event&quot;: &quot;sync_complete&quot;,
  &quot;student_id&quot;: &quot;a1b2c3d4-...&quot;,
  &quot;repo_id&quot;: &quot;e5f6g7h8-...&quot;,
  &quot;duration_ms&quot;: 4200,
  &quot;commits_synced&quot;: 5
}
```

Additional context fields are added per-module as needed.

Log Levels

Level	When to Use
DEBUG	Detailed git command args/outputs (dev only), step-by-step progress
INFO	Normal operations: sync start/complete, handoff steps, app startup/shutdown
WARNING	Recoverable errors: sync retried, credential validation failed (not blocking)
ERROR	Requires operator attention: sync permanently failed, handoff failed, unhandled exception

In production (default): INFO level. In debug mode (`--debug` flag): DEBUG level.

Module-Specific Log Events

sync_service:
{"event": "sync_start", "student_id": "...", "repo_id": &qu
{"event": "sync_complete", "student_id": "...", "commits_synce
{"event": "sync_diverged", "student_id": "...", "ahead": 3
{"event": "sync_failed", "student_id": "...", "error": &quo
{"event": "sync_all_start", "active_students": 4}
{"event": "sync_all_complete", "succeeded": 3, "failed": 1, "sk

handoff_service:
{"event": "handoff_start", "from_student_id": "...", "to_student
{"event": "handoff_step", "step": 3, "description": "Creating ch
{"event": "handoff_complete", "handoff_id": "...", "tag":
{"event": "handoff_failed", "step": 5, "error": "Git clone failed:
{"event": "handoff_rollback", "handoff_id": "...", "reason&quot

credential_service:
{"event": "credential_stored", "account_id": "...", "secret_kind&
{"event": "credential_rotated", "account_id": "..."}
{"event": "credential_validated", "account_id": "...", "result&qu
{"event": "credential_not_found", "secret_ref": "studentflow:github:alice:a

Note: no `value`, `token`, `password`, or similar fields ever appear — `LogSecretFilter` removes them, and callers should not pass them.

git_client:
{"event": "git_op_start", "op": "push", "repo_path": &quo
{"event": "git_op_complete", "op": "push", "duration_ms&quot
{"event": "git_op_timeout", "op": "fetch", "timeout_s": 60
{"event": "git_op_failed", "op": "merge", "returncode": 1

Git stderr is captured but truncated to 500 characters and checked against the secret pattern before logging.

Sync Logs

Sync job history is queryable via the SyncService:

```
SyncService.get_recent_jobs(
    limit: int = 50,
    student_id: str | None = None,
    status: SyncJobStatus | None = None,
    job_type: SyncJobType | None = None
) -&amp;gt; list[SyncJob]
```

Each `SyncJob` record contains:
- `summary`: human-readable outcome, e.g., "Synced 5 commits from alice-dev" (no secrets)
- `error_message`: if failed, the error (truncated to 500 chars, no secrets)
- `commit_hash_before` / `commit_hash_after`: canonical HEAD before and after the sync
- `started_at` / `finished_at`: timestamps for duration calculation

The Logs screen → Sync Log tab queries this and displays it in a table.

Handoff Logs

Handoff history is queryable via HandoffService:

```
HandoffService.get_handoff_history(
    limit: int = 20,
    from_student_id: str | None = None,
    to_student_id: str | None = None
) -&gt; list[HandoffEvent]
```

Each `HandoffEvent` record stores the full handoff context: from/to students, canonical commit hash, checkpoint tag, status, and notes.

The checkpoint tags in the Git repository provide a parallel, independent audit trail that exists outside the database.

Log Retention

Log Type	Location	Retention
ActivityLog (DB)	`~/.studentflow/studentflow.db`	Forever (append-only, never deleted)
Structlog app.log	`~/.studentflow/logs/app.log`	10MB per file, 5 rotated backups (~50MB total)
SyncJob records (DB)	`~/.studentflow/studentflow.db`	Forever (no auto-cleanup)
HandoffEvent records (DB)	`~/.studentflow/studentflow.db`	Forever
Git checkpoint tags	Canonical repo	Forever (never deleted)

The operator can export ActivityLog and SyncJob records to CSV from the Settings screen for archival, but deletion from the DB is not supported through the UI (append-only guarantee).

---

docs/security.md
Розмір: 9,568 байт

Security

Threat Model

StudentFlow is a **single-operator local desktop tool**. It is not a server, has no network-facing interface, and is not multi-user. The operator is fully trusted.

In-scope threats:

Threat	Description	Mitigated By
Credential leak in DB	Secrets accidentally written to SQLite	DB stores only `secret_ref` key names — never values
Credential leak in logs	Tokens/passwords appearing in log files	LogSecretFilter processor strips secret fields before writing
Credential exposure in process args	Tokens visible in `ps aux` output	git operations use env vars or credential helpers, not command-line args
Credential visible in UI	Plain-text display in screens	All credential fields use `QLineEdit.Password` echo mode

| Audit log tampering | Operator (or code bug) deleting/editing audit records | ActivityLog is append-only at the service layer; no `update()` or `delete()` methods exist |
| Accidental template checksum mismatch | Template modified after import | SHA256 checksum computed and compared before use |

Out-of-scope threats:
- Compromised operator machine (beyond this tool's control)
- Git provider account compromise (external)
- Multi-user access or network exposure (tool is single-operator local)
- Encrypted database at rest (SQLite file is not encrypted; OS-level encryption recommended)

Credential Storage

StudentFlow uses the `keyring` library to store all secrets in the OS-native credential store.

Key Naming Convention

studentflow:{provider}:{username}:{account_id[:8]}

Examples:
studentflow:github:alice:a1b2c3d4
studentflow:gitlab:bob-dev:e5f6g7h8
studentflow:bitbucket:carol:i9j0k1l2

The `account_id[:8]` suffix disambiguates accounts if a username appears more than once (e.g., same username on two providers, or a username reused across semesters).

What Goes in Keyring vs DB

Data	DB (Credential table)	Keyring
Key name (`secret_ref`)	YES — stored as `secret_ref` column	NO
PAT / token value	NEVER	YES
SSH private key contents	NEVER	YES (passphrase only; key file path in DB if needed)
App password	NEVER	YES
Account password	NEVER	YES

Credential Types

SSH keys:
- The private key file path is stored in `Credential.secret_ref` alongside the keyring reference for the passphrase.
- Passphrase (if any) stored in keyring: `studentflow:ssh_passphrase:{account_id[:8]}`
- `GIT_SSH_COMMAND` environment variable used to point git to the correct key file.

PAT / App Password / Password:
- Token/password value stored directly in keyring.
- `secret_ref` is the keyring service name.
- Retrieved at runtime via `keyring.get_password(service, username)`.

Git Authentication

Git operations must never expose credentials in process arguments or URLs.

SSH Authentication

```
env = {
    **os.environ,
    &quot;GIT_SSH_COMMAND&quot;: f&quot;ssh -i {key_file_path} -o StrictHostKeyChecking=no&quot;
}
subprocess.run([&quot;git&quot;, &quot;push&quot;, ...], env=env, timeout=config.sync_timeout_seconds)
```

If the key has a passphrase, use `ssh-agent` or `SSH_ASKPASS` pointing to a helper script that reads from keyring.

HTTPS / PAT Authentication

Never do this:
```
# BAD — token visible in ps aux and git logs
remote_url = f&quot;https://{token}@github.com/alice/project.git&quot;
```

Do this instead — credential helper approach:
```
# Write credentials to a temporary netrc file
netrc_content = f&quot;machine github.com login {username} password {token}\n&quot;
with tempfile.NamedTemporaryFile(mode=&apos;w&apos;, suffix=&apos;.netrc&apos;, delete=False) as f:
    f.write(netrc_content)
    netrc_path = f.name
os.chmod(netrc_path, 0o600)

env = {
    **os.environ,
    &quot;GIT_CONFIG_COUNT&quot;: &quot;1&quot;,
    &quot;GIT_CONFIG_KEY_0&quot;: &quot;credential.helper&quot;,
    &quot;GIT_CONFIG_VALUE_0&quot;: f&quot;store --file {netrc_path}&quot;
}
try:
    subprocess.run([&quot;git&quot;, &quot;push&quot;, ...], env=env, timeout=config.sync_timeout_seconds)
finally:
    os.unlink(netrc_path)  # Always clean up
```

Alternatively, use `GIT_ASKPASS` with a helper script that outputs only the password when called:
```
askpass_script = f&quot;&quot;&quot;#!/bin/sh
echo &quot;{token}&quot;
&quot;&quot;&quot;
# Write to temp file, chmod +x, set GIT_ASKPASS env var
# Clean up after operation
```

Authentication Sequence in GitClient

1. Retrieve credential via `CredentialService.retrieve(secret_ref)` → `value`
2. Based on `auth_type`, prepare the env dict (SSH env or HTTPS credential helper)
3. Run subprocess with env dict and timeout
4. Clean up any temp files in a `finally` block
5. Never store `value` in any class attribute beyond the scope of the git call

---

UI Security

Input Fields

All credential input fields must use Qt's password echo mode:
```
line_edit.setEchoMode(QLineEdit.EchoMode.Password)
```

This applies to:
- New credential input in Add Account dialog
- Rotate Credential dialog
- Any field labeled "Token", "Password", "Key passphrase"

Display Fields (Masked)

When showing credential info in read-only fields, use the `mask_secret()` function:

```
# app/security/masking.py
def mask_secret(value: str) -&gt; str:
    if not value or len(value) &lt; 4:
```

```
        return &quot;••••&quot;
    return &quot;••••••••&quot; + value[-4:]
```

Example: `ghp_abc123xyz789` → `••••••••z789`

Credential status in tables shows: `••••••••{last4}` — never the full value.

Copy-to-Clipboard

If the UI ever provides a "Copy credential" button:
- Copy the value to clipboard
- Start a 30-second QTimer
- On timer expiry: `QApplication.clipboard().clear()`
- Show countdown in UI: "Clipboard clears in 23s"

Log Display

The Logs screen shows `ActivityLog.details` as formatted JSON. Since AuditService filters secrets before write, no secret values should appear. However, the Logs screen should also redact any field whose key matches the secret pattern as a defense-in-depth measure.

Audit Trail

Details Field Filtering

Before any `ActivityLog` record is written, `AuditService.log()` filters the `details` dict:

```
SECRET_KEY_PATTERN = re.compile(
    r&apos;password|token|secret|key|credential|pat|passphrase&apos;,
    re.IGNORECASE
)

def _filter_details(details: dict | None) -&gt; dict | None:
    if not details:
        return details
    return {
        k: v for k, v in details.items()
        if not SECRET_KEY_PATTERN.search(k)
    }
```

Filtered before writing, not after. The stored JSON never contains secret values.

Log Files

Structlog output goes to `~/.studentflow/logs/app.log` in JSON Lines format. Each entry includes:

```
{
  &quot;timestamp&quot;: &quot;2024-01-15T14:30:22.145Z&quot;,
  &quot;level&quot;: &quot;info&quot;,
  &quot;module&quot;: &quot;sync_service&quot;,
  &quot;event&quot;: &quot;sync_complete&quot;,
  &quot;student_id&quot;: &quot;a1b2-...&quot;,
  &quot;repo_id&quot;: &quot;c3d4-...&quot;,
  &quot;commits_synced&quot;: 5,
  &quot;duration_ms&quot;: 4200
}
```

The `LogSecretFilter` structlog processor runs before file output and removes any key matching the secret pattern from the log event dict.

What Is Never in DB or Logs

This list is exhaustive — these items must never appear in any persistent storage:

- SSH private key file contents
- PAT (personal access token) values
- Passwords (Git provider or SSH passphrase)

- OAuth tokens
- App passwords
- Any other authentication credential value

Secret Rotation

When a student's credential expires, is compromised, or needs to be changed:

Operator workflow:
1. Open Credentials screen
2. Find the account whose credential needs rotation
3. Click "Rotate" on the row
4. Enter the new value in the Rotate dialog (masked input field)
5. Click "Validate" — CredentialService tests the new credential against the provider
6. If validation passes: click "Save"
7. StudentFlow calls `CredentialService.rotate(credential_id, new_value)`:

```
def rotate(self, credential_id: str, new_value: str) -&gt; None:
    credential = session.get(Credential, credential_id)
    # Write new value to keyring (overwrites old value at same key)
    keyring.set_password(
        service=&quot;studentflow&quot;,
        username=credential.secret_ref,
        password=new_value
    )
    # Update last_validated_at in DB
    credential.last_validated_at = datetime.utcnow()
    credential.updated_at = datetime.utcnow()
    session.commit()
    # Log rotation (no secret value in log)
    audit.log(&apos;operator&apos;, &apos;credential&apos;, credential_id, &apos;rotate&apos;, {
        &apos;account_id&apos;: credential.account_id,
        &apos;secret_kind&apos;: credential.secret_kind.value
    })
```

8. The old value is immediately overwritten in keyring. There is no "old value" retained.
9. Account status is reset to `unvalidated` until the next explicit validation.

Rotation invariant: The `secret_ref` key name does not change during rotation. Only the value stored at that key in the keyring changes. This means no DB migration is needed for credential rotation.

What Is Never Stored in DB or Logs

Summary for clarity:

Item	DB	Logs	Keyring
secret_ref (key name)	YES (Credential.secret_ref)	NO	NO
SSH private key contents	NEVER	NEVER	YES (passphrase only)
PAT / token value	NEVER	NEVER	YES
Password	NEVER	NEVER	YES
Passphrase	NEVER	NEVER	YES
Git remote URLs (no tokens)	YES (Repository.remote_url)	YES	NO
Account username	YES (Account.username)	YES	part of key name
Provider name	YES (Account.provider)	YES	part of key name

---
docs/handoff-protocol.md
Розмір: 10,925 байт

Handoff Protocol

A handoff is the transition of the active project from one student to the next. It is the most consequential operation in StudentFlow — it changes which student is "live", creates a permanent checkpoint in the canonical repo, and updates the next student's working environment.

Handoffs must be atomic at the database level: either all state changes commit together, or none do. Git operations (tagging, push) cannot be rolled back, but their effects are idempotent and safe to leave in place if the DB transaction fails.

Pre-Conditions for Handoff

The following must all be true before a handoff can proceed. `HandoffService.validate_handoff_preconditions(current, next)` checks each and returns a list of error strings. If the list is non-empty, the handoff is blocked.

Pre-condition 1: Current student repo is in a safe state
- `current_student.accounts[current].repository.sync_status` must be `clean` OR
- `current_student.status` must be `exhausted` or `error` (allowing handoff to skip the final sync)
- Error if violated: `"Current student repo is not clean (status: {sync_status}). Run sync first or use force-overwrite."`

Pre-condition 2: Canonical repo is accessible
- The path in `AppSetting.canonical_repo_path` must exist on disk
- It must be a valid git repository (`git -C {path} rev-parse --git-dir` exits 0)
- Error if violated: `"Canonical repo not found at {path}. Check Settings > canonical_repo_path."`

Pre-condition 3: Next student exists in queue
- There must be a student with `queue_position > current_student.queue_position` and `status` in `(active, paused)`
- Error if violated: `"No next student found in queue. Add or activate another student first."`

Pre-condition 4: Next student has a configured Account
- Next student must have at least one Account record
- Error if violated: `"Next student {name} has no account configured. Add an account first."`

Pre-condition 5: Next student has a Credential
- Next student's account must have an associated Credential record
- `CredentialService.validate_exists(credential.secret_ref)` must return True
- Error if violated: `"Next student {name} has no valid credential. Add and validate credentials first."`

Pre-condition 6: Next student has a remote URL
- `Repository.remote_url` must be set (or operator must confirm bootstrap-only mode)
- Error if violated: `"Next student {name} has no repository remote URL configured."`

Handoff Steps (Detailed)

Step 1: VALIDATE

```
errors = validate_handoff_preconditions(current_student, next_student)
if errors:
    raise HandoffValidationError(errors)
```

All 6 pre-conditions are checked. If any fail, the handoff does not proceed. The UI shows the list of errors to the operator.

On failure: HandoffEvent is NOT created. No state changes. Operator fixes the issues and retries.

Step 2: SYNC (Final sync of current student)

```
if current_student.status == StudentStatus.active:
    sync_job = SyncService.sync_student(current_student.id)
    if sync_job.status == SyncJobStatus.failed:
        raise HandoffSyncError(f&quot;Final sync failed: {sync_job.error_message}&quot;)
# If status is exhausted or error: skip sync
```

On failure: `HandoffSyncError` is raised. No HandoffEvent created. Current student status unchanged. Operator must fix sync error (credential issue, diverged repo) before retrying.

Step 3: TAG — Create checkpoint on canonical

```
tag_name = f&quot;handoff/{datetime.utcnow().strftime(&apos;%Y-%m-%d-%H%M%S&apos;)}/from-{from_usern
commit_hash = GitClient.get_commit_hash(canonical_path)
GitClient.create_tag(canonical_path, tag_name)
```

The tag is created on canonical's current HEAD. This is the point of no return — tags cannot be rolled back cleanly (they remain even if the rest of the handoff fails).

Tag format: `handoff/YYYY-MM-DD-HHMMSS/from-{from_username}-to-{to_username}`
For first bootstrap (no prior student):
`handoff/YYYY-MM-DD-HHMMSS/bootstrap-to-{to_username}`

On failure (tag creation fails): `HandoffTagError` raised. No HandoffEvent created. No student status changes. The tag may or may not exist on disk — check canonical before retrying.

Step 4: RECORD — Create HandoffEvent

```
# Begin DB transaction
handoff_event = HandoffEvent(
    from_student_id=current_student.id,  # None if first bootstrap
    to_student_id=next_student.id,
    repo_id=canonical_repo.id,
    commit_hash=commit_hash,
    checkpoint_tag=tag_name,
    status=HandoffStatus.pending,
    created_at=datetime.utcnow()
)
session.add(handoff_event)
session.flush()  # get ID, but do NOT commit yet
```

The event is created with `status=pending`. The DB transaction remains open until Step 9.

On failure: Transaction rolled back. HandoffEvent not persisted. Tag on canonical remains (harmless — it points to a valid commit).

Step 5: BOOTSTRAP NEXT — Update next student's local repo

```
if next_repo.sync_status == SyncStatus.uninitialized:
    # Full clone from canonical
    GitClient.clone(canonical_path, next_repo.local_path, credential=None)
    GitClient.add_remote(next_repo.local_path, &apos;student&apos;, next_repo.remote_url)
else:
    # Pull canonical updates into existing clone
    GitClient.fetch(next_repo.local_path, &apos;origin&apos;)
    GitClient.merge_ff_only(next_repo.local_path, &apos;origin/main&apos;)
```

On failure:

```
handoff_event.status = HandoffStatus.failed
session.commit()  # Commit the failure status
raise HandoffBootstrapError(f&quot;Failed to bootstrap {next_student.full_name}: {error}&quot;)
```

Current student's status is NOT changed. Operator must fix (check disk space, local path, canonical accessibility) and retry from Step 5.

Step 6: PUSH — Push to next student's remote

```
next_credential = CredentialService.retrieve(next_account.credential.secret_ref)
GitClient.push(
    repo_path=next_repo.local_path,
    remote_name=&apos;student&apos;,
    branch=next_repo.default_branch,
    credential=next_credential
)
next_repo.last_commit_hash = commit_hash
next_repo.sync_status = SyncStatus.clean
session.flush()
```

On failure:

```
handoff_event.status = HandoffStatus.failed
session.commit()
raise HandoffPushError(f&quot;Failed to push to {next_student.full_name}&apos;s remote: {error}&quot;)
```

Next student's local repo is correct (Step 5 succeeded); only remote push failed. Operator can re-push manually or fix credential and retry from Step 6.

Step 7: ACTIVATE NEXT — Set next student as current

```
# In same open transaction
next_account.is_current = True
next_student.status = StudentStatus.active
session.flush()
```

Step 8: DEACTIVATE CURRENT — Mark current student as exhausted

```
# In same open transaction
current_account.is_current = False
current_student.status = StudentStatus.exhausted
session.flush()
```

The current student's `queue_position` is retained for historical reference (not set to NULL).

Step 9: COMPLETE — Commit everything and log

```
# Complete the handoff event
handoff_event.status = HandoffStatus.completed
handoff_event.completed_at = datetime.utcnow()

# Commit the entire transaction (Steps 4, 7, 8, 9 all commit atomically)
session.commit()

# Write to audit log AFTER commit (so if audit write fails, handoff is still recorded)
AuditService.log(
    actor=&apos;operator&apos;,
    entity_type=EntityType.handoff,
    entity_id=handoff_event.id,
    action=&apos;complete&apos;,
    details={
        &apos;from_student&apos;: current_student.full_name,
        &apos;to_student&apos;: next_student.full_name,
```

```
        &apos;commit_hash&apos;: commit_hash,
        &apos;tag&apos;: tag_name
    }
)
```

On commit failure: The transaction rolls back. HandoffEvent may be in an inconsistent state in memory. DB is unchanged. Git operations (tag, push) have already happened and cannot be reversed — but they are idempotent (the tag exists, next student's remote is updated). Operator can retry — the DB will create a new HandoffEvent on the next attempt.

Rollback Procedure

If any of Steps 5–9 fail, the rollback procedure is:

```
def rollback_handoff(handoff_event: HandoffEvent) -&gt; None:
    with session.begin():
        # 1. Set handoff event to failed
        handoff_event.status = HandoffStatus.failed

        # 2. Restore current student to active (if we changed it)
        if current_student.status == StudentStatus.exhausted:
            current_student.status = StudentStatus.active

        # 3. Deactivate next student (if we set is_current)
        if next_account.is_current:
            next_account.is_current = False
            next_student.status = StudentStatus.paused  # Not active — partial handoff

        # Note: Git tag is NOT removed — it points to a valid commit and is safe
        # Note: next student&apos;s local repo and remote may have been updated — that is fine

    AuditService.log(&apos;operator&apos;, EntityType.handoff, handoff_event.id, &apos;failed&apos;, {
        &apos;step&apos;: failed_step_number,
        &apos;error&apos;: str(error)
    })
```

What rollback does NOT undo:
- The Git checkpoint tag (permanent by design)
- Any push to next student's remote (idempotent — canonical HEAD is valid)
- Any writes to AuditService (append-only)

What rollback DOES restore:
- Current student status back to `active`
- `is_current` flag restored to current student
- HandoffEvent recorded as `failed`

Retry After Rollback

After a failed handoff, the operator can retry. The second attempt will:
1. Find a new `next_student` (same student, since queue hasn't changed)
2. Create a new HandoffEvent (the failed one remains in DB as `status=failed`)
3. Create a new checkpoint tag (new timestamp, so no collision)
4. Proceed from Step 1

The failed HandoffEvent and its associated tag are permanent audit artifacts. They show the operator attempted a handoff that failed, at what commit, at what time.

Tag Naming

Full format: `handoff/YYYY-MM-DD-HHMMSS/from-{from_username}-to-{to_username}`

Components:
- `YYYY-MM-DD-HHMMSS`: UTC timestamp at handoff initiation (Step 3), e.g., `2024-01-15-143022`
- `from_username`: `Account.username` of current student, e.g., `alice-dev`
- `to_username`: `Account.username` of next student, e.g., `bob-dev`

First bootstrap (no from student):
`handoff/YYYY-MM-DD-HHMMSS/bootstrap-to-{to_username}`

Examples:
handoff/2024-01-15-143022/from-alice-dev-to-bob-dev
handoff/2024-02-03-091545/from-bob-dev-to-csmith
handoff/2024-03-10-160032/bootstrap-to-alice-dev (first ever)
handoff/2024-04-22-083311/from-csmith-to-dkim99

Stored in: `HandoffEvent.checkpoint_tag` (full tag name) and in `HandoffEvent.commit_hash` (the SHA the tag points to).

Invariant: Tags are never deleted. They provide a permanent, external (Git-native) audit trail that exists independently of the StudentFlow database.
---
docs/data-model.md
Розмір: 15,258 байт

Data Model

All entities use UUID string primary keys (`default=lambda: str(uuid.uuid4())`). The database is SQLite at `~/.studentflow/studentflow.db`. Migrations are managed by Alembic.

Student

The primary entity. Represents one student in the workflow queue.

```
id:             UUID PK        — generated on create
full_name:      str NOT NULL   — display name, e.g. &quot;Alice Johnson&quot;
email:          str NULLABLE   — optional contact email
status:         enum NOT NULL DEFAULT active
                active       — currently working or ready to work
                paused       — temporarily suspended (can resume)
                exhausted    — has completed their turn; prior handoff target
                archived     — removed from active roster; kept for history
                error        — manual intervention required
queue_position: int NULLABLE   — NULL means not in queue; lower = earlier
priority:       int NOT NULL DEFAULT 0  — tiebreaker when positions equal
notes:          text NULLABLE  — operator notes
student_number: int NULLABLE UNIQUE — sequential number assigned at creation or migration; used in prompt fil
created_at:     datetime NOT NULL
updated_at:     datetime NOT NULL
```

Relations:
- `accounts: list[Account]` — one student has many accounts (via back_populates)
- `sync_jobs: list[SyncJob]` — history of sync operations for this student
- `handoffs_from: list[HandoffEvent]` — handoffs where this student was the source
- `handoffs_to: list[HandoffEvent]` — handoffs where this student was the target

Indexes:
- `idx_student_status` on `status`
- `idx_student_queue_position` on `queue_position`
- `idx_student_number` on `student_number`

Account

A Git provider account belonging to a student. A student may have multiple accounts (e.g., one GitHub, one GitLab) but at most one is `is_current=true` at any time.

```
id:               UUID PK
student_id:       UUID FK → Student NOT NULL
provider:         enum NOT NULL
                  github / gitlab / bitbucket / gitea / other
username:         str NOT NULL   — provider username, e.g. &quot;alice-dev&quot;
auth_type:        enum NOT NULL
                  ssh / pat / app_password / password
status:           enum NOT NULL DEFAULT unvalidated
                  active / inactive / error / unvalidated
last_validated_at: datetime NULLABLE  — when CredentialService last confirmed access
is_current:       bool NOT NULL DEFAULT false  — is this the active account for handoffs
created_at:       datetime NOT NULL
updated_at:       datetime NOT NULL
```

Relations:
- `student: Student` — parent student
- `repositories: list[Repository]` — repos under this account
- `credential: Credential` — one-to-one (one credential per account)

Indexes:
- `idx_account_student_id` on `student_id`
- `idx_account_is_current` on `is_current`
- `idx_account_status` on `status`

Repository

A Git repository associated with a student account. Also used for the canonical repo record (`is_canonical=true`).

```
id:                 UUID PK
account_id:         UUID FK → Account NOT NULL
repo_name:          str NOT NULL   — human label, e.g. &quot;alice-project-repo&quot;
remote_url:         str NOT NULL   — full remote URL, e.g. &quot;https://github.com/alice/project.git&quot;
local_path:         str NOT NULL   — absolute path on operator&apos;s disk
default_branch:     str NOT NULL DEFAULT main
integration_branch: str NOT NULL DEFAULT main  — branch pushed to canonical
sync_status:        enum NOT NULL DEFAULT uninitialized
                    clean        — local matches canonical HEAD
                    dirty        — local has uncommitted or unsynced changes
                    diverged     — both local and remote have commits the other lacks
                    error        — last sync operation failed
                    uninitialized — repo not yet cloned/bootstrapped
last_commit_hash:   str NULLABLE   — 40-char SHA1 of last known HEAD
template_source_id: UUID FK → TemplateSource NULLABLE  — which template initialized this repo
is_canonical:       bool NOT NULL DEFAULT false  — true for the one canonical repo record
created_at:         datetime NOT NULL
updated_at:         datetime NOT NULL
```

Relations:
- `account: Account` — parent account
- `template_source: TemplateSource | None` — origin template
- `sync_jobs: list[SyncJob]` — sync history for this repo
- `handoff_events: list[HandoffEvent]` — handoffs recorded on this repo

Indexes:
- `idx_repository_account_id` on `account_id`
- `idx_repository_sync_status` on `sync_status`
- `idx_repository_is_canonical` on `is_canonical`

Credential

Holds the keyring reference for an account's secret. One-to-one with Account. The actual secret value lives only in the OS keyring — never in this table.

```
id:               UUID PK
account_id:       UUID FK → Account NOT NULL UNIQUE  — one credential per account
secret_ref:       str NOT NULL  — keyring key name, e.g. &quot;studentflow:github:alice:a1b2c3d4&quot;
secret_kind:      enum NOT NULL
                  ssh_key / pat / app_password / password
is_encrypted:     bool NOT NULL DEFAULT true  — always true; keyring manages encryption
last_validated_at: datetime NULLABLE  — when the stored secret was last confirmed valid
created_at:       datetime NOT NULL
updated_at:       datetime NOT NULL
```

CRITICAL: There is no `value`, `token`, `password`, or `secret` column in this table. The only secret-related field is `secret_ref`, which is a key name (not a value) used to look up the actual secret in the OS keyring.

Relations:
- `account: Account` — parent account (one-to-one)

SyncJob

A record of one sync operation (fetch → merge → push). Created for every sync_student() call and every sync_all() iteration.

```
id:               UUID PK
repo_id:          UUID FK → Repository NOT NULL
student_id:       UUID FK → Student NOT NULL
job_type:         enum NOT NULL
                  sync_current    — normal per-student sync
                  sync_all        — batch sync-all iteration
                  bootstrap       — initial repo setup
                  handoff_push    — push to canonical during handoff
                  handoff_pull    — pull to next student during handoff
status:           enum NOT NULL DEFAULT pending
                  pending / running / success / failed / skipped
started_at:       datetime NULLABLE  — set when status transitions to running
finished_at:      datetime NULLABLE  — set when status transitions to terminal
summary:          text NULLABLE  — human-readable result, e.g. &quot;Synced 5 commits&quot;
error_message:    text NULLABLE  — error details (truncated to 500 chars, no secrets)
commit_hash_before:  str NULLABLE   — canonical HEAD before the sync
commit_hash_after:   str NULLABLE   — canonical HEAD after the sync
created_at:       datetime NOT NULL
```

Note: No `updated_at` — once a job reaches a terminal status (success/failed/skipped), it is not modified.

Relations:
- `repository: Repository` — the repo that was synced
- `student: Student` — the student whose work was synced

Indexes:
- `idx_syncjob_repo_id` on `repo_id`
- `idx_syncjob_student_id` on `student_id`
- `idx_syncjob_status` on `status`
- `idx_syncjob_job_type` on `job_type`

HandoffEvent

A record of one complete handoff from one student to the next. Provides the audit trail of who handed off to whom, at what commit.

```
id:              UUID PK
from_student_id:  UUID FK → Student NULLABLE  — NULL for the first bootstrap
to_student_id:    UUID FK → Student NOT NULL
repo_id:          UUID FK → Repository NOT NULL  — must be the canonical repo
commit_hash:      str NOT NULL  — canonical HEAD at handoff time (40-char SHA1)
checkpoint_tag:   str NOT NULL  — Git tag name, e.g. &quot;handoff/2024-01-15-143022/from-alice-to-bob&quot;
status:           enum NOT NULL DEFAULT pending
                  pending / completed / failed / rolled_back
notes:            text NULLABLE  — operator notes
created_at:       datetime NOT NULL
completed_at:     datetime NULLABLE  — set when status=completed
```

Relations:
- `from_student: Student | None`
- `to_student: Student`
- `repository: Repository`

TemplateSource

A local directory designated as the project template for bootstrapping new student repos.

```
id:          UUID PK
name:        str NOT NULL   — human label, e.g. &quot;Project v2 — Spring 2024&quot;
local_path:  str NOT NULL   — absolute path to the template directory on disk
is_active:   bool NOT NULL DEFAULT true  — whether this template is selectable
checksum:    str NULLABLE   — SHA256 of sorted file contents (recomputed on use)
description: text NULLABLE
created_at:  datetime NOT NULL
updated_at:  datetime NOT NULL
```

Relations:
- `repositories: list[Repository]` — repos bootstrapped from this template

ActivityLog

Append-only audit log. Every significant operator action is recorded here. This table has NO UPDATE and NO DELETE operations — ever.

```
id:          UUID PK       — generated on insert
actor:       str NOT NULL DEFAULT &apos;operator&apos;  — who performed the action
```

entity_type: enum NOT NULL
student / account / repository / credential / sync_job /
handoff / template / system
entity_id: str NULLABLE — UUID of the affected entity (stringified)
action: str NOT NULL — e.g. "create", "archive", "sync", "handoff&quot
details: JSON NULLABLE — structured context; secrets filtered before write
created_at: datetime NOT NULL

CRITICAL INVARIANTS:
1. No `updated_at` column — this record never changes after insert
2. AuditService filters the `details` dict before writing: any key matching the pattern `password|token|secret|key|credential|pat|passphrase` (case-insensitive) is removed before the JSON is stored
3. The only operation permitted on this table is INSERT (read is also permitted for query)

Indexes:
- `idx_activitylog_entity_type` on `entity_type`
- `idx_activitylog_entity_id` on `entity_id`
- `idx_activitylog_created_at` on `created_at`

Example log entries:

actor	entity_type	action	details
operator	student	create	{"student_id": "...", "full_name": "Alice"}
operator	student	archive	{"student_id": "...", "reason": "completed"}
operator	account	validate	{"account_id": "...", "provider": "github", "result": "success"}
operator	credential	rotate	{"account_id": "...", "secret_kind": "pat"}
operator	repository	sync	{"repo_id": "...", "commit_before": "abc123", "commit_after": "def456", "duration_ms": 4200}
operator	handoff	complete	{"handoff_id": "...", "from": "alice", "to": "bob", "tag": "handoff/..."}
operator	handoff	failed	{"handoff_id": "...", "step": 5, "error": "Git push failed"}
operator	repository	sync_failed	{"repo_id": "...", "error_summary": "fetch timeout after 60s"}

AppSetting

Key-value store for operator-configurable settings. Loaded at startup and written through the Settings screen.

key: str PK — setting identifier, e.g. "canonical_repo_path"
value: text NOT NULL — string representation of the value
description: text NULLABLE — human-readable explanation shown in Settings UI
updated_at: datetime NOT NULL

Default keys and values:

key	default value	description
canonical_repo_path	~/.studentflow/canonical	Absolute path to the canonical bare repo
default_branch	main	Default git branch name for new repos
template_dir	~/studentflow-templates	Default directory to search for templates
sync_timeout_seconds	60	Seconds before a git operation is killed
max_sync_retries	2	How many times to retry a failed sync
enable_auto_sync	false	Whether SchedulerService runs periodic sync
auto_sync_interval_minutes	30	Interval for auto sync (if enabled)

QueueState (Derived View — Not a Table)

The queue is not stored as a separate table. It is a derived view computed at query time:

```
SELECT *
FROM student
WHERE status IN (&apos;active&apos;, &apos;paused&apos;)
  AND queue_position IS NOT NULL
ORDER BY queue_position ASC
```

The `is_current` flag on Account identifies the one student currently active. `queue_position` is an explicit integer managed by AccountService — not computed. When positions are reordered (move_up / move_down), AccountService updates the integers directly.

Queue management rules:
- Positions are contiguous integers starting at 1 (gaps are closed after archive/remove)
- NULL queue_position = not in queue (archived or error students)
- Two students may temporarily share a position during a swap (resolved within the same transaction)
- `is_current` on Account overrides queue for display: the Dashboard shows the is_current student as "Active"

Prompt

A single prompt delivery record. Tracks the lifecycle of a Markdown file written to a student's working repository.

```
id:             UUID PK
student_id:     UUID FK → Student NOT NULL
repository_id:  UUID FK → Repository NOT NULL   — student&apos;s working repo (is_canonical=false)
student_number: int NOT NULL    — denormalized from Student for file naming
seq_number:     int NOT NULL    — sequential per student; first prompt = 1
title:          str NOT NULL    — display title; empty string if untitled
slug:           str NOT NULL    — URL-safe slug derived from title; used in filename
content:        text NOT NULL   — full prompt text
file_path:      str NOT NULL    — relative path, e.g. &quot;prompts/student01-001-auth-flow-fix.md&quot;
git_branch:     str NOT NULL    — branch at time of creation, usually &quot;main&quot;
git_commit_hash: str(40) NULLABLE — commit SHA once committed
status:         enum NOT NULL
                draft       — record created, file not yet written
                written     — file written to disk, not yet staged
                committed   — staged and committed locally
                pushed      — successfully pushed to student&apos;s remote
                failed      — any step failed; push_error contains sanitized message
push_error:     text NULLABLE   — sanitized error message (tokens stripped); max 280 chars
created_by:     str NOT NULL DEFAULT &quot;operator&quot;
created_at:     datetime NOT NULL
pushed_at:      datetime NULLABLE — timestamp when push succeeded
```

Relations:
- `student: Student` — the student this prompt is for
- `repository: Repository` — the working repository the file was pushed to

Indexes:
- `idx_prompt_student_id` on `student_id`
- `idx_prompt_status` on `status`
- `idx_prompt_student_seq` on `(student_id, seq_number)` — unique per student

Push remote: Always `"student"` (the student's GitHub/GitLab remote), never `"origin"` (which points to canonical).

---

docs/troubleshooting.md
Розмір: 13,277 байт

Troubleshooting

This guide covers common problems encountered while operating StudentFlow and their recovery procedures.

Sync Status Shows "Diverged"

Symptom: A repository row in the Repositories screen shows a red `■ diverged` badge. Syncing fails with a message like "Sync failed: both repos have divergent commits (ahead=3, behind=2)".

Cause: The student's remote has commits that the local clone does not have, AND the local clone has commits that the student's remote does not have. This is a genuine divergence that cannot be auto-merged.

Diagnosis:
```
# In a terminal:
cd /path/to/student/local/repo
git log --oneline HEAD..student/main   # commits on student remote not in local
git log --oneline student/main..HEAD   # commits in local not on student remote
git log --oneline -10                  # recent local history
```

Resolution — Option 1: Attempt a regular merge (least destructive)
```
cd /path/to/student/local/repo
git fetch student
git merge student/main   # Let git attempt a merge commit
# If there are conflicts: resolve them manually, then:
git add .
git commit -m &quot;Resolve divergence: merge student work&quot;
git push origin main   # Push to canonical
```

Then refresh the Repositories screen — status should return to `clean`.

Resolution — Option 2: Force-overwrite with canonical (discards student divergence)
Use only if the divergent student commits should be abandoned (e.g., student made experimental changes that shouldn't be integrated):

1. In the Repositories screen, click `[Sync Now]` dropdown → "Force Overwrite from Canonical"
2. Confirm the destructive action
3. This is logged in the audit trail as `force_overwrite`

Resolution — Option 3: Restore from canonical manually
```
cd /path/to/student/local/repo
git fetch origin
git reset --hard origin/main
git push student main --force
```

Then run a normal sync from the Repositories screen to confirm status=clean.

Credential Validation Fails

Symptom: Clicking `[Validate]` on an account shows "✗ Validation failed" or the account status badge is red.

Cause: The stored credential is expired, revoked, incorrect, or the provider is unreachable.

Step 1: Check provider status
- GitHub: https://www.githubstatus.com
- GitLab: https://status.gitlab.com
- Bitbucket: https://bitbucket.status.atlassian.com

If the provider has an active incident, wait and retry later.

Step 2: Test the credential manually
For a PAT:
GitHub PAT test:
curl -H "Authorization: token YOUR_PAT" https://api.github.com/user

GitLab PAT test:
curl -H "PRIVATE-TOKEN: YOUR_PAT" https://gitlab.com/api/v4/user

If this returns a 401 or 403, the token is invalid or expired.

Step 3: Generate a new credential
1. Log into the student's Git provider account
2. Go to Developer Settings → Personal Access Tokens
3. Generate a new token with appropriate scopes (repo read+write)
4. Copy the new token

Step 4: Rotate the credential in StudentFlow
1. Accounts screen → `[Rotate Credential]` for the affected account
2. Enter the new token in the masked input
3. Click `[Validate First]` to confirm it works
4. Click `[Save]`

Handoff Failed at Step N

When a handoff fails, the error message specifies which step failed. Use the corresponding recovery below.

Failed at Step 1 (Validate)

The pre-condition check found an issue. The error message lists the specific problems (e.g., "repo not clean", "next student has no credential"). Fix each listed issue and retry the handoff.

Failed at Step 2 (Final Sync)

The final sync of the current student failed before the tag was created. No permanent state changes occurred.

1. Navigate to Repositories screen → find the current student's repo
2. Apply the appropriate sync error recovery (see "Sync Status Shows Diverged" or credential rotation above)
3. Retry the handoff from the Dashboard

Failed at Step 3 (Create Tag)

Tag creation on the canonical repo failed. This is unusual — most likely a filesystem permissions issue.

Check permissions on canonical:
ls -la ~/.studentflow/canonical/
Check you can write to it:
git -C ~/.studentflow/canonical tag test-tag && git -C ~/.studentflow/canonical tag -d test-tag

If permissions are wrong:
chmod -R u+w ~/.studentflow/canonical/

Retry the handoff.

Failed at Step 4 (Create HandoffEvent in DB)

The database write failed. Most likely causes: DB locked, disk full, or corruption.

Check disk space:
df -h ~/.studentflow/

Check if DB is locked (another instance running):
ps aux | grep "python.*app.main"
Verify DB integrity:

sqlite3 ~/.studentflow/studentflow.db "PRAGMA integrity_check;"

Fix the underlying issue, then retry the handoff.

Failed at Step 5 (Bootstrap Next Student)

The git clone or pull of the canonical repo to the next student's local path failed.

1. Check that the local path directory is accessible and writable:
   mkdir -p /path/to/next/student/repo
   ls -la /path/to/next/student/

2. Check disk space: `df -h /path/to/next/student/`
3. Verify canonical is accessible: `git -C ~/.studentflow/canonical status`
4. Navigate to Repositories screen → find next student's repo → click `[Init from Template]` or fix the path
5. Retry the handoff — the HandoffEvent exists in DB with status=failed; a new one will be created on retry

Failed at Step 6 (Push to Next Student's Remote)

The push to the next student's Git remote failed. The local repo was bootstrapped successfully.

1. Go to Accounts screen → find next student's account → click `[Validate]`
2. If credential is invalid: rotate it (see credential rotation procedure above)
3. If network issue: test connectivity to the provider
4. Once fixed, you can push manually:
   cd /path/to/next/student/local/repo
   git push student main

Then in StudentFlow: Repositories screen → `[Validate Remote]` → confirm status
5. Retry the handoff — since the local repo and canonical are already in sync, the retry will re-attempt from validation and proceed quickly

Failed at Steps 7–9 (Activate/Deactivate/Complete)

Database transaction failure during the final state update. The Git work is done — only DB state is inconsistent.

Manual DB recovery:
1. Close StudentFlow
2. Open the database:
   sqlite3 ~/.studentflow/studentflow.db

3. Check current state:
   SELECT id, full_name, status FROM student ORDER BY queue_position;
   SELECT id, account_id, is_current FROM account;
   SELECT id, status, from_student_id, to_student_id FROM handoff_event ORDER BY created_at DESC LIMIT 3;

4. Manually update the stuck state:
   -- Mark handoff as completed (adjust IDs as needed)
   UPDATE handoff_event SET status='completed', completed_at=datetime('now') WHERE

   -- Activate next student
   UPDATE student SET status='active' WHERE id='<next_student_id>';
   UPDATE account SET is_current=1 WHERE student_id='<next_student_id>';

   -- Deactivate current student
   UPDATE student SET status='exhausted' WHERE id='<current_student_id>';
   UPDATE account SET is_current=0 WHERE student_id='<current_student_id>';

5. Close sqlite3 and restart StudentFlow
6. Verify Dashboard shows the correct active student

Keyring Not Available

Symptom: Application crashes on startup or credential operations fail with `keyring.errors.NoKeyringError` or `RuntimeError: No keyring backend found`.

Linux:
Check if SecretService is running:
ps aux | grep gnome-keyring
ps aux | grep kwallet

# Install SecretService support:
sudo apt-get install gnome-keyring python3-secretstorage
# Or for Python package:
pip install secretstorage

Start keyring daemon:
eval $(gnome-keyring-daemon --start --components=secrets)
export SSH_AUTH_SOCK

Install fallback backend for headless Linux:
pip install keyrings.alt
export PYTHON_KEYRING_BACKEND=keyrings.alt.file.EncryptedKeyring

macOS:
If the macOS Keychain is unavailable (rare):
pip uninstall keyring && pip install keyring
Verify the backend:
python3 -c "import keyring; print(keyring.get_keyring())"

Windows:
pip uninstall keyring && pip install keyring
The WinVaultKeyring should be selected automatically
python3 -c "import keyring; print(keyring.get_keyring())"

Verify the keyring is functional:
python3 -c "
import keyring
keyring.set_password('studentflow-test', 'test-user', 'test-value')
result = keyring.get_password('studentflow-test', 'test-user')
print('OK' if result == 'test-value' else 'FAIL')
keyring.delete_password('studentflow-test', 'test-user')
"

Git Operation Timeout

Symptom: Sync fails with "Git operation timed out after 60 seconds". SyncJob.error_message contains "timeout".

Cause: Network latency to the Git provider, large repo, or slow disk.

Resolution:

1. **Increase the timeout in Settings:**
- Navigate to Settings screen
- Change `sync_timeout_seconds` from 60 to 120 (or higher for large repos)
- Click `[Save Settings]`
- Retry the sync

2. **Test network speed to provider:**
   # Measure git fetch time:
   time git -C /path/to/repo fetch student

3. **Check for large files in the repo:**

```
git -C /path/to/repo rev-list --objects --all | sort -k2 | head -20
```

If there are large binary files, consider adding them to `.gitignore` in the template.

4. **SSH vs HTTPS:** SSH is often faster for large pushes. If using HTTPS/PAT, consider switching to SSH authentication.

Database Locked

Symptom: Application shows an error like "sqlite3.OperationalError: database is locked" or "database is locked — another process may be using it".

Cause: Only one instance of StudentFlow should run at a time. A second instance, a zombie process from a crash, or an external tool with the DB file open will cause this.

Resolution:
1. Check for running StudentFlow processes:
```
ps aux | grep &quot;python.*app.main&quot;
```

2. If a zombie process exists, kill it:
```
kill &lt;PID&gt;
```

3. Check for any SQLite browser tool that might have the file open (DB Browser for SQLite, etc.)
4. If no other processes found, the lock may be a stale lock file:
```
ls ~/.studentflow/*.db-shm ~/.studentflow/*.db-wal 2&gt;/dev/null
# If these exist and no process is using the DB, it&apos;s safe to remove them:
rm -f ~/.studentflow/studentflow.db-shm ~/.studentflow/studentflow.db-wal
```

5. Restart StudentFlow

Student Stuck in Error Status

Symptom: A student shows `status=error` and cannot be synced or handed off. The error state was set during a failed operation and was not automatically resolved.

Understanding the error state: The `error` status means the last operation failed and left the student in an indeterminate state. The operator must manually inspect and reset.

Step 1: Check the audit log for this student
1. Navigate to Logs → Audit Log tab
2. Filter by entity type: Student, entity ID: (student's UUID or name search)
3. Read the most recent entries to understand what failed

Step 2: Check the repo status
1. Navigate to Repositories screen
2. Find the student's repo — note the sync_status (error, diverged, etc.)
3. Apply the appropriate repo recovery procedure above

Step 3: Manual status reset

Once the underlying issue is fixed, reset the student's status via the Edit dialog:
1. Students screen → click `[Edit]` on the student
2. Change Status from `error` to `active` (or `paused` if you want to hold them)
3. Click `[Save]`
4. Attempt a sync to confirm recovery

Step 4: If Edit dialog won't save

If the error state is preventing edits through the UI, use the database directly:
```
sqlite3 ~/.studentflow/studentflow.db

SELECT id, full_name, status FROM student WHERE status=&apos;error&apos;;
UPDATE student SET status=&apos;paused&apos;, updated_at=datetime(&apos;now&apos;) WHERE id=&apos;&lt
```

Then run a sync from the Repositories screen to confirm the student's repo is accessible.

Application Won't Start

Symptom: Running `python -m app.main` exits immediately with an error, or the window appears briefly and crashes.

Step 1: Check for Python version:
python --version   # Must be 3.11+
which python   # Confirm you're in the venv

Step 2: Check for missing dependencies:
pip check
pip install -r requirements.txt

Step 3: Check the log file:
cat ~/.studentflow/logs/app.log | tail -50

Step 4: Check Qt display environment (Linux):
PySide6 requires a display
echo $DISPLAY   # Should be set (e.g., :0)
If empty and running headless, set a virtual display:
export DISPLAY=:99
Xvfb :99 -screen 0 1024x768x24 &
python -m app.main

Step 5: Check DB integrity:
sqlite3 ~/.studentflow/studentflow.db "PRAGMA integrity_check;"
Should return: ok

If integrity check fails:
Backup the corrupt DB
cp ~/.studentflow/studentflow.db ~/.studentflow/studentflow.db.bak
Reinitialize (WARNING: loses all data)
python -m app.bootstrap init-db

---
docs/operator-runbook.md
Розмір: 18,036 байт

Operator Runbook

This runbook covers the day-to-day operations of StudentFlow. It assumes the application is already installed and configured (see `docs/setup.md`).

> **v0.1.8 accuracy note:** The Dashboard shows the Active Student (with `[Sync Now]` and `[Hand Off to Next Student]` action buttons), Student Queue, Canonical Repository status, and Recent Sync Activity. When `canonical_remote_url` is configured in Settings, every successful sync also pushes the local canonical repo to that remote. `[Sync All]` and the System Health drill-down are Phase 2. The Settings CSV export is a placeholder.

Canonical Remote Push

When `canonical_remote_url` is configured in **Settings → Canonical remote URL (optional)**, StudentFlow will push the local canonical bare repo to that remote at the end of every successful sync.

How it fits into the workflow

```
active student repo  →  (fetch/merge/push)  →  local canonical bare repo
                                                ↓ (if canonical_remote_url set)
                                              canonical remote on GitHub/GitLab
```

The local canonical bare repo remains the operational integration repository for all student syncs. The canonical remote push is an additional publish step that makes the canonical state available externally (e.g., for Cloudflare Pages auto-deploy).

What "success" means

Outcome	What happened	What operator sees
Full success	Local canonical updated + remote pushed	Sync result: "Synced N commits…"
Partial success	Local canonical updated; remote push failed	Sync result: "Synced N commits…; canonical remote push failed: [reason]"
Remote not configured	Local canonical updated only (expected)	No mention of remote push

Authentication for canonical remote

StudentFlow does **not** manage credentials for the canonical remote. The operator must configure system-level git authentication before the canonical remote push will succeed:

- **SSH:** Ensure `ssh-agent` is running and the key for the canonical remote is loaded
- **HTTPS/PAT:** Add a `~/.netrc` entry or configure the OS git credential manager:
  machine github.com
  login git
  password <your-PAT>

- Verify manually: `git -C ~/.studentflow/canonical push https://github.com/org/repo.git main`

Checkpoint tags (MVP limitation)

Handoff checkpoint tags created during a handoff (`handoff/…/from-…-to-…`) are pushed to the canonical remote only if a subsequent sync occurs after the handoff. To push tags explicitly:
git -C ~/.studentflow/canonical push --tags <canonical_remote_url>

Daily Operations

Morning Checklist

1. **Launch StudentFlow:**
   cd ~/studentflow && source .venv/bin/activate && python -m app.main

2. **Check the Dashboard:**
- Verify the correct student shows as active in the **Active Student** card (green `#NN Name`)
- Confirm the **Canonical Repository** path shows `Path accessible: Yes` (green)
- Scan **Recent Sync Activity** — all entries should show `[success]`; red `[failed]` entries need attention
- Review the **Student Queue** to confirm the ordering is correct

3. **Review any errors from the previous session:**
- Navigate to Logs → Errors tab
- For each error: identify the student, read the error message, apply the appropriate recovery (see recovery procedures below)

4. **Trigger Sync All (optional):**
- If auto-sync is disabled, click `[Sync All]` on the Dashboard queue panel
- Wait for the progress dialog to complete

- Verify the result toast: "N synced, M failed, K skipped"
- If any failures: navigate to Logs → Errors tab

Adding a New Student

This procedure creates a complete student record: Student, Account, Credential, and optionally a Repository.

Step 1: Create the Student Record

1. Navigate to **Students screen** (left nav → "Students")
2. Click **`[+ Add Student]`**
3. Fill in the dialog:
- **Full Name** (required): e.g., "Frank Lee"
- **Email** (optional): e.g., frank.lee@university.edu
- **Notes** (optional): any relevant context
- **Priority**: leave at 0 unless this student should jump the queue
4. Click **`[Save]`**
5. The student appears in the table with status=active, queue position = last in queue

Step 2: Create the Account

1. Navigate to **Accounts screen**
2. Click **`[+ Add Account]`**
3. Fill in the dialog:
- **Student**: select "Frank Lee" from the dropdown
- **Provider**: GitHub / GitLab / Bitbucket / Gitea / Other
- **Username**: the student's provider username (e.g., "frank-lee-dev")
- **Auth Type**: PAT (recommended for GitHub/GitLab), SSH, App Password, or Password
- **Credential Value**: paste the token/password (the field is masked)
4. Click **`[Save]`**
5. The credential is stored in the OS keyring. The account appears with status=unvalidated.

Step 3: Validate the Credential

1. In the Accounts table, find Frank Lee's account
2. Click **`[Validate]`**
3. Wait for the validation (tests a lightweight API call to the provider)
4. Toast shows: "✓ Credential validated" or "✗ Validation failed"
5. If failed: use `[Rotate Credential]` to enter a corrected token and try again

Step 4: Add the Repository

1. Navigate to **Repositories screen**
2. Click **`[Add Repository]`**
3. Fill in the dialog:
- **Account**: select Frank Lee's account
- **Repo Name**: e.g., "frank-project"
- **Remote URL**: e.g., `https://github.com/frank-lee-dev/project.git`
- **Local Path**: e.g., `/home/operator/repos/frank-lee/` (absolute path)
- **Default Branch**: `main`
4. Click **`[Save]`**
5. The repository record is saved with status `uninitialized`.
6. **Manual clone required (v0.1.0-mvp):** Clone the repository manually:
   git clone https://github.com/frank-lee-dev/project.git /home/operator/repos/frank-lee/

> **Phase 2:** Automatic cloning on add and Init from Template are planned for Phase 2.

Step 5: Verify

1. Return to Students screen — Frank Lee shows queue position, account, and last sync (just now)
2. Frank is now ready to be handed off to when his queue position becomes active

---

Switching Active Student / Triggering a Handoff

The handoff moves the project from the current active student to the next one in the queue.

Pre-flight Check

Before triggering a handoff, verify:
1. The current student's repo shows `clean` in the Repositories screen
- If `dirty` or `diverged`: run a sync first (see Recovering from Sync Error below)
2. The next student in the queue has a valid credential (Accounts screen: Validated column shows ✓)
3. The canonical repo is reachable (Dashboard System Health: green check)

Triggering the Handoff

1. Navigate to **Dashboard**
2. In the Active Student card, click **`[Hand Off]`**
3. A confirmation dialog appears:
   Hand off from Alice Johnson to Bob Chen?

   This will:
   • Run a final sync for Alice Johnson
   • Create checkpoint tag: handoff/2024-01-15-143022/from-alice-dev-to-bob-dev
   • Bootstrap Bob Chen's repository from canonical
   • Set Bob Chen as the active student

   [Proceed] [Cancel]

4. Click **`[Proceed]`**
5. A progress dialog shows the steps as they complete:
- "Step 1: Validating pre-conditions…"
- "Step 2: Final sync for Alice Johnson…"
- "Step 3: Creating checkpoint tag…"
- "Step 4: Recording handoff event…"
- "Step 5: Bootstrapping Bob Chen's repo…"
- "Step 6: Pushing to Bob Chen's remote…"
- "Step 7–9: Activating Bob Chen…"
6. On success: Dashboard updates — Bob Chen is now in the Active Student card

After Handoff

1. Verify Dashboard: Bob Chen shows as active, repo status=clean
2. Check Logs → Handoff Log: a new entry with status=completed is present
3. Verify the checkpoint tag in canonical (optional): open a terminal and run:
   git -C ~/.studentflow/canonical tag -l "handoff/*" | tail -1

Recovering from Sync Error

A sync error means `SyncService.sync_student()` failed for a student. The repo status shows `error` or `diverged`.

Scenario 1: Auth Error (credential expired or invalid)

Symptoms: Error message contains "authentication failed", "401", "403", "Permission denied"

Steps:
1. Navigate to **Accounts screen**
2. Find the affected student's account
3. Click **`[Validate]`** — confirm it fails with auth error
4. Click **`[Rotate Credential]`**
5. In the dialog, enter the new token (get a new PAT from GitHub/GitLab settings)
6. Click **`[Validate First]`** — confirm it succeeds
7. Click **`[Save]`**
8. Return to **Repositories screen** or **Dashboard**
9. Click **`[Sync Now]`** for the affected student

10. Verify sync succeeds (status returns to `clean`)

Scenario 2: Network Timeout

Symptoms: Error message contains "timeout", "timed out", "connection refused"

Steps:
1. Check your internet connection
2. Verify the provider (GitHub/GitLab) is operational: check status.github.com or status.gitlab.com
3. Edit `~/.studentflow/config.json` — increase `sync_timeout_seconds` (e.g., from 60 to 120), then restart the app.
> **Note:** Settings screen is a placeholder in v0.1.0-mvp; edit `config.json` directly.
4. Retry the sync: Repositories screen → `[Sync Now]`

Scenario 3: Sync Failure with No Clear Error

Steps:
1. Navigate to **Logs → Sync Log tab**
2. Find the failed SyncJob record, double-click to open detail view
3. Read the full `error_message` field
4. Apply the appropriate fix from the error message
5. Retry sync

Recovering from Auth Error

See Scenario 1 in "Recovering from Sync Error" above. The same procedure applies for auth errors encountered during handoff or validation.

Recovering from a Diverged Repo

A diverged repo (`sync_status=diverged`) means both the student's remote and the local clone have commits the other lacks. This cannot be automatically merged.

Understanding the situation:
- `ahead=3` means the student's remote has 3 commits we haven't pulled yet
- `behind=2` means our local clone has 2 commits the student's remote hasn't seen
- If both are non-zero: diverged

Option A: Manual investigation (recommended first)

1. Open a terminal and check what's in the student's remote:
```
cd /path/to/student/local/repo
git log --oneline HEAD..student/main   # What the student has that we don&apos;t
git log --oneline student/main..HEAD   # What we have that the student doesn&apos;t
```

2. Decide: is the student's work the authoritative version, or is canonical?

Option B: Accept student's work (most common case)

If the student's remote has the correct version:

1. Navigate to **Repositories screen**
2. Find the diverged repo
3. Click `[Sync Now]` — StudentFlow will attempt merge (if the divergence is just a missing push on the local side, this may resolve it)
4. If still diverged: the operator must manually merge in a terminal

Option C: Force-overwrite with canonical (destructive)

If canonical is authoritative and the student's divergent work should be discarded:

1. Navigate to **Repositories screen**
2. Click `[Sync Now]` dropdown → select **"Force Overwrite from Canonical"**
3. Confirmation dialog:

Force overwrite Alice Johnson's repository with canonical HEAD?

This will DISCARD any divergent commits on Alice's remote that
are not in canonical. This action cannot be undone.

Canonical HEAD: a1b2c3d4 (Merged Bob's work — 2024-01-12)

[Yes, Force Overwrite] [Cancel]

4. After confirmation: StudentFlow hard-resets the local clone to canonical HEAD and force-pushes to student's remote
5. The action is logged in the audit log as `force_overwrite`

Reordering the Student Queue

Adjust the queue order to change which student works next, or to resequence the full roster.

Quick reorder (one step at a time):
1. Navigate to **Students screen**
2. Locate the student to move
3. Click **`[↑]`** to move them one position earlier in the queue
4. Click **`[↓]`** to move them one position later in the queue
5. The table refreshes immediately showing the new order
6. `[↑]` is disabled for the first student; `[↓]` is disabled for the last student

Precise reorder (set exact position):
1. Navigate to **Students screen**
2. Click **`[Edit]`** for the student
3. Change the **Queue Position** spinbox to the desired value
4. Click **`[Save]`**
5. Note: if another student already holds that position, both will temporarily share the same number until you reorder one of them

---

Archiving a Student

Archive a student when they have completed their work and should be removed from the active queue.

Precondition: Student should be in status=exhausted (completed their handoff) before archiving. Archiving an active student is allowed but requires confirmation.

Steps:
1. Navigate to **Students screen**
2. Find the student to archive
3. Click **`[Archive]`** in their row actions
4. Confirmation dialog appears:
Archive 'Carol Smith'?

They will be removed from the active queue and their status will be
set to archived. This action can only be reversed by editing the
student record directly.

[Yes] [No]

Default selection is **No** to prevent accidental archives.
5. Click **`[Yes]`** to confirm
6. Student status changes to `archived` and they disappear from the default view

Note: Archiving does not delete any data, repos, or credentials. The student's Git repositories remain on disk. Use the Credentials screen to delete their credentials if they are no longer needed.

Rotating a Credential

Rotate a credential when it expires, is compromised, or the student changes their account password.

Steps:
1. Navigate to **Credentials screen** (or Accounts screen → `[Rotate Credential]`)
2. Find the account whose credential needs rotation
3. Click **`[Rotate]`**
4. In the Rotate Credential dialog:
- Note the current masked value: `••••••••z789`
- Enter the new credential value in the "New credential" field (masked input)
- Click **`[Validate First]`** — this tests the new value without saving it
- If validation fails: check you entered the correct value
- Click **`[Save]`** — the old value is overwritten in the OS keyring
5. Toast: "✓ Credential rotated and validated"
6. The `last_validated_at` timestamp is updated in the table

What happens internally:
- `CredentialService.rotate()` calls `keyring.set_password()` at the same key, overwriting the old value
- DB `Credential.last_validated_at` is updated
- Audit log: `credential.rotate` with no secret value

Exporting the Audit Log

The audit log is the complete record of all operator actions.

From the UI (Phase 2):
The Settings screen Export feature is a placeholder in v0.1.0-mvp and is not yet functional.

Direct database query (v0.1.0-mvp workaround):
```
# Export audit log directly from SQLite
sqlite3 ~/.studentflow/studentflow.db \
  &quot;SELECT timestamp, actor, entity_type, entity_id, action, details FROM activity_log ORDER BY timestamp DE
  | head -50
```

From the CLI (Phase 2):
`export-audit` is an unimplemented stub in v0.1.0-mvp.

CSV format:
```
timestamp,actor,entity_type,entity_id,action,details
2024-01-15T14:30:22Z,operator,repository,abc123,sync,&quot;{&quot;&quot;commit_before&quot;&quot;: &quot;
```

The details column contains JSON (double-quoted for CSV compatibility). No secret values appear.

Delivering Prompts to the Active Student

Prompts are Markdown files written to the student's working repository and pushed to their remote. Students reference the file path in Lovable instead of pasting large text into chat.

Sending a Prompt

1. Navigate to **Prompts screen** (left nav → "Prompts")

2. Verify the banner at the top shows the correct active student (e.g., "Active student: #01 Alice Johnson")
- If it shows "Active student: none", no student is currently active — set one via Students screen
3. Fill in the fields:
- **Title** (optional): a short label, e.g. "Auth Flow Fix". Becomes part of the filename slug.
- **Prompt text** (required): paste the full prompt content.
4. Click **`[Commit & Push]`**
5. If push succeeds: a dialog shows the file path, e.g.:
   prompts/student01-001-auth-flow-fix.md

Give this path to the student — they enter it in Lovable as:
`@prompts/student01-001-auth-flow-fix.md`
6. The history table at the bottom shows all prompts with status badges.

File Naming Convention

Prompt files follow the pattern:
prompts/studentNN-SEQ-slug.md

- `NN` — zero-padded student number (e.g., `01`, `14`)
- `SEQ` — zero-padded sequence number per student (001, 002, ...)
- `slug` — URL-safe slug from the title (max 40 chars), e.g., `auth-flow-fix`

Example: `prompts/student03-007-add-login-form.md`

Push Failed / Retry

If the history table shows `failed` or `committed` status:
1. Select the failed row in the history table
2. The file path appears below the table (the file was committed but not pushed)
3. Click **`[Retry Push]`** to re-attempt the push
4. If retry fails again: check credentials (Credentials screen → Validate / Rotate), then retry

Status meanings:
Status	Meaning
pushed	File is in student's remote repo. Give path to student.
committed	Committed locally, push failed. Use Retry Push.
failed	Write or commit failed. Check Logs for error details.

Prompt File Contents

Each file is a Markdown document with a metadata table:
Auth Flow Fix

Field	Value
Student	#01
Sequence	001
Created at	2026-03-17 14:30:00 UTC
Created by	operator
Target repo	alice-project
Target branch	main

Prompt

[full prompt text here]

---
docs/ui-flows.md
Розмір: 23,092 байт

UI Flows

All screens are children of the main `QMainWindow`. Navigation is via a left sidebar `QListWidget`. The right side is a `QStackedWidget` that switches between 7 screens. The window title is "StudentFlow — Student Workflow Manager". Minimum size is 1200×800.

Dashboard Screen

> **v0.1.0-mvp:** Dashboard is a placeholder. The screen displays "not implemented yet". The full layout described below is the Phase 2 target specification.

Purpose: The operator's primary view. Shows who is currently active, what's next in the queue, system health, and provides the most common quick actions.

Layout: Full-height 3-column layout below a header bar.

```
┌──────────────────────────────────────────────────────────────────────┐
│ HEADER: &quot;Dashboard&quot;  │  Status: Last sync: 14:32 (3 min ago) │
├──────────────────┬──────────────────────┬──────────────────────────┤
│ ACTIVE STUDENT│ QUEUE (next 5)  │ SYSTEM HEALTH          │
│                  │                      │                          │
│ Alice Johnson │ 2. Bob Chen     │ ✓ Sync healthy         │
│ [active]      │ 3. Carol Smith  │ Last sync: success     │
│ Repo: alice-p │ 4. David Kim    │ Canonical: reachable   │
│ Last sync:    │ 5. Eve Wilson   │ Errors (last 24h): 0   │
│ 14:32, clean  │ 6. Frank Lee    │                        │
│                  │                      │ Recent errors:         │
│ [Sync Now]    │                      │ (none)                 │
│ [Hand Off]    │                      │                        │
├──────────────────┴──────────────────────┴──────────────────────────┤
│ RECENT ACTIVITY (last 10 ActivityLog entries)                       │
│ 14:32  operator  repository.sync   alice-project  success           │
│ 13:10  operator  student.create    Frank Lee                        │
│ ...                                                                 │
└──────────────────────────────────────────────────────────────────────┘
```

Active Student Card (Column 1):
- Student full name (large text, 16pt bold)
- Status badge: colored pill label (active=green, paused=yellow, error=red)
- Repository name and sync_status badge (same color coding)
- Last sync timestamp and result (clean / error)
- `[Sync Now]` button: triggers `SyncWorker` for this student; shows `QProgressDialog("Syncing...", None, 0, 0)` while running
- `[Hand Off]` button: opens `HandoffConfirmDialog` before triggering `HandoffWorker`
- If no current student: card shows "No active student — select from queue"

Queue Panel (Column 2):
- Title: "Up Next"
- Shows next 5 students in queue order (position > current)
- Each row: position number, full name, status badge
- Double-click on a student opens their Student detail dialog
- `[Sync All]` button at bottom of panel: triggers sync-all job with progress

System Health Panel (Column 3):
- Title: "System Health"
- Canonical repo reachability: green check or red X with path shown
- Last SyncJob result: status badge + timestamp
- Error count (last 24h): count of SyncJob records with status=failed in last 24h
- Last 3 error messages (truncated to 80 chars each): shown as red text
- `[View All Errors]` link: navigates to Logs screen, Errors tab

Recent Activity (Bottom bar):
- Title: "Recent Activity"
- Scrollable table (not full QTableWidget — use QListWidget for performance)
- Columns: Time | Actor | Entity Type | Action | Entity ID (short) | Details summary
- Shows last 10 ActivityLog entries, newest first
- Auto-refreshes every 30 seconds via QTimer
- `[View Full Log]` link: navigates to Logs screen, Audit Log tab

Signal/Slot connections:
- `SyncWorker.sync_complete` → refresh Active Student card + Recent Activity

- `HandoffWorker.handoff_complete` → refresh all three panels
- QTimer(30s) → query ActivityLog → refresh Recent Activity

Students Screen

Purpose: Full CRUD for Student records and queue management.

Layout: Toolbar at top, QTableWidget below.

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ [+ Add Student]  Status: [All ▼]  Search: [_____________]   │
├──────────────────────────────────────────────────────────────────────────────┤
│ Full Name      │ Status  │ Queue │ Account        │ Last Sync │ Actions │
├────────────────┼─────────┼───────┼────────────────┼───────────┼─────────────┤
│ Alice Johnson  │ active  │  1    │ github/alice   │ 14:32 ✓   │ [Edit][Archive][↑][↓][Repos][Logs] │
│ Bob Chen       │ active  │  2    │ github/bob-dev │ —         │ ... │
│ Carol Smith    │ paused  │  3    │ gitlab/csmith  │ Error     │ ... │
│ David Kim      │ active  │  4    │ github/dkim99  │ 10:15 ✓   │ ... │
└──────────────────────────────────────────────────────────────────────────────┘
```

Toolbar:
- `[+ Add Student]` button: opens `AddStudentDialog`
- Status filter dropdown: All | Active | Paused | Exhausted | Archived | Error
- Search box: live filter on `full_name` (case-insensitive, updates table on keystroke)

Table columns:
Column	Source	Notes
Full Name	Student.full_name	
Status	Student.status	Colored badge: active=green, paused=yellow, exhausted=gray, archived=gray, error=red
Queue	Student.queue_position	"—" if NULL
Account	Account.provider + "/" + Account.username	Primary (is_current) account, or first account
Last Sync	SyncJob.finished_at (latest success)	"—" if never synced; colored: green=success, red=failed
Actions	Inline buttons	

Row Actions:
- `[Edit]`: opens `EditStudentDialog` (prefilled with student fields)
- `[Archive]`: confirmation dialog → `AccountService.archive_student(id, reason)` → row disappears (filtered by default status=active/paused)
- `[↑]` / `[↓]`: move student up or down in queue → `AccountService.move_up(id)` / `move_down(id)` → refresh table
- `[Repos]`: navigates to Repositories screen filtered to this student
- `[Logs]`: navigates to Logs screen filtered to this student

AddStudentDialog / EditStudentDialog fields:
- Full Name (QLineEdit, required)
- Email (QLineEdit, optional, validated as email format)
- Notes (QTextEdit, optional)
- Priority (QSpinBox, 0-100, default 0)
- Status (QComboBox — active/paused — only in Edit dialog, not Add)
- `[Save]` / `[Cancel]` buttons
- Validation: Full Name must not be empty, Email must be valid format if provided

---

Accounts Screen

Purpose: Manage Git provider accounts for each student. Each account has a credential.

Layout: Toolbar + QTableWidget.

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│ [+ Add Account]  Student: [All ▼]  Provider: [All ▼]        │
├──────────────────────────────────────────────────────────────────────────────────────────────┤
│ Username      │ Provider  │ Auth Type │ Validated    │ Student      │ Actions │
├──────────────┼──────────┼──────────┼──────────────┼──────────────┼──────────────────────────┤
│ alice-dev     │ GitHub    │ PAT       │ ✓ 2h ago     │ Alice Johnson│ [Edit][Validate][Rotate Cred][Deactivate] │
│ bob-dev       │ GitHub    │ SSH       │ ✗ Never      │ Bob Chen     │ ... │
│ csmith        │ GitLab    │ PAT       │ ✓ 1d ago     │ Carol Smith  │ ... │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
```

Toolbar:
- `[+ Add Account]` button: opens `AddAccountDialog`
- Student filter dropdown: All + each student name
- Provider filter dropdown: All | GitHub | GitLab | Bitbucket | Gitea | Other

Table columns:
Column	Source	Notes
Username	Account.username	
Provider	Account.provider	Display name: "GitHub", "GitLab", etc.
Auth Type	Account.auth_type	"PAT", "SSH", "App Password", "Password"
Validated	Account.last_validated_at	"✓ {time_ago}" or "✗ Never"
Student	Student.full_name	
Actions	Inline buttons	

Row Actions:
- `[Edit]`: opens `EditAccountDialog` (username, provider, auth_type only — no credential change here)
- `[Validate]`: calls `CredentialService.validate_exists()` + tests git access → shows result toast notification
- `[Rotate Credential]`: opens `RotateCredentialDialog`
- `[Deactivate]`: sets Account.status=inactive, confirmation required

AddAccountDialog fields:
- Student (QComboBox — required, shows all non-archived students)
- Provider (QComboBox — required)
- Username (QLineEdit — required)
- Auth Type (QComboBox — required)
- Credential Value (QLineEdit — required, `setEchoMode(Password)`, placeholder "Enter token / password / passphrase")
- Secret Kind (auto-populated from auth_type: ssh_key / pat / app_password / password)
- `[Save]` / `[Cancel]`

RotateCredentialDialog fields:
- Account info (read-only): username, provider
- Current credential: "••••••••{last4}" (read-only)
- New credential value (QLineEdit, `setEchoMode(Password)`)
- `[Validate First]` button: tests new credential before saving
- `[Save]` (disabled until validated or explicitly bypassed) / `[Cancel]`

Repositories Screen

Purpose: View and manage all student repos, run syncs, check status.

Layout: Toolbar + QTableWidget with colored status badges.

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│ Student: [All ▼]  Status: [All ▼]  [Sync All]               │
├──────────────────────────────────────────────────────────────────────────────────────────────┤
│ Repo Name       │ Student       │ Local Path          │ Status      │ Last Commit │ Actions │
├──────────────────┼──────────────┼──────────────────────┼──────────────┼──────────────┼──────────────────────────┤
│ alice-project   │ Alice Johnson │ ~/sf/alice/project  │ ■ clean     │ a1b2c3d (2h)│ [Sync][Log][Open][Validate][Init]
│ bob-project     │ Bob Chen      │ ~/sf/bob/project    │ ■ dirty     │ e5f6g7h (1d)│ ... │
│ carol-project   │ Carol Smith   │ ~/sf/carol/project  │ ■ diverged  │ i9j0k1l (3d)│ ... │
```

```
│ canonical      │ (canonical)  │ ~/.sf/canonical   │ ■ clean    │ a1b2c3d    │ [Log][Open][Validate] │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────
```

Status color coding:
Status	Color	Badge
clean	Green (#43b581)	■ clean
dirty	Yellow (#faa61a)	■ dirty
diverged	Red (#f04747)	■ diverged
error	Red (#f04747)	■ error
uninitialized	Gray (#72767d)	■ uninitialized

Row Actions:
- `[Sync Now]`: calls `SyncService.sync_student()` via worker, shows progress indicator, refreshes row on completion
- `[View Log]`: opens `SyncLogDialog` showing last 20 SyncJob records for this repo
- `[Open in Explorer]`: `QDesktopServices.openUrl(QUrl.fromLocalFile(repo.local_path))`
- `[Validate Remote]`: calls `RepositoryService.validate_remote(repo)` → shows toast: "✓ Remote accessible" or "✗ Cannot reach remote"
- `[Init from Template]`: only shown for uninitialized repos — opens `InitRepoDialog`

InitRepoDialog fields:
- Template source (QComboBox of active TemplateSource records)
- Local path (QLineEdit with `[Browse]` button)
- Remote URL (QLineEdit)
- `[Initialize]` / `[Cancel]`

Credentials Screen

Purpose: View all stored credentials, validate, rotate, and delete.

Layout: Toolbar + QTableWidget. No credential values ever displayed in full.

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────
│ [+ Add Credential]  Account: [All ▼]                                                                      │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│ Account            │ Secret Kind  │ Last Validated  │ Status     │ Masked Value  │ Actions │
├────────────────────┼──────────────┼─────────────────┼────────────┼───────────────┼─────────────────────┼
│ alice-dev (GitHub) │ PAT          │ ✓ 2h ago        │ active     │ ••••••••z789  │ [Validate][Rotate][Delete] │
│ bob-dev (GitHub)   │ SSH Key      │ ✗ Never         │ unvalidated │ ••••••••(SSH) │ [Validate][Rotate][Delete] │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────
```

Table columns:
Column	Source	Notes
Account	Account.username + " (" + provider + ")"	
Secret Kind	Credential.secret_kind	"PAT", "SSH Key", "App Password", "Password"
Last Validated	Credential.last_validated_at	"✓ {time_ago}" or "✗ Never"
Status	Account.status	colored badge
Masked Value	`mask_secret(keyring.get(secret_ref))`	Shows "••••••••{last4}" or "••••••••(SSH)" for SSH keys
Actions	Inline buttons	

Row Actions:
- `[Validate]`: retrieves from keyring, tests against provider API, updates `last_validated_at`, shows toast
- `[Rotate]`: opens `RotateCredentialDialog` (same as in Accounts screen)
- `[Delete]`: confirmation dialog → removes from keyring, deletes Credential record, sets Account.status=inactive

Security rules enforced by this screen:
- Masked Value column calls `mask_secret()` — never shows full value
- "Copy" is not provided by default (reduces accidental clipboard exposure)
- Rotate dialog uses `setEchoMode(Password)` on input

Prompts Screen

Purpose: Compose prompt files for the active student, commit and push them to the student's working repository, and track delivery history.

Layout: Vertical splitter with compose pane (top) and history pane (bottom).

Active Student Banner
- Shows: `Active student: #NN Full Name` in bold 14px
- If no active student: shows "Active student: none" and disables `[Commit & Push]`

Compose Pane (top)
- **Title** (QLineEdit, optional): prompt title, used to generate file slug
- **Prompt text** (QTextEdit, required): full prompt content
- **`[Commit & Push]`** button: disabled when no active student
- On click: validates content non-empty, calls `PromptService.create_and_push_prompt()`
- While running: button shows "Pushing..." and is disabled
- On success: shows dialog with file path, clears title and content fields
- On push failure: shows warning with committed path — use Retry Push

History Pane (bottom)
Column	Content
#	`NN-SEQ` format, e.g. `01-001`
Title	prompt title
File Path	relative path, e.g. `prompts/student01-001-auth-flow-fix.md`
Status	colored text: pushed=green (#a6e3a1), committed=teal (#89dceb), written=peach (#fab387), draft=gray (#a6adc8), failed=red (#f38ba8)
Pushed At	UTC timestamp truncated to minute, or blank if not pushed

- Selecting a row shows the file path below the table (selectable text for clipboard copy)
- `[Retry Push]` button: enabled only when selected row has status=failed or status=committed
- `[Refresh]` button: reloads active student banner + full history
- Hover/tooltip on a failed row shows sanitized error message (no tokens)

File Naming
prompts/studentNN-SEQ-slug.md

Where NN is zero-padded student number, SEQ is zero-padded sequence, slug is derived from title (lowercase, alphanumeric + hyphens, max 40 chars). If no title: slug is `"prompt"`.

Signal Flow
- `[Commit & Push]` → `PromptsViewModel.create_prompt()` → `PromptService.create_and_push_prompt()` → git add / commit / push to `student` remote
- `[Retry Push]` → `PromptsViewModel.retry_push()` → `PromptService.retry_push()` → push only (no new commit)
- `[Refresh]` → `refresh()` → re-query active student + prompt list

Logs Screen

Purpose: Browse and search all audit, sync, handoff, and error logs.

Layout: 4-tab QTabWidget, each tab has filters + QTableWidget + export button.

Tab 1: Audit Log

Displays `ActivityLog` records.

Filters:
- Date range: from/to QDateEdit widgets
- Entity type: QComboBox (All / Student / Account / Repository / Credential / Handoff / System)
- Action: QLineEdit (text search)

Columns: Timestamp | Actor | Entity Type | Entity ID | Action | Details (JSON, truncated to 80 chars)

Export: `[Export to CSV]` button → `QFileDialog.getSaveFileName()` → write filtered results as CSV.

Tab 2: Sync Log

Displays `SyncJob` records.

Filters:
- Date range
- Student: QComboBox (All + each student)
- Status: QComboBox (All / Pending / Running / Success / Failed / Skipped)
- Job type: QComboBox

Columns: Timestamp | Student | Repo | Job Type | Status (colored) | Duration | Commits Before/After | Summary

Double-click opens `SyncJobDetailDialog` showing full error_message and summary.

Tab 3: Handoff Log

Displays `HandoffEvent` records.

Columns: Date | From Student | To Student | Commit Hash (short) | Checkpoint Tag | Status | Notes

Double-click opens detail view.

Tab 4: Errors

Displays all `SyncJob` records with `status=failed` and `ActivityLog` entries with action containing "fail" or "error".

Columns: Timestamp | Type (Sync/Audit) | Student | Entity | Error Message | `[Retry]` button (for sync errors)

`[Retry]` on a failed SyncJob: calls `SyncService.sync_student()` for that student's current repo.

Settings Screen

Purpose: Configure application-level settings. Non-secret settings only.

Layout: Two-column form layout with section headings.

General Section

Setting	UI Control	AppSetting Key
Canonical Repo Path	QLineEdit + `[Browse]` button	canonical_repo_path
Template Directory	QLineEdit + `[Browse]` button	template_dir
Default Branch	QLineEdit	default_branch

`[Browse]` opens `QFileDialog.getExistingDirectory()`.
After changing `canonical_repo_path`: validate the path is a git repo (or offer to initialize it).

Sync Section

Setting	UI Control	AppSetting Key
Sync Timeout (seconds)	QSpinBox (10–300)	sync_timeout_seconds
Max Sync Retries	QSpinBox (0–5)	max_sync_retries
Enable Auto Sync	QCheckBox	enable_auto_sync
Auto Sync Interval (minutes)	QSpinBox (5–1440, enabled only if auto sync on)	auto_sync_interval_minutes

Security Section (Read-Only)

Field	Value
Keyring Backend	`keyring.get_keyring().__class__.__name__`
DB Path	`~/.studentflow/studentflow.db`
Log Directory	`~/.studentflow/logs/`

These fields are `QLabel` or read-only `QLineEdit` — not editable.

Export Section
- `[Export Audit Log as CSV]`: date range picker → export filtered ActivityLog to CSV
- `[Export Sync Log as CSV]`: date range picker → export SyncJob records to CSV

Buttons
- `[Save Settings]`: write changed AppSetting values to DB, update in-memory Config
- `[Reset to Defaults]`: confirmation dialog → restore all AppSetting keys to their default values
- `[Cancel]`: discard changes (only if unsaved changes exist)

Shared UI Patterns

Status Badges
All status values are displayed as colored pill labels (`QLabel` with border-radius stylesheet):
- Green (#43b581): active, clean, success
- Yellow (#faa61a): paused, dirty, running
- Red (#f04747): error, diverged, failed
- Gray (#72767d): archived, uninitialized, skipped, exhausted
- Blue (#5865f2): pending, unvalidated

Toast Notifications
Short-lived status messages are shown via a `ToastWidget` that appears at the bottom-right of the main window for 3 seconds:
- Success toast: green background, "✓ {message}"
- Error toast: red background, "✗ {message}"
- Info toast: blue background, "{message}"

Confirmation Dialogs
Destructive actions (Archive, Delete, Force Overwrite, Reset Defaults) require a `QMessageBox.question()` confirmation with explicit "Yes, proceed" / "Cancel" buttons. The message describes exactly what will happen.

Progress Indicators
Long-running operations (sync, handoff, bootstrap) show a `QProgressDialog` with:
- Title: "StudentFlow — Working…"
- Label: "{operation_name} for {student_name}…"
- Cancel button: only shown for operations that can be safely interrupted (sync fetch); not shown during handoff commit steps.
- Indeterminate progress bar (setMaximum(0))
---
docs/architecture.md
Розмір: 13,641 байт

Architecture

System Architecture

StudentFlow is a three-tier desktop application running entirely on the operator's local machine.

```
┌──────────────────────────────────────────────────────────────┐
│ UI Layer (PySide6 / Qt6)                                     │
│ MainWindow → 7 Screens → Dialogs → Widgets                   │
│ ViewModels provide data; service calls run synchronously     │
│ (QThread async workers are Phase 2)                          │
└──────────────────────────────────────────────────────────────┘
                        │ method calls
                        ▼
┌──────────────────────────────────────────────────────────────┐
│ Service Layer (Python)                                       │
│ AccountService  RepositoryService  CredentialService         │
```

```
│ SyncService  HandoffService  TemplateService  AuditService  │
│ SchedulerService                                            │
└─────────────────────────────────────────────────────────────┘
                    │ SQLAlchemy + subprocess
┌───────────────────▼─────────────────────────────────────────┐
│ Data Layer                                                  │
│ SQLite DB (~/.studentflow/studentflow.db)                   │
│ Git repos (filesystem: canonical + per-student clones)      │
│ OS Keyring (secrets — never in DB)                          │
└─────────────────────────────────────────────────────────────┘
```

The UI layer never touches the database or runs git commands directly. All such operations go through the service layer. The service layer owns all business logic and orchestration. The data layer is passive — it stores and retrieves state on demand.

Component Map

AccountService (`app/services/account_service.py`)
Manages CRUD for Student and Account entities.

Responsibilities:
- Create, read, update, archive Student records
- Create, read, update, deactivate Account records
- Queue management: get_queue(), move_up(), move_down(), set_position()
- Set/clear `is_current` flag on Account
- Delegate all credential operations to CredentialService

Key methods:
- `create_student(full_name, email, notes) → Student`
- `archive_student(student_id, reason)`
- `get_queue() → list[Student]` (ordered by queue_position ASC, status in active/paused)
- `create_account(student_id, provider, username, auth_type) → Account`
- `set_current_student(student_id)`

RepositoryService (`app/services/repository_service.py`)
Manages repository initialization, validation, and state queries.

Responsibilities:
- Bootstrap a new local repo from a TemplateSource or from canonical
- Validate that a local path is a valid git repo
- Validate that a remote URL is reachable (with credentials)
- Query and update sync_status on Repository records
- Never run git commands directly — delegates to GitClient

Key methods:
- `bootstrap_from_template(student, account, template_source, local_path, remote_url) → Repository`
- `bootstrap_from_canonical(student, account, local_path, remote_url) → Repository`
- `validate_local_path(path: str) → bool`
- `validate_remote(repo: Repository) → bool`
- `get_repo_status(repo: Repository) → SyncStatus`

CredentialService (`app/services/credential_service.py`)
Single interface for all OS keyring operations. The only service that interacts with the keyring.

Responsibilities:
- Store a new credential in keyring, create Credential DB record with secret_ref
- Retrieve a credential value from keyring by secret_ref
- Rotate a credential (write new value, invalidate old)
- Validate that a keyring entry exists
- Delete a credential from keyring and DB

Key naming convention: `studentflow:{provider}:{username}:{account_id[:8]}`

Key methods:
- `store(account_id, secret_kind, value) → Credential`
- `retrieve(secret_ref: str) → str`
- `rotate(credential_id, new_value)`
- `delete(secret_ref: str)`
- `validate_exists(secret_ref: str) → bool`

SyncService (`app/services/sync_service.py`)
Orchestrates the per-student sync pipeline and sync-all batch.

Responsibilities:
- Sync a single student: fetch from student remote → merge into local → push to canonical
- Run sync-all: iterate all active students, isolate failures
- Detect divergence (both repos have commits the other lacks)
- Force-overwrite local with canonical HEAD (manual recovery)
- Create SyncJob records for every operation

Key methods:
- `sync_student(student_id: str) → SyncJob`
- `sync_all() → list[SyncJob]`
- `detect_divergence(repo: Repository) → bool`
- `force_overwrite_from_canonical(repo_id: str) → SyncJob`
- `get_recent_jobs(limit=50, student_id=None, status=None) → list[SyncJob]`

HandoffService (`app/services/handoff_service.py`)
Implements the 9-step handoff workflow from current student to next student.

Responsibilities:
- Validate all pre-conditions before any state change
- Run final sync on current student (if active)
- Create checkpoint tag on canonical
- Bootstrap or update next student's repo from canonical
- Update student statuses and queue
- Create HandoffEvent record
- Roll back all changes if any step fails

Key methods:
- `handoff_to_next(current_student_id: str) → HandoffEvent`
- `find_next_student(current_position: int) → Student | None`
- `validate_handoff_preconditions(current: Student, next: Student) → list[str]`
- `rollback_handoff(handoff_event: HandoffEvent)`

TemplateService (`app/services/template_service.py`)
Manages project template sources used to initialize new student repos.

Responsibilities:
- Import a local directory as a TemplateSource, compute SHA256 checksum
- Validate that a template directory has the expected structure
- List available templates, set active template

Key methods:
- `import_from_path(path: str, name: str) → TemplateSource`
- `validate_template(template_source: TemplateSource) → bool`
- `list_templates() → list[TemplateSource]`
- `set_active(template_id: str)`
- `compute_checksum(path: str) → str`

AuditService (`app/services/audit_service.py`)
Provides the append-only audit log interface. All other services call this to record actions.

Responsibilities:
- Write ActivityLog entries (append only — no updates, no deletes)
- Filter secrets from details JSON before writing
- Query audit log with filters

Key invariant: `log()` never raises. If writing to DB fails, it writes to structlog instead.

Key methods:
- `log(actor, entity_type, entity_id, action, details: dict | None) → ActivityLog`
- `query(entity_type=None, entity_id=None, action=None, limit=100) → list[ActivityLog]`

SchedulerService (`app/jobs/scheduler.py`)
Manages APScheduler background jobs for periodic sync.

Responsibilities:
- Start/stop the APScheduler instance
- Register periodic sync-all job (configurable interval)
- Emit Qt signals when jobs complete so UI can refresh

Key methods:
- `start()`
- `stop()`
- `schedule_sync_all(interval_minutes: int)`
- `cancel_sync_all()`

Data Flow

Flow 1: Create a Student and Account

```
Operator → AccountsScreen
  → AccountService.create_student(full_name, email)
    → DB: INSERT Student
    → AuditService.log(&apos;operator&apos;, &apos;student&apos;, id, &apos;create&apos;, {...})
  → AccountService.create_account(student_id, provider, username, auth_type)
    → DB: INSERT Account
  → CredentialService.store(account_id, secret_kind, value)
    → keyring.set_password(service, username, value)
    → DB: INSERT Credential(secret_ref=key_name) — no value stored
    → AuditService.log(&apos;operator&apos;, &apos;credential&apos;, id, &apos;store&apos;, {...})
```

Flow 2: Initialize Student Repository from Template

```
Operator → RepositoriesScreen → &quot;Init from Template&quot;
  → RepositoryService.bootstrap_from_template(student, account, template_source, local_path, remote_url)
    → TemplateService.validate_template(template_source)
    → GitClient.init_from_dir(template_source.local_path, local_path)
      # copies files, git init, initial commit
    → GitClient.clone(canonical_path, local_path)  # if canonical exists
    → GitClient.push(local_path, &apos;student&apos;, &apos;main&apos;, credential)
    → DB: INSERT Repository(sync_status=clean)
    → AuditService.log(...)
```

Flow 3: Sync Current Student

```
Operator → Dashboard → &quot;Sync Now&quot;
  → SyncService.sync_student(student_id)
    → DB: INSERT SyncJob(status=pending)
    → CredentialService.retrieve(credential.secret_ref)  → token
    → GitClient.fetch(local_path, &apos;student&apos;)
    → GitClient.check_divergence(local_path, &apos;student/main&apos;)  → (ahead, behind)
    → if clean: GitClient.merge_ff_only(local_path, &apos;student/main&apos;)
    → GitClient.push(local_path, &apos;origin&apos;, &apos;main&apos;, canonical_cred)
    → DB: UPDATE Repository.last_commit_hash, sync_status=clean
    → DB: UPDATE SyncJob(status=success, finished_at=now)
    → AuditService.log(&apos;operator&apos;, &apos;repository&apos;, repo_id, &apos;sync&apos;, {...})
```

Flow 4: Handoff to Next Student

Operator → Dashboard → "Hand Off"
  → HandoffService.handoff_to_next(current_student_id)
    → validate_handoff_preconditions(current, next) → [] (no errors)
    → SyncService.sync_student(current) # final sync
    → GitClient.create_tag(canonical_path, 'handoff/2024-01-15-143022/from-alice-to-bob')
    → DB: INSERT HandoffEvent(status=pending)
    → GitClient.clone or pull canonical → next_student.local_path
    → GitClient.push(next_local_path, 'student', 'main', next_credential)
    → DB: UPDATE Student(current=exhausted, is_current=false)
    → DB: UPDATE Student(next=active, is_current=true)
    → DB: UPDATE HandoffEvent(status=completed)
    → AuditService.log('operator', 'handoff', id, 'complete', {...})

Flow 5: Next Student Pulls from Canonical (Next Session)

Next student has already been bootstrapped by the handoff.
At start of next session, operator verifies repo state.
Operator → RepositoriesScreen → "Validate Remote"
  → RepositoryService.validate_remote(repo) → True
Optionally sync again to catch any operator updates
Operator → Dashboard → "Sync Now"
  → SyncService.sync_student(next_student_id)

Canonical Repo Strategy

The canonical bare repository lives at `~/.studentflow/canonical/` (configurable in AppSetting). It is the single integration hub for all student work.

Structure:
- One bare repo (or a non-bare mirror repo) — no working tree needed
- Branch `main` (configurable as `default_branch` in AppSetting) is the integration branch
- Checkpoint tags accumulate permanently: `handoff/YYYY-MM-DD-HHMMSS/from-X-to-Y`

Rules:
- Every student's sync pushes to canonical's main branch
- Every new student's repo is bootstrapped from canonical's current HEAD
- No force-push to canonical. Merge-only (--ff-only preferred, --no-ff fallback)
- Student branches in their own remote: `student/{username}` (the student controls their remote)
- The canonical repo is the only repo StudentFlow directly pushes to (aside from student remotes during bootstrap)

Init:
- If canonical does not exist: TemplateService creates it from the template directory during first-time setup
- If canonical already exists: new student repos are cloned from it

---

Process Boundaries

Git Operations
- All Git operations run in a subprocess with a configurable timeout (`sync_timeout_seconds`, default 60s)
- GitPython is used for programmatic operations (log reading, status checks, tag creation)
- `subprocess.run()` with explicit timeout is the fallback for operations GitPython cannot handle
- No Git operation blocks the UI thread: all are wrapped in QThread workers that emit signals on completion

Threading Model
- Main Qt thread: UI only — no blocking calls

- `SyncWorker(QThread)`: wraps SyncService calls, emits `sync_complete(SyncJob)` and `sync_error(str)`
- `HandoffWorker(QThread)`: wraps HandoffService, emits `handoff_complete(HandoffEvent)` and `handoff_error(str)`
- APScheduler runs in a separate thread managed by SchedulerService

Keyring Access
- Keyring operations are synchronous (OS API calls)
- They are fast enough to run on-demand (not pre-cached)
- CredentialService is the only module that calls keyring — never called from UI directly

Database Access
- Session-per-operation pattern: each service method opens a session, commits or rolls back, closes
- No long-lived sessions held open across operations
- SQLAlchemy 2.x with explicit `with session.begin()` blocks for transaction safety
- No raw SQL — all access through ORM

Configuration
- `~/.studentflow/config.json` holds non-secret settings (repo paths, timeouts, defaults)
- Loaded at startup by `Config` dataclass in `app/config.py`
- In-memory for the process lifetime; reloaded on explicit "Reload Config" action
---
docs/setup.md
Розмір: 8,291 байт

Setup Guide

Prerequisites

Before installing StudentFlow, ensure the following are available on your system:

Required:
- **Python 3.11 or later** — check with `python3 --version`
- **Git 2.30 or later** — check with `git --version`
- **pip 23+** — comes with Python 3.11; check with `pip --version`
- **OS keyring support** — see platform-specific notes below

Optional but recommended:
- A virtual environment tool (`python -m venv` is built-in)
- A terminal with UTF-8 locale (for status badge characters)

Installation

Step 1: Clone the Repository

```
git clone &lt;studentflow-repo-url&gt; studentflow
cd studentflow
```

Step 2: Create and Activate a Virtual Environment

```
python3.11 -m venv .venv

# Linux / macOS:
source .venv/bin/activate

# Windows (Command Prompt):
.venv\Scripts\activate.bat
# Windows (PowerShell):
```

```
.venv\Scripts\Activate.ps1
```

You should see `(.venv)` in your prompt when the venv is active.

Step 3: Install Dependencies

```
pip install --upgrade pip
pip install -r requirements.txt
```

This installs all pinned dependencies including PySide6, SQLAlchemy, Alembic, GitPython, keyring, structlog, and APScheduler.

Note on PySide6: PySide6 6.7.0 requires approximately 200MB of disk space. On Linux, ensure `libxcb` and related Qt6 system libraries are installed:

```
# Debian/Ubuntu:
sudo apt-get install libxcb-cursor0 libxcb-xinerama0 libegl1

# Fedora/RHEL:
sudo dnf install xcb-util-cursor mesa-libEGL
```

Step 4: Initialize the Database

```
python -m app.bootstrap init-db
```

This command:
1. Creates the `~/.studentflow/` directory and subdirectories
2. Initializes the SQLite database at `~/.studentflow/studentflow.db`
3. Runs all Alembic migrations to the latest schema
4. Writes default `AppSetting` values to the database
5. Creates `~/.studentflow/config.json` with default non-secret settings
6. Creates `~/.studentflow/logs/` for log files
7. Creates `~/.studentflow/canonical/` as an empty directory (canonical repo is initialized in the UI)

Expected output:
```
[INFO] Creating ~/.studentflow/ ...
[INFO] Running database migrations ...
[INFO] Writing default AppSettings ...
[INFO] Database initialized at ~/.studentflow/studentflow.db
[INFO] Done.
```

First Run

```
QT_QPA_PLATFORM=xcb python -m app.main
```

The PySide6 window opens directly to the main screen. **There is no setup wizard in v0.1.0-mvp.** First-time configuration is done manually through the menu screens:

1. **Students → Add Student** — create student records with queue positions.
2. **Accounts → Add Account** — link a Git provider account to each student.
3. **Credentials → Add Credential** — store tokens/passwords in the OS keyring.
4. **Repositories → Add Repository** — register a local path and remote URL.

The canonical repo path defaults to `~/.studentflow/canonical/` and can be changed in `~/.studentflow/config.json`.

> **Note:** A first-run setup wizard is planned for Phase 2.

CLI Mode (Phase 2 — stub only in v0.1.0-mvp)

The `--cli` flag is parsed but **all CLI subcommands are unimplemented stubs in v0.1.0-mvp**. Running `python -m app.main --cli` exits silently without error.

```
# Accepted by the argument parser but does nothing in v0.1.0-mvp:
python -m app.main --cli
```

Full CLI subcommands (`queue`, `sync-all`, `sync`, `handoff`, `status`, `bootstrap`, `export-audit`) are planned for Phase 2.

Configuration File

`~/.studentflow/config.json` holds non-secret settings loaded at startup. It is created with defaults during `init-db` and can be edited manually or through the Settings screen.

Example `config.json`:

```
{
  &quot;canonical_repo_path&quot;: &quot;/home/operator/.studentflow/canonical&quot;,
  &quot;template_dir&quot;: &quot;/home/operator/studentflow-templates&quot;,
  &quot;default_branch&quot;: &quot;main&quot;,
  &quot;sync_timeout_seconds&quot;: 60,
  &quot;max_sync_retries&quot;: 2,
  &quot;enable_auto_sync&quot;: false,
  &quot;auto_sync_interval_minutes&quot;: 30,
  &quot;log_level&quot;: &quot;INFO&quot;
}
```

Important: Do not store any secrets (tokens, passwords, SSH passphrases) in `config.json`. Use the Credentials screen or `CredentialService` programmatically.

The in-memory `Config` object is loaded once at startup. To reload it after manual edits: restart the app, or use Settings screen → `[Save Settings]` (which writes AppSetting to DB and reloads Config).

Platform-Specific Keyring Setup

Linux

StudentFlow uses the `keyring` library which defaults to the `SecretService` backend on Linux. This requires a running SecretService implementation.

GNOME Keyring (most common):

```
# Install if not present
sudo apt-get install gnome-keyring libsecret-tools

# Check if running
ps aux | grep gnome-keyring
```

GNOME Keyring typically starts automatically with a desktop session. If running headless or in a minimal environment:

```
# Start gnome-keyring-daemon manually
eval $(gnome-keyring-daemon --start --components=secrets)
export SSH_AUTH_SOCK
```

KWallet (KDE):
KWallet is supported automatically when running a KDE desktop. No extra setup required.

libsecret backend (non-GNOME/KDE):

pip install keyrings.alt # Install alternative backends
Or install python3-secretstorage for DBus SecretService
pip install secretstorage

Headless Linux (servers):
For truly headless environments, use the `keyrings.alt` package with an encrypted file backend:
pip install keyrings.alt

Then set: `PYTHON_KEYRING_BACKEND=keyrings.alt.file.PlaintextKeyring` (or the encrypted variant).

Warning: Headless keyring backends may be less secure than the desktop keyring. Use with caution.

Verify keyring works:
python3 -c "import keyring; keyring.set_password('test', 'user', 'value')
Should print: value

macOS

macOS uses the system Keychain automatically. No additional setup is required. The keyring library integrates with `Security.framework` via `keyring`'s built-in macOS backend.

Verify
python3 -c "import keyring; print(keyring.get_keyring())"
Should show: <keyring.backends.macOS.Keyring object>

If prompted for keychain access on first run, click "Always Allow" to avoid repeated prompts.

Windows

Windows uses the Windows Credential Manager automatically via `keyring`'s built-in Windows backend.

# Verify in Python
import keyring
print(keyring.get_keyring())
# Should show: <keyring.backends.Windows.WinVaultKeyring object>

Windows App Password: Tokens are stored under "Windows Credentials" → "Generic Credentials". They can be viewed (but not easily read in plain text) in Control Panel → Credential Manager.

Verifying the Installation

Run the full test suite:
pytest tests/ -v

Run a quick smoke test:
python -m app.main --help

Check the database is accessible:
python3 -c "
from app.bootstrap import get_db_session
from app.db.models import AppSetting
with get_db_session() as session:
    settings = session.query(AppSetting).all()
    print(f'AppSettings: {len(settings)} records found')
"

Upgrading

When a new version is released:

```
# Pull latest code
git pull origin main

# Install any new/updated dependencies
pip install -r requirements.txt

# Re-initialize database (applies schema changes, idempotent)
python -m app.bootstrap init-db

# Restart the application
QT_QPA_PLATFORM=xcb python -m app.main
```

Alembic handles all schema changes. Never manually edit the SQLite schema. If `migrate` reports an error, check `docs/troubleshooting.md`.

Uninstalling

```
# Remove the virtual environment and source code
rm -rf /path/to/studentflow/

# Remove application data (WARNING: this deletes all students, sync history, audit log)
rm -rf ~/.studentflow/

# Remove credentials from OS keyring
python3 -c &quot;
import keyring
# Remove all studentflow credentials (requires knowing the key names)
# Better: use the Credentials screen in the app to delete each credential first
&quot;
```

Recommendation: Before uninstalling, export the audit log (Settings screen → Export) and note all `secret_ref` key names from the Credentials screen. This allows you to manually clean up keyring entries.

---

docs/sync-strategy.md

Розмір: 10,412 байт

Sync Strategy

Source of Truth

The canonical bare repository at `~/.studentflow/canonical/` (path configurable in AppSetting `canonical_repo_path`) is the single source of truth for all project state. This principle is absolute and never overridden.

Implications:

- Every student's local clone is a downstream copy. The student's remote is a backup/submission mechanism.
- If a student's local repo and the canonical repo disagree, canonical wins.

- The operator's role is to pull student work *into* canonical, not to push canonical *over* student work (except for the explicit force-overwrite recovery operation).
- No student repo is ever designated as a source of truth. Two students never sync to each other directly — all integration happens through canonical.

Canonical branch: `main` (configurable via `default_branch` AppSetting). Student-facing branches in their own remote: typically `main` or `student/{username}`.

Init from Template

Used when setting up the canonical repo for the first time, or when a new template version needs to seed a fresh canonical state.

Steps:
1. Operator specifies a template source: a local directory path containing the project files.
2. `TemplateService.import_from_path(path, name)` registers it as a `TemplateSource` record and computes its SHA256 checksum.
3. To initialize the canonical repo from this template:
- Copy template files to a temp directory.
- `git init temp_dir`
- `git -C temp_dir add .`
- `git -C temp_dir commit -m "Initial project template"`
- If canonical does not exist: `git clone --bare temp_dir canonical_path`
- Record `TemplateSource` as the origin on the canonical `Repository` record.
4. If canonical already exists, the template is only used for explicit "Init from Template" bootstrapping of a specific new student's repo — it does not overwrite canonical.

Checksum validation: Before using a template, `TemplateService.validate_template()` recomputes the SHA256 and compares against the stored `checksum`. If the template directory has been modified since import, a warning is shown but the operator can proceed.

First Repo Bootstrap (New Student)

When a new student is added and their repo needs to be initialized for the first time.

Option A: Bootstrap from Canonical (standard — used when canonical exists)

```
# 1. Clone canonical to student&apos;s local path
git clone &lt;canonical_path&gt; &lt;local_path&gt;

# 2. Add student&apos;s remote
git -C &lt;local_path&gt; remote add student &lt;student_remote_url&gt;

# 3. Push to student remote
git -C &lt;local_path&gt; push student main

# 4. Verify
git -C &lt;local_path&gt; log --oneline -1
```

After clone, `origin` points to canonical. `student` remote points to the student's GitHub/GitLab/etc. repo.

Option B: Bootstrap from Template (used when canonical doesn't exist or operator explicitly requests it)

```
# TemplateService copies files, git init, initial commit → temp dir
# GitClient.init_from_dir(template_path, local_path)
git init &lt;local_path&gt;
git -C &lt;local_path&gt; add .
git -C &lt;local_path&gt; commit -m &quot;Initial project template&quot;
git -C &lt;local_path&gt; remote add origin &lt;canonical_path&gt;
```

```
git -C &lt;local_path&gt; remote add student &lt;student_remote_url&gt;
git -C &lt;local_path&gt; push origin main
git -C &lt;local_path&gt; push student main
```

Post-bootstrap state:
- `Repository.sync_status = clean`
- `Repository.last_commit_hash = <HEAD SHA>`
- `SyncJob` created with `job_type=bootstrap, status=success`
- `ActivityLog` entry written

Sync Current Student

The standard operation: pull the student's submitted work into their local clone, then push to canonical.

Detailed steps:

```
# 1. Fetch latest from student&apos;s remote
git -C &lt;local_path&gt; fetch student

# 2. Check divergence
ahead = git rev-list --count HEAD..student/main   # student has new commits
behind = git rev-list --count student/main..HEAD  # local is ahead of student

# 3. Assess state
if ahead == 0 and behind == 0:
    # Nothing to sync. Status stays clean. SyncJob = skipped.
    return

if ahead &gt; 0 and behind &gt; 0:
    # DIVERGED — both sides have unique commits. Cannot auto-merge.
    repo.sync_status = diverged
    sync_job.status = failed
    # Operator must manually resolve (see force_overwrite_from_canonical)
    return

if ahead &gt; 0 and behind == 0:
    # Student has new work. Merge it in.
    git -C &lt;local_path&gt; merge --ff-only student/main
    # If ff-only fails (shouldn&apos;t happen since we checked counts):
    git -C &lt;local_path&gt; merge --no-ff student/main -m &quot;Merge student work&quot;

# 4. Push to canonical
git -C &lt;local_path&gt; push origin main

# 5. Update state
repo.last_commit_hash = git -C &lt;local_path&gt; rev-parse HEAD
repo.sync_status = clean
sync_job.status = success
sync_job.summary = f&quot;Synced {ahead} commits&quot;
```

Error handling:
- If `fetch` fails (auth error, network timeout): `SyncJob.status=failed`, `SyncJob.error_message=<stderr>`, `repo.sync_status=error`
- If `merge` fails unexpectedly: same error handling
- If `push` to canonical fails: `SyncJob.status=failed`, `repo.sync_status=error`
- Credentials are retrieved from keyring just before the git operation (not cached session-wide)

Handoff to Next Student

The full handoff is orchestrated by `HandoffService`. The sync component is:

1. **Final sync on current student** (if status=active): call `SyncService.sync_student(current_student_id)`. If this sync fails, handoff is aborted (unless student status is already exhausted or error — in which case skip).
2. **Canonical is now up to date.** Create checkpoint tag (see below).
3. **Bootstrap or update next student's repo from canonical:**

```
if next_student.repository does not exist:
    # Full bootstrap from canonical (Option A above)
    git clone &lt;canonical_path&gt; &lt;next_local_path&gt;
    git -C &lt;next_local_path&gt; remote add student &lt;next_remote_url&gt;
else:
    # Update existing local clone from canonical
    git -C &lt;next_local_path&gt; fetch origin
    git -C &lt;next_local_path&gt; merge --ff-only origin/main
```

4. **Push to next student's remote:**

```
git -C &lt;next_local_path&gt; push student main
```

Merge vs Rebase Policy

Rule: Always merge. Never rebase.

Rationale: With 20 student accounts, each with their own commit history, rebase would rewrite commits and create confusion when comparing histories. Merge preserves the full audit trail of who committed what and when.

Merge preference order:
1. `--ff-only`: preferred. Keeps history linear. Used when the student branch is a clean fast-forward of the local HEAD.
2. `--no-ff`: fallback if ff-only fails. Creates an explicit merge commit. Used when the student has commits that diverge from the local but are compatible (no conflicting changes).
3. If `--no-ff` would produce conflicts: abort. Set `sync_status=diverged`. Require operator resolution.

Force-overwrite (manual recovery for diverged repos):

```
# Operator explicitly approves losing student&apos;s uncommitted divergence
git -C &lt;local_path&gt; fetch origin
git -C &lt;local_path&gt; reset --hard origin/main
git -C &lt;local_path&gt; push student main --force
```

This is logged as `action="force_overwrite"` in AuditService. It is a destructive operation — only available through an explicit confirmation dialog in the UI.

Divergence Detection

```
# How many commits does the student remote have that local lacks?
ahead=$(git -C &lt;local_path&gt; rev-list --count HEAD..student/main)

# How many commits does local have that student/main lacks?
behind=$(git -C &lt;local_path&gt; rev-list --count student/main..HEAD)
```

Interpretation:

ahead	behind	Meaning	Action

0	0	In sync	SyncJob=skipped
>0	0	Student has new work	Normal sync: merge + push to canonical
0	>0	Local is ahead of student	Student hasn't pushed yet — sync does nothing to canonical
>0	>0	**Diverged**	Set status=diverged; require manual resolution

The `GitClient.check_divergence(repo_path, remote_branch) → tuple[int, int]` method returns `(ahead, behind)`.

Sync All

`SyncService.sync_all()` syncs every student with `status=active`.

Algorithm:
```
def sync_all() -&gt; list[SyncJob]:
    students = session.query(Student).filter(Student.status == &apos;active&apos;).all()
    results = []
    for student in students:
        try:
            job = sync_student(student.id)
            results.append(job)
        except Exception as e:
            # Log error but NEVER abort the loop
            logger.error(&quot;sync_all: student failed&quot;, student_id=student.id, error=str(e))
            error_job = SyncJob(status=failed, error_message=str(e)[:500], ...)
            results.append(error_job)
    return results
```

Isolation guarantee: One student's failure (auth error, git error, network timeout) does not affect any other student's sync. The `try/except` wraps the entire per-student operation.

Order: Students are processed in `queue_position ASC` order. This matters because canonical is the integration point — earlier-queue students' work merges first.

Concurrency: Sync-all runs students sequentially (not in parallel). Parallel pushes to canonical would risk conflicts. Sequential processing avoids canonical merge conflicts.

Report: The return value is a list of SyncJob records. The UI counts: N succeeded, M failed, K skipped.

Checkpoint Tags

Every handoff creates a permanent Git tag on canonical.

Format: `handoff/YYYY-MM-DD-HHMMSS/from-{from_username}-to-{to_username}`

Examples:
handoff/2024-01-15-143022/from-alice-to-bob
handoff/2024-02-03-091545/from-bob-to-carol
handoff/2024-03-10-160000/from-carol-to-david

For the first bootstrap (no prior student):
`handoff/YYYY-MM-DD-HHMMSS/bootstrap-to-{username}`

Commands:
```
# Create tag
git -C &lt;canonical_path&gt; tag handoff/2024-01-15-143022/from-alice-to-bob

# These tags accumulate on canonical — never deleted
git -C &lt;canonical_path&gt; tag -l &quot;handoff/*&quot;
```

Invariant: Checkpoint tags are never deleted. They form the permanent audit trail of all handoffs. Even if a handoff is "rolled back" (students reactivated), the tag remains as evidence that the handoff was attempted.

Stored in DB: The tag name is stored in `HandoffEvent.checkpoint_tag`. The commit SHA at tagging time is stored in `HandoffEvent.commit_hash`.
---
docs/uk/README.md
Розмір: 1,202 байт

StudentFlow — Документація (Українська)

StudentFlow — локальний десктопний інструмент для оператора, який керує послідовною роботою ~20 студентів над спільним навчальним Git-проєктом.

Для кого

Для одного довіреного оператора, який запускає застосунок локально на своїй машині. Це не серверний застосунок і не має мережевого інтерфейсу.

Документи

Документ	Опис
rozgortannya.md	Встановлення, налаштування, запуск, оновлення
metodyka-zastosuvannya.md	Методика роботи оператора з ~20 студентами

Англійська документація

Повна документація англійською мовою: [README.md (корінь репозиторію)](../../README.md)

Додаткові технічні документи: [docs/](../)
---
docs/uk/rozgortannya.md
Розмір: 9,607 байт

Розгортання StudentFlow

Призначення системи

StudentFlow — десктопний застосунок (PySide6) для оператора навчального процесу. Оператор керує чергою студентів, які послідовно працюють над спільним Git-репозиторієм. Застосунок:

- зберігає облікові дані студентів у системному сховищі ключів (OS keyring);
- синхронізує код між локальними клонами студентів і центральним канонічним репозиторієм;
- веде журнал усіх операцій;
- підтримує передачу проєкту від одного студента до наступного (handoff).

Це локальний інструмент. Інтернет-доступ потрібен лише для синхронізації з Git-провайдерами (GitHub, GitLab тощо).

Системні вимоги

Компонент	Мінімум
Python	3.11 або новіший
Git	2.30 або новіший
ОС	Linux (Debian/Ubuntu/RPM), macOS, Windows
Дисковий простір	~400 МБ (PySide6 ~200 МБ + залежності)
Keyring	Системне сховище ключів (GNOME Keyring / KWallet / macOS Keychain / Windows Credential Manager)

Перевірте версії:

```
python3 --version   # очікується 3.11+
git --version       # очікується 2.30+
```

Встановлення

1. Клонування репозиторію

```
git clone https://github.com/maxfraieho/studentflow-operator.git studentflow
cd studentflow
```

2. Створення і активація віртуального середовища

```
python3.11 -m venv .venv

# Linux / macOS:
source .venv/bin/activate

# Windows (PowerShell):
.venv\Scripts\Activate.ps1
```

Ознака активації: `(.venv)` з'являється в запрошенні командного рядка.

3. Встановлення залежностей

```
pip install --upgrade pip
pip install -r requirements.txt
```

Встановлюються: PySide6, SQLAlchemy, Alembic, GitPython, keyring, structlog та інші.

Linux — обов'язкові системні бібліотеки для Qt:

```
# Debian / Ubuntu:
sudo apt-get install libxcb-cursor0 libxcb-xinerama0 libegl1

# Fedora / RHEL:
sudo dnf install xcb-util-cursor mesa-libEGL
```

4. Ініціалізація бази даних

```
python -m app.bootstrap init-db
```

Команда:
1. Створює директорію `~/.studentflow/` і піддиректорії.

2. Ініціалізує SQLite-базу: `~/.studentflow/studentflow.db`.
3. Виконує всі Alembic-міграції до актуальної схеми.
4. Записує значення за замовчуванням в `AppSetting`.
5. Створює `~/.studentflow/config.json`.
6. Створює `~/.studentflow/logs/` для журналів.
7. Створює `~/.studentflow/canonical/` (порожня директорія для канонічного репозиторію).

5. (Необов'язково) Завантаження демонстраційних даних

```
python -m app.db.seeds
```

Створює 5 демо-студентів. Безпечно для повторного запуску (ідемпотентна операція).

Запуск застосунку

```
QT_QPA_PLATFORM=xcb python -m app.main
```

Відкривається вікно StudentFlow. Жодного майстра налаштування немає — усе налаштовується через меню.

Перший запуск (порожня база):

1. **Студенти → Додати студента** — додайте першого студента.
2. **Облікові записи → Додати обліковий запис** — прив'яжіть Git-провайдер (GitHub/GitLab тощо).
3. **Облікові дані → Додати облікові дані** — збережіть токен або пароль у системному сховищі.
4. **Репозиторії → Додати репозиторій** — вкажіть локальний шлях і URL.

Розташування локальних даних

Шлях	Вміст
`~/.studentflow/studentflow.db`	SQLite-база (студенти, облікові записи, журнали)
`~/.studentflow/config.json`	Конфігурація (не зберігає секретів)
`~/.studentflow/logs/`	Журнали застосунку (structlog)
`~/.studentflow/canonical/`	Канонічний Git-репозиторій

Системне сховище ключів (keyring)

Усі облікові дані (токени, SSH-паролі тощо) зберігаються **виключно** в системному сховищі — ніколи в базі даних.

Linux — GNOME Keyring

```
# Встановлення (якщо відсутнє):
sudo apt-get install gnome-keyring libsecret-tools

# Перевірка роботи:
python3 -c &quot;import keyring; keyring.set_password(&apos;test&apos;,&apos;u&apos;,&apos;v&apos;); print(k
# Очікується: v
```

Якщо GNOME Keyring не запускається автоматично (headless / мінімальне середовище):

```
eval $(gnome-keyring-daemon --start --components=secrets)
export SSH_AUTH_SOCK
```

KDE: KWallet підтримується автоматично.

Headless Linux (без desktop-сесії):

```
pip install keyrings.alt
export PYTHON_KEYRING_BACKEND=keyrings.alt.file.EncryptedKeyring
```

macOS

Використовується системний Keychain автоматично. Додаткового налаштування не потрібно. При першому запуску macOS може запросити дозвіл — оберіть «Завжди дозволяти».

Windows

Використовується Windows Credential Manager автоматично. Перегляд збережених записів: Панель керування → Credential Manager → Generic Credentials.

Перевірка встановлення

```
QT_QPA_PLATFORM=offscreen .venv/bin/pytest -q --tb=short
# Очікується: 83 passed
```

Оновлення до нової версії

```
# 1. Отримати оновлення коду
git pull origin main

# 2. Оновити залежності
pip install -r requirements.txt

# 3. Застосувати міграції бази даних (якщо є зміни схеми)
python -m app.bootstrap init-db

# 4. Перезапустити застосунок
QT_QPA_PLATFORM=xcb python -m app.main
```

> **Увага:** Ніколи не редагуйте схему SQLite вручну. Alembic керує всіма змінами схеми.

Конфігурація

Файл `~/.studentflow/config.json` містить несекретні налаштування. Редагується вручну (Settings screen у v0.1.0 є заглушкою):

```
{
  &quot;canonical_repo_path&quot;: &quot;/home/operator/.studentflow/canonical&quot;,
  &quot;default_branch&quot;: &quot;main&quot;,
  &quot;sync_timeout_seconds&quot;: 60,
  &quot;log_level&quot;: &quot;INFO&quot;
}
```

Зміни набирають чинності після перезапуску застосунку.

Видалення

```
# Видалити вихідний код і віртуальне середовище
rm -rf /шлях/до/studentflow/

# Видалити дані застосунку (УВАГА: видаляє всіх студентів, журнали, синхронізації)
rm -rf ~/.studentflow/
```

Перед видаленням рекомендується видалити облікові дані через екран Credentials, щоб очистити записи в системному сховищі.

Основні проблеми та рішення

Симптом	Причина	Рішення
`qt.qpa.plugin: Could not load the Qt platform plugin`	Відсутні системні бібліотеки Qt	`sudo apt-get install libxcb-cursor0 libxcb-xinerama0 libegl1`
`keyring.errors.NoKeyringError`	Немає системного сховища ключів	Встановіть GNOME Keyring або `pip install keyrings.alt`
Застосунок зависає під час Git-операцій	Відомо (Phase 1)	Зачекайте, v0.1 блокує UI-потік під час git-операцій. Вирішено буде у Phase 2.
`ModuleNotFoundError: No module named 'app'`	Запуск поза активованим venv або поза коренем репозиторію	`source .venv/bin/activate` і запускайте з кореня репозиторію
База даних не ініціалізована	`init-db` не виконувалась	`python -m app.bootstrap init-db`

Детальніше: [docs/troubleshooting.md](../troubleshooting.md)
---
docs/uk/metodyka-zastosuvannya.md
Розмір: 15,766 байт

Методика застосування StudentFlow

Цей документ описує практичну методику роботи оператора зі StudentFlow для навчального процесу з ~20 студентами, які послідовно працюють над спільним Git-проєктом.

Концепція роботи системи

Що таке канонічний репозиторій

Канонічний репозиторій (`~/.studentflow/canonical/`) — єдине джерело правди для всього проєкту. Кожен студент має власний Git-провайдер (GitHub, GitLab тощо) і локальний клон. Синхронізація завжди відбувається через канонічний репозиторій:

Студент A (remote) ←→ локальний клон A ←→ canonical ←→ локальний клон B ←→ Студент B (remote)

Оператор ніколи не взаємодіє з Git безпосередньо — StudentFlow виконує всі операції автоматично.

Черга студентів

Студенти мають **queue_position** (позицію в черзі). Лише один студент є активним у будь-який момент. Коли активний студент завершує свою частину роботи, виконується **handoff** — передача проєкту наступному за чергою.

Цикл роботи

[Додати студента] → [Додати обліковий запис] → [Зберегти облікові дані]
→ [Додати репозиторій] → [Синхронізація] → [Handoff] → [наступний студент]

Рекомендований порядок початкового налаштування

1. Спочатку — студенти

Перейдіть: **Students → Add Student**

Для кожного студента заповніть:
- **Full Name** — повне ім'я (унікальне в системі)
- **Queue Position** — порядок у черзі (1 = перший активний)
- **Notes** — будь-які нотатки оператора

Почніть зі всіх студентів до того, як переходити до облікових записів — це спрощує прив'язку.

2. Облікові записи

Перейдіть: **Accounts → Add Account**

Для кожного студента:
- **Student** — оберіть зі списку
- **Provider** — GitHub / GitLab / Bitbucket / Gitea / Other
- **Username** — логін студента у провайдера
- **Auth Type** — PAT (рекомендовано для GitHub/GitLab), SSH, App Password

3. Облікові дані

Перейдіть: **Credentials → Add Credential**

- Введіть токен або пароль — він зберігається **виключно** в системному сховищі ключів.
- База даних зберігає лише рядок-посилання (`secret_ref`), але не сам секрет.
- Після збереження натисніть **Validate** в екрані Accounts, щоб перевірити присутність секрету в keyring.

> **Важливо:** Validate у v0.1.0 перевіряє лише присутність секрету в keyring — **не** перевіряє мережевий доступ до провайдера. Мережева перевірка — Phase 2.

4. Репозиторії

Перейдіть: **Repositories → Add Repository**

- **Account** — оберіть обліковий запис студента
- **Repo Name** — назва репозиторію
- **Remote URL** — URL студентського репозиторію (наприклад, `https://github.com/student/project.git`)
- **Local Path** — абсолютний шлях для локального клону (наприклад, `/home/operator/repos/alice/`)
- **Default Branch** — зазвичай `main`

> **Обмеження v0.1.0:** Після збереження репозиторій **не клонується автоматично**. Клонування — ручний крок (Phase 2). Виконайте вручну: `git clone <remote-url> <local-path>`

Управління чергою

Принципи

- Призначайте `queue_position` від 1 (найближчий) до N (останній).
- Студенти з однаковою позицією впорядковуються за іменем.
- Студенти зі статусом `archived` виключаються з черги автоматично.
- Оновити позицію: **Students → Edit Student → Queue Position**.

Рекомендований розклад для 20 студентів

Плануйте час роботи кожного студента перед його активацією. Зміна позиції в черзі не потребує перезапуску — застосунок читає поточний стан при кожному handoff.

Синхронізація (Sync)

Синхронізація підтягує зміни з репозиторію студента в канонічний репозиторій.

Ручна синхронізація (v0.1.0):
- Перейдіть: **Repositories**
- Натисніть **Sync** для потрібного студента

Кожна синхронізація створює запис `SyncJob` у базі. Перегляд результатів: **Logs**.

> **Обмеження v0.1.0:** Auto-sync заглушений — лише ручна синхронізація. Планувальник — Phase 2.

Handoff — передача проєкту

Handoff виконується, коли активний студент завершив свою частину:

1. Переконайтесь, що синхронізація активного студента пройшла успішно.
2. Переконайтесь, що наступний студент у черзі має валідні облікові дані та репозиторій.
3. Виконайте handoff через відповідну кнопку (Dashboard або Logs).

Що відбувається при handoff:
- Виконується фінальна синхронізація поточного студента.
- Створюється тег-чекпоінт у канонічному репозиторії.
- Записується подія `HandoffEvent`.
- Наступний студент стає активним.

> **Обмеження v0.1.0:** `resume_handoff()` заблокований — при спробі відновити перерваний handoff система поверне `HandoffError`. У разі переривання handoff перезапустіть його вручну після усунення причини.

Перевірка журналів

Перейдіть: **Logs**

Журнал відображає всі операції: синхронізації, handoff, помилки. Регулярно перевіряйте:
- Наявність помилок синхронізації (статус `error`).
- Результати handoff (статус `completed` або `failed`).
- Записи `ActivityLog` — усі дії оператора фіксуються.

Безпечне управління обліковими даними

Правила

1. **Ніколи** не записуйте токени або паролі в `config.json`, нотатках студента або будь-якому файлі.
2. Усі секрети вводяться **тільки** через екран Credentials — вони автоматично потрапляють у keyring.
3. Після завершення роботи студента **ротуйте** його облікові дані: **Credentials → Rotate**.
4. Якщо токен скомпрометований — негайно відкличте його на стороні провайдера (GitHub/GitLab Settings), потім ротуйте в StudentFlow.
5. Перед видаленням студента з системи — видаліть його облікові дані через екран Credentials.

Ротація облікових даних

1. **Credentials → Rotate** (або Accounts → Rotate Credential)
2. Введіть новий токен.
3. Натисніть **Validate** — переконайтесь, що новий токен є в keyring.
4. Збережіть.

Стара версія токену автоматично перезаписується в keyring. Секрет ніколи не з'являється в базі даних.

Архівування студента

Коли студент завершив роботу і більше не буде активним:

1. **Students → Archive** для відповідного студента.
2. Статус змінюється на `archived`, студент виключається з черги.
3. Дані (журнали, handoff-події, аудит) зберігаються.
4. Репозиторії на диску залишаються — видаліть вручну, якщо потрібно.
5. Видаліть облікові дані через **Credentials** — очищує запис в keyring.

Доставка промптів студентам (Prompts)

Промпти — це Markdown-файли, які оператор записує прямо в репозиторій активного студента через StudentFlow. Студент вказує шлях до файлу в Lovable замість того, щоб вставляти великий текст у чат.

Як надіслати промпт

1. Перейдіть: **Prompts** (ліва навігаційна панель)
2. Перевірте банер вгорі — там має бути вказаний активний студент, наприклад: `Active student: #01 Аліса Іванченко`
Якщо показано "Active student: none" — спочатку активуйте студента через **Students**.
3. Заповніть поля:
- **Title** (необов'язково) — коротка назва, наприклад `Auth Flow Fix`. Буде частиною імені файлу.
- **Prompt text** (обов'язково) — повний текст промпту.
4. Натисніть **`[Commit & Push]`**.
5. При успіху з'явиться діалог із шляхом до файлу, наприклад:
   prompts/student01-001-auth-flow-fix.md

6. Передайте цей шлях студенту — він вводить його в Lovable як: `@prompts/student01-001-auth-flow-fix.md`

Формат імені файлу

prompts/studentNN-SEQ-slug.md

- `NN` — номер студента з нулем (01, 14)
- `SEQ` — порядковий номер промпту (001, 002, ...)
- `slug` — назва у URL-форматі, макс. 40 символів

Якщо push не вдався

1. Оберіть рядок зі статусом `failed` або `committed` в таблиці.
2. Нижче таблиці з'явиться шлях до файлу (файл вже збережено в локальному репозиторії).
3. Натисніть **`[Retry Push]`**.
4. Якщо знову не вдалося — перевірте облікові дані (**Credentials → Validate / Rotate**), потім повторіть.

Статуси промптів

Статус	Значення
pushed	Файл у репозиторії студента. Шлях можна передавати.
committed	Зафіксовано локально, push не вдався. Використайте Retry Push.
failed	Помилка запису або фіксації. Перевірте Logs.

Обмеження v0.1.0-mvp і Phase 2

Обмеження	Стан	Phase 2
Dashboard	Показує статус канонічного репозиторію та кнопку оновлення	Повна реалізація з чергою, health panel
Settings screen	Редагована форма: canonical path, default branch, sync timeout, retries	Розширені налаштування Phase 2
Auto-sync	Вимкнено — лише ручна синхронізація	Планувальник (APScheduler)
Блокування UI	Git-операції блокують UI-потік	QThread-воркери
`resume_handoff()`	Повертає `HandoffError`	Повна реалізація
Клонування репозиторію	Ручний крок після додавання	Автоматичне клонування
Перевірка облікових даних	Перевіряє лише keyring (не мережу)	Мережева перевірка
Блокування UI при push промпту	Відомо — push блокує UI-потік	QThread-воркери Phase 2

Рекомендована операційна дисципліна для ~20 студентів

На початку проєкту

- [ ] Додати всіх студентів з правильними `queue_position`.
- [ ] Додати облікові записи та облікові дані для всіх студентів.
- [ ] Перевірити облікові дані через **Validate** для кожного.
- [ ] Додати репозиторії та клонувати їх вручну (`git clone`).
- [ ] Переконатись, що канонічний репозиторій ініціалізований.

Перед кожним handoff

- [ ] Запустити ручну синхронізацію активного студента.
- [ ] Перевірити Logs — немає помилок.
- [ ] Перевірити, що наступний студент має валідні дані.
- [ ] Виконати handoff.
- [ ] Перевірити Logs — подія handoff зафіксована зі статусом `completed`.

Регулярно

- [ ] Перевіряти журнали на помилки.
- [ ] Ротувати облікові дані студентів за потреби.
- [ ] Архівувати студентів, що завершили роботу.

Чого не робити

- Не редагуйте SQLite-базу вручну (`~/.studentflow/studentflow.db`).
- Не зберігайте токени або паролі поза keyring.
- Не запускайте кілька екземплярів StudentFlow одночасно.
- Не видаляйте `~/.studentflow/canonical/` без резервної копії.
- Не ігноруйте помилки в Logs — вони вказують на реальні проблеми.

---

docs/plans/2026-03-17-canonical-repo-ui.md
Розмір: 26,123 байт

Canonical Repository UI Implementation Plan

> **For Claude:** REQUIRED SUB-SKILL: Use superpowers:executing-plans to implement this plan task-by-task.

Goal: Make the canonical repository configuration visible and editable in the UI so an operator can discover, view, and change which repository is canonical without touching config files.

Architecture: Three targeted UI changes — Settings screen (edit AppSettings), Dashboard screen (show canonical repo status), Repositories screen (show canonical badge + Set as Canonical action). All canonical data is already in the DB (`Repository.is_canonical`, `AppSetting["canonical_repo_path"]`); we only add UI to expose it. No schema changes needed.

Tech Stack: Python 3.11, PySide6 (Qt6), SQLAlchemy, pytest, pytest-qt

---

Diagnosis (do not implement — for reference only)

- `Repository.is_canonical` field exists (`app/db/models.py:111`)
- `AppSetting["canonical_repo_path"]` seeded on bootstrap (`app/bootstrap.py:79`)
- `RepositoryService.get_canonical_repo()` exists (`app/services/repository_service.py:184`)
- `_get_canonical_path()` used by 3 services (`repository_service.py:31`, `handoff_service.py:252`, `sync_service.py`)
- `SettingsScreen` and `DashboardScreen` are pure placeholders
- `RepositoriesScreen` shows repos but has no canonical indicator

Task 1: SettingsViewModel

Files:
- Create: `app/ui/viewmodels/settings_viewmodel.py`
- Test: `tests/test_settings_viewmodel.py`

Step 1: Write the failing test

```
# tests/test_settings_viewmodel.py
import pytest
from sqlalchemy.orm import sessionmaker
from app.db.models import AppSetting
from app.ui.viewmodels.settings_viewmodel import SettingsViewModel


def test_get_setting_returns_value(db_session):
    db_session.add(AppSetting(key=&quot;canonical_repo_path&quot;, value=&quot;/tmp/canon&quot;, descriptio
    db_session.commit()
    factory = sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)
    vm = SettingsViewModel(session_factory=factory)
    assert vm.get_setting(&quot;canonical_repo_path&quot;) == &quot;/tmp/canon&quot;


def test_get_setting_missing_returns_none(db_session):
    factory = sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)
    vm = SettingsViewModel(session_factory=factory)
    assert vm.get_setting(&quot;nonexistent&quot;) is None
def test_set_setting_creates_and_updates(db_session):
    db_session.add(AppSetting(key=&quot;canonical_repo_path&quot;, value=&quot;/old&quot;, description=&quo
```

```
    db_session.commit()
    factory = sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)
    vm = SettingsViewModel(session_factory=factory)
    vm.set_setting(&quot;canonical_repo_path&quot;, &quot;/new&quot;)
    assert vm.get_setting(&quot;canonical_repo_path&quot;) == &quot;/new&quot;


def test_set_setting_rejects_empty_string(db_session):
    factory = sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)
    vm = SettingsViewModel(session_factory=factory)
    with pytest.raises(ValueError, match=&quot;cannot be empty&quot;):
        vm.set_setting(&quot;canonical_repo_path&quot;, &quot;&quot;)


def test_list_operator_settings_returns_expected_keys(db_session):
    for key, val in [
        (&quot;canonical_repo_path&quot;, &quot;/tmp/c&quot;),
        (&quot;default_branch&quot;, &quot;main&quot;),
        (&quot;sync_timeout_seconds&quot;, &quot;60&quot;),
    ]:
        db_session.add(AppSetting(key=key, value=val, description=&quot;&quot;))
    db_session.commit()
    factory = sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)
    vm = SettingsViewModel(session_factory=factory)
    result = vm.list_operator_settings()
    keys = [r[&quot;key&quot;] for r in result]
    assert &quot;canonical_repo_path&quot; in keys
    assert &quot;default_branch&quot; in keys
    assert &quot;sync_timeout_seconds&quot; in keys
```

Step 2: Run test to verify it fails

```
cd /home/vokov/projects/studentflow
python -m pytest tests/test_settings_viewmodel.py -v 2&gt;&amp;1 | head -20
```

Expected: `ModuleNotFoundError: No module named 'app.ui.viewmodels.settings_viewmodel'`

Step 3: Write minimal implementation

```
# app/ui/viewmodels/settings_viewmodel.py
&quot;&quot;&quot;ViewModel for SettingsScreen — read/write AppSetting records.&quot;&quot;&quot;
from __future__ import annotations

from app.db.base import SessionLocal
from app.db.models import AppSetting

# Keys exposed to the operator in the Settings UI
OPERATOR_SETTINGS = [
    &quot;canonical_repo_path&quot;,
    &quot;default_branch&quot;,
    &quot;sync_timeout_seconds&quot;,
    &quot;max_sync_retries&quot;,
]


class SettingsViewModel:
    def __init__(self, session_factory=None):
        self._factory = session_factory or SessionLocal

    def get_setting(self, key: str) -&gt; str | None:
        with self._factory() as session:
            row = session.get(AppSetting, key)
            return row.value if row is not None else None

    def set_setting(self, key: str, value: str) -&gt; None:
        if not value or not value.strip():
            raise ValueError(f&quot;Setting &apos;{key}&apos; cannot be empty&quot;)
```

```
        with self._factory() as session:
            row = session.get(AppSetting, key)
            if row is None:
                row = AppSetting(key=key, value=value.strip(), description=&quot;&quot;)
                session.add(row)
            else:
                row.value = value.strip()
            session.commit()

    def list_operator_settings(self) -&gt; list[dict]:
        &quot;&quot;&quot;Return operator-visible settings as list of {key, value, description}.&quot;&quot;&quot;
        with self._factory() as session:
            rows = session.query(AppSetting).filter(
                AppSetting.key.in_(OPERATOR_SETTINGS)
            ).order_by(AppSetting.key).all()
            return [
                {&quot;key&quot;: r.key, &quot;value&quot;: r.value, &quot;description&quot;: r.description or &quot;&
                for r in rows
            ]
```

Step 4: Run test to verify it passes

```
cd /home/vokov/projects/studentflow
python -m pytest tests/test_settings_viewmodel.py -v
```

Expected: 5 tests PASS

Step 5: Commit

```
git add app/ui/viewmodels/settings_viewmodel.py tests/test_settings_viewmodel.py
git commit -m &quot;feat: add SettingsViewModel for AppSetting read/write&quot;
```

Task 2: Settings Screen Implementation

Files:
- Modify: `app/ui/windows/settings_screen.py`
- No new test file needed (UI widget; tested via existing `test_ui_skeleton.py` + manual smoke)

Step 1: Read the current placeholder

Current content of `app/ui/windows/settings_screen.py` is a pure placeholder with a label "Settings content - not implemented yet". Replace it entirely.

Step 2: Write the implementation

```
# app/ui/windows/settings_screen.py
&quot;&quot;&quot;Settings screen — operator-visible AppSetting configuration.&quot;&quot;&quot;
from __future__ import annotations

from PySide6.QtWidgets import (
    QFormLayout,
    QGroupBox,
    QHBoxLayout,
    QLabel,
    QLineEdit,
    QMessageBox,
    QPushButton,
    QVBoxLayout,
    QWidget,
)

from app.ui.viewmodels.settings_viewmodel import SettingsViewModel

_LABELS = {
```

```
    &quot;canonical_repo_path&quot;: &quot;Canonical repo path&quot;,
    &quot;default_branch&quot;: &quot;Default branch&quot;,
    &quot;sync_timeout_seconds&quot;: &quot;Sync timeout (seconds)&quot;,
    &quot;max_sync_retries&quot;: &quot;Max sync retries&quot;,
}


class SettingsScreen(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._vm = SettingsViewModel()
        self._fields: dict[str, QLineEdit] = {}
        self._build_ui()
        self._load()

    def _build_ui(self) -&gt; None:
        layout = QVBoxLayout(self)
        layout.setContentsMargins(24, 24, 24, 24)
        layout.setSpacing(16)

        title = QLabel(&quot;Settings&quot;, self)
        title.setStyleSheet(&quot;font-size: 20px; font-weight: bold;&quot;)
        layout.addWidget(title)

        group = QGroupBox(&quot;Canonical Repository &amp; Sync&quot;, self)
        form = QFormLayout(group)
        form.setLabelAlignment(form.labelAlignment())
        form.setRowWrapPolicy(QFormLayout.RowWrapPolicy.WrapLongRows)

        for key in [&quot;canonical_repo_path&quot;, &quot;default_branch&quot;, &quot;sync_timeout_seconds&quo
            label_text = _LABELS.get(key, key)
            field = QLineEdit(self)
            field.setObjectName(key)
            form.addRow(label_text + &quot;:&quot;, field)
            self._fields[key] = field

        layout.addWidget(group)

        btn_row = QWidget(self)
        btn_layout = QHBoxLayout(btn_row)
        btn_layout.setContentsMargins(0, 0, 0, 0)
        save_btn = QPushButton(&quot;Save&quot;, btn_row)
        save_btn.clicked.connect(self._on_save)
        btn_layout.addWidget(save_btn)
        reset_btn = QPushButton(&quot;Reset to saved&quot;, btn_row)
        reset_btn.clicked.connect(self._load)
        btn_layout.addWidget(reset_btn)
        btn_layout.addStretch(1)
        layout.addWidget(btn_row)

        layout.addStretch(1)

    def _load(self) -&gt; None:
        try:
            rows = {r[&quot;key&quot;]: r[&quot;value&quot;] for r in self._vm.list_operator_settings()}
            for key, field in self._fields.items():
                field.setText(rows.get(key, &quot;&quot;))
        except Exception as e:
            QMessageBox.critical(self, &quot;Error&quot;, str(e))

    def _on_save(self) -&gt; None:
        errors = []
        for key, field in self._fields.items():
            value = field.text().strip()
            if not value:
                errors.append(f&quot;&apos;{_LABELS.get(key, key)}&apos; cannot be empty&quot;)
        if errors:
```

```
            QMessageBox.warning(self, &quot;Validation&quot;, &quot;\n&quot;.join(errors))
            return
        try:
            for key, field in self._fields.items():
                self._vm.set_setting(key, field.text().strip())
            QMessageBox.information(self, &quot;Saved&quot;, &quot;Settings saved successfully.&quot;)
        except Exception as e:
            QMessageBox.critical(self, &quot;Error&quot;, str(e))
```

Step 3: Run UI skeleton tests to verify no regression

```
cd /home/vokov/projects/studentflow
python -m pytest tests/test_ui_skeleton.py -v
```

Expected: all tests PASS (Settings screen still navigable)

Step 4: Commit

```
git add app/ui/windows/settings_screen.py
git commit -m &quot;feat: implement Settings screen with canonical repo and sync configuration&quot;
```

Task 3: DashboardViewModel

Files:
- Create: `app/ui/viewmodels/dashboard_viewmodel.py`
- Test: `tests/test_dashboard_viewmodel.py`

Step 1: Write the failing test

```
# tests/test_dashboard_viewmodel.py
import pytest
from pathlib import Path
from sqlalchemy.orm import sessionmaker
from app.db.models import AppSetting, Repository, Account, Student
from app.ui.viewmodels.dashboard_viewmodel import DashboardViewModel


def _make_factory(db_session):
    return sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)


def test_get_canonical_summary_no_setting(db_session):
    vm = DashboardViewModel(session_factory=_make_factory(db_session))
    summary = vm.get_canonical_summary()
    assert summary[&quot;path&quot;] is None or isinstance(summary[&quot;path&quot;], str)
    assert &quot;path_valid&quot; in summary
    assert &quot;canonical_repo&quot; in summary


def test_get_canonical_summary_with_setting(db_session, tmp_path):
    db_session.add(AppSetting(key=&quot;canonical_repo_path&quot;, value=str(tmp_path), description=&quot;x&
    db_session.commit()
    # Create a git marker so validate returns True
    (tmp_path / &quot;HEAD&quot;).write_text(&quot;ref: refs/heads/main\n&quot;)
    vm = DashboardViewModel(session_factory=_make_factory(db_session))
    summary = vm.get_canonical_summary()
    assert summary[&quot;path&quot;] == str(tmp_path)
    assert summary[&quot;path_valid&quot;] is True


def test_get_canonical_summary_invalid_path(db_session, tmp_path):
    db_session.add(AppSetting(key=&quot;canonical_repo_path&quot;, value=&quot;/nonexistent/path&quot;, desc
    db_session.commit()
```

```
    vm = DashboardViewModel(session_factory=_make_factory(db_session))
    summary = vm.get_canonical_summary()
    assert summary[&quot;path_valid&quot;] is False


def test_get_canonical_summary_includes_repo_record(db_session, tmp_path):
    student = Student(full_name=&quot;Test Student&quot;)
    db_session.add(student)
    db_session.flush()
    account = Account(student_id=student.id, provider=&quot;github&quot;, username=&quot;op&quot;, auth_typ
    db_session.add(account)
    db_session.flush()
    repo = Repository(
        account_id=account.id,
        repo_name=&quot;canonical-repo&quot;,
        remote_url=&quot;https://github.com/op/project.git&quot;,
        local_path=str(tmp_path),
        default_branch=&quot;main&quot;,
        is_canonical=True,
    )
    db_session.add(repo)
    db_session.add(AppSetting(key=&quot;canonical_repo_path&quot;, value=str(tmp_path), description=&quot;x&
    db_session.commit()
    vm = DashboardViewModel(session_factory=_make_factory(db_session))
    summary = vm.get_canonical_summary()
    assert summary[&quot;canonical_repo&quot;] is not None
    assert summary[&quot;canonical_repo&quot;].repo_name == &quot;canonical-repo&quot;
```

Step 2: Run test to verify it fails

```
cd /home/vokov/projects/studentflow
python -m pytest tests/test_dashboard_viewmodel.py -v 2&gt;&amp;1 | head -20
```

Expected: `ModuleNotFoundError`

Step 3: Write minimal implementation

```
# app/ui/viewmodels/dashboard_viewmodel.py
&quot;&quot;&quot;ViewModel for DashboardScreen — canonical repo summary.&quot;&quot;&quot;
from __future__ import annotations

from pathlib import Path

from app.db.base import SessionLocal
from app.db.models import AppSetting, Repository


class DashboardViewModel:
    def __init__(self, session_factory=None):
        self._factory = session_factory or SessionLocal

    def get_canonical_summary(self) -&gt; dict:
        &quot;&quot;&quot;Return a dict with keys: path, path_valid, canonical_repo.&quot;&quot;&quot;
        with self._factory() as session:
            setting = session.get(AppSetting, &quot;canonical_repo_path&quot;)
            path = setting.value if setting is not None else None

            path_valid = False
            if path:
                p = Path(path)
                path_valid = p.exists() and ((p / &quot;.git&quot;).exists() or (p / &quot;HEAD&quot;).exists())

            canonical_repo = (
                session.query(Repository).filter_by(is_canonical=True).first()
            )
            if canonical_repo is not None:
                session.expunge(canonical_repo)
```

```
        return {
            &quot;path&quot;: path,
            &quot;path_valid&quot;: path_valid,
            &quot;canonical_repo&quot;: canonical_repo,
        }
```

Step 4: Run test to verify it passes

```
cd /home/vokov/projects/studentflow
python -m pytest tests/test_dashboard_viewmodel.py -v
```

Expected: 4 tests PASS

Step 5: Commit

```
git add app/ui/viewmodels/dashboard_viewmodel.py tests/test_dashboard_viewmodel.py
git commit -m &quot;feat: add DashboardViewModel for canonical repo summary&quot;
```

Task 4: Dashboard Screen Implementation

Files:
- Modify: `app/ui/windows/dashboard_screen.py`

Step 1: Write the implementation

```
# app/ui/windows/dashboard_screen.py
&quot;&quot;&quot;Dashboard screen — canonical repository status at a glance.&quot;&quot;&quot;
from __future__ import annotations

from PySide6.QtWidgets import (
    QFormLayout,
    QGroupBox,
    QLabel,
    QPushButton,
    QVBoxLayout,
    QWidget,
)

from app.ui.viewmodels.dashboard_viewmodel import DashboardViewModel


class DashboardScreen(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._vm = DashboardViewModel()
        self._build_ui()
        self.refresh()

    def _build_ui(self) -&gt; None:
        layout = QVBoxLayout(self)
        layout.setContentsMargins(24, 24, 24, 24)
        layout.setSpacing(16)

        title = QLabel(&quot;Dashboard&quot;, self)
        title.setStyleSheet(&quot;font-size: 20px; font-weight: bold;&quot;)
        layout.addWidget(title)

        group = QGroupBox(&quot;Canonical Repository&quot;, self)
        self._form = QFormLayout(group)
        self._path_label = QLabel(&quot;—&quot;, self)
        self._path_label.setWordWrap(True)
        self._form.addRow(&quot;Path:&quot;, self._path_label)
        self._valid_label = QLabel(&quot;—&quot;, self)
```

```
        self._form.addRow(&quot;Path accessible:&quot;, self._valid_label)

        self._repo_label = QLabel(&quot;—&quot;, self)
        self._form.addRow(&quot;Registered as:&quot;, self._repo_label)

        self._branch_label = QLabel(&quot;—&quot;, self)
        self._form.addRow(&quot;Branch:&quot;, self._branch_label)

        self._commit_label = QLabel(&quot;—&quot;, self)
        self._form.addRow(&quot;Last commit:&quot;, self._commit_label)

        layout.addWidget(group)

        refresh_btn = QPushButton(&quot;Refresh&quot;, self)
        refresh_btn.clicked.connect(self.refresh)
        layout.addWidget(refresh_btn)

        layout.addStretch(1)

    def refresh(self) -&gt; None:
        try:
            summary = self._vm.get_canonical_summary()
        except Exception as e:
            self._path_label.setText(f&quot;Error: {e}&quot;)
            return

        path = summary.get(&quot;path&quot;) or &quot;(not configured)&quot;
        self._path_label.setText(path)

        valid = summary.get(&quot;path_valid&quot;, False)
        if valid:
            self._valid_label.setText(&quot;Yes&quot;)
            self._valid_label.setStyleSheet(&quot;color: #a6e3a1;&quot;)
        else:
            self._valid_label.setText(&quot;No — path does not exist or is not a git repo&quot;)
            self._valid_label.setStyleSheet(&quot;color: #f38ba8;&quot;)

        repo = summary.get(&quot;canonical_repo&quot;)
        if repo is not None:
            self._repo_label.setText(repo.repo_name)
            self._branch_label.setText(repo.default_branch or &quot;—&quot;)
            self._commit_label.setText((repo.last_commit_hash or &quot;&quot;)[:8] or &quot;—&quot;)
        else:
            self._repo_label.setText(&quot;(no canonical repo record in database)&quot;)
            self._repo_label.setStyleSheet(&quot;color: #f9e2af;&quot;)
            self._branch_label.setText(&quot;—&quot;)
            self._commit_label.setText(&quot;—&quot;)
```

Step 2: Run UI skeleton tests to verify no regression

```
cd /home/vokov/projects/studentflow
python -m pytest tests/test_ui_skeleton.py -v
```

Expected: PASS

Step 3: Commit

```
git add app/ui/windows/dashboard_screen.py
git commit -m &quot;feat: implement Dashboard screen with canonical repo status&quot;
```

Task 5: RepoViewModel — set_canonical method

Files:
- Modify: `app/ui/viewmodels/repos_viewmodel.py`
- Test: `tests/test_repos_viewmodel.py` (create new)

Step 1: Write the failing test

```
# tests/test_repos_viewmodel.py
import pytest
from sqlalchemy.orm import sessionmaker
from app.db.models import Account, Repository, Student
from app.db.enums import SyncStatus
from app.ui.viewmodels.repos_viewmodel import RepoViewModel


def _make_factory(db_session):
    return sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)


def _add_repo(db_session, name: str, is_canonical: bool = False) -&gt; Repository:
    student = Student(full_name=name)
    db_session.add(student)
    db_session.flush()
    account = Account(student_id=student.id, provider=&quot;github&quot;, username=name, auth_type=&quot;p
    db_session.add(account)
    db_session.flush()
    repo = Repository(
        account_id=account.id,
        repo_name=name,
        remote_url=f&quot;https://github.com/{name}/project.git&quot;,
        local_path=f&quot;/tmp/{name}&quot;,
        default_branch=&quot;main&quot;,
        sync_status=SyncStatus.clean.value,
        is_canonical=is_canonical,
    )
    db_session.add(repo)
    db_session.commit()
    return repo


def test_set_canonical_marks_repo(db_session):
    repo = _add_repo(db_session, &quot;alice&quot;)
    factory = _make_factory(db_session)
    vm = RepoViewModel(session_factory=factory)
    vm.set_canonical(repo.id)
    updated = db_session.get(Repository, repo.id)
    assert updated.is_canonical is True


def test_set_canonical_clears_previous_canonical(db_session):
    old = _add_repo(db_session, &quot;old&quot;, is_canonical=True)
    new = _add_repo(db_session, &quot;new&quot;, is_canonical=False)
    factory = _make_factory(db_session)
    vm = RepoViewModel(session_factory=factory)
    vm.set_canonical(new.id)
    db_session.expire_all()
    assert db_session.get(Repository, old.id).is_canonical is False
    assert db_session.get(Repository, new.id).is_canonical is True


def test_set_canonical_unknown_id_raises(db_session):
    factory = _make_factory(db_session)
    vm = RepoViewModel(session_factory=factory)
    with pytest.raises(ValueError, match=&quot;not found&quot;):
        vm.set_canonical(&quot;nonexistent-id&quot;)
```

Step 2: Run test to verify it fails

```
cd /home/vokov/projects/studentflow
python -m pytest tests/test_repos_viewmodel.py -v 2&gt;&amp;1 | head -20
```

Expected: `AttributeError: 'RepoViewModel' object has no attribute 'set_canonical'`

Step 3: Add set_canonical to RepoViewModel

Add after the `list_accounts` method in `app/ui/viewmodels/repos_viewmodel.py`:

```
    def set_canonical(self, repo_id: str) -&gt; None:
        &quot;&quot;&quot;Mark repo_id as canonical, clearing any previous canonical flag.&quot;&quot;&quot;
        with self._factory() as session:
            repo = session.get(Repository, repo_id)
            if repo is None:
                raise ValueError(f&quot;Repository {repo_id} not found&quot;)
            # Clear existing canonical flag
            session.query(Repository).filter(
                Repository.is_canonical == True  # noqa: E712
            ).update({&quot;is_canonical&quot;: False})
            repo.is_canonical = True
            session.commit()
```

Also update the constructor to accept `session_factory`:

```
class RepoViewModel:
    def __init__(self, session_factory=None):
        self._factory = session_factory or SessionLocal
        self._service = RepositoryService(session_factory=self._factory)
```

Step 4: Run test to verify it passes

```
cd /home/vokov/projects/studentflow
python -m pytest tests/test_repos_viewmodel.py -v
```

Expected: 3 tests PASS

Step 5: Commit

```
git add app/ui/viewmodels/repos_viewmodel.py tests/test_repos_viewmodel.py
git commit -m &quot;feat: add set_canonical() to RepoViewModel&quot;
```

Task 6: Repositories Screen — Canonical Indicator + Set as Canonical Action

Files:
- Modify: `app/ui/windows/repositories_screen.py`

Step 1: Modify the Repositories screen

Changes:
1. In `_fill_table`: prefix repo name with `★ ` if `r.is_canonical` is True
2. In the actions cell: add a "Set as Canonical" button (disabled if already canonical)

The full updated `_fill_table` and action cell section:

In `_fill_table`, replace the first `setItem` line and the actions cell widget block:

```
    def _fill_table(self) -&gt; None:
        repos = self._repos
        self._table.setVisible(bool(repos))
        self._empty.setVisible(not bool(repos))
        self._table.setRowCount(0)
        for row, r in enumerate(repos):
            self._table.insertRow(row)
```

```
        # Repo name with canonical indicator
        name_text = f&quot;★ {r.repo_name}&quot; if getattr(r, &quot;is_canonical&quot;, False) else r.repo_nam
        name_item = QTableWidgetItem(name_text)
        if getattr(r, &quot;is_canonical&quot;, False):
            name_item.setForeground(QColor(&quot;#cba6f7&quot;))  # purple highlight
        self._table.setItem(row, 0, name_item)

        self._table.setItem(row, 1, QTableWidgetItem(r.local_path))
        self._table.setItem(row, 2, QTableWidgetItem(r.remote_url))
        self._table.setItem(row, 3, QTableWidgetItem(r.default_branch))

        sync_item = QTableWidgetItem(r.sync_status)
        sync_item.setForeground(QColor(_SYNC_COLORS.get(r.sync_status, &quot;#cdd6f4&quot;)))
        self._table.setItem(row, 4, sync_item)

        self._table.setItem(row, 5, QTableWidgetItem((r.last_commit_hash or &quot;&quot;)[:8]))

        cell = QWidget()
        cl = QHBoxLayout(cell)
        cl.setContentsMargins(2, 1, 2, 1)
        cl.setSpacing(4)

        vl_btn = QPushButton(&quot;Validate Path&quot;)
        vl_btn.clicked.connect(lambda _=False, path=r.local_path: self._validate_path(path))
        cl.addWidget(vl_btn)

        vr_btn = QPushButton(&quot;Validate Remote&quot;)
        vr_btn.clicked.connect(lambda _=False, rid=r.id: self._validate_remote(rid))
        cl.addWidget(vr_btn)

        fm_btn = QPushButton(&quot;Open Folder&quot;)
        fm_btn.clicked.connect(lambda _=False, path=r.local_path: self._open_folder(path))
        cl.addWidget(fm_btn)

        sc_btn = QPushButton(&quot;Set as Canonical&quot;)
        sc_btn.setEnabled(not getattr(r, &quot;is_canonical&quot;, False))
        sc_btn.clicked.connect(lambda _=False, rid=r.id: self._set_canonical(rid))
        cl.addWidget(sc_btn)

        self._table.setCellWidget(row, 6, cell)
```

Add this new method to the class:

```
    def _set_canonical(self, repo_id: str) -&gt; None:
        try:
            self._vm.set_canonical(repo_id)
            self.refresh()
        except Exception as e:
            QMessageBox.critical(self, &quot;Error&quot;, str(e))
```

Step 2: Run all tests to verify no regressions

```
cd /home/vokov/projects/studentflow
python -m pytest tests/ -v --tb=short 2&gt;&amp;1 | tail -30
```

Expected: all existing tests PASS, new tests PASS

Step 3: Commit

```
git add app/ui/windows/repositories_screen.py
git commit -m &quot;feat: show canonical indicator and Set as Canonical button in Repositories screen&quot;
```

Task 7: Full Test Suite Run + Regression Check

Step 1: Run full test suite

cd /home/vokov/projects/studentflow
python -m pytest tests/ -v 2>&1 | tail -40

Expected: All tests pass. Count should be >= prior 80 tests + new tests added in Tasks 1, 3, 5.

Step 2: If any failures, diagnose and fix before proceeding

Do not mark complete until all tests pass.

Acceptance Criteria

1. `python -m pytest tests/ -v` — all tests pass
2. Settings screen shows and saves `canonical_repo_path`, `default_branch`, `sync_timeout_seconds`, `max_sync_retries`
3. Dashboard screen shows canonical repo path, whether it is accessible, and the canonical repo name/branch/commit if a `Repository.is_canonical=True` record exists
4. Repositories screen marks canonical repo with `★` prefix and purple text
5. "Set as Canonical" button is disabled on the current canonical repo and enabled on all others
6. Clicking "Set as Canonical" on a non-canonical repo updates the DB and refreshes the table
7. No changes to sync/handoff logic (services are untouched)
8. No schema migrations needed
---
docs/plans/2026-03-17-prompt-delivery.md
Розмір: 63,369 байт

Prompt Delivery Implementation Plan

> **For Claude:** REQUIRED SUB-SKILL: Use superpowers:executing-plans to implement this plan task-by-task.

Goal: Add a prompt-management workflow so the operator can create, commit, and push prompts into the active student's git repository for Lovable-style reference.

Architecture: Six tasks in order: GitClient extension → DB model + migration → PromptService → ViewModel → UI screen → Docs. Each task is independently testable and committable. The Prompt entity stores full history; push happens to the `"student"` remote (GitHub/GitLab) using existing credential infrastructure.

Tech Stack: Python 3.11, PySide6 6.x, SQLAlchemy, Alembic, GitPython, pytest, structlog

---

Architecture Notes (read before implementing)

- **Active student query**: `Student.status == "active"`, order by `queue_position asc, created_at asc`, `.first()`
- **Student → Repo**: Student → Account (prefer `is_current=True`, fallback to first) → Repository (where `is_canonical=False`)
- **Push remote**: `"student"` (not `"origin"`) for GitHub/GitLab
- **GitClient**: uses `_run_command(["git", "-C", repo_path, ...])` internally. No `add_file` or `commit` methods yet.
- **AuditService.log signature**: `log(action, entity_type, entity_id=None, actor="operator", details=None)`
- **AuditService._VALID_ENTITY_TYPES**: hardcoded set at line 23 — must add `"prompt"` there
- **Enum pattern**: `class XStatus(str, enum.Enum): value = "value"`
- **NAV_ITEMS** currently has 7 items (indices 0–6); Prompts goes at index 7

- **conftest.py**: provides `db_session`, `session_factory`, `mock_git_client`, `mock_audit` fixtures
- **Migration**: only one exists (`7ca526e3ce7e_initial_schema.py`). New migration chains off it.
- **Student.student_number**: does NOT exist yet — must be added (stable operator-facing number)

Task 1: GitClient — add_file() and commit()

Files:
- Modify: `app/git/client.py`
- Test: `tests/test_git_client.py`

Step 1: Write the failing tests

```
# Add to tests/test_git_client.py

def test_add_file_stages_specific_file(tmp_path):
    &quot;&quot;&quot;add_file stages exactly one file, not the whole tree.&quot;&quot;&quot;
    from app.git.client import GitClient
    import subprocess
    # Init a real git repo
    subprocess.run([&quot;git&quot;, &quot;init&quot;, str(tmp_path)], check=True, capture_output=True)
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;config&quot;, &quot;user.email&quot;,
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;config&quot;, &quot;user.name&quot;,
    # Create initial commit so HEAD exists
    (tmp_path / &quot;readme.txt&quot;).write_text(&quot;hello&quot;)
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;add&quot;, &quot;.&quot;], check=True
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;commit&quot;, &quot;-m&quot;, &quot;
    # Create two files: one to stage, one to leave unstaged
    (tmp_path / &quot;prompts&quot;).mkdir()
    (tmp_path / &quot;prompts&quot; / &quot;student01-001-test.md&quot;).write_text(&quot;# Test&quot;)
    (tmp_path / &quot;dirty.txt&quot;).write_text(&quot;should not be staged&quot;)

    client = GitClient()
    client.add_file(str(tmp_path), &quot;prompts/student01-001-test.md&quot;)

    result = subprocess.run(
        [&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;diff&quot;, &quot;--cached&quot;, &quot;--name-only
        capture_output=True, text=True, check=True
    )
    staged = result.stdout.strip().splitlines()
    assert staged == [&quot;prompts/student01-001-test.md&quot;]
    # dirty.txt must NOT be staged
    result2 = subprocess.run(
        [&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;status&quot;, &quot;--porcelain&quot;],
        capture_output=True, text=True, check=True
    )
    assert &quot;dirty.txt&quot; in result2.stdout
    assert result2.stdout.count(&quot;??&quot;) &gt;= 1  # untracked


def test_commit_returns_hash_and_creates_commit(tmp_path):
    &quot;&quot;&quot;commit() commits staged files and returns 40-char SHA1.&quot;&quot;&quot;
    from app.git.client import GitClient
    import subprocess
    subprocess.run([&quot;git&quot;, &quot;init&quot;, str(tmp_path)], check=True, capture_output=True)
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;config&quot;, &quot;user.email&quot;,
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;config&quot;, &quot;user.name&quot;,
    (tmp_path / &quot;readme.txt&quot;).write_text(&quot;hello&quot;)
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;add&quot;, &quot;.&quot;], check=True
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;commit&quot;, &quot;-m&quot;, &quot;

    (tmp_path / &quot;prompts&quot;).mkdir()
    (tmp_path / &quot;prompts&quot; / &quot;f.md&quot;).write_text(&quot;# Hello&quot;)
```

```
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;add&quot;, &quot;prompts/f.md&quot;]

    client = GitClient()
    commit_hash = client.commit(str(tmp_path), &quot;prompt: student01-001-test&quot;)

    assert len(commit_hash) == 40
    assert all(c in &quot;0123456789abcdef&quot; for c in commit_hash)

    # Verify the commit message
    log = subprocess.run(
        [&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;log&quot;, &quot;--oneline&quot;, &quot;-1&quot;],
        capture_output=True, text=True, check=True
    )
    assert &quot;student01-001-test&quot; in log.stdout


def test_add_file_raises_on_missing_file(tmp_path):
    &quot;&quot;&quot;add_file raises GitOperationError for a non-existent path.&quot;&quot;&quot;
    from app.git.client import GitClient, GitOperationError
    import subprocess
    subprocess.run([&quot;git&quot;, &quot;init&quot;, str(tmp_path)], check=True, capture_output=True)
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;config&quot;, &quot;user.email&quot;,
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;config&quot;, &quot;user.name&quot;,
    (tmp_path / &quot;readme.txt&quot;).write_text(&quot;hello&quot;)
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;add&quot;, &quot;.&quot;], check=True
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;commit&quot;, &quot;-m&quot;, &quot;

    client = GitClient()
    with pytest.raises(GitOperationError):
        client.add_file(str(tmp_path), &quot;nonexistent/file.md&quot;)
```

Step 2: Run to verify it fails

```
cd /home/vokov/projects/studentflow
.venv/bin/pytest tests/test_git_client.py::test_add_file_stages_specific_file -v
```

Expected: `AttributeError: 'GitClient' object has no attribute 'add_file'`

Step 3: Add methods to GitClient

Add after `get_log()` method (before `_make_netrc_env`), at approximately line 233:

```
def add_file(self, repo_path: str, file_path: str) -&gt; None:
    &quot;&quot;&quot;Stage a single file by relative path. Raises GitOperationError if file doesn&apos;t exist.&quot
    self._run_command([&quot;git&quot;, &quot;-C&quot;, repo_path, &quot;add&quot;, &quot;--&quot;, file_path])

def commit(self, repo_path: str, message: str) -&gt; str:
    &quot;&quot;&quot;Commit currently staged changes. Returns the 40-char commit hash.&quot;&quot;&quot;
    self._run_command([&quot;git&quot;, &quot;-C&quot;, repo_path, &quot;commit&quot;, &quot;-m&quot;, messa
    result = self._run_command([&quot;git&quot;, &quot;-C&quot;, repo_path, &quot;rev-parse&quot;, &quot;HEAD
    return result.stdout.strip()
```

Step 4: Run tests to verify they pass

```
.venv/bin/pytest tests/test_git_client.py::test_add_file_stages_specific_file tests/test_git_client.py::test_commit_retu
```

Expected: 3 PASS

Step 5: Run full suite to check no regressions

```
.venv/bin/pytest tests/ -q
```

Expected: all pass.

Step 6: Commit

```
git add app/git/client.py tests/test_git_client.py
git commit -m &quot;feat: add GitClient.add_file() and commit() for targeted staging&quot;
```

Task 2: Enum, Student.student_number, Prompt model, Alembic migration

Files:
- Modify: `app/db/enums.py`
- Modify: `app/db/models.py`
- Modify: `app/services/audit_service.py` (add "prompt" to `_VALID_ENTITY_TYPES`)
- Create: `app/db/migrations/versions/b1a2c3d4e5f6_add_prompt_and_student_number.py`
- Test: `tests/test_models.py` (add to existing file)

Step 1: Write failing tests

```
# Add to tests/test_models.py

def test_prompt_model_creation(db_session):
    &quot;&quot;&quot;Prompt model can be created and persisted.&quot;&quot;&quot;
    from app.db.models import Student, Account, Repository, Prompt
    from app.db.enums import SyncStatus, PromptStatus

    student = Student(full_name=&quot;Test Student&quot;, student_number=1)
    db_session.add(student)
    db_session.flush()

    account = Account(student_id=student.id, provider=&quot;github&quot;, username=&quot;tester&quot;, auth_
    db_session.add(account)
    db_session.flush()

    repo = Repository(
        account_id=account.id,
        repo_name=&quot;tester-project&quot;,
        remote_url=&quot;https://github.com/tester/project.git&quot;,
        local_path=&quot;/tmp/tester&quot;,
        default_branch=&quot;main&quot;,
        sync_status=SyncStatus.clean.value,
    )
    db_session.add(repo)
    db_session.flush()

    prompt = Prompt(
        student_id=student.id,
        repository_id=repo.id,
        student_number=1,
        seq_number=1,
        title=&quot;Auth Flow Fix&quot;,
        slug=&quot;auth-flow-fix&quot;,
        content=&quot;Please implement authentication.&quot;,
        file_path=&quot;prompts/student01-001-auth-flow-fix.md&quot;,
        git_branch=&quot;main&quot;,
        status=PromptStatus.draft.value,
        created_by=&quot;operator&quot;,
    )
    db_session.add(prompt)
    db_session.commit()

    fetched = db_session.get(Prompt, prompt.id)
    assert fetched is not None
    assert fetched.seq_number == 1
    assert fetched.student_number == 1
    assert fetched.status == &quot;draft&quot;
    assert fetched.git_commit_hash is None
    assert fetched.pushed_at is None
```

```
def test_student_has_student_number(db_session):
    &quot;&quot;&quot;Student model has student_number field.&quot;&quot;&quot;
    from app.db.models import Student

    student = Student(full_name=&quot;Numbered Student&quot;, student_number=42)
    db_session.add(student)
    db_session.commit()

    fetched = db_session.get(Student, student.id)
    assert fetched.student_number == 42


def test_prompt_status_enum_values():
    &quot;&quot;&quot;PromptStatus has required values.&quot;&quot;&quot;
    from app.db.enums import PromptStatus
    assert PromptStatus.draft.value == &quot;draft&quot;
    assert PromptStatus.written.value == &quot;written&quot;
    assert PromptStatus.committed.value == &quot;committed&quot;
    assert PromptStatus.pushed.value == &quot;pushed&quot;
    assert PromptStatus.failed.value == &quot;failed&quot;
```

Step 2: Run to verify they fail

```
.venv/bin/pytest tests/test_models.py::test_prompt_model_creation tests/test_models.py::test_student_has_studen
```

Expected: `ImportError` or `AttributeError`.

Step 3: Add PromptStatus to enums.py

Add at the end of `app/db/enums.py`:

```
class PromptStatus(str, enum.Enum):
    draft = &quot;draft&quot;
    written = &quot;written&quot;
    committed = &quot;committed&quot;
    pushed = &quot;pushed&quot;
    failed = &quot;failed&quot;
```

Step 4: Add student_number to Student model

In `app/db/models.py`, in the `Student` class, add after `notes` column (before `created_at`):

```
student_number = Column(Integer, nullable=True, unique=True)
```

Also update `__table_args__` to add an index:

```
Index(&quot;idx_student_number&quot;, &quot;student_number&quot;),
```

Step 5: Add Prompt model to models.py

Add the following class after `AppSetting` (before `__all__`):

```
class Prompt(Base):
    __tablename__ = &quot;prompt&quot;

    id = Column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
    student_id = Column(String(36), ForeignKey(&quot;student.id&quot;), nullable=False)
    repository_id = Column(String(36), ForeignKey(&quot;repository.id&quot;), nullable=False)
    student_number = Column(Integer, nullable=False)
    seq_number = Column(Integer, nullable=False)
    title = Column(String(255), nullable=False, default=&quot;&quot;)
    slug = Column(String(255), nullable=False)
    content = Column(Text, nullable=False)
    file_path = Column(String(255), nullable=False)
    git_branch = Column(String(255), nullable=False)
    git_commit_hash = Column(String(40), nullable=True)
    status = Column(String(20), nullable=False, default=&quot;draft&quot;)
    push_error = Column(Text, nullable=True)
```

```
    created_by = Column(String(100), nullable=False, default=&quot;operator&quot;)
    created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
    pushed_at = Column(DateTime, nullable=True)

    student = relationship(&quot;Student&quot;, back_populates=&quot;prompts&quot;)
    repository = relationship(&quot;Repository&quot;, back_populates=&quot;prompts&quot;)

    __table_args__ = (
        Index(&quot;idx_prompt_student_id&quot;, &quot;student_id&quot;),
        Index(&quot;idx_prompt_status&quot;, &quot;status&quot;),
        Index(&quot;idx_prompt_student_seq&quot;, &quot;student_id&quot;, &quot;seq_number&quot;),
    )
```

Also add to Student model: `prompts = relationship("Prompt", back_populates="student")`
Also add to Repository model: `prompts = relationship("Prompt", back_populates="repository")`

Also update `__all__` list to include `"Prompt"` and `"PromptStatus"`.

Also update imports at top of models.py — `PromptStatus` is needed in enums imports block:

```
from app.db.enums import (
    ...
    PromptStatus,
)
```

Step 6: Add "prompt" to AuditService._VALID_ENTITY_TYPES

In `app/services/audit_service.py`, find `_VALID_ENTITY_TYPES` set (lines 23-32) and add `"prompt"`:

```
_VALID_ENTITY_TYPES = {
    &quot;student&quot;,
    &quot;account&quot;,
    &quot;repository&quot;,
    &quot;credential&quot;,
    &quot;sync_job&quot;,
    &quot;handoff_event&quot;,
    &quot;app_setting&quot;,
    &quot;prompt&quot;,
    &quot;system&quot;,
}
```

Step 7: Create Alembic migration

Create `app/db/migrations/versions/b1a2c3d4e5f6_add_prompt_and_student_number.py`:

```
&quot;&quot;&quot;add_prompt_and_student_number

Revision ID: b1a2c3d4e5f6
Revises: 7ca526e3ce7e
Create Date: 2026-03-17 12:00:00.000000

&quot;&quot;&quot;
from typing import Sequence, Union

from alembic import op
import sqlalchemy as sa


revision: str = &apos;b1a2c3d4e5f6&apos;
down_revision: Union[str, None] = &apos;7ca526e3ce7e&apos;
branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None


def upgrade() -&gt; None:
    # Add student_number to student table
    op.add_column(&apos;student&apos;, sa.Column(&apos;student_number&apos;, sa.Integer(), nullable=True))
    op.create_index(&apos;idx_student_number&apos;, &apos;student&apos;, [&apos;student_number&apos;], unic
```

```
    # Assign sequential student_numbers to existing students ordered by created_at
    op.execute(&quot;&quot;&quot;
        UPDATE student SET student_number = (
            SELECT COUNT(*) FROM student s2
            WHERE s2.created_at &lt; student.created_at
            OR (s2.created_at = student.created_at AND s2.id &lt;= student.id)
        )
    &quot;&quot;&quot;)

    # Create prompt table
    op.create_table(
        &apos;prompt&apos;,
        sa.Column(&apos;id&apos;, sa.String(36), nullable=False),
        sa.Column(&apos;student_id&apos;, sa.String(36), nullable=False),
        sa.Column(&apos;repository_id&apos;, sa.String(36), nullable=False),
        sa.Column(&apos;student_number&apos;, sa.Integer(), nullable=False),
        sa.Column(&apos;seq_number&apos;, sa.Integer(), nullable=False),
        sa.Column(&apos;title&apos;, sa.String(255), nullable=False),
        sa.Column(&apos;slug&apos;, sa.String(255), nullable=False),
        sa.Column(&apos;content&apos;, sa.Text(), nullable=False),
        sa.Column(&apos;file_path&apos;, sa.String(255), nullable=False),
        sa.Column(&apos;git_branch&apos;, sa.String(255), nullable=False),
        sa.Column(&apos;git_commit_hash&apos;, sa.String(40), nullable=True),
        sa.Column(&apos;status&apos;, sa.String(20), nullable=False),
        sa.Column(&apos;push_error&apos;, sa.Text(), nullable=True),
        sa.Column(&apos;created_by&apos;, sa.String(100), nullable=False),
        sa.Column(&apos;created_at&apos;, sa.DateTime(), nullable=False),
        sa.Column(&apos;pushed_at&apos;, sa.DateTime(), nullable=True),
        sa.ForeignKeyConstraint([&apos;student_id&apos;], [&apos;student.id&apos;]),
        sa.ForeignKeyConstraint([&apos;repository_id&apos;], [&apos;repository.id&apos;]),
        sa.PrimaryKeyConstraint(&apos;id&apos;),
    )
    op.create_index(&apos;idx_prompt_student_id&apos;, &apos;prompt&apos;, [&apos;student_id&apos;])
    op.create_index(&apos;idx_prompt_status&apos;, &apos;prompt&apos;, [&apos;status&apos;])
    op.create_index(&apos;idx_prompt_student_seq&apos;, &apos;prompt&apos;, [&apos;student_id&apos;, &apos;


def downgrade() -&gt; None:
    op.drop_table(&apos;prompt&apos;)
    op.drop_index(&apos;idx_student_number&apos;, table_name=&apos;student&apos;)
    op.drop_column(&apos;student&apos;, &apos;student_number&apos;)
```

Step 8: Run failing tests — now they should pass

```
.venv/bin/pytest tests/test_models.py::test_prompt_model_creation tests/test_models.py::test_student_has_studen
```

Expected: 3 PASS

Step 9: Run full suite

```
.venv/bin/pytest tests/ -q
```

Expected: all pass.

Step 10: Commit

```
git add app/db/enums.py app/db/models.py app/services/audit_service.py \
    app/db/migrations/versions/b1a2c3d4e5f6_add_prompt_and_student_number.py \
    tests/test_models.py
git commit -m &quot;feat: add PromptStatus enum, student_number, Prompt model, migration&quot;
```

Task 3: PromptService

Files:
- Create: `app/services/prompt_service.py`
- Test: `tests/test_prompt_service.py`

Step 1: Write failing tests

```
# tests/test_prompt_service.py
&quot;&quot;&quot;Tests for PromptService.&quot;&quot;&quot;
from __future__ import annotations

import re
from datetime import datetime
from pathlib import Path
from unittest.mock import MagicMock, call

import pytest
from sqlalchemy.orm import sessionmaker

from app.db.enums import PromptStatus, StudentStatus, SyncStatus
from app.db.models import Account, Prompt, Repository, Student


def _factory(db_session):
    return sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)


def _make_student(db_session, name=&quot;Alice&quot;, queue_pos=1, student_number=1):
    s = Student(
        full_name=name,
        status=StudentStatus.active.value,
        queue_position=queue_pos,
        student_number=student_number,
    )
    db_session.add(s)
    db_session.flush()
    a = Account(
        student_id=s.id, provider=&quot;github&quot;, username=name.lower(), auth_type=&quot;pat&quot;
    )
    db_session.add(a)
    db_session.flush()
    r = Repository(
        account_id=a.id,
        repo_name=f&quot;{name.lower()}-project&quot;,
        remote_url=f&quot;https://github.com/{name.lower()}/project.git&quot;,
        local_path=f&quot;/tmp/{name.lower()}&quot;,
        default_branch=&quot;main&quot;,
        sync_status=SyncStatus.clean.value,
        is_canonical=False,
    )
    db_session.add(r)
    from app.db.models import Credential
    cred = Credential(
        account_id=a.id,
        secret_ref=f&quot;studentflow:github:{name.lower()}:abc123&quot;,
        secret_kind=&quot;pat&quot;,
    )
    db_session.add(cred)
    db_session.commit()
    return s, a, r


# ── slug generation ──────────────────────────────────────────────


def test_make_slug_basic():
    from app.services.prompt_service import _make_slug
    assert _make_slug(&quot;Auth Flow Fix&quot;) == &quot;auth-flow-fix&quot;
```

```
def test_make_slug_empty_returns_prompt():
    from app.services.prompt_service import _make_slug
    assert _make_slug(&quot;&quot;) == &quot;prompt&quot;
    assert _make_slug(&quot;   &quot;) == &quot;prompt&quot;


def test_make_slug_strips_special_chars():
    from app.services.prompt_service import _make_slug
    slug = _make_slug(&quot;Hello, World! (2026)&quot;)
    assert re.match(r&apos;^[a-z0-9-]+$&apos;, slug)


def test_make_slug_truncates_to_40_chars():
    from app.services.prompt_service import _make_slug
    long_title = &quot;a&quot; * 100
    assert len(_make_slug(long_title)) &lt;= 40


# ── file path generation ──────────────────────────────────────────


def test_make_file_path_format():
    from app.services.prompt_service import _make_file_path
    result = _make_file_path(student_number=3, seq_number=1, slug=&quot;auth-fix&quot;)
    assert result == &quot;prompts/student03-001-auth-fix.md&quot;


def test_make_file_path_zero_pads():
    from app.services.prompt_service import _make_file_path
    assert _make_file_path(7, 14, &quot;test&quot;) == &quot;prompts/student07-014-test.md&quot;
    assert _make_file_path(12, 100, &quot;big&quot;) == &quot;prompts/student12-100-big.md&quot;


# ── file content generation ───────────────────────────────────────


def test_make_file_content_contains_metadata():
    from app.services.prompt_service import _make_file_content
    content = _make_file_content(
        title=&quot;Auth Flow Fix&quot;,
        student_number=3,
        seq_number=1,
        created_by=&quot;operator&quot;,
        repo_name=&quot;alice-project&quot;,
        branch=&quot;main&quot;,
        prompt_text=&quot;Fix the login.&quot;,
        created_at=datetime(2026, 3, 17, 12, 0, 0),
    )
    assert &quot;# Auth Flow Fix&quot; in content
    assert &quot;#03&quot; in content
    assert &quot;001&quot; in content
    assert &quot;operator&quot; in content
    assert &quot;alice-project&quot; in content
    assert &quot;main&quot; in content
    assert &quot;Fix the login.&quot; in content


# ── service: get_active_student ──────────────────────────────────


def test_get_active_student_returns_first_by_queue(db_session):
    from app.services.prompt_service import PromptService
    s1, _, _ = _make_student(db_session, &quot;Alice&quot;, queue_pos=2, student_number=1)
    s2, _, _ = _make_student(db_session, &quot;Bob&quot;, queue_pos=1, student_number=2)

    svc = PromptService(session_factory=_factory(db_session))
    with _factory(db_session)() as session:
```

```
        result = svc._get_active_student(session)
    assert result.id == s2.id  # Bob has queue_pos=1


def test_get_active_student_raises_when_none(db_session):
    from app.services.prompt_service import PromptService
    svc = PromptService(session_factory=_factory(db_session))
    with _factory(db_session)() as session:
        with pytest.raises(ValueError, match=&quot;No active student&quot;):
            svc._get_active_student(session)


# ── service: _next_seq ──────────────────────────────────────────────


def test_next_seq_starts_at_1(db_session):
    from app.services.prompt_service import PromptService
    s, _, _ = _make_student(db_session)
    svc = PromptService(session_factory=_factory(db_session))
    with _factory(db_session)() as session:
        assert svc._next_seq(session, s.id) == 1


def test_next_seq_increments(db_session):
    from app.services.prompt_service import PromptService
    s, _, r = _make_student(db_session)
    p = Prompt(
        student_id=s.id, repository_id=r.id, student_number=1, seq_number=1,
        title=&quot;t&quot;, slug=&quot;t&quot;, content=&quot;c&quot;, file_path=&quot;prompts/s.md&quot;,
        git_branch=&quot;main&quot;, status=PromptStatus.pushed.value, created_by=&quot;operator&quot;,
        created_at=datetime.utcnow(),
    )
    db_session.add(p)
    db_session.commit()
    svc = PromptService(session_factory=_factory(db_session))
    with _factory(db_session)() as session:
        assert svc._next_seq(session, s.id) == 2


# ── service: create_and_push_prompt ──────────────────────────────────


def test_create_and_push_prompt_success(db_session, tmp_path):
    &quot;&quot;&quot;Full happy path: file written, committed, pushed, DB updated.&quot;&quot;&quot;
    import subprocess
    from app.services.prompt_service import PromptService

    # Set up a real git repo at tmp_path
    subprocess.run([&quot;git&quot;, &quot;init&quot;, str(tmp_path)], check=True, capture_output=True)
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;config&quot;, &quot;user.email&quot;,
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;config&quot;, &quot;user.name&quot;,
    (tmp_path / &quot;readme.txt&quot;).write_text(&quot;hello&quot;)
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;add&quot;, &quot;.&quot;], check=True
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(tmp_path), &quot;commit&quot;, &quot;-m&quot;, &quot;

    s, a, r = _make_student(db_session)
    # Override local_path to our tmp real repo
    r.local_path = str(tmp_path)
    db_session.commit()

    mock_git = MagicMock()
    mock_git.add_file.return_value = None
    mock_git.commit.return_value = &quot;a&quot; * 40
    mock_git.push.return_value = None

    mock_creds = MagicMock()
    mock_creds.retrieve.return_value = &quot;fake_token&quot;
```

```
    mock_audit = MagicMock()

    svc = PromptService(
        git_client=mock_git,
        credential_service=mock_creds,
        audit_service=mock_audit,
        session_factory=_factory(db_session),
    )

    prompt = svc.create_and_push_prompt(
        title=&quot;Auth Flow Fix&quot;,
        content=&quot;Fix the login.&quot;,
    )

    assert prompt.status == PromptStatus.pushed.value
    assert prompt.git_commit_hash == &quot;a&quot; * 40
    assert prompt.pushed_at is not None
    assert prompt.file_path == &quot;prompts/student01-001-auth-flow-fix.md&quot;
    assert prompt.seq_number == 1
    assert prompt.student_number == 1
    assert prompt.push_error is None

    # Verify git calls were made in order
    mock_git.add_file.assert_called_once()
    mock_git.commit.assert_called_once()
    mock_git.push.assert_called_once()

    # Verify audit was called
    assert mock_audit.log.call_count &gt;= 2  # at least committed + pushed


def test_create_and_push_prompt_push_failure_stores_error(db_session, tmp_path):
    &quot;&quot;&quot;If push fails, status=failed and push_error is stored safely.&quot;&quot;&quot;
    from app.git.client import GitAuthError
    from app.services.prompt_service import PromptService

    s, a, r = _make_student(db_session)
    r.local_path = str(tmp_path)
    db_session.commit()

    mock_git = MagicMock()
    mock_git.add_file.return_value = None
    mock_git.commit.return_value = &quot;b&quot; * 40
    mock_git.push.side_effect = GitAuthError(&quot;authentication failed&quot;)

    mock_creds = MagicMock()
    mock_creds.retrieve.return_value = &quot;fake_token&quot;

    svc = PromptService(
        git_client=mock_git,
        credential_service=mock_creds,
        audit_service=MagicMock(),
        session_factory=_factory(db_session),
    )

    prompt = svc.create_and_push_prompt(title=&quot;Test&quot;, content=&quot;content&quot;)

    assert prompt.status == PromptStatus.failed.value
    assert prompt.push_error is not None
    assert len(prompt.push_error) &lt; 300  # concise, no secrets
    assert prompt.git_commit_hash == &quot;b&quot; * 40  # committed before push
    assert prompt.pushed_at is None


def test_push_error_never_contains_token(db_session, tmp_path):
    &quot;&quot;&quot;push_error must never contain credential values.&quot;&quot;&quot;
    from app.git.client import GitOperationError
```

```
    from app.services.prompt_service import PromptService

    s, a, r = _make_student(db_session)
    r.local_path = str(tmp_path)
    db_session.commit()

    secret = &quot;ghp_supersecrettoken12345678901234567&quot;
    mock_git = MagicMock()
    mock_git.add_file.return_value = None
    mock_git.commit.return_value = &quot;c&quot; * 40
    mock_git.push.side_effect = GitOperationError(f&quot;push failed: {secret}&quot;)

    mock_creds = MagicMock()
    mock_creds.retrieve.return_value = secret

    svc = PromptService(
        git_client=mock_git,
        credential_service=mock_creds,
        audit_service=MagicMock(),
        session_factory=_factory(db_session),
    )

    prompt = svc.create_and_push_prompt(title=&quot;Test&quot;, content=&quot;c&quot;)
    assert secret not in (prompt.push_error or &quot;&quot;)


def test_list_prompts_all(db_session):
    &quot;&quot;&quot;list_prompts returns all prompts when no student_id filter.&quot;&quot;&quot;
    from app.services.prompt_service import PromptService
    s1, _, r1 = _make_student(db_session, &quot;Alice&quot;, student_number=1)
    s2, _, r2 = _make_student(db_session, &quot;Bob&quot;, queue_pos=2, student_number=2)

    for s, r, n in [(s1, r1, 1), (s2, r2, 2)]:
        db_session.add(Prompt(
            student_id=s.id, repository_id=r.id, student_number=n,
            seq_number=1, title=&quot;t&quot;, slug=&quot;t&quot;, content=&quot;c&quot;,
            file_path=&quot;prompts/s.md&quot;, git_branch=&quot;main&quot;,
            status=PromptStatus.pushed.value, created_by=&quot;operator&quot;,
            created_at=datetime.utcnow(),
        ))
    db_session.commit()

    svc = PromptService(session_factory=_factory(db_session))
    results = svc.list_prompts()
    assert len(results) == 2


def test_list_prompts_filtered_by_student(db_session):
    from app.services.prompt_service import PromptService
    s1, _, r1 = _make_student(db_session, &quot;Alice&quot;, student_number=1)
    s2, _, r2 = _make_student(db_session, &quot;Bob&quot;, queue_pos=2, student_number=2)
    db_session.add(Prompt(
        student_id=s1.id, repository_id=r1.id, student_number=1,
        seq_number=1, title=&quot;t&quot;, slug=&quot;t&quot;, content=&quot;c&quot;,
        file_path=&quot;p.md&quot;, git_branch=&quot;main&quot;,
        status=PromptStatus.pushed.value, created_by=&quot;op&quot;,
        created_at=datetime.utcnow(),
    ))
    db_session.commit()
    svc = PromptService(session_factory=_factory(db_session))
    assert len(svc.list_prompts(student_id=s1.id)) == 1
    assert len(svc.list_prompts(student_id=s2.id)) == 0
```

Step 2: Run to verify they fail

```
.venv/bin/pytest tests/test_prompt_service.py -v 2&gt;&amp;1 | head -20
```

Expected: `ModuleNotFoundError: No module named 'app.services.prompt_service'`

Step 3: Implement PromptService

Create `app/services/prompt_service.py`:

```
&quot;&quot;&quot;PromptService: create, commit, and push prompt files for the active student.&quot;&quot;&qu
from __future__ import annotations

import re
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

from app.db.base import SessionLocal
from app.db.enums import PromptStatus, StudentStatus
from app.db.models import Account, Credential, Prompt, Repository, Student


# ── helpers ──────────────────────────────────────────────────────────────────

def _make_slug(title: str) -&gt; str:
    &quot;&quot;&quot;Derive a filesystem-safe slug from a title.&quot;&quot;&quot;
    if not title or not title.strip():
        return &quot;prompt&quot;
    s = title.lower().strip()
    s = re.sub(r&quot;[^a-z0-9\s-]&quot;, &quot;&quot;, s)
    s = re.sub(r&quot;[\s-]+&quot;, &quot;-&quot;, s)
    s = s.strip(&quot;-&quot;)
    return s[:40] or &quot;prompt&quot;


def _make_file_path(student_number: int, seq_number: int, slug: str) -&gt; str:
    &quot;&quot;&quot;Build the prompts/ relative file path.&quot;&quot;&quot;
    return f&quot;prompts/student{student_number:02d}-{seq_number:03d}-{slug}.md&quot;


def _make_file_content(
    title: str,
    student_number: int,
    seq_number: int,
    created_by: str,
    repo_name: str,
    branch: str,
    prompt_text: str,
    created_at: datetime,
) -&gt; str:
    &quot;&quot;&quot;Render the markdown file content for a prompt.&quot;&quot;&quot;
    display_title = title.strip() or &quot;Prompt&quot;
    ts = created_at.strftime(&quot;%Y-%m-%d %H:%M:%S UTC&quot;)
    return (
        f&quot;# {display_title}\n\n&quot;
        f&quot;| Field | Value |\n&quot;
        f&quot;|---|---|\n&quot;
        f&quot;| Student | #{student_number:02d} |\n&quot;
        f&quot;| Sequence | {seq_number:03d} |\n&quot;
        f&quot;| Created at | {ts} |\n&quot;
        f&quot;| Created by | {created_by} |\n&quot;
        f&quot;| Target repo | {repo_name} |\n&quot;
        f&quot;| Target branch | {branch} |\n&quot;
        f&quot;\n## Prompt\n\n{prompt_text}\n&quot;
    )


def _sanitize_error(message: str) -&gt; str:
    &quot;&quot;&quot;Strip potential secrets from error text before storing.&quot;&quot;&quot;
```

```
    # Remove anything that looks like a token
    cleaned = re.sub(r&quot;ghp_[A-Za-z0-9]{20,}&quot;, &quot;[token]&quot;, message)
    cleaned = re.sub(r&quot;glpat-[A-Za-z0-9\-_]{10,}&quot;, &quot;[token]&quot;, cleaned)
    cleaned = re.sub(r&quot;github_pat_[A-Za-z0-9_]{20,}&quot;, &quot;[token]&quot;, cleaned)
    return cleaned[:280]


# ── service ────────────────────────────────────────────────────────────────────

class PromptService:
    &quot;&quot;&quot;Manage prompt file lifecycle: write, commit, push, record.&quot;&quot;&quot;

    def __init__(
        self,
        git_client=None,
        credential_service=None,
        audit_service=None,
        session_factory=None,
    ):
        from app.git.client import GitClient
        from app.services.audit_service import AuditService
        from app.services.credential_service import CredentialService

        self._git = git_client or GitClient()
        self._creds = credential_service or CredentialService(session_factory=session_factory)
        self._audit = audit_service or AuditService(session_factory=session_factory)
        self._session_factory = session_factory or SessionLocal

    # ── internal helpers ──────────────────────────────────────────────────────

    def _get_active_student(self, session) -&gt; Student:
        student = (
            session.query(Student)
            .filter(Student.status == StudentStatus.active.value)
            .order_by(Student.queue_position.asc(), Student.created_at.asc())
            .first()
        )
        if student is None:
            raise ValueError(&quot;No active student found&quot;)
        return student

    def _get_student_repo(self, session, student_id: str) -&gt; tuple[Repository, Credential]:
        account = (
            session.query(Account).filter_by(student_id=student_id, is_current=True).first()
            or session.query(Account).filter_by(student_id=student_id).first()
        )
        if account is None:
            raise ValueError(f&quot;No account for student {student_id}&quot;)
        repo = session.query(Repository).filter_by(
            account_id=account.id, is_canonical=False
        ).first()
        if repo is None:
            raise ValueError(f&quot;No repository for student {student_id}&quot;)
        cred = session.query(Credential).filter_by(account_id=account.id).first()
        if cred is None:
            raise ValueError(f&quot;No credential for account {account.id}&quot;)
        return repo, cred

    def _next_seq(self, session, student_id: str) -&gt; int:
        from sqlalchemy import func
        result = (
            session.query(func.max(Prompt.seq_number))
            .filter(Prompt.student_id == student_id)
            .scalar()
        )
        return (result or 0) + 1
    # ── public API ────────────────────────────────────────────────────────────
```

```
def create_and_push_prompt(
    self,
    title: str,
    content: str,
    created_by: str = &quot;operator&quot;,
) -&gt; Prompt:
    &quot;&quot;&quot;Write prompt to student repo, commit, push. Returns persisted Prompt.&quot;&quot;&qu
    with self._session_factory() as session:
        student = self._get_active_student(session)
        repo, cred = self._get_student_repo(session, student.id)

        student_number = student.student_number or 0
        seq_number = self._next_seq(session, student.id)
        slug = _make_slug(title)
        file_path = _make_file_path(student_number, seq_number, slug)
        branch = repo.default_branch or &quot;main&quot;
        created_at = datetime.utcnow()

        file_content = _make_file_content(
            title=title,
            student_number=student_number,
            seq_number=seq_number,
            created_by=created_by,
            repo_name=repo.repo_name,
            branch=branch,
            prompt_text=content,
            created_at=created_at,
        )

        # Persist initial draft record
        prompt = Prompt(
            student_id=student.id,
            repository_id=repo.id,
            student_number=student_number,
            seq_number=seq_number,
            title=title.strip(),
            slug=slug,
            content=content,
            file_path=file_path,
            git_branch=branch,
            status=PromptStatus.draft.value,
            created_by=created_by,
            created_at=created_at,
        )
        session.add(prompt)
        session.commit()
        session.refresh(prompt)
        prompt_id = prompt.id

        # Capture values needed outside session
        local_path = repo.local_path
        remote_url = repo.remote_url
        secret_ref = cred.secret_ref
        auth_type = cred.secret_kind

    # Write file outside session (filesystem op)
    abs_file = Path(local_path) / file_path
    abs_file.parent.mkdir(parents=True, exist_ok=True)
    abs_file.write_text(file_content, encoding=&quot;utf-8&quot;)

    # Update status: written
    with self._session_factory() as session:
        prompt = session.get(Prompt, prompt_id)
        prompt.status = PromptStatus.written.value
        session.commit()

    # Git add (only this file)
```

```
try:
    self._git.add_file(local_path, file_path)
except Exception as exc:
    with self._session_factory() as session:
        prompt = session.get(Prompt, prompt_id)
        prompt.status = PromptStatus.failed.value
        prompt.push_error = _sanitize_error(str(exc))
        session.commit()
        session.expunge(prompt)
        return prompt

# Git commit
commit_hash: str | None = None
try:
    commit_msg = f&quot;prompt: {Path(file_path).stem}&quot;
    commit_hash = self._git.commit(local_path, commit_msg)
except Exception as exc:
    with self._session_factory() as session:
        prompt = session.get(Prompt, prompt_id)
        prompt.status = PromptStatus.failed.value
        prompt.push_error = _sanitize_error(str(exc))
        session.commit()
        session.expunge(prompt)
        return prompt

with self._session_factory() as session:
    prompt = session.get(Prompt, prompt_id)
    prompt.status = PromptStatus.committed.value
    prompt.git_commit_hash = commit_hash
    session.commit()

self._audit.log(
    &quot;prompt_committed&quot;,
    &quot;prompt&quot;,
    prompt_id,
    details={&quot;file_path&quot;: file_path, &quot;commit_hash&quot;: commit_hash},
)

# Git push
try:
    credential_value = self._creds.retrieve(secret_ref)
    self._git.push(
        local_path,
        &quot;student&quot;,
        branch,
        credential=credential_value,
        auth_type=auth_type,
    )
except Exception as exc:
    with self._session_factory() as session:
        prompt = session.get(Prompt, prompt_id)
        prompt.status = PromptStatus.failed.value
        prompt.push_error = _sanitize_error(str(exc))
        session.commit()
        session.expunge(prompt)

    self._audit.log(
        &quot;prompt_push_failed&quot;,
        &quot;prompt&quot;,
        prompt_id,
        details={&quot;file_path&quot;: file_path, &quot;error&quot;: _sanitize_error(str(exc))},
    )
    return prompt

# Success
with self._session_factory() as session:
    prompt = session.get(Prompt, prompt_id)
```

```
            prompt.status = PromptStatus.pushed.value
            prompt.pushed_at = datetime.utcnow()
            session.commit()
            session.expunge(prompt)

        self._audit.log(
            &quot;prompt_pushed&quot;,
            &quot;prompt&quot;,
            prompt_id,
            details={&quot;file_path&quot;: file_path, &quot;student_number&quot;: student_number},
        )
        return prompt

    def retry_push(self, prompt_id: str) -&gt; Prompt:
        &quot;&quot;&quot;Re-attempt push for a failed prompt.&quot;&quot;&quot;
        with self._session_factory() as session:
            prompt = session.get(Prompt, prompt_id)
            if prompt is None:
                raise ValueError(f&quot;Prompt {prompt_id} not found&quot;)
            if prompt.status not in (PromptStatus.failed.value, PromptStatus.committed.value):
                raise ValueError(f&quot;Prompt {prompt_id} is not in a retryable state&quot;)
            repo, cred = self._get_student_repo(session, prompt.student_id)
            local_path = repo.local_path
            branch = prompt.git_branch
            secret_ref = cred.secret_ref
            auth_type = cred.secret_kind
            file_path = prompt.file_path

        try:
            credential_value = self._creds.retrieve(secret_ref)
            self._git.push(local_path, &quot;student&quot;, branch, credential=credential_value, auth_type=auth_type
        except Exception as exc:
            with self._session_factory() as session:
                prompt = session.get(Prompt, prompt_id)
                prompt.status = PromptStatus.failed.value
                prompt.push_error = _sanitize_error(str(exc))
                session.commit()
                session.expunge(prompt)
            return prompt

        with self._session_factory() as session:
            prompt = session.get(Prompt, prompt_id)
            prompt.status = PromptStatus.pushed.value
            prompt.pushed_at = datetime.utcnow()
            prompt.push_error = None
            session.commit()
            session.expunge(prompt)

        self._audit.log(&quot;prompt_pushed&quot;, &quot;prompt&quot;, prompt_id, details={&quot;retry&quot;: Tr
        return prompt

    def list_prompts(self, student_id: str | None = None) -&gt; list[Prompt]:
        &quot;&quot;&quot;Return prompts ordered by created_at desc. Optionally filter by student.&quot;&quot;&qu
        with self._session_factory() as session:
            q = session.query(Prompt).order_by(Prompt.created_at.desc())
            if student_id is not None:
                q = q.filter(Prompt.student_id == student_id)
            prompts = q.all()
            session.expunge_all()
            return prompts
```

Step 4: Run tests

```
.venv/bin/pytest tests/test_prompt_service.py -v
```

Expected: All pass.

Step 5: Run full suite

.venv/bin/pytest tests/ -q

Expected: all pass.

Step 6: Commit

```
git add app/services/prompt_service.py tests/test_prompt_service.py
git commit -m &quot;feat: implement PromptService with create_and_push_prompt and retry_push&quot;
```

Task 4: PromptsViewModel + NAV_ITEMS

Files:
- Create: `app/ui/viewmodels/prompts_viewmodel.py`
- Modify: `app/ui/windows/main_window.py` (add Prompts nav item at index 7)

Step 1: Write failing tests

```
# tests/test_prompts_viewmodel.py
import pytest
from unittest.mock import MagicMock, patch
from sqlalchemy.orm import sessionmaker
from datetime import datetime

from app.db.enums import StudentStatus, SyncStatus, PromptStatus
from app.db.models import Account, Repository, Student, Prompt


def _factory(db_session):
    return sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)


def _make_active_student(db_session, name=&quot;Alice&quot;, student_number=1):
    s = Student(full_name=name, status=StudentStatus.active.value,
                queue_position=1, student_number=student_number)
    db_session.add(s)
    db_session.flush()
    a = Account(student_id=s.id, provider=&quot;github&quot;, username=name.lower(), auth_type=&quot;pat&qu
    db_session.add(a)
    db_session.flush()
    r = Repository(account_id=a.id, repo_name=f&quot;{name.lower()}-project&quot;,
                   remote_url=&quot;https://github.com/a/b.git&quot;,
                   local_path=&quot;/tmp/a&quot;, default_branch=&quot;main&quot;,
                   sync_status=SyncStatus.clean.value, is_canonical=False)
    db_session.add(r)
    db_session.commit()
    return s, a, r


def test_get_active_info_returns_student_and_repo(db_session):
    from app.ui.viewmodels.prompts_viewmodel import PromptsViewModel
    s, _, r = _make_active_student(db_session)
    vm = PromptsViewModel(session_factory=_factory(db_session))
    info = vm.get_active_info()
    assert info[&quot;student&quot;] is not None
    assert info[&quot;student&quot;].full_name == &quot;Alice&quot;
    assert info[&quot;repo&quot;] is not None
    assert info[&quot;repo&quot;].repo_name == &quot;alice-project&quot;


def test_get_active_info_returns_none_when_no_active(db_session):
    from app.ui.viewmodels.prompts_viewmodel import PromptsViewModel
```

```
    vm = PromptsViewModel(session_factory=_factory(db_session))
    info = vm.get_active_info()
    assert info[&quot;student&quot;] is None
    assert info[&quot;repo&quot;] is None


def test_list_prompts_delegates_to_service(db_session):
    from app.ui.viewmodels.prompts_viewmodel import PromptsViewModel
    mock_svc = MagicMock()
    mock_svc.list_prompts.return_value = []
    vm = PromptsViewModel(session_factory=_factory(db_session))
    vm._service = mock_svc
    result = vm.list_prompts()
    mock_svc.list_prompts.assert_called_once_with(student_id=None)
    assert result == []


def test_create_and_push_delegates_to_service(db_session):
    from app.ui.viewmodels.prompts_viewmodel import PromptsViewModel
    mock_svc = MagicMock()
    fake_prompt = MagicMock()
    mock_svc.create_and_push_prompt.return_value = fake_prompt
    vm = PromptsViewModel(session_factory=_factory(db_session))
    vm._service = mock_svc
    result = vm.create_and_push(&quot;My Title&quot;, &quot;Content here&quot;)
    mock_svc.create_and_push_prompt.assert_called_once_with(
        title=&quot;My Title&quot;, content=&quot;Content here&quot;
    )
    assert result is fake_prompt
```

Step 2: Run to verify they fail

```
.venv/bin/pytest tests/test_prompts_viewmodel.py -v 2&gt;&amp;1 | head -15
```

Step 3: Implement PromptsViewModel

Create `app/ui/viewmodels/prompts_viewmodel.py`:

```
&quot;&quot;&quot;ViewModel for PromptsScreen.&quot;&quot;&quot;
from __future__ import annotations

from app.db.base import SessionLocal
from app.db.enums import StudentStatus
from app.db.models import Account, Prompt, Repository, Student
from app.services.prompt_service import PromptService


class PromptsViewModel:
    def __init__(self, session_factory=None):
        self._factory = session_factory or SessionLocal
        self._service = PromptService(session_factory=self._factory)

    def get_active_info(self) -&gt; dict:
        &quot;&quot;&quot;Return {&apos;student&apos;: Student|None, &apos;repo&apos;: Repository|None}.&quo
        with self._factory() as session:
            student = (
                session.query(Student)
                .filter(Student.status == StudentStatus.active.value)
                .order_by(Student.queue_position.asc(), Student.created_at.asc())
                .first()
            )
            if student is None:
                return {&quot;student&quot;: None, &quot;repo&quot;: None}

            session.expunge(student)

            account = (
```

```
            session.query(Account)
            .filter_by(student_id=student.id, is_current=True)
            .first()
            or session.query(Account).filter_by(student_id=student.id).first()
        )
        if account is None:
            return {&quot;student&quot;: student, &quot;repo&quot;: None}

        repo = session.query(Repository).filter_by(
            account_id=account.id, is_canonical=False
        ).first()
        if repo is not None:
            session.expunge(repo)
        return {&quot;student&quot;: student, &quot;repo&quot;: repo}

    def create_and_push(self, title: str, content: str) -&gt; Prompt:
        return self._service.create_and_push_prompt(title=title, content=content)

    def retry_push(self, prompt_id: str) -&gt; Prompt:
        return self._service.retry_push(prompt_id)

    def list_prompts(self, student_id: str | None = None) -&gt; list[Prompt]:
        return self._service.list_prompts(student_id=student_id)
```

Step 4: Add Prompts to NAV_ITEMS in main_window.py

In `app/ui/windows/main_window.py`, update `NAV_ITEMS` to add index 7:

```
NAV_ITEMS = [
    (0, &quot;Dashboard&quot;, QStyle.StandardPixmap.SP_DesktopIcon),
    (1, &quot;Students&quot;, QStyle.StandardPixmap.SP_FileDialogDetailedView),
    (2, &quot;Accounts&quot;, QStyle.StandardPixmap.SP_DirHomeIcon),
    (3, &quot;Repositories&quot;, QStyle.StandardPixmap.SP_DirIcon),
    (4, &quot;Credentials&quot;, QStyle.StandardPixmap.SP_DialogApplyButton),
    (5, &quot;Prompts&quot;, QStyle.StandardPixmap.SP_FileDialogNewFolder),
    (6, &quot;Logs&quot;, QStyle.StandardPixmap.SP_FileDialogInfoView),
    (7, &quot;Settings&quot;, QStyle.StandardPixmap.SP_FileDialogContentsView),
]
```

Note: Prompts is inserted at index 5, pushing Logs to 6, Settings to 7.

Also update `_load_screens()` to import and add `PromptsScreen` at the right position (index 5):

```
def _load_screens(self) -&gt; None:
    from app.ui.windows.accounts_screen import AccountsScreen
    from app.ui.windows.credentials_screen import CredentialsScreen
    from app.ui.windows.dashboard_screen import DashboardScreen
    from app.ui.windows.logs_screen import LogsScreen
    from app.ui.windows.prompts_screen import PromptsScreen
    from app.ui.windows.repositories_screen import RepositoriesScreen
    from app.ui.windows.settings_screen import SettingsScreen
    from app.ui.windows.students_screen import StudentsScreen

    screens = [
        DashboardScreen,   # 0
        StudentsScreen,    # 1
        AccountsScreen,    # 2
        RepositoriesScreen,# 3
        CredentialsScreen, # 4
        PromptsScreen,     # 5
        LogsScreen,        # 6
        SettingsScreen,    # 7
    ]
    for screen_cls in screens:
        self._stack.addWidget(screen_cls(self))
```

Step 5: Run tests

.venv/bin/pytest tests/test_prompts_viewmodel.py tests/test_ui_skeleton.py -v

Expected: all pass. Note: `test_navigation_switches_screens` checks index 6 (was Settings, now Logs) — verify it still passes or update it.

Step 6: Commit

git add app/ui/viewmodels/prompts_viewmodel.py tests/test_prompts_viewmodel.py app/ui/windows/main_window
git commit -m "feat: add PromptsViewModel and Prompts nav item at index 5"

Task 5: PromptsScreen UI

Files:
- Create: `app/ui/windows/prompts_screen.py`

Step 1: Implement PromptsScreen

Create `app/ui/windows/prompts_screen.py`:

"""Prompts screen — create and push prompt files for the active student."""
from __future__ import annotations

from PySide6.QtGui import QColor
from PySide6.QtWidgets import (
    QGroupBox,
    QHBoxLayout,
    QLabel,
    QLineEdit,
    QMessageBox,
    QPushButton,
    QSizePolicy,
    QTableWidget,
    QTableWidgetItem,
    QTextEdit,
    QVBoxLayout,
    QWidget,
)

from app.ui.viewmodels.prompts_viewmodel import PromptsViewModel

_STATUS_COLORS = {
    "pushed": "#a6e3a1",
    "committed": "#89b4fa",
    "written": "#cba6f7",
    "draft": "#f9e2af",
    "failed": "#f38ba8",
}

_HISTORY_COLUMNS = ["#", "Seq", "Title", "File path", "Status

class PromptsScreen(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._vm = PromptsViewModel()
        self._build_ui()
        self.refresh()

    def _build_ui(self) -> None:
        layout = QVBoxLayout(self)
        layout.setContentsMargins(24, 24, 24, 24)
        layout.setSpacing(16)
        title = QLabel("Prompts", self)

```
title.setStyleSheet(&quot;font-size: 20px; font-weight: bold;&quot;)
layout.addWidget(title)

# ── Active student info bar ──────────────────────────────
info_group = QGroupBox(&quot;Active Student&quot;, self)
info_layout = QHBoxLayout(info_group)
self._student_label = QLabel(&quot;—&quot;, self)
self._student_label.setStyleSheet(&quot;font-size: 14px; font-weight: 600;&quot;)
info_layout.addWidget(self._student_label)
info_layout.addStretch(1)
self._repo_label = QLabel(&quot;—&quot;, self)
self._repo_label.setStyleSheet(&quot;color: #a6adc8;&quot;)
info_layout.addWidget(self._repo_label)
self._branch_label = QLabel(&quot;—&quot;, self)
self._branch_label.setStyleSheet(&quot;color: #89b4fa;&quot;)
info_layout.addWidget(self._branch_label)
layout.addWidget(info_group)

# ── Compose form ─────────────────────────────────────────
compose_group = QGroupBox(&quot;New Prompt&quot;, self)
compose_layout = QVBoxLayout(compose_group)

title_row = QHBoxLayout()
title_row.addWidget(QLabel(&quot;Title:&quot;, self))
self._title_edit = QLineEdit(self)
self._title_edit.setPlaceholderText(&quot;e.g. Auth Flow Fix&quot;)
self._title_edit.textChanged.connect(self._update_preview)
title_row.addWidget(self._title_edit, 1)
compose_layout.addLayout(title_row)

compose_layout.addWidget(QLabel(&quot;Prompt content:&quot;, self))
self._content_edit = QTextEdit(self)
self._content_edit.setPlaceholderText(&quot;Paste the full prompt text here…&quot;)
self._content_edit.setMinimumHeight(120)
compose_layout.addWidget(self._content_edit)

# Preview row
preview_row = QHBoxLayout()
preview_row.addWidget(QLabel(&quot;File path:&quot;, self))
self._preview_path = QLabel(&quot;—&quot;, self)
self._preview_path.setStyleSheet(&quot;color: #cba6f7; font-family: monospace;&quot;)
preview_row.addWidget(self._preview_path, 1)
copy_btn = QPushButton(&quot;Copy path&quot;, self)
copy_btn.clicked.connect(self._copy_path)
preview_row.addWidget(copy_btn)
compose_layout.addLayout(preview_row)

push_btn = QPushButton(&quot;Create and Push Prompt&quot;, self)
push_btn.setStyleSheet(
    &quot;QPushButton { background-color: #89b4fa; color: #1e1e2e; font-weight: 700; padding: 8px 20px; }
    &quot;QPushButton:hover { background-color: #b4d0fa; }&quot;
    &quot;QPushButton:disabled { background-color: #313244; color: #45475a; }&quot;
)
push_btn.clicked.connect(self._on_push)
compose_layout.addWidget(push_btn)
layout.addWidget(compose_group)

# ── History table ────────────────────────────────────────
history_group = QGroupBox(&quot;Prompt History&quot;, self)
history_layout = QVBoxLayout(history_group)
self._table = QTableWidget(self)
self._table.setColumnCount(len(_HISTORY_COLUMNS))
self._table.setHorizontalHeaderLabels(_HISTORY_COLUMNS)
self._table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
self._table.setSelectionBehavior(QTableWidget.SelectionBehavior.SelectRows)
self._table.horizontalHeader().setStretchLastSection(True)
history_layout.addWidget(self._table)
```

```
    layout.addWidget(history_group)

def refresh(self) -&gt; None:
    self._refresh_info()
    self._refresh_table()

def _refresh_info(self) -&gt; None:
    try:
        info = self._vm.get_active_info()
    except Exception as e:
        self._student_label.setText(f&quot;Error: {e}&quot;)
        return

    student = info.get(&quot;student&quot;)
    repo = info.get(&quot;repo&quot;)
    if student is None:
        self._student_label.setText(&quot;No active student&quot;)
        self._student_label.setStyleSheet(&quot;color: #f38ba8; font-size: 14px;&quot;)
        self._repo_label.setText(&quot;—&quot;)
        self._branch_label.setText(&quot;—&quot;)
    else:
        num = student.student_number or &quot;?&quot;
        self._student_label.setText(f&quot;#{num:02d}  {student.full_name}&quot; if isinstance(num, int) else f&q
        self._student_label.setStyleSheet(&quot;color: #a6e3a1; font-size: 14px; font-weight: 600;&quot;)
        if repo:
            self._repo_label.setText(repo.repo_name)
            self._branch_label.setText(f&quot;branch: {repo.default_branch}&quot;)
        else:
            self._repo_label.setText(&quot;(no repo)&quot;)
            self._branch_label.setText(&quot;—&quot;)
    self._update_preview()

def _update_preview(self) -&gt; None:
    try:
        info = self._vm.get_active_info()
    except Exception:
        return
    student = info.get(&quot;student&quot;)
    if student is None:
        self._preview_path.setText(&quot;—&quot;)
        return
    from app.services.prompt_service import _make_file_path, _make_slug
    num = student.student_number or 0
    slug = _make_slug(self._title_edit.text()) if self._title_edit.text().strip() else &quot;prompt&quot;
    # seq is preview only — show &quot;NNN&quot; as placeholder for next seq
    self._preview_path.setText(f&quot;prompts/student{num:02d}-NNN-{slug}.md&quot;)

def _copy_path(self) -&gt; None:
    from PySide6.QtWidgets import QApplication
    text = self._preview_path.text()
    if text and text != &quot;—&quot;:
        QApplication.clipboard().setText(text)

def _on_push(self) -&gt; None:
    title = self._title_edit.text().strip()
    content = self._content_edit.toPlainText().strip()
    if not content:
        QMessageBox.warning(self, &quot;Validation&quot;, &quot;Prompt content cannot be empty.&quot;)
        return
    try:
        prompt = self._vm.create_and_push(title=title, content=content)
    except Exception as e:
        QMessageBox.critical(self, &quot;Error&quot;, str(e))
        return

    if prompt.status == &quot;pushed&quot;:
        QMessageBox.information(
```

```
            self, &quot;Pushed&quot;,
            f&quot;Prompt saved and pushed.\nFile: {prompt.file_path}&quot;
        )
        self._title_edit.clear()
        self._content_edit.clear()
    else:
        QMessageBox.warning(
            self, &quot;Push Failed&quot;,
            f&quot;Committed but push failed.\nFile: {prompt.file_path}\nError: {prompt.push_error or &apos;unknc
        )
    self._refresh_table()

def _refresh_table(self) -&gt; None:
    try:
        prompts = self._vm.list_prompts()
    except Exception as e:
        return
    self._table.setRowCount(0)
    for row, p in enumerate(prompts):
        self._table.insertRow(row)
        self._table.setItem(row, 0, QTableWidgetItem(f&quot;#{p.student_number:02d}&quot; if p.student_numbe
        self._table.setItem(row, 1, QTableWidgetItem(f&quot;{p.seq_number:03d}&quot;))
        self._table.setItem(row, 2, QTableWidgetItem(p.title or &quot;—&quot;))
        self._table.setItem(row, 3, QTableWidgetItem(p.file_path))

        status_item = QTableWidgetItem(p.status)
        status_item.setForeground(QColor(_STATUS_COLORS.get(p.status, &quot;#cdd6f4&quot;)))
        self._table.setItem(row, 4, status_item)

        pushed = p.pushed_at.strftime(&quot;%Y-%m-%d %H:%M&quot;) if p.pushed_at else &quot;—&quot;
        self._table.setItem(row, 5, QTableWidgetItem(pushed))
        self._table.setItem(row, 6, QTableWidgetItem((p.git_commit_hash or &quot;&quot;)[:8] or &quot;—&quot;))

        cell = QWidget()
        cl = QHBoxLayout(cell)
        cl.setContentsMargins(2, 1, 2, 1)
        cl.setSpacing(4)

        cp_btn = QPushButton(&quot;Copy path&quot;)
        cp_btn.clicked.connect(lambda _=False, fp=p.file_path: self._copy_to_clipboard(fp))
        cl.addWidget(cp_btn)

        if p.status == &quot;failed&quot;:
            retry_btn = QPushButton(&quot;Retry Push&quot;)
            retry_btn.clicked.connect(lambda _=False, pid=p.id: self._retry(pid))
            cl.addWidget(retry_btn)

        self._table.setCellWidget(row, 7, cell)

def _copy_to_clipboard(self, text: str) -&gt; None:
    from PySide6.QtWidgets import QApplication
    QApplication.clipboard().setText(text)

def _retry(self, prompt_id: str) -&gt; None:
    try:
        prompt = self._vm.retry_push(prompt_id)
        if prompt.status == &quot;pushed&quot;:
            QMessageBox.information(self, &quot;Retry Success&quot;, &quot;Push succeeded.&quot;)
        else:
            QMessageBox.warning(self, &quot;Retry Failed&quot;, prompt.push_error or &quot;unknown error&quot
    except Exception as e:
        QMessageBox.critical(self, &quot;Error&quot;, str(e))
    self._refresh_table()
```

Step 2: Run UI skeleton tests

```
.venv/bin/pytest tests/test_ui_skeleton.py -v
```

Expected: all pass.

Step 3: Run full suite

.venv/bin/pytest tests/ -q

Expected: all pass.

Step 4: Commit

git add app/ui/windows/prompts_screen.py
git commit -m "feat: implement PromptsScreen with create/push/history UI"

Task 6: Documentation updates

Files:
- Modify: `README.md`
- Modify: `docs/operator-runbook.md` (create if missing)
- Modify: `docs/ui-flows.md`

Step 1: Add to README.md

Find the MVP features section and add a bullet for Prompts. Find limitations section and remove "Prompts" if listed there.

Add under "Features" or "What it does":

Prompt Delivery

Operators can write and push prompt files directly into the active student's Git repository from the **Prompt

- Prompts are stored as Markdown files under `prompts/` in the student's repo
- Filename format: `prompts/studentNN-SEQ-slug.md` (e.g. `prompts/student03-001-auth-flow-fix.md`)
- On save, StudentFlow commits and pushes the file to the student's GitHub/GitLab remote
- The operator receives the exact file path to paste into Lovable instead of a large prompt block
- All prompts are recorded in the database for later analysis

Step 2: Add to docs/ui-flows.md

Append a section for the Prompts screen:

Prompts Screen

Purpose: Create, commit, and push prompt files for the currently active student.

Layout:
- Active Student bar: shows student number, name, target repo, branch
- New Prompt form: Title field, content textarea, live file-path preview, Create and Push Prompt button
- Prompt History table: student#, seq, title, file path, status, pushed at, commit hash, actions (Copy path, Retry Pu

Workflow:
1. Operator opens Prompts screen — active student info is shown automatically
2. Operator fills in Title (optional) and prompt Content
3. File path preview updates live (`prompts/studentNN-NNN-slug.md`)
4. Operator clicks "Create and Push Prompt"
5. StudentFlow writes the file, commits it, pushes to `student` remote
6. On success: confirmation dialog with file path; history table updates
7. On push failure: warning dialog with error; prompt appears in history with status `failed`
8. Operator can click "Retry Push" on failed prompts

Operator hands off to student:

In Lovable, reference the file at: prompts/student03-001-auth-flow-fix.md

Step 3: Commit

git add README.md docs/
git commit -m "docs: document Prompt Delivery feature in README and ui-flows"

Acceptance Criteria Checklist

Before marking the feature complete, verify all of the following:

[ ] .venv/bin/pytest tests/ -q — all tests pass
[ ] GitClient.add_file() stages only the specified file
[ ] GitClient.commit() returns 40-char hash
[ ] PromptStatus enum has: draft, written, committed, pushed, failed
[ ] Student.student_number field exists and is indexed
[ ] Prompt table exists in DB with all required columns
[ ] Alembic migration chains correctly off 7ca526e3ce7e
[ ] PromptService.create_and_push_prompt() writes file, commits, pushes, stores hash
[ ] push_error never contains raw credentials
[ ] Failed push stores status=failed and safe error text
[ ] list_prompts() returns all or filtered by student_id
[ ] PromptsViewModel.get_active_info() returns active student and repo
[ ] Prompts appears in NAV_ITEMS at index 5
[ ] PromptsScreen shows active student, compose form, history table
[ ] Copy path button works
[ ] Retry Push appears on failed prompts
[ ] Docs updated in README and ui-flows.md
[ ] No unrelated files accidentally staged by git add_file()
[ ] Full suite passes: .venv/bin/pytest tests/ -q

---
codex-tasks/T13-tests.md
Розмір: 8,041 байт

T13: Write Comprehensive Test Suite

Goal

Write a complete unit test suite covering all services (AccountService, RepositoryService, CredentialService, SyncService, HandoffService, TemplateService, AuditService), GitClient, GitValidator, and Pydantic schemas. All tests must use the `db_session` fixture from `tests/conftest.py`. All Git operations must be mocked — no actual git repos created.

Context

- Project root: `/home/vokov/projects/studentflow/`
- All services (T02–T09) and validators (T12) are complete
- `tests/conftest.py` already provides `temp_db` and `db_session` fixtures
- Services accept a `session` as first constructor argument — inject `db_session` from fixture
- AuditService injection: use a `MagicMock` for `audit_service` in most service tests

Constraints

- NO test may create an actual git repository on the filesystem
- NO test may call `keyring.get_password()` or `keyring.set_password()` — mock `keyring` at the module level in `tests/conftest.py`
- Each test file must run independently: `pytest tests/test_X.py -v` must pass without running other files first
- No test may write to `~/.studentflow/` — use `tmp_path` from pytest for any filesystem needs
- Test isolation: each test gets a fresh DB session via the `db_session` fixture (rolls back after test)
- Coverage target: all public methods of all services must have at least one test

Files to Create/Update

```
tests/conftest.py              — add keyring mock fixture (update existing file)
tests/test_account_service.py   — AccountService tests
tests/test_repository_service.py — RepositoryService tests
tests/test_credential_service.py — CredentialService tests (already exists, extend)
tests/test_template_service.py  — TemplateService tests
tests/test_sync_service.py      — SyncService tests (already exists, extend)
tests/test_handoff_service.py   — HandoffService tests (already exists, extend)
tests/test_audit_service.py     — AuditService tests (already exists, extend)
tests/test_git_client.py        — GitClient tests
tests/test_validators.py        — GitValidator tests (already exists from T12)
tests/test_schemas.py           — Schema tests (already exists from T12)
```

Implementation Requirements

tests/conftest.py additions

Add to the existing file:

```
import pytest
from unittest.mock import MagicMock, patch

@pytest.fixture(autouse=True)
def mock_keyring():
    &quot;&quot;&quot;Mock keyring globally for all tests to prevent touching the OS keyring.&quot;&quot;&quot;
    with patch(&apos;keyring.get_password&apos;, return_value=None) as mock_get, \
         patch(&apos;keyring.set_password&apos;, return_value=None) as mock_set, \
         patch(&apos;keyring.delete_password&apos;, return_value=None) as mock_del:
        yield {&apos;get&apos;: mock_get, &apos;set&apos;: mock_set, &apos;delete&apos;: mock_del}


@pytest.fixture
def mock_git_client():
    &quot;&quot;&quot;Return a MagicMock with all GitClient methods pre-configured.&quot;&quot;&quot;
    client = MagicMock()
    client.clone.return_value = None
    client.fetch.return_value = None
    client.pull.return_value = None
    client.push.return_value = None
    client.merge.return_value = None
    client.tag.return_value = None
    client.get_head_commit.return_value = &quot;abc1234567890000&quot;
    client.is_diverged.return_value = False
    return client


@pytest.fixture
def mock_audit():
    &quot;&quot;&quot;Return a MagicMock AuditService that accepts .log() calls.&quot;&quot;&quot;
    audit = MagicMock()
    audit.log.return_value = None
```

return audit

tests/test_account_service.py

Minimum test cases:
1. `test_create_student_success` — creates Student, verifies in DB
2. `test_create_student_duplicate_name_same_status` — raises `DuplicateStudentError`
3. `test_list_students_returns_all` — creates 3 students, list returns 3
4. `test_archive_student` — sets status to `archived`, not returned in active list
5. `test_set_queue_position` — changes queue_position, reflected in DB
6. `test_get_next_in_queue` — 3 students with positions 1, 2, 3; returns student with position 1
7. `test_create_account_success` — creates Account linked to Student
8. `test_deactivate_account` — sets account status to inactive

tests/test_credential_service.py

Minimum test cases:
1. `test_store_credential_calls_keyring_set` — `mock_keyring['set']` was called with correct service name and username
2. `test_retrieve_credential_calls_keyring_get` — `mock_keyring['get']` called, returns mocked value
3. `test_retrieve_missing_credential_raises` — keyring returns None → raises `CredentialNotFoundError`
4. `test_rotate_credential` — old key deleted, new key set, Credential.last_validated_at updated
5. `test_secret_ref_format` — `secret_ref` matches pattern `studentflow:{account_id}:{auth_type}`
6. `test_validate_credential_ssh` — validation calls GitClient with correct credential, returns True on success
7. `test_validate_credential_marks_db_timestamp` — after successful validation, `Credential.last_validated_at` is updated

tests/test_sync_service.py

Minimum test cases:
1. `test_sync_student_creates_sync_job_record`
2. `test_sync_student_marks_job_completed_on_success`
3. `test_sync_student_marks_job_failed_on_git_error`
4. `test_sync_all_continues_after_one_student_fails` — 3 students, first raises, other 2 complete
5. `test_sync_job_error_message_truncated_to_500` — error message longer than 500 chars is truncated
6. `test_sync_job_error_message_no_credentials` — error message never contains PAT/token patterns

tests/test_handoff_service.py

Minimum test cases (extend existing file):
1. `test_handoff_success`
2. `test_handoff_no_active_student`
3. `test_handoff_no_next_student`
4. `test_handoff_rollback_on_push_failure`
5. `test_get_next_student_skips_archived`
6. `test_checkpoint_tag_format`
7. `test_handoff_creates_handoff_event_record` — verify HandoffEvent in DB after success
8. `test_handoff_sets_from_student_exhausted` — from_student.status == exhausted after handoff

tests/test_template_service.py

Minimum test cases:
1. `test_import_template_from_local_folder` — copies files from `tmp_path/template/` to `tmp_path/dest/`
2. `test_import_template_missing_source` — raises `TemplateNotFoundError`
3. `test_list_templates` — returns only is_active templates
4. `test_template_checksum_stored` — TemplateSource.checksum is non-empty after import

tests/test_git_client.py

`GitClient` wraps `subprocess` calls — mock at `subprocess.run` level.

Minimum test cases:
1. `test_clone_calls_subprocess_with_correct_args`
2. `test_fetch_calls_subprocess`

3. `test_push_calls_subprocess`
4. `test_tag_includes_correct_tag_name`
5. `test_get_head_commit_returns_hash_string`
6. `test_git_error_raises_git_operation_error` — subprocess returncode != 0 raises `GitOperationError`

Acceptance Criteria

1. `pytest tests/ -v --tb=short` — all tests pass with 0 failures, 0 errors
2. `pytest tests/ --co -q` — shows at least 45 collected tests
3. No test writes to `~/.studentflow/` (verified by checking `tmp_path` usage)
4. `pytest tests/ -v -k "credential"` — all credential tests pass, `mock_keyring` fixture confirms no real keyring calls
5. Running `pytest tests/test_sync_service.py` in isolation passes without importing other test files

Notes for Codex

- When mocking `subprocess.run`, use `unittest.mock.patch('subprocess.run')` — not `app.git.client.subprocess.run`, unless GitClient imports subprocess at module level (check the import)
- `mock_keyring` is `autouse=True` — it applies to ALL tests automatically; you do not need to add it to every test's arguments
- For `test_sync_job_error_message_no_credentials`: create an error message containing `ghp_abc123xyz789testtoken0000000000`, call sync_student, verify the stored `error_message` does NOT contain that string
- Tests that check DB state must query via `db_session` not via the service, to avoid false positives from caching
- The `mock_git_client` fixture returns a `MagicMock` — set `.return_value` explicitly for each method you test

---
codex-tasks/T03-credential-service.md
Розмір: 14,976 байт

T03: Implement CredentialService

Goal

Implement secure keyring-based credential storage in `app/services/credential_service.py` and `app/security/vault.py`. CredentialService is the only module in the codebase that reads or writes the OS keyring. It also manages `Credential` DB records (key references only — never values).

Context

- Project root: `/home/vokov/projects/studentflow/`
- Security spec: `docs/security.md` — read the "Credential Storage" and "Git Authentication" sections
- Models: `app/db/models.py` — `Credential` and `Account` entities (from T02)
- Enums: `app/db/enums.py` — `SecretKind`, `AccountProvider`
- Stack: `keyring==25.2.1`

Constraints

- CredentialService must be the ONLY module that calls `keyring.get_password()` and `keyring.set_password()`
- The `value` parameter to `store()` and `rotate()` must never be logged, stored in DB, or stored in any instance variable
- `validate_exists()` must call `keyring.get_password()` — NOT a real auth test (that's a separate method)
- `retrieve()` must return the raw value from keyring; callers are responsible for not logging it
- The key naming convention must be strictly followed: `studentflow:{provider}:{username}:{account_id[:8]}`
- `mask_secret()` in `masking.py` must be implemented and tested separately

Files to Create/Update

```
app/security/vault.py            — SecretVault (thin keyring wrapper)
app/security/masking.py          — mask_secret() function (replace stub from T01)
app/services/credential_service.py  — CredentialService (replace stub from T01)
tests/test_credential_service.py    — unit tests with mocked keyring
```

Implementation Requirements

```
### app/security/vault.py

&quot;&quot;&quot;SecretVault: thin wrapper around the OS keyring via the keyring library.&quot;&quot;&quot;
import keyring
import keyring.errors
from typing import Optional


KEYRING_SERVICE = &quot;studentflow&quot;


class SecretVault:
    &quot;&quot;&quot;Single interface for all OS keyring operations.

    All methods take a fully-qualified key name (secret_ref) as used
    in the Credential.secret_ref DB field.

    Key format: &quot;studentflow:{provider}:{username}:{account_id[:8]}&quot;
    Example:    &quot;studentflow:github:alice-dev:a1b2c3d4&quot;
    &quot;&quot;&quot;

    def set(self, secret_ref: str, value: str) -&gt; None:
        &quot;&quot;&quot;Store or overwrite a secret in the OS keyring.

        Args:
            secret_ref: The fully-qualified key name (e.g., &quot;studentflow:github:alice:a1b2c3d4&quot;)
            value: The secret value to store (PAT, password, passphrase)
        &quot;&quot;&quot;
        keyring.set_password(KEYRING_SERVICE, secret_ref, value)

    def get(self, secret_ref: str) -&gt; Optional[str]:
        &quot;&quot;&quot;Retrieve a secret from the OS keyring.

        Returns:
            The stored value, or None if the key does not exist.
        &quot;&quot;&quot;
        return keyring.get_password(KEYRING_SERVICE, secret_ref)

    def delete(self, secret_ref: str) -&gt; None:
        &quot;&quot;&quot;Delete a secret from the OS keyring.
```

```
        Raises:
            keyring.errors.PasswordDeleteError if the key does not exist.
        &quot;&quot;&quot;
        keyring.delete_password(KEYRING_SERVICE, secret_ref)

    def exists(self, secret_ref: str) -&gt; bool:
        &quot;&quot;&quot;Check whether a key exists in the keyring (does not validate the value).

        Returns:
            True if keyring.get_password returns a non-None value.
        &quot;&quot;&quot;
        return keyring.get_password(KEYRING_SERVICE, secret_ref) is not None

    @staticmethod
    def make_key(provider: str, username: str, account_id: str) -&gt; str:
        &quot;&quot;&quot;Construct the standardized secret_ref key name.

        Args:
            provider: e.g., &quot;github&quot;, &quot;gitlab&quot;, &quot;bitbucket&quot;
            username: e.g., &quot;alice-dev&quot;
            account_id: UUID string; first 8 chars used

        Returns:
            e.g., &quot;studentflow:github:alice-dev:a1b2c3d4&quot;
        &quot;&quot;&quot;
        return f&quot;studentflow:{provider}:{username}:{account_id[:8]}&quot;


### app/security/masking.py

&quot;&quot;&quot;Credential masking utilities for safe display in UI and logs.&quot;&quot;&quot;


def mask_secret(value: str) -&gt; str:
    &quot;&quot;&quot;Return a masked version of a secret, showing only the last 4 characters.

    Used in UI display fields and log output. Never call this with the intent
    to log the result — the unmasked value must never appear in logs at all.

    Examples:
        mask_secret(&quot;ghp_abc123xyz789&quot;) -&gt; &quot;••••••••z789&quot;
        mask_secret(&quot;ab&quot;) -&gt; &quot;••••&quot;
        mask_secret(&quot;&quot;) -&gt; &quot;••••&quot;
        mask_secret(&quot;abcd&quot;) -&gt; &quot;••••&quot;  (exactly 4 chars: show dots only)
    &quot;&quot;&quot;
    if not value or len(value) &lt;= 4:
        return &quot;••••&quot;
    return &quot;••••••••&quot; + value[-4:]


def mask_url(url: str) -&gt; str:
    &quot;&quot;&quot;Remove any embedded credentials from a git remote URL.

    Converts: https://token:x-oauth-basic@github.com/user/repo.git
    To:       https://github.com/user/repo.git

    Used before logging remote URLs.
    &quot;&quot;&quot;
    import re
    return re.sub(r&apos;https?://[^@]+@&apos;, lambda m: m.group(0).split(&apos;//&apos;)[0] + &apos;//&apos;


### app/services/credential_service.py

&quot;&quot;&quot;CredentialService: manages keyring-based credentials and Credential DB records.

This is the only service that interacts with the OS keyring directly.
All other services that need credentials must call this service.
```

```
&quot;&quot;&quot;
from __future__ import annotations
import logging
from datetime import datetime
from typing import Optional

import structlog

from app.db.base import SessionLocal
from app.db.models import Credential, Account
from app.db.enums import SecretKind
from app.security.vault import SecretVault

logger = structlog.get_logger(__name__)


class CredentialService:
    &quot;&quot;&quot;Manages OS keyring secrets and their corresponding Credential DB records.

    Key naming: studentflow:{provider}:{username}:{account_id[:8]}
    DB stores: only the key name (secret_ref) — never the secret value.
    &quot;&quot;&quot;

    def __init__(self, vault: Optional[SecretVault] = None):
        self._vault = vault or SecretVault()

    def store(self, account_id: str, secret_kind: SecretKind, value: str) -&gt; Credential:
        &quot;&quot;&quot;Store a new credential in keyring and create a Credential DB record.

        Args:
            account_id: UUID of the Account this credential belongs to
            secret_kind: Type of secret (pat, ssh_key, app_password, password)
            value: The actual secret — stored ONLY in keyring, never in DB

        Returns:
            The created Credential record (contains secret_ref, not value)

        Raises:
            ValueError: if an Account with account_id does not exist
            ValueError: if a Credential already exists for this account (use rotate() instead)
        &quot;&quot;&quot;
        with SessionLocal() as session:
            account = session.get(Account, account_id)
            if account is None:
                raise ValueError(f&quot;Account {account_id} not found&quot;)

            # Check for existing credential
            existing = session.query(Credential).filter_by(account_id=account_id).first()
            if existing is not None:
                raise ValueError(
                    f&quot;Credential already exists for account {account_id}. Use rotate() to update.&quot;
                )

            secret_ref = SecretVault.make_key(
                provider=account.provider,
                username=account.username,
                account_id=account_id,
            )

            # Store in keyring FIRST — if this fails, no DB record is created
            self._vault.set(secret_ref, value)

            credential = Credential(
                account_id=account_id,
                secret_ref=secret_ref,
                secret_kind=secret_kind.value if isinstance(secret_kind, SecretKind) else secret_kind,
                is_encrypted=True,
```

```
            created_at=datetime.utcnow(),
            updated_at=datetime.utcnow(),
        )
        session.add(credential)
        session.commit()

        logger.info(&quot;credential_stored&quot;, account_id=account_id, secret_kind=str(secret_kind))
        return credential

    def retrieve(self, secret_ref: str) -&gt; str:
        &quot;&quot;&quot;Retrieve a credential value from the OS keyring.

        Args:
            secret_ref: The keyring key name (from Credential.secret_ref)

        Returns:
            The stored secret value

        Raises:
            KeyError: if no value found at secret_ref in keyring
        &quot;&quot;&quot;
        value = self._vault.get(secret_ref)
        if value is None:
            logger.warning(&quot;credential_not_found&quot;, secret_ref=secret_ref)
            raise KeyError(f&quot;No credential found for key: {secret_ref}&quot;)
        return value

    def rotate(self, credential_id: str, new_value: str) -&gt; None:
        &quot;&quot;&quot;Replace an existing credential value in the keyring.

        The secret_ref (key name) does not change. Only the stored value changes.
        Updates Credential.last_validated_at and updated_at.

        Args:
            credential_id: UUID of the Credential record to rotate
            new_value: The new secret value — stored ONLY in keyring, never in DB

        Raises:
            ValueError: if no Credential with credential_id exists
        &quot;&quot;&quot;
        with SessionLocal() as session:
            credential = session.get(Credential, credential_id)
            if credential is None:
                raise ValueError(f&quot;Credential {credential_id} not found&quot;)

            # Overwrite in keyring first
            self._vault.set(credential.secret_ref, new_value)

            # Update timestamps
            credential.last_validated_at = datetime.utcnow()
            credential.updated_at = datetime.utcnow()
            session.commit()

            logger.info(
                &quot;credential_rotated&quot;,
                credential_id=credential_id,
                account_id=credential.account_id,
            )

    def delete(self, secret_ref: str) -&gt; None:
        &quot;&quot;&quot;Remove a credential from keyring and delete the Credential DB record.

        Args:
            secret_ref: The keyring key name (from Credential.secret_ref)
        &quot;&quot;&quot;
        with SessionLocal() as session:
            credential = session.query(Credential).filter_by(secret_ref=secret_ref).first()
```

```
        # Delete from keyring first
        try:
            self._vault.delete(secret_ref)
        except Exception as e:
            logger.warning(&quot;credential_delete_keyring_error&quot;, secret_ref=secret_ref, error=str(e))

        if credential is not None:
            session.delete(credential)
            session.commit()
            logger.info(&quot;credential_deleted&quot;, account_id=credential.account_id)

    def validate_exists(self, secret_ref: str) -&gt; bool:
        &quot;&quot;&quot;Check that a keyring entry exists for the given secret_ref.

        This does NOT test the credential against the provider.
        It only checks the keyring contains a non-None value.

        Returns:
            True if the keyring has a value for this key, False otherwise
        &quot;&quot;&quot;
        return self._vault.exists(secret_ref)

    def get_by_account(self, account_id: str) -&gt; Optional[Credential]:
        &quot;&quot;&quot;Retrieve the Credential DB record for an account (not the value).

        Returns:
            Credential record (with secret_ref), or None if no credential configured
        &quot;&quot;&quot;
        with SessionLocal() as session:
            return session.query(Credential).filter_by(account_id=account_id).first()
```

Tests (tests/test_credential_service.py)

Use `unittest.mock.patch` to mock `keyring.get_password`, `keyring.set_password`, `keyring.delete_password`.

Write these tests:

test_store_creates_credential_record:
- Create a Student + Account in test DB
- Call `CredentialService.store(account_id, SecretKind.pat, "ghp_test123")`
- Assert `keyring.set_password` was called with service="studentflow", username=secret_ref, password="ghp_test123"
- Assert the returned Credential has `secret_ref` set
- Assert the returned Credential has NO attribute `value`, `token`, or `password`
- Query the Credential from DB — verify `secret_ref` is stored but no secret value

test_retrieve_returns_value:
- Mock `keyring.get_password` to return "ghp_fake_token"
- Call `CredentialService.retrieve("studentflow:github:alice:a1b2c3d4")`
- Assert return value is "ghp_fake_token"

test_retrieve_raises_on_missing:
- Mock `keyring.get_password` to return `None`
- Call `CredentialService.retrieve("studentflow:github:missing:xxxxxxxx")`
- Assert `KeyError` is raised

test_rotate_updates_keyring:
- Create Student + Account + Credential (via store())
- Mock keyring, call `rotate(credential_id, "new_ghp_token")`
- Assert `keyring.set_password` called with the new value
- Assert `Credential.last_validated_at` is not None after rotate

test_delete_removes_from_keyring_and_db:
- Create Student + Account + Credential
- Call `delete(credential.secret_ref)`

- Assert `keyring.delete_password` was called
- Assert querying Credential from DB returns None

test_validate_exists_true:
- Mock `keyring.get_password` to return "some_value"
- Assert `validate_exists("studentflow:github:alice:a1b2c3d4")` returns True

test_validate_exists_false:
- Mock `keyring.get_password` to return None
- Assert `validate_exists("studentflow:github:missing:xxxxxxxx")` returns False

test_mask_secret:
from app.security.masking import mask_secret
def test_mask_secret():
    assert mask_secret("ghp_abc123xyz789") == "••••••••z789"
    assert mask_secret("abcd") == "••••"
    assert mask_secret("") == "••••"
    assert mask_secret("abc") == "••••"
    assert mask_secret("abcde") == "••••••••bcde"

test_secret_key_naming:
from app.security.vault import SecretVault
def test_secret_key_naming():
    key = SecretVault.make_key("github", "alice-dev", "a1b2c3d4-e5f6-7890-abcd-ef12
    assert key == "studentflow:github:alice-dev:a1b2c3d"
    # Uses first 8 chars of UUID: a1b2c3d4

Wait — correct the test to verify: `account_id[:8]` of "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
is "a1b2c3d4":
key = SecretVault.make_key("github", "alice-dev", "a1b2c3d4-e5f6-7890-abcd-ef1234
assert key == "studentflow:github:alice-dev:a1b2c3d4"

Acceptance Criteria

1. `from app.services.credential_service import CredentialService` — imports without error
2. `from app.security.vault import SecretVault` — imports without error
3. `from app.security.masking import mask_secret, mask_url` — imports without error
4. All 9 tests pass: `pytest tests/test_credential_service.py -v` shows 9 passed
5. No call to `keyring.get_password` appears outside `app/security/vault.py`
6. No secret value appears in any structlog output (verify by checking log output in tests)
7. `Credential` record returned by `store()` has no attribute named `value`, `token`, `password`, or `secret`

---
codex-tasks/T09-audit-service.md
Розмір: 5,351 байт

T09: Implement AuditService

Goal

Implement `AuditService` in `app/services/audit_service.py`. Every state-changing action in StudentFlow (create, update, delete, sync, handoff, credential rotation, error) must emit a structured `ActivityLog` record to the SQLite DB and a structured JSON line to the log file via `structlog`. The service must never block the caller — it swallows its own exceptions and logs them to stderr only.

Context

- Project root: `/home/vokov/projects/studentflow/`
- Read `docs/audit-and-logging.md` in full before writing any code.
- Model: `ActivityLog` in `app/db/models.py` (T02)
- Structlog is configured in `app/bootstrap.py::init_logging()` — use `structlog.get_logger(__name__)`
- All other services inject `AuditService` as a dependency — it must be safe to call from any service

Constraints

- `AuditService.log()` must NEVER raise — catch all exceptions, emit to stderr, return silently
- Credential values must never appear in `details` — callers are responsible for masking, but `AuditService` also strips common secret patterns as a last-resort guard (see masking section below)
- `actor` field: use `"operator"` as default; accept string override for future automation
- `entity_type` must be one of: `student`, `account`, `repository`, `credential`, `sync_job`, `handoff_event`, `app_setting`, `system`
- `details` must be a `dict[str, Any]` — serialised to JSON string in the DB column
- `ActivityLog.id` is a UUID string
- Each `log()` call results in EXACTLY one DB insert — no batching

Files to Create/Update

app/services/audit_service.py — full implementation
tests/test_audit_service.py — unit tests

Implementation Requirements

AuditService class

```
import structlog
from typing import Any

log = structlog.get_logger(__name__)

_SECRET_PATTERNS = [
    # Strips common token prefixes from dict values
    r&apos;ghp_[A-Za-z0-9]{36}&apos;,
    r&apos;glpat-[A-Za-z0-9\-_]{20}&apos;,
    r&apos;github_pat_[A-Za-z0-9_]{82}&apos;,
]


class AuditService:
    def __init__(self, session):
        self._session = session

    def log(
        self,
        action: str,
        entity_type: str,
        entity_id: str | None = None,
        actor: str = &quot;operator&quot;,
        details: dict[str, Any] | None = None,
    ) -&gt; None:
        &quot;&quot;&quot;
        Write one ActivityLog record to DB and one JSON line to structlog.
        Never raises. If DB write fails, falls back to structlog only.
```

"""

    def get_logs(
        self,
        entity_type: str | None = None,
        entity_id: str | None = None,
        action: str | None = None,
        limit: int = 200,
        offset: int = 0,
) -> list[ActivityLog]:
        """Query ActivityLog with optional filters. Returns list, never raises."""

    def _sanitise_details(self, details: dict[str, Any]) -> dict[str, Any]:
        """Strip values matching _SECRET_PATTERNS. Returns new dict, does not mutate input.&quo

Log record structure in structlog output
Every call to `log()` emits a structlog event with these fields:
{
  "event": "audit",
  "action": "student_created",
  "entity_type": "student",
  "entity_id": "uuid-here",
  "actor": "operator",
  "details": {"full_name": "Alice", "status": "active"},
  "timestamp": "2025-01-01T12:00:00Z",
  "level": "info"
}

Standard action names (use these exactly across all services)
student_created, student_updated, student_archived
account_created, account_updated, account_deactivated
repo_created, repo_updated, repo_deleted
credential_stored, credential_rotated, credential_validated, credential_validation_failed
sync_started, sync_completed, sync_failed
handoff_started, handoff_completed, handoff_failed
template_imported
settings_updated
error

Tests

File: `tests/test_audit_service.py`

Test cases:

1. `test_log_creates_activity_log_record` — after `log('student_created', 'student', ...)`, one ActivityLog record exists in DB
2. `test_log_does_not_raise_on_db_error` — patch session.add to raise, verify `log()` returns silently
3. `test_log_sanitises_secrets_in_details` — pass a details dict containing a `ghp_` token, verify it's not stored literally
4. `test_get_logs_filters_by_entity_type` — create 3 logs of different types, filter by one type, get 1 result
5. `test_get_logs_filters_by_action` — filter by action name
6. `test_get_logs_respects_limit` — create 10 logs, limit=3 returns 3

Acceptance Criteria

1. `AuditService.log(...)` called with valid args creates one `ActivityLog` row
2. `AuditService.log(...)` with a broken DB session does NOT raise — returns None silently

3. A `ghp_`-pattern value in `details` is redacted before being stored
4. `AuditService.get_logs(entity_type='student')` returns only student-entity logs
5. All 6 tests pass: `pytest tests/test_audit_service.py -v`

Notes for Codex

- `AuditService` is the only service that must be entirely non-throwing — all others may raise
- `_sanitise_details` uses `re.sub` on JSON-serialised values, not key inspection (simpler and safer)
- The `details` column in `ActivityLog` is `Text` — store `json.dumps(sanitised_details)`
- Structlog call must come AFTER the DB insert attempt, not before, so we log what actually happened
- Do not import `keyring` in this file — no credential operations here

---
codex-tasks/T10-ui-skeleton.md
Розмір: 7,041 байт

T10: Implement Desktop UI Skeleton

Goal

Implement the PySide6 `MainWindow` with a sidebar navigation, a central stacked widget, and seven placeholder screen widgets. No real data yet — all screens show a title label and "Not implemented yet" placeholder. The window must launch, display, navigate between screens, and not crash.

Context

- Project root: `/home/vokov/projects/studentflow/`
- Read `docs/ui-flows.md` in full before writing any code.
- Entry point: `app/main.py` already imports `from app.ui.windows.main_window import MainWindow`
- Stack: PySide6 6.7+, no external UI libraries (no Qt Designer .ui files — code-first only)
- Theme: dark-ish neutral desktop app — use QPalette with dark background (#1e1e2e), light text (#cdd6f4), accent (#89b4fa). No stylesheets longer than 30 lines.

Constraints

- MainWindow must be resizable, minimum size 1024×700
- Sidebar must show 7 navigation items (icons + labels): Dashboard, Students, Accounts, Repositories, Credentials, Logs, Settings
- Clicking a sidebar item switches the central stacked widget to that screen
- The active sidebar item must be visually highlighted
- No hardcoded pixel positions — use layouts only (QVBoxLayout, QHBoxLayout, QSplitter)
- Every screen widget lives in its own file (one class per file)
- Screen widgets must accept no constructor arguments other than `parent=None`
- `MainWindow` must not import any service or DB class — UI layer only in this task

Files to Create/Update

app/ui/windows/main_window.py — MainWindow class
app/ui/windows/dashboard_screen.py — DashboardScreen widget
app/ui/windows/students_screen.py — StudentsScreen widget

```
app/ui/windows/accounts_screen.py     — AccountsScreen widget
app/ui/windows/repos_screen.py        — RepositoriesScreen widget
app/ui/windows/credentials_screen.py  — CredentialsScreen widget
app/ui/windows/logs_screen.py         — LogsScreen widget
app/ui/windows/settings_screen.py     — SettingsScreen widget
```

Implementation Requirements

```
### MainWindow structure

from PySide6.QtWidgets import (
    QMainWindow, QWidget, QHBoxLayout, QVBoxLayout,
    QStackedWidget, QPushButton, QLabel, QSizePolicy, QSplitter
)
from PySide6.QtCore import Qt
from PySide6.QtGui import QPalette, QColor

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle(&quot;StudentFlow&quot;)
        self.setMinimumSize(1024, 700)
        self._setup_palette()
        self._build_ui()

    def _setup_palette(self) -&gt; None:
        &quot;&quot;&quot;Apply dark theme via QPalette.&quot;&quot;&quot;
        palette = QPalette()
        palette.setColor(QPalette.ColorRole.Window, QColor(&quot;#1e1e2e&quot;))
        palette.setColor(QPalette.ColorRole.WindowText, QColor(&quot;#cdd6f4&quot;))
        palette.setColor(QPalette.ColorRole.Base, QColor(&quot;#181825&quot;))
        palette.setColor(QPalette.ColorRole.AlternateBase, QColor(&quot;#313244&quot;))
        palette.setColor(QPalette.ColorRole.Button, QColor(&quot;#313244&quot;))
        palette.setColor(QPalette.ColorRole.ButtonText, QColor(&quot;#cdd6f4&quot;))
        palette.setColor(QPalette.ColorRole.Highlight, QColor(&quot;#89b4fa&quot;))
        palette.setColor(QPalette.ColorRole.HighlightedText, QColor(&quot;#1e1e2e&quot;))
        self.setPalette(palette)

    def _build_ui(self) -&gt; None:
        &quot;&quot;&quot;Construct sidebar + stacked widget layout.&quot;&quot;&quot;
        # Central widget
        central = QWidget()
        self.setCentralWidget(central)
        root_layout = QHBoxLayout(central)
        root_layout.setContentsMargins(0, 0, 0, 0)
        root_layout.setSpacing(0)

        # Sidebar
        self._sidebar = self._build_sidebar()
        root_layout.addWidget(self._sidebar)

        # Stacked screens
        self._stack = QStackedWidget()
        self._load_screens()
        root_layout.addWidget(self._stack, stretch=1)

    def _build_sidebar(self) -&gt; QWidget: ...
        # Returns a 200px-wide QWidget with vertical list of nav buttons

    def _load_screens(self) -&gt; None: ...
        # Imports and instantiates each screen, adds to self._stack

    def _navigate_to(self, index: int) -&gt; None: ...
        # Switches stacked widget, updates active button highlight
```

Sidebar nav items (in order)
```
NAV_ITEMS = [
    (0, &quot;Dashboard&quot;),
    (1, &quot;Students&quot;),
    (2, &quot;Accounts&quot;),
    (3, &quot;Repositories&quot;),
    (4, &quot;Credentials&quot;),
    (5, &quot;Logs&quot;),
    (6, &quot;Settings&quot;),
]
```

Each button: `QPushButton`, full-width, flat style, left-aligned text, highlighted when active with accent colour background.

Each screen widget (placeholder pattern)
```
from PySide6.QtWidgets import QWidget, QVBoxLayout, QLabel
from PySide6.QtCore import Qt

class DashboardScreen(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        layout = QVBoxLayout(self)
        layout.setAlignment(Qt.AlignmentFlag.AlignTop)
        title = QLabel(&quot;Dashboard&quot;)
        title.setStyleSheet(&quot;font-size: 22px; font-weight: bold; padding: 16px 0;&quot;)
        layout.addWidget(title)
        placeholder = QLabel(&quot;Dashboard content — not implemented yet&quot;)
        placeholder.setStyleSheet(&quot;color: #6c7086; padding: 8px 0;&quot;)
        layout.addWidget(placeholder)
```

Apply same pattern to all 7 screens (change class name and title text).

Status bar
Add a `QStatusBar` to `MainWindow`. Set initial message: `"StudentFlow ready."` This will later show sync status.

Tests

File: `tests/ui/test_main_window.py`

Use `pytest-qt` (`qtbot` fixture). Tests must NOT require a display (`QApplication` is created by `qtbot`).

Test cases:

1. `test_main_window_opens` — `MainWindow()` constructs without error, `window.isVisible()` is False (not shown in test)
2. `test_sidebar_has_seven_buttons` — sidebar widget has exactly 7 `QPushButton` children
3. `test_navigate_to_students` — click Students button, `stack.currentIndex() == 1`
4. `test_navigate_to_settings` — click Settings button, `stack.currentIndex() == 6`
5. `test_window_minimum_size` — `minimumWidth() >= 1024` and `minimumHeight() >= 700`

Acceptance Criteria

1. `python -m app.main` launches and shows MainWindow without errors (manual check)
2. All 7 sidebar nav buttons are visible in a vertical list
3. Clicking each nav button switches the central content area to the corresponding screen
4. Active nav button is visually distinct from inactive buttons
5. Window title is "StudentFlow"
6. All 5 pytest-qt tests pass: `pytest tests/ui/test_main_window.py -v`

Notes for Codex

- Do NOT use Qt Designer `.ui` files — all layout must be code
- Do NOT import any `app.services.*` or `app.db.*` in UI files in this task
- On Linux without a display, set `QT_QPA_PLATFORM=offscreen` in the test environment — `pytest-qt` handles this automatically with the `qtbot` fixture
- Keep total stylesheet string length under 300 characters across all files combined — prefer QPalette over heavy CSS
- The `_load_screens()` method must use lazy imports inside the function to avoid circular dependencies at module level

---
codex-tasks/T11-ui-screens.md
Розмір: 7,862 байт

T11: Implement UI Screens — Students, Accounts, Repositories, Credentials, Logs

Goal

Replace the five placeholder screen widgets (StudentsScreen, AccountsScreen, RepositoriesScreen, CredentialsScreen, LogsScreen) with functional data-driven screens. Each screen loads data from the DB via the appropriate service, displays it in a table, and provides action buttons. Dialogs for create/edit open modal windows. DashboardScreen and SettingsScreen are deferred to Phase 2 (leave placeholders).

Context

- Project root: `/home/vokov/projects/studentflow/`
- Read `docs/ui-flows.md` — especially the screen-by-screen spec sections
- Read `docs/security.md` — credential masking rules apply to CredentialsScreen
- UI skeleton (T10) must be complete — this task replaces content of existing screen files
- Services: AccountService (T02/T04), RepositoryService (T05), CredentialService (T03), AuditService (T09)
- ViewModels: each screen has a corresponding ViewModel in `app/ui/viewmodels/`
- Signal/slot pattern: service calls happen in the ViewModel; the screen only calls ViewModel methods

Constraints

- All service calls must be wrapped in try/except — never crash the UI on DB or Git errors
- Errors shown via `QMessageBox.critical(self, "Error", str(e))` — never printed to stdout
- Credential values (tokens, passwords) MUST be displayed masked: `mask_secret()` from `app/security/masking.py`
- Tables must use `QTableWidget` (not `QTableView` + model for MVP simplicity)
- No async/QThread in this task — service calls are synchronous from the UI thread
- Action buttons: always placed in a toolbar QWidget above the table, not in table cells
- Each screen file must not exceed ~250 lines — extract dialogs to separate files if needed

Files to Create/Update

app/ui/windows/students_screen.py — full implementation

```
app/ui/windows/accounts_screen.py          — full implementation
app/ui/windows/repos_screen.py             — full implementation
app/ui/windows/credentials_screen.py       — full implementation
app/ui/windows/logs_screen.py              — full implementation (read-only)
app/ui/viewmodels/students_viewmodel.py    — StudentViewModel
app/ui/viewmodels/accounts_viewmodel.py    — AccountViewModel
app/ui/viewmodels/repos_viewmodel.py       — RepoViewModel
app/ui/viewmodels/credentials_viewmodel.py — CredentialViewModel
app/ui/viewmodels/logs_viewmodel.py        — LogViewModel
app/ui/dialogs/student_dialog.py           — Add/Edit Student dialog
app/ui/dialogs/account_dialog.py           — Add/Edit Account dialog
app/ui/dialogs/credential_dialog.py        — Add/Rotate Credential dialog
```

Implementation Requirements

```
### ViewModel pattern (use for all 5 screens)

# app/ui/viewmodels/students_viewmodel.py
from app.db.base import SessionLocal
from app.services.account_service import AccountService

class StudentViewModel:
    def __init__(self):
        self._session = SessionLocal()
        self._service = AccountService(self._session)

    def list_students(self) -&gt; list:
        return self._service.list_students()

    def create_student(self, data: dict) -&gt; None:
        self._service.create_student(**data)

    def update_student(self, student_id: str, data: dict) -&gt; None:
        self._service.update_student(student_id, **data)

    def archive_student(self, student_id: str) -&gt; None:
        self._service.archive_student(student_id)

    def reorder_queue(self, student_id: str, new_position: int) -&gt; None:
        self._service.set_queue_position(student_id, new_position)

    def close(self) -&gt; None:
        self._session.close()


### StudentsScreen columns
| # | Full Name | Status | Queue Position | Notes | Actions |

- Actions column: two `QPushButton` per row: "Edit", "Archive"
- Toolbar buttons: "Add Student", "Refresh"
- Row double-click opens Edit dialog
- Status shown as coloured badge label: active=green, paused=yellow, exhausted=grey,
error=red, archived=dim

### AccountsScreen columns
| Username | Provider | Auth Type | Status | Linked Student | Last Validated | Actions |

- Actions: "Edit", "Validate"
- Toolbar: "Add Account", "Refresh"

### RepositoriesScreen columns
| Repo Name | Local Path | Remote URL | Branch | Sync Status | Last Commit | Actions |

- Actions: "Validate Path", "Validate Remote", "Open in File Manager"
- Toolbar: "Add Repository", "Refresh"
```

- Sync status shown as coloured text: ok=green, error=red, pending=yellow, unknown=grey

CredentialsScreen columns
| Account | Auth Type | Secret (masked) | Last Validated | Actions |

- `Secret (masked)` column: always show output of `mask_secret(value)` — NEVER raw value
- Actions: "Validate", "Rotate"
- Toolbar: "Add Credential", "Refresh"
- Rotate opens `CredentialDialog` with password field (input mode: `QLineEdit.Password`)

LogsScreen columns
| Timestamp | Actor | Action | Entity Type | Entity ID | Details |

- Read-only, no edit actions
- Toolbar: filter by entity_type (QComboBox), filter by action (QLineEdit), "Refresh", "Export CSV"
- Export CSV writes to `~/.studentflow/logs/export-{timestamp}.csv`
- Loads last 200 records by default via `AuditService.get_logs(limit=200)`

Add/Edit Student dialog (student_dialog.py)
Fields:
- Full Name: `QLineEdit` (required)
- Status: `QComboBox` (active, paused, exhausted, archived, error)
- Queue Position: `QSpinBox` (1–100)
- Notes: `QTextEdit` (optional, max 1000 chars)
- Buttons: Save, Cancel

Add/Edit Account dialog (account_dialog.py)
Fields:
- Linked Student: `QComboBox` (populated from DB, shows full_name)
- Provider: `QComboBox` (github, gitlab, bitbucket, other)
- Username: `QLineEdit` (required)
- Auth Type: `QComboBox` (ssh, pat, app_password, password)
- Buttons: Save, Cancel

Add/Rotate Credential dialog (credential_dialog.py)
Fields:
- Auth Type: `QComboBox` (read-only, shows current value)
- New Secret: `QLineEdit` in `Password` echo mode
- Confirm Secret: `QLineEdit` in `Password` echo mode (must match)
- Buttons: Save, Cancel

Validation: if New Secret != Confirm Secret, show inline `QLabel` error "Secrets do not match", disable Save button.

Tests

No UI tests for this task (full integration requires running QApplication with fixtures — deferred to Phase 2). Manual verification only.

Acceptance Criteria

1. StudentsScreen loads and displays student rows from DB (use seed data from T14)
2. "Add Student" button opens dialog, filling form and clicking Save creates a new student row
3. "Archive" button sets student status to archived and row disappears from active view
4. CredentialsScreen shows masked secret values (`••••••••xxxx` pattern) — never raw tokens
5. LogsScreen loads last 200 logs and supports filter by entity_type
6. No `AttributeError` or `RuntimeError` when switching between all 5 screens rapidly (manual check)
7. `python -m app.main` with seed data: all 5 screens show data, no error dialogs on open

Notes for Codex

- Do NOT load secrets from keyring in the UI thread — CredentialsScreen only shows metadata (account, type, masked key ref). The actual secret is retrieved only on "Validate" action, immediately masked before display.
- `QTableWidget.setItem()` indices are (row, column) — be careful not to transpose
- When `AccountService.list_students()` returns an empty list, show a `QLabel("No students yet. Click Add Student to begin.")` in place of the empty table
- The `Details` column in LogsScreen: store as `json.dumps(dict)` in DB, display as truncated string (max 80 chars) — full value shown in a tooltip (`setToolTip`)
- Each ViewModel's `close()` method must be called in the screen's `closeEvent()` or when the screen is hidden — add `def hideEvent(self, event): self._vm.close()` to each screen class
---
codex-tasks/T06-git-client.md
Розмір: 18,363 байт

T06: Implement GitClient

Goal

Implement `GitClient` in `app/git/client.py` — the single module responsible for all Git operations in StudentFlow. GitClient uses GitPython where convenient and `subprocess.run()` as a fallback. All operations respect the configured timeout. Authentication is passed via environment variables (never command-line arguments).

Context

- Project root: `/home/vokov/projects/studentflow/`
- Security spec: `docs/security.md` — "Git Authentication" section
- Sync strategy: `docs/sync-strategy.md` — divergence detection commands
- Stack: GitPython==3.1.43

Constraints

- NEVER pass tokens or passwords in URL query strings: `https://token@host` is forbidden
- NEVER log credential values — mask or omit from all log output
- ALL subprocess calls must use `timeout=config.sync_timeout_seconds` (or a passed `timeout` param)
- SSH auth: use `GIT_SSH_COMMAND` env var
- HTTPS/PAT auth: use temp netrc file or `GIT_ASKPASS` helper, cleaned up in `finally` block
- `check_divergence()` must run `git rev-list --count` (not any GitPython equivalent that may differ)
- `clone()` of a local path (canonical → local) does NOT require credentials
- GitClient must NOT import from services layer (no circular imports)

Files to Create/Update

```
app/git/client.py           — full GitClient implementation
app/git/validators.py       — GitValidator (local and remote validation helpers)
app/git/policies.py         — SyncPolicy (merge strategy logic)
tests/test_git_client.py    — unit and integration tests using real temp git repos
```

Implementation Requirements

GitClient class: full method signatures and docstrings

```
class GitClient:
    &quot;&quot;&quot;All Git operations for StudentFlow.

    Uses GitPython for high-level operations and subprocess.run() as fallback.
    Authentication is always passed via environment variables (GIT_SSH_COMMAND
    for SSH, temp netrc file for HTTPS/PAT). Credentials are never passed
    as URL components or command-line arguments.
    &quot;&quot;&quot;

    def __init__(self, timeout: int = 60):
        &quot;&quot;&quot;
        Args:
            timeout: Seconds before a git subprocess is killed (default: 60)
        &quot;&quot;&quot;
        self._timeout = timeout

    def clone(
        self,
        remote_url: str,
        local_path: str,
        credential: str | None = None,
        auth_type: str | None = None,
        ssh_key_path: str | None = None,
    ) -&gt; None:
        &quot;&quot;&quot;Clone a repository to local_path.

        For local-to-local clones (canonical → local): credential is None.
        For remote clones: uses credential for authentication.

        Args:
            remote_url: URL or local path to clone from
            local_path: Destination path (must not already exist)
            credential: PAT/password value (never passed as URL component)
            auth_type: &quot;pat&quot;, &quot;ssh&quot;, &quot;app_password&quot;, &quot;password&quot; — dete
            ssh_key_path: Path to SSH private key file (only for auth_type=&quot;ssh&quot;)
        &quot;&quot;&quot;

    def fetch(self, repo_path: str, remote_name: str = &quot;student&quot;) -&gt; None:
        &quot;&quot;&quot;Fetch from a remote without merging.

        Args:
            repo_path: Absolute path to the local git repo
            remote_name: Name of the remote to fetch (default: &quot;student&quot;)
        &quot;&quot;&quot;

    def merge_ff_only(self, repo_path: str, branch: str) -&gt; None:
        &quot;&quot;&quot;Attempt a fast-forward-only merge.

        Args:
            repo_path: Local repo path
            branch: Branch to merge (e.g., &quot;student/main&quot;)

        Raises:
            GitMergeError: if --ff-only fails (diverged commits)
        &quot;&quot;&quot;

    def merge_no_ff(self, repo_path: str, branch: str, message: str) -&gt; None:
        &quot;&quot;&quot;Merge with an explicit merge commit (--no-ff).

        Args:
            repo_path: Local repo path
            branch: Branch to merge
            message: Commit message for the merge commit

        Raises:
```

```
        GitMergeConflictError: if merge produces conflicts
    &quot;&quot;&quot;

def push(
    self,
    repo_path: str,
    remote_name: str,
    branch: str,
    credential: str | None = None,
    auth_type: str | None = None,
    ssh_key_path: str | None = None,
    force: bool = False,
) -&gt; None:
    &quot;&quot;&quot;Push a branch to a remote.

    For local-to-local pushes (to canonical bare repo): credential is None.
    For pushes to student remotes: uses credential for auth.

    Args:
        force: If True, adds --force flag (used only for force_overwrite recovery)
    &quot;&quot;&quot;

def add_remote(self, repo_path: str, remote_name: str, remote_url: str) -&gt; None:
    &quot;&quot;&quot;Add a named remote to a repository.&quot;&quot;&quot;

def create_tag(self, repo_path: str, tag_name: str, message: str | None = None) -&gt; None:
    &quot;&quot;&quot;Create a lightweight or annotated tag at HEAD.

    Args:
        tag_name: Full tag name (e.g., &quot;handoff/2024-01-15-143022/from-alice-to-bob&quot;)
        message: If provided, creates annotated tag; otherwise lightweight
    &quot;&quot;&quot;

def get_commit_hash(self, repo_path: str, ref: str = &quot;HEAD&quot;) -&gt; str:
    &quot;&quot;&quot;Return the full 40-char SHA1 of a git ref.

    Args:
        ref: Git ref (default: &quot;HEAD&quot;)

    Returns:
        40-character hex SHA1

    Raises:
        ValueError: if repo_path is not a git repo or ref doesn&apos;t exist
    &quot;&quot;&quot;

def check_divergence(self, repo_path: str, remote_branch: str) -&gt; tuple[int, int]:
    &quot;&quot;&quot;Count commits ahead and behind between HEAD and a remote branch.

    Runs:
        git rev-list --count HEAD..{remote_branch}  → ahead
        git rev-list --count {remote_branch}..HEAD  → behind

    Args:
        repo_path: Local repo path (must have been fetched recently)
        remote_branch: Remote tracking branch (e.g., &quot;student/main&quot;)

    Returns:
        (ahead, behind) — number of commits HEAD is ahead/behind of remote_branch
    &quot;&quot;&quot;

def init_from_dir(self, src_path: str, dest_path: str) -&gt; None:
    &quot;&quot;&quot;Create a new git repo at dest_path initialized with files from src_path.

    Steps:
    1. Copy src_path contents to dest_path (shutil.copytree)
    2. git init dest_path
```

```
        3. git -C dest_path add .
        4. git -C dest_path commit -m &quot;Initial project template&quot;

        Args:
            src_path: Directory to copy files from (template source)
            dest_path: Path for the new git repo (must not already exist)
        &quot;&quot;&quot;

    def validate_remote(self, remote_url: str, credential: str | None = None, auth_type: str | None = None, ssh_key_p
        &quot;&quot;&quot;Test connectivity to a remote URL.

        Uses `git ls-remote {url}` to check reachability.

        Returns:
            True if remote is accessible, False otherwise (never raises)
        &quot;&quot;&quot;

    def reset_hard(self, repo_path: str, ref: str = &quot;HEAD&quot;) -&gt; None:
        &quot;&quot;&quot;Hard reset the working tree to a given ref (used for force_overwrite).&quot;&quot;&quot;

    def get_log(self, repo_path: str, limit: int = 10) -&gt; list[dict]:
        &quot;&quot;&quot;Return recent commit log as list of dicts with keys: hash, message, author, date.&quot;&q
```

Authentication implementation

For HTTPS/PAT:

```
import tempfile
import os
import stat

def _make_netrc_env(self, remote_url: str, credential: str, username: str = &quot;git&quot;) -&gt; tuple[dict, str]:
    &quot;&quot;&quot;Create a temp netrc file and return (env_dict, temp_file_path).&quot;&quot;&quot;
    from urllib.parse import urlparse
    host = urlparse(remote_url).hostname or &quot;github.com&quot;

    netrc_content = f&quot;machine {host}\nlogin {username}\npassword {credential}\n&quot;

    with tempfile.NamedTemporaryFile(mode=&quot;w&quot;, suffix=&quot;.netrc&quot;, delete=False) as f:
        f.write(netrc_content)
        temp_path = f.name
    os.chmod(temp_path, stat.S_IRUSR | stat.S_IWUSR)  # 0o600

    env = {
        **os.environ,
        &quot;GIT_CONFIG_COUNT&quot;: &quot;1&quot;,
        &quot;GIT_CONFIG_KEY_0&quot;: &quot;credential.helper&quot;,
        &quot;GIT_CONFIG_VALUE_0&quot;: f&quot;store --file {temp_path}&quot;,
        &quot;HOME&quot;: str(Path.home()),  # Ensure git finds the netrc
    }
    return env, temp_path

def _run_with_credential(self, cmd: list[str], remote_url: str, credential: str, auth_type: str, ssh_key_path: str | None
    &quot;&quot;&quot;Run a git command with appropriate credential environment setup.&quot;&quot;&quot;
    if auth_type == &quot;ssh&quot; and ssh_key_path:
        env = {
            **os.environ,
            &quot;GIT_SSH_COMMAND&quot;: f&quot;ssh -i {ssh_key_path} -o StrictHostKeyChecking=no -o BatchMod
        }
        try:
            return subprocess.run(cmd, env=env, capture_output=True, text=True, timeout=self._timeout, check=Tru
        finally:
            pass  # No temp file to clean up for SSH

    elif auth_type in (&quot;pat&quot;, &quot;app_password&quot;, &quot;password&quot;):
        env, temp_path = self._make_netrc_env(remote_url, credential)
        try:
            return subprocess.run(cmd, env=env, capture_output=True, text=True, timeout=self._timeout, check=Tru
```

```
        finally:
            try:
                os.unlink(temp_path)
            except OSError:
                pass
    else:
        return subprocess.run(cmd, capture_output=True, text=True, timeout=self._timeout, check=True)


### check_divergence implementation

def check_divergence(self, repo_path: str, remote_branch: str) -&gt; tuple[int, int]:
    import subprocess
    ahead_result = subprocess.run(
        [&quot;git&quot;, &quot;-C&quot;, repo_path, &quot;rev-list&quot;, &quot;--count&quot;, f&quot;HEAD..{rem
        capture_output=True, text=True, timeout=self._timeout
    )
    behind_result = subprocess.run(
        [&quot;git&quot;, &quot;-C&quot;, repo_path, &quot;rev-list&quot;, &quot;--count&quot;, f&quot;{remote_bra
        capture_output=True, text=True, timeout=self._timeout
    )
    if ahead_result.returncode != 0 or behind_result.returncode != 0:
        raise GitOperationError(
            f&quot;check_divergence failed: {ahead_result.stderr or behind_result.stderr}&quot;
        )
    ahead = int(ahead_result.stdout.strip())
    behind = int(behind_result.stdout.strip())
    return ahead, behind


### Custom Exceptions

Define in `app/git/client.py`:
class GitOperationError(Exception):
    &quot;&quot;&quot;Raised when a git operation fails (non-zero exit, timeout, etc.)&quot;&quot;&quot;
    pass

class GitMergeError(GitOperationError):
    &quot;&quot;&quot;Raised when --ff-only merge fails due to divergent history.&quot;&quot;&quot;
    pass

class GitMergeConflictError(GitOperationError):
    &quot;&quot;&quot;Raised when --no-ff merge produces conflicts.&quot;&quot;&quot;
    pass

class GitAuthError(GitOperationError):
    &quot;&quot;&quot;Raised when a git operation fails due to authentication error.&quot;&quot;&quot;
    pass


### app/git/validators.py

&quot;&quot;&quot;GitValidator: validates local repo paths and remote URLs.&quot;&quot;&quot;
import subprocess
from pathlib import Path


class GitValidator:
    &quot;&quot;&quot;Utility class for validating git repository paths and remote URLs.&quot;&quot;&quot;

    @staticmethod
    def is_git_repo(path: str) -&gt; bool:
        &quot;&quot;&quot;Return True if path is a valid git repository (bare or normal).&quot;&quot;&quot;
        p = Path(path)
        if not p.exists():
            return False
        result = subprocess.run(
            [&quot;git&quot;, &quot;-C&quot;, str(p), &quot;rev-parse&quot;, &quot;--git-dir&quot;],
            capture_output=True, text=True
```

```
        )
        return result.returncode == 0

    @staticmethod
    def is_bare_repo(path: str) -&gt; bool:
        &quot;&quot;&quot;Return True if path is a bare git repository.&quot;&quot;&quot;
        result = subprocess.run(
            [&quot;git&quot;, &quot;-C&quot;, path, &quot;rev-parse&quot;, &quot;--is-bare-repository&quot;],
            capture_output=True, text=True
        )
        return result.returncode == 0 and result.stdout.strip() == &quot;true&quot;


### app/git/policies.py

&quot;&quot;&quot;SyncPolicy: merge strategy logic for StudentFlow.

Policy: always merge, never rebase.
Preferred: --ff-only. Fallback: --no-ff with explicit merge commit.
Never: --force on canonical.
&quot;&quot;&quot;
from dataclasses import dataclass
from enum import Enum


class MergeStrategy(str, Enum):
    ff_only = &quot;ff_only&quot;       # Preferred: fast-forward only
    no_ff = &quot;no_ff&quot;           # Fallback: explicit merge commit
    force_overwrite = &quot;force_overwrite&quot;  # Manual recovery only


@dataclass
class SyncPolicyResult:
    strategy: MergeStrategy
    reason: str


class SyncPolicy:
    &quot;&quot;&quot;Determines the merge strategy based on divergence state.&quot;&quot;&quot;

    def get_strategy(self, ahead: int, behind: int) -&gt; SyncPolicyResult:
        &quot;&quot;&quot;
        Args:
            ahead: commits student remote has that local lacks
            behind: commits local has that student remote lacks

        Returns:
            SyncPolicyResult with recommended strategy and reason
        &quot;&quot;&quot;
        if ahead == 0 and behind == 0:
            return SyncPolicyResult(MergeStrategy.ff_only, &quot;Already in sync — nothing to merge&quot;)

        if ahead &gt; 0 and behind == 0:
            return SyncPolicyResult(MergeStrategy.ff_only, f&quot;Student has {ahead} new commits — fast-forward a

        if ahead == 0 and behind &gt; 0:
            return SyncPolicyResult(MergeStrategy.ff_only, f&quot;Local is {behind} ahead of student — no merge nee

        # Both ahead and behind: diverged
        # Cannot auto-merge — return ff_only as intent, caller will detect failure
        return SyncPolicyResult(
            MergeStrategy.ff_only,
            f&quot;Diverged: student ahead={ahead}, local behind={behind} — ff-only will fail, manual resolution req
        )
```

Tests (tests/test_git_client.py)

Use `pytest`'s `tmp_path` fixture to create real local git repos. All tests use only local operations (no network). SSH/PAT tests mock subprocess.

test_init_from_dir_creates_repo:
```
def test_init_from_dir_creates_repo(tmp_path):
    src = tmp_path / &quot;template&quot;
    src.mkdir()
    (src / &quot;README.md&quot;).write_text(&quot;# Project\n&quot;)
    (src / &quot;main.py&quot;).write_text(&quot;print(&apos;hello&apos;)\n&quot;)

    dest = tmp_path / &quot;student_repo&quot;
    client = GitClient()
    client.init_from_dir(str(src), str(dest))

    assert (dest / &quot;.git&quot;).exists()
    assert (dest / &quot;README.md&quot;).exists()
    assert (dest / &quot;main.py&quot;).exists()
```

test_get_commit_hash_returns_40_chars:
```
def test_get_commit_hash_returns_40_chars(tmp_path):
    src = tmp_path / &quot;src&quot;
    src.mkdir()
    (src / &quot;f.txt&quot;).write_text(&quot;content&quot;)
    dest = tmp_path / &quot;repo&quot;
    client = GitClient()
    client.init_from_dir(str(src), str(dest))
    commit_hash = client.get_commit_hash(str(dest))
    assert len(commit_hash) == 40
    assert all(c in &quot;0123456789abcdef&quot; for c in commit_hash)
```

test_check_divergence_clean:
```
def test_check_divergence_clean(tmp_path):
    import subprocess
    # Create a bare &quot;canonical&quot; repo and two clones
    bare = tmp_path / &quot;bare&quot;
    subprocess.run([&quot;git&quot;, &quot;init&quot;, &quot;--bare&quot;, str(bare)], check=True)
    clone1 = tmp_path / &quot;clone1&quot;
    subprocess.run([&quot;git&quot;, &quot;clone&quot;, str(bare), str(clone1)], check=True, capture_output=True
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(clone1), &quot;config&quot;, &quot;user.email&quot;, &q
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(clone1), &quot;config&quot;, &quot;user.name&quot;, &q
    (clone1 / &quot;f.txt&quot;).write_text(&quot;initial&quot;)
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(clone1), &quot;add&quot;, &quot;.&quot;], check=True, c
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(clone1), &quot;commit&quot;, &quot;-m&quot;, &quot;ini
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(clone1), &quot;push&quot;, &quot;origin&quot;, &quot;ma

    clone2 = tmp_path / &quot;clone2&quot;
    subprocess.run([&quot;git&quot;, &quot;clone&quot;, str(bare), str(clone2)], check=True, capture_output=True
    subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(clone2), &quot;remote&quot;, &quot;rename&quot;, &quo

    client = GitClient()
    ahead, behind = client.check_divergence(str(clone2), &quot;student/main&quot;)
    assert ahead == 0
    assert behind == 0
```

test_check_divergence_student_ahead:
```
def test_check_divergence_student_ahead(tmp_path):
    import subprocess
    # Setup: student remote is 2 commits ahead of local HEAD
    # ... (use git operations to create the divergent state)
    # Then:
    client = GitClient()
    ahead, behind = client.check_divergence(str(local_path), &quot;student/main&quot;)
    assert ahead == 2
    assert behind == 0
```

test_create_tag:
```
def test_create_tag(tmp_path):
    src = tmp_path / &quot;src&quot;
    src.mkdir()
    (src / &quot;f.txt&quot;).write_text(&quot;content&quot;)
    repo = tmp_path / &quot;repo&quot;
    client = GitClient()
    client.init_from_dir(str(src), str(repo))
    client.create_tag(str(repo), &quot;handoff/2024-01-15-143022/from-alice-to-bob&quot;)
    import subprocess
    result = subprocess.run([&quot;git&quot;, &quot;-C&quot;, str(repo), &quot;tag&quot;, &quot;-l&quot;], capture
    assert &quot;handoff/2024-01-15-143022/from-alice-to-bob&quot; in result.stdout
```

test_netrc_file_cleaned_up_after_push:
```
def test_netrc_file_cleaned_up_after_push(tmp_path):
    import tempfile
    import os
    # Count temp files before
    tmpdir = tempfile.gettempdir()
    before = set(f for f in os.listdir(tmpdir) if f.endswith(&quot;.netrc&quot;))
    # Attempt a push that will fail (no real remote) but catch the exception
    client = GitClient()
    try:
        client._make_netrc_env(&quot;https://github.com/test/repo.git&quot;, &quot;fake_token&quot;)
    except Exception:
        pass
    # Verify no leftover netrc files (the _make_netrc_env creates but the run cleans up)
    # Note: this test is best done by patching subprocess.run to succeed
    # and verifying os.unlink is called in the finally block
```

Acceptance Criteria

1. `from app.git.client import GitClient, GitOperationError, GitMergeError` — imports without error
2. `from app.git.validators import GitValidator` — imports without error
3. `from app.git.policies import SyncPolicy, MergeStrategy` — imports without error
4. Tests `test_init_from_dir_creates_repo`, `test_get_commit_hash_returns_40_chars`, `test_create_tag` pass using real git repos
5. `check_divergence()` uses `git rev-list --count` (not GitPython's alternative)
6. No credential values appear in any log output from GitClient operations
7. Temp netrc files are always cleaned up in `finally` blocks — no temp files left after any operation
8. All subprocess calls use the `timeout` parameter
9. All 5+ tests pass: `pytest tests/test_git_client.py -v`

---
codex-tasks/T01-scaffold.md
Розмір: 15,464 байт

T01: Scaffold Project Structure

Goal

Create the complete project skeleton with all directories, `__init__.py` files, `requirements.txt`, `pyproject.toml`, `.gitignore`, and bootstrap entry points. No business logic — only structure and dependencies.

Context

Project: studentflow — a PySide6 desktop app for managing student Git repo workflows.
Stack: Python 3.11+, PySide6, SQLAlchemy 2.x, Alembic, GitPython, keyring, structlog, APScheduler.
Root: `/home/vokov/projects/studentflow/`

Constraints

- Do NOT implement any business logic
- Do NOT create UI windows yet
- Create `__init__.py` in every package directory
- `requirements.txt` must use pinned versions
- `pyproject.toml` must have a `[project]` metadata section
- `bootstrap.py` must expose: `init_db()`, `init_logging()`, `get_db_session()`
- `app/main.py` must accept `--help` and exit 0

Files to Create

requirements.txt
pyproject.toml
.gitignore
app/__init__.py
app/main.py
app/bootstrap.py
app/config.py
app/db/__init__.py
app/db/base.py
app/db/migrations/ (directory with .gitkeep)
app/services/__init__.py
app/services/account_service.py
app/services/repository_service.py
app/services/credential_service.py
app/services/sync_service.py
app/services/handoff_service.py
app/services/template_service.py
app/services/audit_service.py
app/git/__init__.py
app/git/client.py
app/git/policies.py
app/git/validators.py
app/ui/__init__.py
app/ui/windows/__init__.py
app/ui/dialogs/__init__.py
app/ui/widgets/__init__.py
app/ui/viewmodels/__init__.py
app/jobs/__init__.py
app/jobs/scheduler.py
app/jobs/workers.py
app/security/__init__.py
app/security/vault.py
app/security/masking.py
app/security/crypto.py
app/schemas/__init__.py
app/schemas/student.py
app/schemas/account.py
app/schemas/repository.py
tests/__init__.py
tests/conftest.py

requirements.txt content (exact pinned versions):

```
PySide6==6.7.0
SQLAlchemy==2.0.30
alembic==1.13.1
GitPython==3.1.43
keyring==25.2.1
structlog==24.2.0
APScheduler==3.10.4
pydantic==2.7.1
pytest==8.2.0
pytest-qt==4.4.0
```

Implementation Requirements

requirements.txt
Write the exact pinned versions listed above, one per line, no extras.

pyproject.toml
```
[project]
name = &quot;studentflow&quot;
version = &quot;0.1.0&quot;
description = &quot;Desktop operator tool for managing student Git repo workflows&quot;
requires-python = &quot;&gt;=3.11&quot;
license = {text = &quot;MIT&quot;}

[project.scripts]
studentflow = &quot;app.main:main&quot;

[tool.pytest.ini_options]
testpaths = [&quot;tests&quot;]
python_files = [&quot;test_*.py&quot;]
python_classes = [&quot;Test*&quot;]
python_functions = [&quot;test_*&quot;]
```

.gitignore
Include patterns for:
- Python: `__pycache__/`, `*.pyc`, `*.pyo`, `*.pyd`, `.Python`, `build/`, `dist/`, `*.egg-info/`, `.eggs/`
- Virtual envs: `.venv/`, `venv/`, `env/`
- SQLite: `*.db`, `*.db-shm`, `*.db-wal`
- IDE: `.idea/`, `.vscode/`, `*.swp`
- OS: `.DS_Store`, `Thumbs.db`
- App data: `~/.studentflow/` is NOT in the project, so no need to gitignore it
- Logs: `logs/`, `*.log`
- Alembic compiled: `app/db/migrations/__pycache__/`

app/__init__.py
Empty file with module docstring: `"""StudentFlow — Student workflow management desktop app."""`

app/main.py
```
&quot;&quot;&quot;Entry point for StudentFlow. Supports GUI mode (default) and --cli mode.&quot;&quot;&quot;
import argparse
import sys


def main() -&gt; None:
    parser = argparse.ArgumentParser(
        prog=&quot;studentflow&quot;,
        description=&quot;StudentFlow — Student workflow management desktop app&quot;
```

```
    )
    parser.add_argument(
        &quot;--cli&quot;,
        action=&quot;store_true&quot;,
        help=&quot;Run in headless CLI mode (no GUI)&quot;
    )
    parser.add_argument(
        &quot;--debug&quot;,
        action=&quot;store_true&quot;,
        help=&quot;Enable debug logging&quot;
    )
    args = parser.parse_args()

    # Bootstrap: init DB, logging, config, directories
    from app.bootstrap import init_db, init_logging, create_app_dirs, get_config
    create_app_dirs()
    config = get_config()
    init_logging(debug=args.debug)
    init_db()

    if args.cli:
        from app.cli import run_cli
        run_cli(args)
    else:
        from PySide6.QtWidgets import QApplication
        app = QApplication(sys.argv)
        app.setApplicationName(&quot;StudentFlow&quot;)
        app.setApplicationVersion(&quot;0.1.0&quot;)
        # MainWindow import deferred to avoid importing PySide6 in CLI mode
        from app.ui.windows.main_window import MainWindow
        window = MainWindow()
        window.show()
        sys.exit(app.exec())


if __name__ == &quot;__main__&quot;:
    main()


### app/bootstrap.py
&quot;&quot;&quot;Application bootstrap: initialize DB, logging, config, and filesystem directories.&quot;&quot;&q
import sys
from pathlib import Path
from contextlib import contextmanager


APP_DIR = Path.home() / &quot;.studentflow&quot;
LOG_DIR = APP_DIR / &quot;logs&quot;
CANONICAL_DIR = APP_DIR / &quot;canonical&quot;
DB_PATH = APP_DIR / &quot;studentflow.db&quot;
CONFIG_PATH = APP_DIR / &quot;config.json&quot;


def create_app_dirs() -&gt; None:
    &quot;&quot;&quot;Create ~/.studentflow/ directory structure if it doesn&apos;t exist.&quot;&quot;&quot;
    APP_DIR.mkdir(exist_ok=True)
    LOG_DIR.mkdir(exist_ok=True)
    CANONICAL_DIR.mkdir(exist_ok=True)


def get_config():
    &quot;&quot;&quot;Load Config from ~/.studentflow/config.json (creates with defaults if missing).&quot;&quot;&
    from app.config import Config
    return Config.load(CONFIG_PATH)


def init_logging(debug: bool = False) -&gt; None:
```

```
    &quot;&quot;&quot;Configure structlog for JSON output to ~/.studentflow/logs/app.log.&quot;&quot;&quot;
    import logging
    import logging.handlers
    import structlog

    level = logging.DEBUG if debug else logging.INFO

    handler = logging.handlers.RotatingFileHandler(
        filename=LOG_DIR / &quot;app.log&quot;,
        maxBytes=10 * 1024 * 1024,
        backupCount=5,
        encoding=&quot;utf-8&quot;,
    )
    handler.setLevel(level)

    logging.basicConfig(
        format=&quot;%(message)s&quot;,
        handlers=[handler, logging.StreamHandler(sys.stdout)],
        level=level,
    )

    structlog.configure(
        processors=[
            structlog.stdlib.add_log_level,
            structlog.stdlib.add_logger_name,
            structlog.processors.TimeStamper(fmt=&quot;iso&quot;),
            structlog.processors.JSONRenderer(),
        ],
        context_class=dict,
        logger_factory=structlog.stdlib.LoggerFactory(),
        wrapper_class=structlog.stdlib.BoundLogger,
        cache_logger_on_first_use=True,
    )


def init_db() -&gt; None:
    &quot;&quot;&quot;Run Alembic migrations and seed default AppSettings if missing.&quot;&quot;&quot;
    from app.db.base import engine, Base
    # Create all tables (used before Alembic is configured, or as fallback)
    Base.metadata.create_all(bind=engine)
    _seed_default_settings()


def _seed_default_settings() -&gt; None:
    &quot;&quot;&quot;Insert default AppSetting records if they don&apos;t already exist.&quot;&quot;&quot;
    from app.db.base import SessionLocal
    defaults = {
        &quot;canonical_repo_path&quot;: str(CANONICAL_DIR),
        &quot;default_branch&quot;: &quot;main&quot;,
        &quot;template_dir&quot;: str(Path.home() / &quot;studentflow-templates&quot;),
        &quot;sync_timeout_seconds&quot;: &quot;60&quot;,
        &quot;max_sync_retries&quot;: &quot;2&quot;,
        &quot;enable_auto_sync&quot;: &quot;false&quot;,
        &quot;auto_sync_interval_minutes&quot;: &quot;30&quot;,
    }
    descriptions = {
        &quot;canonical_repo_path&quot;: &quot;Absolute path to the canonical bare git repository&quot;,
        &quot;default_branch&quot;: &quot;Default git branch name for new repositories&quot;,
        &quot;template_dir&quot;: &quot;Default directory to search for project templates&quot;,
        &quot;sync_timeout_seconds&quot;: &quot;Seconds before a git operation is killed&quot;,
        &quot;max_sync_retries&quot;: &quot;How many times to retry a failed sync&quot;,
        &quot;enable_auto_sync&quot;: &quot;Whether SchedulerService runs periodic sync (true/false)&quot;,
        &quot;auto_sync_interval_minutes&quot;: &quot;Interval for auto sync if enabled&quot;,
    }
    with SessionLocal() as session:
        from app.db.models import AppSetting
        from datetime import datetime
```

```
        for key, value in defaults.items():
            existing = session.get(AppSetting, key)
            if existing is None:
                session.add(AppSetting(
                    key=key,
                    value=value,
                    description=descriptions.get(key),
                    updated_at=datetime.utcnow(),
                ))
        session.commit()


@contextmanager
def get_db_session():
    &quot;&quot;&quot;Context manager that yields a SQLAlchemy session and handles commit/rollback.&quot;&qu
    from app.db.base import SessionLocal
    with SessionLocal() as session:
        try:
            yield session
            session.commit()
        except Exception:
            session.rollback()
            raise


# Allow running as: python -m app.bootstrap init-db
if __name__ == &quot;__main__&quot;:
    import sys as _sys
    if len(_sys.argv) &gt; 1 and _sys.argv[1] == &quot;init-db&quot;:
        create_app_dirs()
        config = get_config()
        init_logging()
        init_db()
        print(f&quot;Database initialized at {DB_PATH}&quot;)
    else:
        print(f&quot;Usage: python -m app.bootstrap init-db&quot;)


### app/config.py
&quot;&quot;&quot;Application configuration loaded from ~/.studentflow/config.json.&quot;&quot;&quot;
from __future__ import annotations
import json
from dataclasses import dataclass, asdict, field
from pathlib import Path


@dataclass
class Config:
    canonical_repo_path: str = str(Path.home() / &quot;.studentflow&quot; / &quot;canonical&quot;)
    template_dir: str = str(Path.home() / &quot;studentflow-templates&quot;)
    default_branch: str = &quot;main&quot;
    sync_timeout_seconds: int = 60
    max_sync_retries: int = 2
    enable_auto_sync: bool = False
    auto_sync_interval_minutes: int = 30
    log_level: str = &quot;INFO&quot;

    @classmethod
    def load(cls, path: Path) -&gt; &quot;Config&quot;:
        &quot;&quot;&quot;Load config from JSON file, creating with defaults if missing.&quot;&quot;&quot;
        if not path.exists():
            config = cls()
            config.save(path)
            return config
        with open(path, &quot;r&quot;, encoding=&quot;utf-8&quot;) as f:
            data = json.load(f)
        return cls(**{k: v for k, v in data.items() if k in cls.__dataclass_fields__})
```

```
    def save(self, path: Path) -&gt; None:
        &quot;&quot;&quot;Write config to JSON file.&quot;&quot;&quot;
        path.parent.mkdir(parents=True, exist_ok=True)
        with open(path, &quot;w&quot;, encoding=&quot;utf-8&quot;) as f:
            json.dump(asdict(self), f, indent=2)


### app/db/base.py
&quot;&quot;&quot;SQLAlchemy engine, session factory, and declarative base.&quot;&quot;&quot;
# Convention: all PKs are UUID strings generated as str(uuid.uuid4())
import uuid
from pathlib import Path
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

_DB_PATH = Path.home() / &quot;.studentflow&quot; / &quot;studentflow.db&quot;
DATABASE_URL = f&quot;sqlite:///{_DB_PATH}&quot;

engine = create_engine(
    DATABASE_URL,
    connect_args={&quot;check_same_thread&quot;: False},
    echo=False,
)

SessionLocal = sessionmaker(
    bind=engine,
    autocommit=False,
    autoflush=False,
    expire_on_commit=False,
)

Base = declarative_base()


### app/services/ (all service files)
Each service file contains a class stub with `pass`. Example for `account_service.py`:
&quot;&quot;&quot;AccountService: CRUD for Student and Account entities, queue management.&quot;&quot;&qu


class AccountService:
    pass

Write the same pattern for all 7 service files (AccountService, RepositoryService,
CredentialService, SyncService, HandoffService, TemplateService, AuditService).

### app/git/client.py
&quot;&quot;&quot;GitClient: all Git operations via GitPython and subprocess fallback.&quot;&quot;&quot;


class GitClient:
    pass


### app/git/policies.py
&quot;&quot;&quot;SyncPolicy: merge/rebase policy logic for StudentFlow.&quot;&quot;&quot;


class SyncPolicy:
    pass


### app/git/validators.py
&quot;&quot;&quot;GitValidator: local path and remote URL validation.&quot;&quot;&quot;


class GitValidator:
    pass
```

app/jobs/scheduler.py
"""SchedulerService: APScheduler-based background job management."""

class SchedulerService:
    pass

app/jobs/workers.py
"""QThread workers for async Git and sync operations."""
try:
    from PySide6.QtCore import QThread
except ImportError:
    QThread = object # Allow import in non-GUI/test contexts

class SyncWorker(QThread):
    pass

app/security/vault.py
"""SecretVault: OS keyring interface via the keyring library."""

class SecretVault:
    pass

app/security/masking.py
"""Credential masking utilities for safe display in UI and logs."""

def mask_secret(value: str) -> str:
    """Return a masked version of a secret, showing only the last 4 characters.

    Example: "ghp_abc123xyz789" -> "••••••••z789"
    """
    if not value:
        return "••••"
    if len(value) <= 4:
        return "••••"
    return "••••••••" + value[-4:]

app/security/crypto.py
"""Placeholder for future encryption utilities."""
Reserved for future use.
Current architecture stores secrets in the OS keyring (see vault.py).
This module may be used for encrypting local SQLite data at rest
in a future release.

app/schemas/ (all schema files)
Each schema file is empty with a module docstring. Example for `student.py`:
"""TypedDict and dataclass schemas for Student data transfer objects."""
To be implemented in T02 alongside models.

tests/conftest.py
"""Pytest fixtures shared across all test modules."""
import pytest
import tempfile
from pathlib import Path
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

@pytest.fixture(scope="function")
def temp_db(tmp_path):

```
    &quot;&quot;&quot;Create a temporary in-memory SQLite database for testing.

    Returns the engine. Use db_session fixture for a session.
    &quot;&quot;&quot;
    from app.db.base import Base
    db_path = tmp_path / &quot;test.db&quot;
    test_engine = create_engine(
        f&quot;sqlite:///{db_path}&quot;,
        connect_args={&quot;check_same_thread&quot;: False},
    )
    Base.metadata.create_all(bind=test_engine)
    yield test_engine
    Base.metadata.drop_all(bind=test_engine)


@pytest.fixture(scope=&quot;function&quot;)
def db_session(temp_db):
    &quot;&quot;&quot;Yield a SQLAlchemy session bound to the temp test database.

    Automatically rolls back after each test.
    &quot;&quot;&quot;
    TestSessionLocal = sessionmaker(bind=temp_db, autocommit=False, autoflush=False)
    session = TestSessionLocal()
    try:
        yield session
    finally:
        session.rollback()
        session.close()
```

Acceptance Criteria

1. `python -m app.main --help` prints help text and exits with code 0
2. `python -m app.bootstrap init-db` creates `~/.studentflow/` directory and subdirs (or confirms they exist), prints "Database initialized"
3. All imports work without error:
   from app.services.account_service import AccountService
   from app.services.sync_service import SyncService
   from app.git.client import GitClient
   from app.security.masking import mask_secret
   from app.config import Config

4. `pytest tests/ --collect-only` discovers `tests/conftest.py` without errors
5. No circular import errors when running `python -c "import app"`

Notes for Codex

- Convention for all future IDs: `default=lambda: str(uuid.uuid4())` — note this in `app/db/base.py` as a comment
- `bootstrap.py` should be runnable as `python -m app.bootstrap init-db` (the `if __name__ == "__main__"` block handles this)
- The `app/db/migrations/` directory should contain a `.gitkeep` file to be tracked by git
- `workers.py` guards the PySide6 import to allow running in CLI/test environments without a display
- Default config values must match exactly what is specified in `Config` dataclass defaults
---
codex-tasks/T14-seed-data.md
Розмір: 10,311 байт

T14: Create Demo Seed Data

Goal

Implement `app/db/seeds.py` with a `seed_demo_data()` function that populates the DB with realistic demo data: 5 students, one account per student, one repository per account, one credential reference per account, and 10 sample audit log entries. Also implement `app/db/enums.py` with all enum classes referenced by the models.

Context

- Project root: `/home/vokov/projects/studentflow/`
- Models in `app/db/models.py` (T02) must all be defined before this task
- Enums referenced: `StudentStatus`, `AccountStatus`, `AuthType`, `SyncStatus`, `SyncJobType`, `SyncJobStatus`, `HandoffStatus`, `CredentialKind`
- Seed data must be idempotent: calling `seed_demo_data()` twice must not create duplicates

Constraints

- Seed data must NOT call `keyring` — use placeholder `secret_ref` values like `"demo-token-ref-{i}"`
- Seed data must NOT create actual git repos or directories
- Credential values must NEVER appear in seed data — only `secret_ref` strings
- `seed_demo_data()` must be idempotent: if students with seed names already exist, skip
- The function must be runnable as: `python -m app.db.seeds`

Files to Create/Update

```
app/db/enums.py    — all enum classes (new file)
app/db/seeds.py    — seed_demo_data() function
```

Implementation Requirements

app/db/enums.py

"""All SQLAlchemy-compatible enum classes for StudentFlow models."""
import enum

class StudentStatus(str, enum.Enum):
    active = "active"
    paused = "paused"
    exhausted = "exhausted"
    archived = "archived"
    error = "error"

class AccountStatus(str, enum.Enum):
    active = "active"
    inactive = "inactive"
    error = "error"

class AuthType(str, enum.Enum):
    ssh = "ssh"

```
    pat = &quot;pat&quot;
    app_password = &quot;app_password&quot;
    password = &quot;password&quot;


class SyncStatus(str, enum.Enum):
    ok = &quot;ok&quot;
    pending = &quot;pending&quot;
    error = &quot;error&quot;
    unknown = &quot;unknown&quot;
    diverged = &quot;diverged&quot;


class SyncJobType(str, enum.Enum):
    sync_student = &quot;sync_student&quot;
    sync_all = &quot;sync_all&quot;
    handoff = &quot;handoff&quot;
    validate = &quot;validate&quot;


class SyncJobStatus(str, enum.Enum):
    pending = &quot;pending&quot;
    running = &quot;running&quot;
    completed = &quot;completed&quot;
    failed = &quot;failed&quot;
    cancelled = &quot;cancelled&quot;


class HandoffStatus(str, enum.Enum):
    pending = &quot;pending&quot;
    completed = &quot;completed&quot;
    failed = &quot;failed&quot;
    rolled_back = &quot;rolled_back&quot;


class CredentialKind(str, enum.Enum):
    ssh_passphrase = &quot;ssh_passphrase&quot;
    pat = &quot;pat&quot;
    app_password = &quot;app_password&quot;
    password = &quot;password&quot;


### app/db/seeds.py

&quot;&quot;&quot;Demo seed data for StudentFlow. Idempotent — safe to run multiple times.&quot;&quot;&quot
import uuid
from datetime import datetime, timezone
from app.db.base import SessionLocal
from app.db.enums import (
    StudentStatus, AccountStatus, AuthType,
    SyncStatus, CredentialKind
)

DEMO_STUDENTS = [
    {&quot;full_name&quot;: &quot;Alice Kovalenko&quot;,  &quot;queue_position&quot;: 1, &quot;status&quot;: St
    {&quot;full_name&quot;: &quot;Bohdan Marchenko&quot;, &quot;queue_position&quot;: 2, &quot;status&quot;
    {&quot;full_name&quot;: &quot;Daria Lysenko&quot;,    &quot;queue_position&quot;: 3, &quot;status&quot;: St
    {&quot;full_name&quot;: &quot;Evhen Petrenko&quot;,   &quot;queue_position&quot;: 4, &quot;status&quot;: S
    {&quot;full_name&quot;: &quot;Fedir Bondarenko&quot;, &quot;queue_position&quot;: 5, &quot;status&quot;: S
]

DEMO_ACCOUNTS = [
    {&quot;provider&quot;: &quot;github&quot;, &quot;username&quot;: &quot;alice-kovalenko&quot;,  &quot;auth
    {&quot;provider&quot;: &quot;github&quot;, &quot;username&quot;: &quot;b-marchenko&quot;,      &quot;auth
    {&quot;provider&quot;: &quot;gitlab&quot;, &quot;username&quot;: &quot;daria.lysenko&quot;,    &quot;auth_
    {&quot;provider&quot;: &quot;github&quot;, &quot;username&quot;: &quot;evhen-petrenko&quot;,   &quot;aut
    {&quot;provider&quot;: &quot;bitbucket&quot;, &quot;username&quot;: &quot;fbondarenko&quot;,   &quot;aut
]
```

```
def seed_demo_data() -&gt; None:
    &quot;&quot;&quot;Insert demo students, accounts, repos, credentials, and audit logs.
    Safe to call multiple times — existing records are not duplicated.
    &quot;&quot;&quot;
    from app.db.models import Student, Account, Repository, Credential, ActivityLog, AppSetting

    with SessionLocal() as session:
        # Check idempotency
        existing_names = {s.full_name for s in session.query(Student).all()}

        student_ids = []
        for i, s_data in enumerate(DEMO_STUDENTS):
            if s_data[&quot;full_name&quot;] in existing_names:
                # Find existing
                student = session.query(Student).filter_by(full_name=s_data[&quot;full_name&quot;]).first()
            else:
                student = Student(
                    id=str(uuid.uuid4()),
                    full_name=s_data[&quot;full_name&quot;],
                    status=s_data[&quot;status&quot;],
                    queue_position=s_data[&quot;queue_position&quot;],
                    notes=f&quot;Demo student #{i+1}&quot;,
                    created_at=datetime.now(timezone.utc),
                    updated_at=datetime.now(timezone.utc),
                )
                session.add(student)
            student_ids.append(student)

        session.flush()  # Get IDs for new students

        for i, (student, acc_data) in enumerate(zip(student_ids, DEMO_ACCOUNTS)):
            # Check if account already exists for this student
            existing_acc = session.query(Account).filter_by(
                student_id=student.id,
                username=acc_data[&quot;username&quot;]
            ).first()

            if existing_acc:
                account = existing_acc
            else:
                account = Account(
                    id=str(uuid.uuid4()),
                    student_id=student.id,
                    provider=acc_data[&quot;provider&quot;],
                    username=acc_data[&quot;username&quot;],
                    auth_type=acc_data[&quot;auth_type&quot;],
                    status=AccountStatus.active,
                    active=True,
                    created_at=datetime.now(timezone.utc),
                )
                session.add(account)
                session.flush()

                # Repository for this account
                repo = Repository(
                    id=str(uuid.uuid4()),
                    account_id=account.id,
                    repo_name=f&quot;student-project-{i+1}&quot;,
                    remote_url=f&quot;https://github.com/{acc_data[&apos;username&apos;]}/student-project.git&quot;
                    local_path=f&quot;/tmp/studentflow-demo/repos/student-{i+1}&quot;,
                    default_branch=&quot;main&quot;,
                    integration_branch=&quot;integration&quot;,
                    sync_status=SyncStatus.unknown,
                    last_commit_hash=None,
                    created_at=datetime.now(timezone.utc),
                )
                session.add(repo)
```

```
            # Credential reference (no actual secret stored here)
            credential = Credential(
                id=str(uuid.uuid4()),
                account_id=account.id,
                secret_ref=f&quot;studentflow:{account.id}:{acc_data[&apos;auth_type&apos;].value}&quot;,
                secret_kind=CredentialKind(acc_data[&quot;auth_type&quot;].value)
                    if acc_data[&quot;auth_type&quot;].value in CredentialKind._value2member_map_
                    else CredentialKind.pat,
                is_encrypted=True,
                last_validated_at=None,
            )
            session.add(credential)

    session.flush()

    # Sample audit log entries
    audit_count = session.query(ActivityLog).count()
    if audit_count &lt; 5:
        actions = [
            (&quot;student_created&quot;, &quot;student&quot;, &quot;Alice added to the system&quot;),
            (&quot;sync_completed&quot;, &quot;sync_job&quot;, &quot;Alice sync: 3 commits pushed to canonical
            (&quot;handoff_completed&quot;, &quot;handoff_event&quot;, &quot;Handoff Alice → Bohdan, tag check
            (&quot;student_created&quot;, &quot;student&quot;, &quot;Bohdan added to the system&quot;),
            (&quot;sync_started&quot;, &quot;sync_job&quot;, &quot;Bohdan initial sync started&quot;),
            (&quot;credential_stored&quot;, &quot;credential&quot;, &quot;PAT stored for b-marchenko&quot;),
            (&quot;credential_validated&quot;, &quot;credential&quot;, &quot;PAT validated OK for b-marchenko&q
            (&quot;repo_created&quot;, &quot;repository&quot;, &quot;student-project-2 created from template&qu
            (&quot;sync_completed&quot;, &quot;sync_job&quot;, &quot;Bohdan sync: up to date&quot;),
            (&quot;settings_updated&quot;, &quot;app_setting&quot;, &quot;canonical_repo_path updated&quot;),
        ]
        for action, entity_type, details_text in actions:
            log_entry = ActivityLog(
                id=str(uuid.uuid4()),
                actor=&quot;operator&quot;,
                entity_type=entity_type,
                entity_id=None,
                action=action,
                details=f&apos;{{&quot;message&quot;: &quot;{details_text}&quot;}}&apos;,
                created_at=datetime.now(timezone.utc),
            )
            session.add(log_entry)

    session.commit()
    print(f&quot;Seed data inserted: {len(DEMO_STUDENTS)} students, accounts, repos, credentials, 10 audit ent


if __name__ == &quot;__main__&quot;:
    from app.bootstrap import create_app_dirs, init_db, init_logging
    create_app_dirs()
    init_logging()
    init_db()
    seed_demo_data()
```

Acceptance Criteria

1. `python -m app.db.seeds` runs without error and prints the summary line
2. After running: `SELECT COUNT(*) FROM students` returns 5 (or more if run on existing data)
3. Running `python -m app.db.seeds` a second time does NOT insert duplicate students
4. `ActivityLog` table has at least 10 records after seeding
5. No `Credential` record contains a `secret_ref` that looks like an actual token (`ghp_`, `glpat-`, etc.)
6. `from app.db.enums import StudentStatus; StudentStatus.active.value == "active"` — True

Notes for Codex

- `CredentialKind` values may not exactly match `AuthType` values — use a mapping or try/except for the conversion
- `local_path` values in seed data use `/tmp/studentflow-demo/` — these paths do NOT need to exist on disk (they are demo only)
- The `if __name__ == "__main__"` block allows direct execution: `python -m app.db.seeds`
- `enums.py` must be importable without any DB connection — it only defines Python enum classes
- When `app/db/models.py` (T02) references enums, it must import from `app.db.enums`, not redefine them
---
codex-tasks/T15-table-density-canonical-auth.md
Розмір: 17,554 байт

T15 — Table Row Density Fix + Canonical Remote Account Binding

Title

Table readability improvements + canonical remote authentication/account binding in Settings

Goal

Three scoped improvements to StudentFlow:
- **Part A:** Fix table row density so rows are visibly taller and less cramped across all major screens
- **Part B:** Add explicit canonical remote account binding in Settings (operator selects which saved account to use for canonical remote pushes)
- **Part C:** Wire the account binding into `_try_push_canonical_remote` so canonical pushes use the bound credential when configured

Tech Stack

Python 3.11, PySide6 6.x, SQLAlchemy, pytest, structlog

Context

Current Codebase State

All list screens (Students, Accounts, Repositories, Credentials, Prompts, Logs) share the same table initialization pattern:
self._table.verticalHeader().setDefaultSectionSize(36)
self._table.verticalHeader().setMinimumSectionSize(32)

And all action-cell widgets use:
cl.setContentsMargins(4, 4, 4, 4)

Global QSS in `app/ui/windows/main_window.py` has:
QTableWidget::item { padding: 8px 10px; min-height: 20px; }
QHeaderView::section { padding: 7px 10px; }

The `_try_push_canonical_remote` in `app/services/sync_service.py` currently does:
self._git.push_to_url(canonical_path, canonical_remote_url, branch)
# No credential — relies on system git auth

The `AppSetting` model (`app/db/models.py`) is a simple key/value store — no schema migration needed, just new keys.

The `Account` model has: `id`, `username`, `provider`, `auth_type`, `status`, `is_current`
The `Credential` model has: `account_id`, `secret_ref`, `secret_kind`
`CredentialService.retrieve(secret_ref)` returns the plaintext credential value.

The `push_to_url` method signature already supports credentials:
def push_to_url(self, repo_path, remote_url, branch, credential=None, auth_type=None, force=False)

Settings Currently
`OPERATOR_SETTINGS` in `app/ui/viewmodels/settings_viewmodel.py`:
OPERATOR_SETTINGS = [
    "canonical_repo_path",
    "canonical_remote_url",
    "default_branch",
    "sync_timeout_seconds",
    "max_sync_retries",
]
OPTIONAL_SETTINGS = {"canonical_remote_url"}

Settings screen renders all these as QLineEdit fields except no ComboBox/account picker exists yet.

Constraints

1. Do NOT embed raw tokens in URLs
2. Do NOT add new DB migration — AppSetting uses a key/value table, no schema change needed
3. Do NOT break existing student synchronization logic
4. Do NOT rewrite sync_service — minimal extension only
5. Do NOT create new files unless absolutely necessary
6. Preserve all existing behavior when `canonical_remote_account_id` is empty/not set
7. Use `.venv/bin/pytest` to run tests (the venv is at `/home/vokov/projects/studentflow/.venv/`)

Part A — Table Row Density Fix

Files to modify for Part A:

File 1: `app/ui/windows/main_window.py`

In the `_APP_QSS` string, change:
QTableWidget::item {
    padding: 8px 10px;
    border: none;
    min-height: 20px;
}

→
QTableWidget::item {
    padding: 10px 12px;
    border: none;
    min-height: 24px;

```
}
```

Change:
```
QHeaderView::section {
    background-color: #1e1e2e;
    color: #89b4fa;
    border: none;
    border-right: 1px solid #313244;
    border-bottom: 2px solid #45475a;
    padding: 7px 10px;
    font-size: 12px;
    font-weight: 700;
    letter-spacing: 0.5px;
}
```

→
```
QHeaderView::section {
    background-color: #1e1e2e;
    color: #89b4fa;
    border: none;
    border-right: 1px solid #313244;
    border-bottom: 2px solid #45475a;
    padding: 10px 12px;
    font-size: 12px;
    font-weight: 700;
    letter-spacing: 0.5px;
}
```

Files 2–7: All six screen files listed below

For each file, make exactly these two changes:

Change `setDefaultSectionSize(36)` → `setDefaultSectionSize(44)`
Change `setMinimumSectionSize(32)` → `setMinimumSectionSize(40)`

And for any action cell widget layout that has `cl.setContentsMargins(4, 4, 4, 4)`, change to `cl.setContentsMargins(4, 6, 4, 6)`.

Screens to update:
- `app/ui/windows/students_screen.py`
- `app/ui/windows/accounts_screen.py`
- `app/ui/windows/repositories_screen.py`
- `app/ui/windows/credentials_screen.py`
- `app/ui/windows/logs_screen.py`
- `app/ui/windows/prompts_screen.py`

Part B — Canonical Remote Account Binding

Step B1: `app/ui/viewmodels/settings_viewmodel.py`

Add `"canonical_remote_account_id"` to `OPERATOR_SETTINGS` list (after `"canonical_remote_url"`):
```
OPERATOR_SETTINGS = [
    &quot;canonical_repo_path&quot;,
    &quot;canonical_remote_url&quot;,
    &quot;canonical_remote_account_id&quot;,
    &quot;default_branch&quot;,
    &quot;sync_timeout_seconds&quot;,
    &quot;max_sync_retries&quot;,
]
```

Add it to `OPTIONAL_SETTINGS`:
```
OPTIONAL_SETTINGS = {&quot;canonical_remote_url&quot;, &quot;canonical_remote_account_id&quot;}
```

Add a new method `list_accounts()` to `SettingsViewModel`:
def list_accounts(self) -> list[dict]:
    """Return all accounts for the canonical remote account dropdown."""
    from app.db.models import Account
    with self._factory() as session:
        accounts = session.query(Account).order_by(Account.username).all()
        return [
            {"id": a.id, "username": a.username, "provider": a.provider, "auth_
            for a in accounts
        ]

Step B2: `app/ui/windows/settings_screen.py`

The Settings screen currently renders all settings as QLineEdit fields using a loop.

Change the approach: Instead of rendering `canonical_remote_account_id` as a QLineEdit, render it as a QComboBox populated with accounts.

Replace the `_build_ui` method with an updated version that:

1. Imports `QComboBox` in addition to existing imports
2. In the loop over settings keys, skips `"canonical_remote_account_id"` (handled separately)
3. After the `canonical_remote_url` row in the form, adds a ComboBox row for "Canonical remote account":
- First item: `("— None (system auth) —", "")`
- Then each account: display text = `"username (provider / auth_type)"`, stored data = account ID
4. Stores the ComboBox as `self._account_combo: QComboBox`
5. Loads accounts in `_load()` — populate the combo from `self._vm.list_accounts()`, then select the one matching the saved `canonical_remote_account_id` value
6. In `_on_save()`, reads the combo's current data and saves it as `canonical_remote_account_id`

Here is the complete replacement for `app/ui/windows/settings_screen.py`:

"""Settings screen — operator-visible AppSetting configuration."""
from __future__ import annotations

from PySide6.QtWidgets import (
    QComboBox,
    QFormLayout,
    QGroupBox,
    QHBoxLayout,
    QLabel,
    QLineEdit,
    QMessageBox,
    QPushButton,
    QVBoxLayout,
    QWidget,
)

from app.ui.viewmodels.settings_viewmodel import SettingsViewModel

_LABELS = {
    "canonical_repo_path": "Canonical local path",
    "canonical_remote_url": "Canonical remote URL (optional)",
    "default_branch": "Default branch",
    "sync_timeout_seconds": "Sync timeout (seconds)",
    "max_sync_retries": "Max sync retries",
}

_OPTIONAL_KEYS = {"canonical_remote_url"}

# Keys rendered as QLineEdit (account picker is handled separately)
_LINE_EDIT_KEYS = [
    "canonical_repo_path",
    "canonical_remote_url",

```
    &quot;default_branch&quot;,
    &quot;sync_timeout_seconds&quot;,
    &quot;max_sync_retries&quot;,
]


class SettingsScreen(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._vm = SettingsViewModel()
        self._fields: dict[str, QLineEdit] = {}
        self._account_combo: QComboBox | None = None
        self._build_ui()
        self._load()

    def _build_ui(self) -&gt; None:
        layout = QVBoxLayout(self)
        layout.setContentsMargins(24, 24, 24, 24)
        layout.setSpacing(16)

        title = QLabel(&quot;Settings&quot;, self)
        title.setObjectName(&quot;screenTitle&quot;)
        layout.addWidget(title)

        group = QGroupBox(&quot;Canonical Repository &amp; Sync&quot;, self)
        form = QFormLayout(group)
        form.setLabelAlignment(form.labelAlignment())
        form.setRowWrapPolicy(QFormLayout.RowWrapPolicy.WrapLongRows)

        for key in _LINE_EDIT_KEYS:
            field = QLineEdit(self)
            field.setObjectName(key)
            if key in _OPTIONAL_KEYS:
                field.setPlaceholderText(&quot;e.g. https://github.com/org/canonical-project.git&quot;)
            form.addRow(f&quot;{_LABELS.get(key, key)}:&quot;, field)
            self._fields[key] = field

            # Insert account ComboBox right after canonical_remote_url
            if key == &quot;canonical_remote_url&quot;:
                self._account_combo = QComboBox(self)
                self._account_combo.setObjectName(&quot;canonical_remote_account_id&quot;)
                form.addRow(&quot;Canonical remote account:&quot;, self._account_combo)

        layout.addWidget(group)

        button_row = QWidget(self)
        button_layout = QHBoxLayout(button_row)
        button_layout.setContentsMargins(0, 0, 0, 0)

        save_btn = QPushButton(&quot;Save&quot;, button_row)
        save_btn.setProperty(&quot;role&quot;, &quot;primary&quot;)
        save_btn.clicked.connect(self._on_save)
        button_layout.addWidget(save_btn)

        reset_btn = QPushButton(&quot;Reset to saved&quot;, button_row)
        reset_btn.clicked.connect(self._load)
        button_layout.addWidget(reset_btn)

        button_layout.addStretch(1)
        layout.addWidget(button_row)
        layout.addStretch(1)

    def _load(self) -&gt; None:
        try:
            rows = {row[&quot;key&quot;]: row[&quot;value&quot;] for row in self._vm.list_operator_settings()}
            for key, field in self._fields.items():
                field.setText(rows.get(key, &quot;&quot;))
```

```
            self._reload_account_combo(rows.get(&quot;canonical_remote_account_id&quot;, &quot;&quot;))
        except Exception as exc:
            QMessageBox.critical(self, &quot;Error&quot;, str(exc))

    def _reload_account_combo(self, selected_id: str) -&gt; None:
        if self._account_combo is None:
            return
        self._account_combo.clear()
        self._account_combo.addItem(&quot;— None (system auth) —&quot;, &quot;&quot;)
        try:
            for acct in self._vm.list_accounts():
                label = f&quot;{acct[&apos;username&apos;]} ({acct[&apos;provider&apos;]} / {acct[&apos;auth_type
                self._account_combo.addItem(label, acct[&quot;id&quot;])
        except Exception:
            pass  # Accounts unavailable — combo stays with &quot;None&quot; only
        # Select the saved account
        for i in range(self._account_combo.count()):
            if self._account_combo.itemData(i) == selected_id:
                self._account_combo.setCurrentIndex(i)
                break

    def _on_save(self) -&gt; None:
        errors = []
        for key, field in self._fields.items():
            value = field.text().strip()
            if not value and key not in _OPTIONAL_KEYS:
                errors.append(f&quot;&apos;{_LABELS.get(key, key)}&apos; cannot be empty&quot;)

        if errors:
            QMessageBox.warning(self, &quot;Validation&quot;, &quot;\n&quot;.join(errors))
            return

        try:
            for key, field in self._fields.items():
                self._vm.set_setting(key, field.text().strip())
            # Save canonical_remote_account_id from combo
            if self._account_combo is not None:
                account_id = self._account_combo.currentData() or &quot;&quot;
                self._vm.set_setting(&quot;canonical_remote_account_id&quot;, account_id)
            QMessageBox.information(self, &quot;Saved&quot;, &quot;Settings saved successfully.&quot;)
        except Exception as exc:
            QMessageBox.critical(self, &quot;Error&quot;, str(exc))
```

Part C — Wire Account Binding into Canonical Remote Push

File: `app/services/sync_service.py`

Replace the `_try_push_canonical_remote` method with this updated version:

```
def _try_push_canonical_remote(self, branch: str) -&gt; str | None:
    &quot;&quot;&quot;Push the local canonical bare repo to canonical_remote_url if configured.

    Returns None when:
      - canonical_remote_url is not configured (empty or missing AppSetting)
      - the push succeeds

    Returns a warning string when the push fails for any reason.
    Failure is NON-FATAL.

    Authentication: if canonical_remote_account_id is configured, the bound
    account&apos;s stored credential is used. Otherwise falls back to system-level
    git authentication (SSH agent, ~/.netrc, credential manager).
    &quot;&quot;&quot;
```

```
with self._session_factory() as session:
    url_setting = session.get(AppSetting, &quot;canonical_remote_url&quot;)
    canonical_remote_url = (
        url_setting.value.strip()
        if url_setting and url_setting.value and url_setting.value.strip()
        else None
    )
    if not canonical_remote_url:
        return None

    path_setting = session.get(AppSetting, &quot;canonical_repo_path&quot;)
    canonical_path = (
        path_setting.value.strip()
        if path_setting and path_setting.value and path_setting.value.strip()
        else None
    )

    account_id_setting = session.get(AppSetting, &quot;canonical_remote_account_id&quot;)
    canonical_account_id = (
        account_id_setting.value.strip()
        if account_id_setting and account_id_setting.value and account_id_setting.value.strip()
        else None
    )

if not canonical_path:
    return &quot;canonical_remote_url is set but canonical_repo_path is not configured&quot;

# Resolve bound account credential (if any)
credential_value: str | None = None
auth_type: str | None = None

if canonical_account_id:
    with self._session_factory() as session:
        account = session.get(Account, canonical_account_id)
        if account is None:
            return (
                f&quot;canonical remote account (id={canonical_account_id!r}) not found — &quot;
                &quot;update Settings to fix the binding&quot;
            )
        credential = session.query(Credential).filter_by(account_id=account.id).first()
        if credential is None:
            return (
                f&quot;canonical remote account &apos;{account.username}&apos; has no stored credential — &quo
                &quot;add a credential in the Credentials screen&quot;
            )
        bound_secret_ref = credential.secret_ref
        bound_auth_type = account.auth_type

    try:
        credential_value = self._creds.retrieve(bound_secret_ref)
        auth_type = bound_auth_type
    except KeyError as exc:
        return f&quot;canonical remote credential retrieval failed: {exc}&quot;

try:
    self._git.push_to_url(
        canonical_path,
        canonical_remote_url,
        branch,
        credential=credential_value,
        auth_type=auth_type,
    )
    logger.info(
        &quot;canonical_remote_push_succeeded&quot;,
        remote=canonical_remote_url,
        auth=&quot;account_binding&quot; if canonical_account_id else &quot;system_auth&quot;,
    )
```

```
        return None
    except Exception as exc:
        warning = f&quot;canonical remote push failed: {str(exc)[:150]}&quot;
        logger.warning(
            &quot;canonical_remote_push_failed&quot;,
            remote=canonical_remote_url,
            error=str(exc)[:200],
        )
        return warning
```

Important: The `Account` and `Credential` imports are already in `sync_service.py` at line 11:
from app.db.models import Account, AppSetting, Credential, Repository, Student, SyncJob

No new imports are required.

Test Requirements

After all changes, run:
.venv/bin/pytest tests/ -q --tb=short

All 155 existing tests must pass.

Additionally, verify these specific test files pass cleanly:
- `tests/test_sync_service.py` — check `_try_push_canonical_remote` is still invoked correctly
- Any test files for Settings or UI viewmodels

If any test fails due to Part B/C changes (e.g., settings viewmodel tests expect exactly the old `OPERATOR_SETTINGS` list), update those tests to include `canonical_remote_account_id`.

Acceptance Criteria

1. **Part A**: All 6 table screens have `setDefaultSectionSize(44)` and `setMinimumSectionSize(40)`
2. **Part A**: All action cell widgets have `setContentsMargins(4, 6, 4, 6)`
3. **Part A**: QSS item padding is `10px 12px` and header padding is `10px 12px`
4. **Part B**: `OPERATOR_SETTINGS` includes `"canonical_remote_account_id"` after `"canonical_remote_url"`
5. **Part B**: `OPTIONAL_SETTINGS` includes `"canonical_remote_account_id"`
6. **Part B**: `SettingsViewModel.list_accounts()` method exists and returns account dicts
7. **Part B**: Settings screen shows a QComboBox for "Canonical remote account" after the URL field
8. **Part B**: ComboBox first item is "— None (system auth) —" with empty data
9. **Part B**: ComboBox populates from `list_accounts()` and selects saved value on load
10. **Part B**: Save correctly stores `canonical_remote_account_id` as AppSetting
11. **Part C**: `_try_push_canonical_remote` reads `canonical_remote_account_id` AppSetting
12. **Part C**: When account bound: retrieves credential and passes to `push_to_url`
13. **Part C**: When account not found or no credential: returns descriptive warning string (non-fatal)
14. **Part C**: When no account bound: behavior is identical to before (system auth, `credential=None`)
15. **Tests**: All 155 existing tests pass

Deliverable

All modified files saved to disk. No new files created unless strictly necessary. Tests passing.

---
codex-tasks/T07-sync-service.md
Розмір: 16,888 байт

T07: Implement SyncService

Goal

Implement `SyncService` in `app/services/sync_service.py` with the full per-student sync pipeline, sync-all batch, divergence detection, and force-overwrite recovery. Every sync operation creates a `SyncJob` record in the DB. Failures are isolated per student — one failure never aborts sync-all.

Context

- Project root: `/home/vokov/projects/studentflow/`
- Sync strategy: `docs/sync-strategy.md` — read ALL sections before coding
- Models: `SyncJob`, `Repository`, `Student`, `Account`, `Credential`, `AppSetting` in `app/db/models.py` (T02)
- Enums: `SyncJobType`, `SyncJobStatus`, `SyncStatus` in `app/db/enums.py` (T02)
- GitClient: `app/git/client.py` (T06) — inject as dependency
- CredentialService: `app/services/credential_service.py` (T03) — inject as dependency
- AuditService: `app/services/audit_service.py` (T09) — inject as dependency

Constraints

- SyncService must NOT call `keyring` directly — use `CredentialService.retrieve()`
- `sync_all()` must NOT abort on a single student failure — catch per-student, continue
- All Git operations must go through the injected `GitClient` — no direct `subprocess` or `git`
- SyncJob records must be created for EVERY operation (pending → running → terminal)
- `SyncJob.error_message` must be truncated to 500 characters
- `SyncJob.error_message` must NEVER contain credential values
- `sync_student()` must NOT be called from `sync_all()` without wrapping in try/except

Files to Create/Update

app/services/sync_service.py — full implementation
tests/test_sync_service.py — unit tests with mocked GitClient

Implementation Requirements

SyncService class

class SyncService:
    """Per-student sync pipeline and sync-all batch.

    Each sync operation fetches from the student's remote, merges into the local clone,
    and pushes to the canonical repo. Results are recorded as SyncJob DB records.

    Dependency injection: git_client, credential_service, audit_service — injectable
    for testing. Production code uses defaults.
    """

```
def __init__(
    self,
    git_client=None,
    credential_service=None,
    audit_service=None,
):
    from app.git.client import GitClient
    from app.services.credential_service import CredentialService
    from app.services.audit_service import AuditService
    self._git = git_client or GitClient()
    self._creds = credential_service or CredentialService()
    self._audit = audit_service or AuditService()

def sync_student(self, student_id: str) -&gt; SyncJob:
    &quot;&quot;&quot;Run the full sync pipeline for one student.

    Pipeline:
    1. Load student → account → repository → credential
    2. Create SyncJob(status=pending, job_type=sync_current)
    3. Set SyncJob.status=running, record started_at
    4. Retrieve credential from keyring
    5. GitClient.fetch(repo.local_path, &apos;student&apos;)
    6. ahead, behind = GitClient.check_divergence(repo.local_path, &apos;student/main&apos;)
    7. If both ahead&gt;0 and behind&gt;0: set sync_status=diverged, SyncJob=failed, return
    8. If ahead==0 and behind==0: SyncJob=skipped, return
    9. If ahead&gt;0: GitClient.merge_ff_only(repo.local_path, &apos;student/main&apos;)
       If merge_ff_only raises GitMergeError: set sync_status=diverged, SyncJob=failed, return
    10. GitClient.push(repo.local_path, &apos;origin&apos;, repo.integration_branch, credential=None)
        (push to canonical local bare repo — no credential needed for local push)
    11. new_hash = GitClient.get_commit_hash(repo.local_path)
    12. Update repo.last_commit_hash=new_hash, repo.sync_status=clean, repo.updated_at
    13. SyncJob.status=success, finished_at=now, commit_hash_after=new_hash
    14. summary = f&quot;Synced {ahead} commits from {account.username}&quot;
    15. AuditService.log(...)
    16. Return SyncJob

    Args:
        student_id: UUID of Student to sync

    Returns:
        SyncJob record with final status (success, failed, or skipped)

    Raises:
        ValueError: if student, account, or repository not found (not a sync failure)
    &quot;&quot;&quot;

def sync_all(self) -&gt; list[SyncJob]:
    &quot;&quot;&quot;Run sync_student for all students with status=active.

    Iterates in queue_position ASC order. Catches ALL exceptions per student —
    one failure never aborts the loop.

    Returns:
        List of SyncJob records (one per active student with a repo)
        May include failed SyncJobs for students that errored.
    &quot;&quot;&quot;

def detect_divergence(self, repo_id: str) -&gt; bool:
    &quot;&quot;&quot;Check if a repository is diverged (both sides have unique commits).

    Requires a recent fetch to be accurate.

    Returns:
        True if diverged (ahead&gt;0 AND behind&gt;0)
        False otherwise
    &quot;&quot;&quot;
def force_overwrite_from_canonical(self, repo_id: str) -&gt; SyncJob:
```

```
        &quot;&quot;&quot;Hard reset a student&apos;s local repo to canonical HEAD, force-push to their remote.

        This is a destructive recovery operation. It discards any divergent commits
        on the student&apos;s side that are not in canonical.

        Pipeline:
        1. Create SyncJob(job_type=sync_current, status=running)
        2. Load canonical path from AppSetting
        3. GitClient.fetch(repo.local_path, &apos;origin&apos;)  (fetch canonical updates)
        4. GitClient.reset_hard(repo.local_path, &apos;origin/main&apos;)
        5. Load credential, GitClient.push(repo.local_path, &apos;student&apos;, branch, credential, force=True)
        6. Update repo.sync_status=clean, last_commit_hash
        7. SyncJob.status=success
        8. AuditService.log(..., action=&apos;force_overwrite&apos;, ...)
        9. Return SyncJob
        &quot;&quot;&quot;

    def get_recent_jobs(
        self,
        limit: int = 50,
        student_id: str | None = None,
        status: str | None = None,
        job_type: str | None = None,
    ) -&gt; list[SyncJob]:
        &quot;&quot;&quot;Query recent SyncJob records with optional filters.

        Returns:
            List of SyncJob records ordered by created_at DESC
        &quot;&quot;&quot;


### sync_student complete implementation:

def sync_student(self, student_id: str) -&gt; SyncJob:
    import structlog
    logger = structlog.get_logger(__name__)

    with SessionLocal() as session:
        student = session.get(Student, student_id)
        if student is None:
            raise ValueError(f&quot;Student {student_id} not found&quot;)

        # Get current account and its repo
        account = session.query(Account).filter_by(
            student_id=student_id, is_current=True
        ).first()
        if account is None:
            # Fall back to first active account
            account = session.query(Account).filter_by(student_id=student_id).first()
        if account is None:
            raise ValueError(f&quot;No account found for student {student_id}&quot;)

        repo = session.query(Repository).filter_by(
            account_id=account.id, is_canonical=False
        ).first()
        if repo is None:
            raise ValueError(f&quot;No repository found for account {account.id}&quot;)

        credential = session.query(Credential).filter_by(account_id=account.id).first()
        if credential is None:
            raise ValueError(f&quot;No credential for account {account.id}&quot;)

        commit_hash_before = repo.last_commit_hash

        # Create SyncJob record
        sync_job = SyncJob(
            repo_id=repo.id,
            student_id=student_id,
```

```
        job_type=SyncJobType.sync_current.value,
        status=SyncJobStatus.pending.value,
        commit_hash_before=commit_hash_before,
        created_at=datetime.utcnow(),
    )
    session.add(sync_job)
    session.flush()
    job_id = sync_job.id

    # Transition to running
    sync_job.status = SyncJobStatus.running.value
    sync_job.started_at = datetime.utcnow()
    session.commit()

    # Capture values before session closes
    repo_id = repo.id
    local_path = repo.local_path
    integration_branch = repo.integration_branch
    secret_ref = credential.secret_ref
    username = account.username

# Git operations (outside DB session)
try:
    cred_value = self._creds.retrieve(secret_ref)
except KeyError as e:
    return self._fail_job(job_id, repo_id, str(e), SyncStatus.error)

try:
    self._git.fetch(local_path, &quot;student&quot;)
except Exception as e:
    error_msg = f&quot;fetch failed: {str(e)[:450]}&quot;
    logger.error(&quot;sync_fetch_failed&quot;, student_id=student_id, error=str(e)[:200])
    return self._fail_job(job_id, repo_id, error_msg, SyncStatus.error)

try:
    ahead, behind = self._git.check_divergence(local_path, &quot;student/main&quot;)
except Exception as e:
    return self._fail_job(job_id, repo_id, f&quot;divergence check failed: {str(e)[:450]}&quot;, SyncStatus.error)

if ahead &gt; 0 and behind &gt; 0:
    logger.warning(&quot;sync_diverged&quot;, student_id=student_id, ahead=ahead, behind=behind)
    return self._fail_job(job_id, repo_id,
        f&quot;Diverged: student ahead={ahead}, local behind={behind}. Manual resolution required.&quot;,
        SyncStatus.diverged)

if ahead == 0:
    # Nothing to sync
    return self._complete_job(job_id, repo_id, skipped=True,
        summary=f&quot;Already up to date ({username})&quot;, commit_hash_after=commit_hash_before)

# ahead &gt; 0 and behind == 0: merge
try:
    self._git.merge_ff_only(local_path, &quot;student/main&quot;)
except Exception as e:
    return self._fail_job(job_id, repo_id, f&quot;merge failed: {str(e)[:450]}&quot;, SyncStatus.diverged)

# Push to canonical
try:
    self._git.push(local_path, &quot;origin&quot;, integration_branch, credential=None)
except Exception as e:
    return self._fail_job(job_id, repo_id, f&quot;push to canonical failed: {str(e)[:450]}&quot;, SyncStatus.error)

# Get new commit hash
try:
    new_hash = self._git.get_commit_hash(local_path)
except Exception:
    new_hash = None
```

```
    # Update repo and job
    with SessionLocal() as session:
        repo = session.get(Repository, repo_id)
        repo.last_commit_hash = new_hash
        repo.sync_status = SyncStatus.clean.value
        repo.updated_at = datetime.utcnow()

        job = session.get(SyncJob, job_id)
        job.status = SyncJobStatus.success.value
        job.finished_at = datetime.utcnow()
        job.commit_hash_after = new_hash
        job.summary = f&quot;Synced {ahead} commit(s) from {username}&quot;
        session.commit()
        final_job = job

    self._audit.log(
        &quot;operator&quot;, &quot;repository&quot;, repo_id, &quot;sync&quot;,
        {&quot;job_id&quot;: job_id, &quot;student_id&quot;: student_id, &quot;commits_synced&quot;: ahead,
         &quot;commit_before&quot;: commit_hash_before, &quot;commit_after&quot;: new_hash}
    )
    logger.info(&quot;sync_complete&quot;, student_id=student_id, commits_synced=ahead)
    return final_job


### Helper methods

def _fail_job(self, job_id: str, repo_id: str, error_msg: str, repo_status: SyncStatus) -&gt; SyncJob:
    &quot;&quot;&quot;Mark a SyncJob as failed and update repo status.&quot;&quot;&quot;
    with SessionLocal() as session:
        job = session.get(SyncJob, job_id)
        job.status = SyncJobStatus.failed.value
        job.finished_at = datetime.utcnow()
        job.error_message = error_msg[:500]  # Truncate to 500 chars

        repo = session.get(Repository, repo_id)
        if repo:
            repo.sync_status = repo_status.value
            repo.updated_at = datetime.utcnow()

        session.commit()
        return job


def _complete_job(self, job_id: str, repo_id: str, skipped: bool, summary: str, commit_hash_after: str | None) -&gt; S
    &quot;&quot;&quot;Mark a SyncJob as success or skipped.&quot;&quot;&quot;
    with SessionLocal() as session:
        job = session.get(SyncJob, job_id)
        job.status = SyncJobStatus.skipped.value if skipped else SyncJobStatus.success.value
        job.finished_at = datetime.utcnow()
        job.summary = summary
        job.commit_hash_after = commit_hash_after
        session.commit()
        return job


### sync_all implementation

def sync_all(self) -&gt; list[SyncJob]:
    import structlog
    logger = structlog.get_logger(__name__)

    with SessionLocal() as session:
        students = session.query(Student).filter(
            Student.status == StudentStatus.active.value,
            Student.queue_position.isnot(None)
        ).order_by(Student.queue_position.asc()).all()
        student_ids = [s.id for s in students]
    logger.info(&quot;sync_all_start&quot;, active_students=len(student_ids))
```

```
    results = []
    succeeded = 0
    failed = 0

    for sid in student_ids:
        try:
            job = self.sync_student(sid)
            results.append(job)
            if job.status == SyncJobStatus.success.value:
                succeeded += 1
            elif job.status == SyncJobStatus.failed.value:
                failed += 1
        except ValueError as e:
            # Configuration error — student not properly set up
            logger.warning(&quot;sync_all_skip&quot;, student_id=sid, reason=str(e))
        except Exception as e:
            logger.error(&quot;sync_all_student_failed&quot;, student_id=sid, error=str(e)[:200])
            failed += 1
            # Create a failed SyncJob record for visibility
            with SessionLocal() as session:
                error_job = SyncJob(
                    repo_id=&quot;unknown&quot;,
                    student_id=sid,
                    job_type=SyncJobType.sync_all.value,
                    status=SyncJobStatus.failed.value,
                    error_message=str(e)[:500],
                    created_at=datetime.utcnow(),
                    started_at=datetime.utcnow(),
                    finished_at=datetime.utcnow(),
                )
                try:
                    session.add(error_job)
                    session.commit()
                    results.append(error_job)
                except Exception:
                    pass  # Don&apos;t let error logging fail the loop

    logger.info(&quot;sync_all_complete&quot;, succeeded=succeeded, failed=failed, skipped=len(student_ids) - su
    return results
```

Tests (tests/test_sync_service.py)

All tests use mocked GitClient and CredentialService.

test_sync_student_success:

```
def test_sync_student_success(db_session):
    from unittest.mock import MagicMock, patch
    # Create Student + Account + Repository + Credential in test DB
    # Mock GitClient: fetch returns None, check_divergence returns (5, 0),
    # merge_ff_only returns None, push returns None, get_commit_hash returns &quot;b&quot;*40
    # Mock CredentialService.retrieve returns &quot;fake_token&quot;
    # Call SyncService(git_client=mock_git, credential_service=mock_creds).sync_student(student.id)
    # Assert: job.status == &quot;success&quot;, job.summary contains &quot;5&quot;, repo.sync_status == &quo
```

test_sync_student_diverged:

```
def test_sync_student_diverged(db_session):
    # check_divergence returns (3, 2)
    # Assert: job.status == &quot;failed&quot;, job.error_message contains &quot;Diverged&quot;
    # Assert: repo.sync_status == &quot;diverged&quot;
```

test_sync_student_git_fetch_error:

```
def test_sync_student_git_fetch_error(db_session):
```

```
    # mock_git.fetch raises GitOperationError(&quot;Connection refused&quot;)
    # Assert: job.status == &quot;failed&quot;
    # Assert: job.error_message contains &quot;fetch failed&quot;
    # Assert: credential value does NOT appear in job.error_message
```

test_sync_all_continues_on_failure:

```
def test_sync_all_continues_on_failure(db_session):
    # Create 3 students with active status and queue positions 1, 2, 3
    # Each has account, repo, credential
    # Mock: student 2&apos;s fetch raises GitOperationError
    # Mock: students 1 and 3 succeed (ahead=2, behind=0)
    # Call sync_all()
    # Assert: 3 SyncJob records returned
    # Assert: student 1&apos;s job is success
    # Assert: student 2&apos;s job is failed
    # Assert: student 3&apos;s job is success
```

test_sync_all_empty:

```
def test_sync_all_empty(db_session):
    # No students in DB (or none with status=active)
    service = SyncService(git_client=MagicMock(), credential_service=MagicMock())
    results = service.sync_all()
    assert results == []
```

Acceptance Criteria

1. `from app.services.sync_service import SyncService` — imports without error
2. All 5 tests pass: `pytest tests/test_sync_service.py -v` shows 5 passed
3. `sync_all()` never propagates an exception from a single student's sync
4. `SyncJob` is created for EVERY sync_student() call (even if it ultimately fails)
5. `SyncJob.error_message` is never longer than 500 characters
6. No credential values appear in `SyncJob.summary` or `SyncJob.error_message` fields
7. No direct `subprocess` or `git` calls in this file — only through `self._git`

---

codex-tasks/T02-models.md

Розмір: 12,500 байт

T02: Implement SQLAlchemy Models

Goal

Implement all 9 SQLAlchemy ORM models from `docs/data-model.md` with correct enums, relationships, indexes, and an Alembic initial migration. All models must be importable and the database must be creatable from scratch.

Context

- Project root: `/home/vokov/projects/studentflow/`
- Full data model spec: `docs/data-model.md` — read this before coding
- Stack: Python 3.11, SQLAlchemy 2.0.30, Alembic 1.13.1, SQLite
- Base is defined in `app/db/base.py` (created in T01)
- All PKs are UUID strings: `default=lambda: str(uuid.uuid4())`
- Enums: use Python `enum.Enum` classes, stored as SQLAlchemy `String` columns (not native SQL enums for SQLite portability)

Constraints

- Do NOT implement any service layer — models and migrations only
- Do NOT add any fields beyond those specified in `docs/data-model.md`
- `ActivityLog` must have NO `update()` method and NO `updated_at` field
- `Credential` model must have NO field for storing secret values — only `secret_ref`
- All enum values must exactly match those in `docs/data-model.md`
- No circular imports between `enums.py` and `models.py`

Files to Create/Update

```
app/db/enums.py              — all enum classes
app/db/models.py             — all 9 models
app/db/migrations/env.py       — alembic env (after alembic init)
app/db/migrations/script.py.mako  — alembic template
app/db/migrations/alembic.ini  — alembic config (at project root level)
alembic.ini                  — alembic config at project root
tests/test_models.py           — unit tests
```

Implementation Requirements

Step 1: Create app/db/enums.py

Define all enum classes:

```
&quot;&quot;&quot;All enum types for StudentFlow database models.&quot;&quot;&quot;
import enum


class StudentStatus(str, enum.Enum):
    active = &quot;active&quot;
    paused = &quot;paused&quot;
    exhausted = &quot;exhausted&quot;
    archived = &quot;archived&quot;
    error = &quot;error&quot;


class AccountProvider(str, enum.Enum):
    github = &quot;github&quot;
    gitlab = &quot;gitlab&quot;
    bitbucket = &quot;bitbucket&quot;
    gitea = &quot;gitea&quot;
    other = &quot;other&quot;


class AuthType(str, enum.Enum):
    ssh = &quot;ssh&quot;
    pat = &quot;pat&quot;
    app_password = &quot;app_password&quot;
    password = &quot;password&quot;


class AccountStatus(str, enum.Enum):
    active = &quot;active&quot;
    inactive = &quot;inactive&quot;
    error = &quot;error&quot;
    unvalidated = &quot;unvalidated&quot;
class SyncStatus(str, enum.Enum):
    clean = &quot;clean&quot;
```

```
    dirty = &quot;dirty&quot;
    diverged = &quot;diverged&quot;
    error = &quot;error&quot;
    uninitialized = &quot;uninitialized&quot;


class SyncJobType(str, enum.Enum):
    sync_current = &quot;sync_current&quot;
    sync_all = &quot;sync_all&quot;
    bootstrap = &quot;bootstrap&quot;
    handoff_push = &quot;handoff_push&quot;
    handoff_pull = &quot;handoff_pull&quot;


class SyncJobStatus(str, enum.Enum):
    pending = &quot;pending&quot;
    running = &quot;running&quot;
    success = &quot;success&quot;
    failed = &quot;failed&quot;
    skipped = &quot;skipped&quot;


class HandoffStatus(str, enum.Enum):
    pending = &quot;pending&quot;
    completed = &quot;completed&quot;
    failed = &quot;failed&quot;
    rolled_back = &quot;rolled_back&quot;


class EntityType(str, enum.Enum):
    student = &quot;student&quot;
    account = &quot;account&quot;
    repository = &quot;repository&quot;
    credential = &quot;credential&quot;
    sync_job = &quot;sync_job&quot;
    handoff = &quot;handoff&quot;
    template = &quot;template&quot;
    system = &quot;system&quot;


class SecretKind(str, enum.Enum):
    ssh_key = &quot;ssh_key&quot;
    pat = &quot;pat&quot;
    app_password = &quot;app_password&quot;
    password = &quot;password&quot;


### Step 2: Create app/db/models.py

Implement all 9 models. Key patterns:

**Student model:**
class Student(Base):
    __tablename__ = &quot;student&quot;

    id = Column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
    full_name = Column(String(255), nullable=False)
    email = Column(String(255), nullable=True)
    status = Column(String(20), nullable=False, default=StudentStatus.active.value)
    queue_position = Column(Integer, nullable=True)
    priority = Column(Integer, nullable=False, default=0)
    notes = Column(Text, nullable=True)
    created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
    updated_at = Column(DateTime, nullable=False, default=datetime.utcnow, onupdate=datetime.utcnow)

    # Relationships
    accounts = relationship(&quot;Account&quot;, back_populates=&quot;student&quot;, cascade=&quot;all, delet
    sync_jobs = relationship(&quot;SyncJob&quot;, back_populates=&quot;student&quot;)
```

```
    handoffs_from = relationship(&quot;HandoffEvent&quot;, foreign_keys=&quot;HandoffEvent.from_student_id&q
    handoffs_to = relationship(&quot;HandoffEvent&quot;, foreign_keys=&quot;HandoffEvent.to_student_id&quot;,

    __table_args__ = (
        Index(&quot;idx_student_status&quot;, &quot;status&quot;),
        Index(&quot;idx_student_queue_position&quot;, &quot;queue_position&quot;),
    )
```

Apply the same pattern for all other models. Enum values stored as `String` columns. Use `Column(String(N))` for enums, not `Enum()` type (for SQLite portability).

ActivityLog model (append-only — no updated_at):

```
class ActivityLog(Base):
    __tablename__ = &quot;activity_log&quot;

    id = Column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
    actor = Column(String(100), nullable=False, default=&quot;operator&quot;)
    entity_type = Column(String(50), nullable=False)
    entity_id = Column(String(36), nullable=True)
    action = Column(String(100), nullable=False)
    details = Column(JSON, nullable=True)
    created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
    # NO updated_at — this table is append-only

    __table_args__ = (
        Index(&quot;idx_activitylog_entity_type&quot;, &quot;entity_type&quot;),
        Index(&quot;idx_activitylog_entity_id&quot;, &quot;entity_id&quot;),
        Index(&quot;idx_activitylog_created_at&quot;, &quot;created_at&quot;),
    )
```

AppSetting model (string PK, no UUID):

```
class AppSetting(Base):
    __tablename__ = &quot;app_setting&quot;

    key = Column(String(100), primary_key=True)
    value = Column(Text, nullable=False)
    description = Column(Text, nullable=True)
    updated_at = Column(DateTime, nullable=False, default=datetime.utcnow, onupdate=datetime.utcnow)
```

Credential model (no secret value field):

```
class Credential(Base):
    __tablename__ = &quot;credential&quot;

    id = Column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
    account_id = Column(String(36), ForeignKey(&quot;account.id&quot;), nullable=False, unique=True)
    secret_ref = Column(String(255), nullable=False)
    # secret_kind stores &quot;ssh_key&quot;, &quot;pat&quot;, &quot;app_password&quot;, or &quot;password&q
    secret_kind = Column(String(20), nullable=False)
    is_encrypted = Column(Boolean, nullable=False, default=True)
    last_validated_at = Column(DateTime, nullable=True)
    created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
    updated_at = Column(DateTime, nullable=False, default=datetime.utcnow, onupdate=datetime.utcnow)
    # NO value, token, password, or secret column — ever

    account = relationship(&quot;Account&quot;, back_populates=&quot;credential&quot;)
```

HandoffEvent model (nullable from_student_id for first bootstrap):

```
class HandoffEvent(Base):
    __tablename__ = &quot;handoff_event&quot;

    id = Column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
    from_student_id = Column(String(36), ForeignKey(&quot;student.id&quot;), nullable=True)
    to_student_id = Column(String(36), ForeignKey(&quot;student.id&quot;), nullable=False)
    repo_id = Column(String(36), ForeignKey(&quot;repository.id&quot;), nullable=False)
    commit_hash = Column(String(40), nullable=False)
```

```
    checkpoint_tag = Column(String(255), nullable=False)
    status = Column(String(20), nullable=False, default=HandoffStatus.pending.value)
    notes = Column(Text, nullable=True)
    created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
    completed_at = Column(DateTime, nullable=True)

    from_student = relationship(&quot;Student&quot;, foreign_keys=[from_student_id], back_populates=&quot;han
    to_student = relationship(&quot;Student&quot;, foreign_keys=[to_student_id], back_populates=&quot;handoffs_
    repository = relationship(&quot;Repository&quot;, back_populates=&quot;handoff_events&quot;)
```

Step 3: Initialize Alembic

```
cd /home/vokov/projects/studentflow
alembic init app/db/migrations
```

Update `alembic.ini`:
- Set `sqlalchemy.url = sqlite:///%(here)s/../../../.studentflow/studentflow.db`

Update `app/db/migrations/env.py`:
- Import `Base` from `app.db.base`
- Set `target_metadata = Base.metadata`
- Configure to use the same `DATABASE_URL` as `app/db/base.py`

Step 4: Create initial migration

```
alembic revision --autogenerate -m &quot;initial_schema&quot;
```

Step 5: Verify migration runs

```
alembic upgrade head
```

Must complete without errors on a fresh SQLite database.

Tests (tests/test_models.py)

Write these exact 5 tests:

test_student_create:
```
def test_student_create(db_session):
    from app.db.models import Student
    from app.db.enums import StudentStatus
    student = Student(full_name=&quot;Alice Johnson&quot;, email=&quot;alice@example.com&quot;)
    db_session.add(student)
    db_session.commit()
    queried = db_session.query(Student).filter_by(full_name=&quot;Alice Johnson&quot;).first()
    assert queried is not None
    assert queried.email == &quot;alice@example.com&quot;
    assert queried.status == StudentStatus.active.value
    assert queried.queue_position is None
    assert len(queried.id) == 36  # UUID format
```

test_account_student_relationship:
```
def test_account_student_relationship(db_session):
    from app.db.models import Student, Account
    student = Student(full_name=&quot;Bob Chen&quot;)
    db_session.add(student)
    db_session.flush()
    account = Account(
        student_id=student.id,
        provider=&quot;github&quot;,
        username=&quot;bob-dev&quot;,
        auth_type=&quot;pat&quot;,
    )
    db_session.add(account)
```

```
    db_session.commit()
    queried_student = db_session.get(Student, student.id)
    assert len(queried_student.accounts) == 1
    assert queried_student.accounts[0].username == &quot;bob-dev&quot;
    assert queried_student.accounts[0].student.full_name == &quot;Bob Chen&quot;
```

test_credential_no_secret_field:

```
def test_credential_no_secret_field():
    from app.db.models import Credential
    from sqlalchemy import inspect
    mapper = inspect(Credential)
    column_names = [col.key for col in mapper.attrs]
    forbidden = {&quot;value&quot;, &quot;password&quot;, &quot;token&quot;, &quot;secret&quot;, &quot;privat
    found = forbidden.intersection(set(column_names))
    assert found == set(), f&quot;Credential model has forbidden secret fields: {found}&quot;
    assert &quot;secret_ref&quot; in column_names, &quot;secret_ref column must exist&quot;
```

test_activity_log_append_only:

```
def test_activity_log_append_only():
    from app.db.models import ActivityLog
    from sqlalchemy import inspect
    mapper = inspect(ActivityLog)
    column_names = [col.key for col in mapper.attrs]
    assert &quot;updated_at&quot; not in column_names, &quot;ActivityLog must not have updated_at&quot;
    assert &quot;modified_at&quot; not in column_names, &quot;ActivityLog must not have modified_at&quot;
    assert &quot;created_at&quot; in column_names, &quot;ActivityLog must have created_at&quot;
    # Verify no update method exists on the class
    assert not hasattr(ActivityLog, &quot;update&quot;), &quot;ActivityLog must not have an update() method&quo
```

test_all_enums_importable:

```
def test_all_enums_importable():
    from app.db.enums import (
        StudentStatus, AccountProvider, AuthType, AccountStatus,
        SyncStatus, SyncJobType, SyncJobStatus, HandoffStatus,
        EntityType, SecretKind
    )
    assert StudentStatus.active.value == &quot;active&quot;
    assert AccountProvider.github.value == &quot;github&quot;
    assert SyncStatus.diverged.value == &quot;diverged&quot;
    assert HandoffStatus.rolled_back.value == &quot;rolled_back&quot;
    assert EntityType.handoff.value == &quot;handoff&quot;
    assert SecretKind.pat.value == &quot;pat&quot;
```

Acceptance Criteria

1. `from app.db.models import Student, Account, Repository, Credential, SyncJob, HandoffEvent, TemplateSource, ActivityLog, AppSetting` — all 9 imports succeed
2. `from app.db.enums import StudentStatus, AccountProvider, AuthType, AccountStatus, SyncStatus, SyncJobType, SyncJobStatus, HandoffStatus, EntityType, SecretKind` — all 10 imports succeed
3. `alembic upgrade head` on a fresh DB creates all tables without error
4. All 5 tests pass: `pytest tests/test_models.py -v` shows 5 passed
5. No SQLAlchemy warnings about unmapped relationships or missing back_populates
6. `Credential` model has `secret_ref` but no `value`, `token`, `password`, or `secret` field
7. `ActivityLog` model has `created_at` but no `updated_at`

---

codex-tasks/T05-repository-service.md

Розмір: 13,254 байт

T05: Implement RepositoryService

Goal

Implement `RepositoryService` in `app/services/repository_service.py`. This service handles repository bootstrap from template or canonical, remote URL and local path validation, and sync status queries. All Git operations are delegated to `GitClient` (injected as a dependency).

Context

- Project root: `/home/vokov/projects/studentflow/`
- Architecture: `docs/architecture.md` — RepositoryService section
- Sync strategy: `docs/sync-strategy.md` — "First Repo Bootstrap" section
- Models: `Repository`, `Account`, `Student`, `TemplateSource` in `app/db/models.py` (T02)
- GitClient stub: `app/git/client.py` (T01 stub, real implementation in T06)

Constraints

- RepositoryService must NOT call `subprocess` or `git` directly — all Git ops via `GitClient`
- RepositoryService must NOT call `keyring` — credential retrieval is `CredentialService`'s job
- `bootstrap_from_template()` and `bootstrap_from_canonical()` must create a `Repository` DB record only AFTER all git operations succeed
- `validate_local_path()` must not raise — returns True/False
- `validate_remote()` delegates to `GitClient.validate_remote()` which accepts a credential; RepositoryService retrieves the credential via `CredentialService`

Files to Create/Update

app/services/repository_service.py — full implementation
tests/test_repository_service.py — unit tests with mocked GitClient

Implementation Requirements

RepositoryService class

```
class RepositoryService:
    &quot;&quot;&quot;Manages repository bootstrap, validation, and status queries.

    GitClient is injected to allow mocking in tests. All Git operations
    delegate to GitClient — this service handles only DB state and orchestration.
    &quot;&quot;&quot;

    def __init__(
        self,
        git_client=None,       # GitClient instance (injected)
        credential_service=None,  # CredentialService instance (injected)
        template_service=None,    # TemplateService instance (injected)
    ):
        from app.git.client import GitClient
        from app.services.credential_service import CredentialService
        from app.services.template_service import TemplateService
        self._git = git_client or GitClient()
        self._creds = credential_service or CredentialService()
        self._templates = template_service or TemplateService()
    def bootstrap_from_template(
```

```
    self,
    student_id: str,
    account_id: str,
    template_source_id: str,
    local_path: str,
    remote_url: str,
) -&gt; Repository:
    &quot;&quot;&quot;Initialize a new student repository from a template source.

    Steps:
    1. Load and validate template source
    2. Validate local_path (must NOT already exist as a git repo)
    3. GitClient.init_from_dir(template_source.local_path, local_path)
    4. GitClient.add_remote(local_path, &apos;student&apos;, remote_url)
    5. Retrieve credential, GitClient.push(local_path, &apos;student&apos;, branch, credential)
    6. Get canonical path from AppSetting, GitClient.add_remote(local_path, &apos;origin&apos;, canonical_path)
    7. GitClient.push(local_path, &apos;origin&apos;, branch, canonical_credential=None)  # local push to bare re
    8. Get HEAD commit hash
    9. Create Repository DB record with sync_status=clean
    10. Return Repository record

    Args:
        student_id: UUID of Student
        account_id: UUID of Account (must belong to student_id)
        template_source_id: UUID of TemplateSource to use
        local_path: Absolute path where local clone will be created (must not exist)
        remote_url: Student&apos;s Git remote URL (GitHub/GitLab/etc.)

    Raises:
        ValueError: if student, account, or template_source not found
        ValueError: if local_path already exists
        RuntimeError: if any git operation fails
    &quot;&quot;&quot;

def bootstrap_from_canonical(
    self,
    student_id: str,
    account_id: str,
    local_path: str,
    remote_url: str,
) -&gt; Repository:
    &quot;&quot;&quot;Initialize a new student repository by cloning from canonical.

    Steps:
    1. Load account, verify credential exists
    2. Get canonical path from AppSetting
    3. Validate canonical is a git repo
    4. GitClient.clone(canonical_path, local_path, credential=None)  # local clone
    5. GitClient.add_remote(local_path, &apos;student&apos;, remote_url)
    6. Retrieve credential, GitClient.push(local_path, &apos;student&apos;, branch, credential)
    7. Get HEAD commit hash
    8. Create Repository DB record with sync_status=clean
    9. Return Repository record

    Raises:
        ValueError: if student or account not found
        ValueError: if no canonical repo at canonical_repo_path setting
        RuntimeError: if any git operation fails
    &quot;&quot;&quot;

def validate_local_path(self, path: str) -&gt; bool:
    &quot;&quot;&quot;Check if a path is a valid, accessible git repository.

    Returns:
        True if path exists, is a directory, and contains a .git dir (or is bare)
        False otherwise (missing path, not a directory, not a git repo)
    Note: Does NOT raise on invalid paths — returns False.
```

```
        &quot;&quot;&quot;
        from pathlib import Path
        p = Path(path)
        if not p.exists() or not p.is_dir():
            return False
        return (p / &quot;.git&quot;).exists() or (p / &quot;HEAD&quot;).exists()

    def validate_remote(self, repo_id: str) -&gt; bool:
        &quot;&quot;&quot;Test connectivity to the student&apos;s remote repository.

        Retrieves the credential and delegates to GitClient.validate_remote().

        Returns:
            True if remote is reachable with current credentials
            False if unreachable (auth error, network error, URL invalid)
        &quot;&quot;&quot;

    def get_repo_status(self, repo_id: str) -&gt; SyncStatus:
        &quot;&quot;&quot;Return the current sync_status of a repository.

        Note: This returns the CACHED status from DB. To get live status,
        call SyncService.detect_divergence() which runs a git fetch.

        Returns:
            SyncStatus enum value from Repository.sync_status
        &quot;&quot;&quot;

    def get_canonical_repo(self) -&gt; Repository | None:
        &quot;&quot;&quot;Return the canonical Repository record (is_canonical=True).&quot;&quot;&quot;

    def list_repos_for_student(self, student_id: str) -&gt; list[Repository]:
        &quot;&quot;&quot;Return all Repository records for a given student (via their accounts).&quot;&quot;&quot;

    def update_sync_status(self, repo_id: str, status: SyncStatus, commit_hash: str | None = None) -&gt; None:
        &quot;&quot;&quot;Update Repository.sync_status and optionally last_commit_hash.&quot;&quot;&quot;


### Helper: get_canonical_path()

def _get_canonical_path(self, session) -&gt; str:
    &quot;&quot;&quot;Read canonical_repo_path from AppSetting.&quot;&quot;&quot;
    setting = session.get(AppSetting, &quot;canonical_repo_path&quot;)
    if setting is None:
        from pathlib import Path
        return str(Path.home() / &quot;.studentflow&quot; / &quot;canonical&quot;)
    return setting.value


### bootstrap_from_canonical complete steps:

def bootstrap_from_canonical(self, student_id, account_id, local_path, remote_url):
    with SessionLocal() as session:
        account = session.get(Account, account_id)
        if account is None or account.student_id != student_id:
            raise ValueError(f&quot;Account {account_id} not found or not owned by student {student_id}&quot;)

        credential = session.query(Credential).filter_by(account_id=account_id).first()
        if credential is None:
            raise ValueError(f&quot;No credential configured for account {account_id}&quot;)

        canonical_path = self._get_canonical_path(session)
        if not self.validate_local_path(canonical_path):
            raise ValueError(f&quot;Canonical repo not found at {canonical_path}&quot;)

        # Get the default branch
        branch_setting = session.get(AppSetting, &quot;default_branch&quot;)
        branch = branch_setting.value if branch_setting else &quot;main&quot;
    # Git operations (outside DB session to avoid long-held connections)
```

```
    self._git.clone(canonical_path, local_path, credential=None)
    self._git.add_remote(local_path, &quot;student&quot;, remote_url)

    cred_value = self._creds.retrieve(credential.secret_ref)
    self._git.push(local_path, &quot;student&quot;, branch, cred_value)

    commit_hash = self._git.get_commit_hash(local_path)

    with SessionLocal() as session:
        repo = Repository(
            account_id=account_id,
            repo_name=f&quot;{account.username}-project&quot;,
            remote_url=remote_url,
            local_path=local_path,
            default_branch=branch,
            integration_branch=branch,
            sync_status=SyncStatus.clean.value,
            last_commit_hash=commit_hash,
            template_source_id=None,
            is_canonical=False,
            created_at=datetime.utcnow(),
            updated_at=datetime.utcnow(),
        )
        session.add(repo)
        session.commit()
        return repo
```

Tests (tests/test_repository_service.py)

Use `unittest.mock.MagicMock` to mock GitClient. All Git operations on the mock return sensible defaults.

test_validate_local_path_with_git_dir:

```
def test_validate_local_path_with_git_dir(tmp_path):
    (tmp_path / &quot;.git&quot;).mkdir()
    service = RepositoryService()
    assert service.validate_local_path(str(tmp_path)) is True
```

test_validate_local_path_missing_dir:

```
def test_validate_local_path_missing_dir(tmp_path):
    service = RepositoryService()
    assert service.validate_local_path(str(tmp_path / &quot;nonexistent&quot;)) is False
```

test_validate_local_path_not_git_repo:

```
def test_validate_local_path_not_git_repo(tmp_path):
    (tmp_path / &quot;some_file.txt&quot;).write_text(&quot;not a repo&quot;)
    service = RepositoryService()
    assert service.validate_local_path(str(tmp_path)) is False
```

test_bootstrap_from_canonical_creates_repo_record:

```
def test_bootstrap_from_canonical_creates_repo_record(tmp_path, db_session):
    from unittest.mock import MagicMock, patch

    # Setup: create Student + Account + Credential in test DB
    # (use factories or direct model creation)
    student = Student(full_name=&quot;Test Student&quot;)
    db_session.add(student)
    db_session.flush()
    account = Account(student_id=student.id, provider=&quot;github&quot;, username=&quot;testuser&quot;, aut
    db_session.add(account)
    db_session.flush()
    # Credential mock: secret_ref points to keyring
```

```
    credential = Credential(account_id=account.id, secret_ref=&quot;studentflow:github:testuser:xxxxxxxx&quot;,
    db_session.add(credential)
    db_session.commit()

    # Mock GitClient
    mock_git = MagicMock()
    mock_git.get_commit_hash.return_value = &quot;a&quot; * 40
    mock_git.clone.return_value = None
    mock_git.add_remote.return_value = None
    mock_git.push.return_value = None

    # Mock CredentialService
    mock_creds = MagicMock()
    mock_creds.retrieve.return_value = &quot;fake_token&quot;

    # Patch AppSetting lookup
    with patch(&quot;app.services.repository_service.SessionLocal&quot;) as mock_session_factory:
        # ... setup mock session to return AppSetting(&quot;canonical_repo_path&quot;, str(tmp_path))
        # and the Account/Credential records
        pass

    service = RepositoryService(git_client=mock_git, credential_service=mock_creds)
    # Note: full integration test requires real DB; this tests GitClient calls
    mock_git.clone.assert_called_once()
    mock_git.push.assert_called_once()
```

test_bootstrap_from_canonical_raises_if_no_credential:

```
def test_bootstrap_from_canonical_raises_if_no_credential(db_session):
    # Create Student + Account with NO Credential
    student = Student(full_name=&quot;No Cred Student&quot;)
    db_session.add(student)
    db_session.flush()
    account = Account(
        student_id=student.id, provider=&quot;github&quot;,
        username=&quot;nocred&quot;, auth_type=&quot;pat&quot;
    )
    db_session.add(account)
    db_session.commit()

    service = RepositoryService()
    with pytest.raises(ValueError, match=&quot;No credential&quot;):
        service.bootstrap_from_canonical(student.id, account.id, &quot;/tmp/test&quot;, &quot;https://github.com/tes
```

test_validate_remote_returns_false_on_git_error:

```
def test_validate_remote_returns_false_on_git_error(db_session):
    from unittest.mock import MagicMock
    mock_git = MagicMock()
    mock_git.validate_remote.side_effect = Exception(&quot;Connection refused&quot;)

    service = RepositoryService(git_client=mock_git)
    # Should catch the exception and return False
    result = service.validate_remote(&quot;nonexistent-repo-id&quot;)
    assert result is False
```

Acceptance Criteria

1. `from app.services.repository_service import RepositoryService` — imports without error
2. All 5 tests pass: `pytest tests/test_repository_service.py -v` shows 5 passed
3. No `subprocess` or `git` commands called directly in this file
4. No `keyring` import in this file
5. `bootstrap_from_canonical()` raises `ValueError` (not generic Exception) for missing account or credential

6. `validate_local_path()` returns `bool` and never raises
7. Repository DB record is created only after all git operations succeed (tested by verifying mock_git methods called before DB insert)
---
codex-tasks/T12-validation-error-handling.md
Розмір: 9,986 байт

T12: Implement Validation and Error Handling

Goal

Add input validation, service-level guards, and a consistent error handling policy across all services. This task hardens the system without changing any happy-path logic. It adds: `GitValidator` implementation, Pydantic schemas for input DTOs, service-layer pre-condition checks, and a central `errors.py` exception hierarchy.

Context

- Project root: `/home/vokov/projects/studentflow/`
- Read `docs/security.md` — validation is part of the security model
- All services from T03–T09 are complete
- `app/git/validators.py` is currently a stub (`class GitValidator: pass`)
- `app/schemas/` files are stubs
- This is a hardening pass — do NOT change service method signatures or logic flow

Constraints

- All custom exceptions must inherit from `StudentFlowError` (base class defined in `app/errors.py`)
- Pydantic models are for input validation only — do NOT use them as ORM models
- `GitValidator` must NOT perform any git operations — filesystem and regex checks only
- Service pre-condition checks must raise specific exceptions, not generic `ValueError`
- Error messages must never contain credential values — validate this in tests
- Do NOT add validation for internal function-to-function calls — only at public service method boundaries

Files to Create/Update

```
app/errors.py                    — exception hierarchy (new file)
app/git/validators.py            — GitValidator full implementation
app/schemas/student.py           — StudentCreate, StudentUpdate Pydantic models
app/schemas/account.py           — AccountCreate, AccountUpdate Pydantic models
app/schemas/repository.py        — RepositoryCreate Pydantic model
tests/test_validators.py         — GitValidator tests
tests/test_schemas.py            — Pydantic schema validation tests
```

Implementation Requirements

app/errors.py — exception hierarchy

"""Central exception hierarchy for StudentFlow."""
class StudentFlowError(Exception):

```
    &quot;&quot;&quot;Base class for all StudentFlow exceptions.&quot;&quot;&quot;


# Account / Student
class StudentNotFoundError(StudentFlowError): ...
class AccountNotFoundError(StudentFlowError): ...
class DuplicateStudentError(StudentFlowError): ...
class StudentInWrongStateError(StudentFlowError): ...


# Repository
class RepositoryNotFoundError(StudentFlowError): ...
class LocalPathNotFoundError(StudentFlowError): ...
class LocalPathNotDirectoryError(StudentFlowError): ...
class RemoteURLInvalidError(StudentFlowError): ...
class RepositoryAlreadyExistsError(StudentFlowError): ...


# Credential
class CredentialNotFoundError(StudentFlowError): ...
class CredentialValidationError(StudentFlowError): ...
class SecretStorageError(StudentFlowError): ...


# Git / Sync
class GitOperationError(StudentFlowError): ...
class SyncConflictError(StudentFlowError): ...
class SyncTimeoutError(StudentFlowError): ...
class DivergedRepositoryError(StudentFlowError): ...


# Handoff
class HandoffError(StudentFlowError): ...
class NoActiveStudentError(HandoffError): ...
class NoNextStudentError(HandoffError): ...


# Template
class TemplateNotFoundError(StudentFlowError): ...
class TemplateCopyError(StudentFlowError): ...


# Config / Settings
class ConfigError(StudentFlowError): ...
```

Update T08 `handoff_service.py`: import `HandoffError`, `NoActiveStudentError`, `NoNextStudentError` from `app.errors` instead of defining locally.

app/git/validators.py — full implementation

```
&quot;&quot;&quot;GitValidator: filesystem and URL validation. No git operations.&quot;&quot;&quot;
import re
from pathlib import Path
from app.errors import (
    LocalPathNotFoundError,
    LocalPathNotDirectoryError,
    RemoteURLInvalidError,
)

# Patterns for supported remote URL formats
_SSH_PATTERN = re.compile(r&apos;^git@[\w\.\-]+:[\w\.\-/]+\.git$&apos;)
_HTTPS_PATTERN = re.compile(r&apos;^https?://[\w\.\-]+(:\d+)?/[\w\.\-/]+\.git$&apos;)
_HTTPS_NO_SUFFIX = re.compile(r&apos;^https?://[\w\.\-]+(:\d+)?/[\w\.\-/]+$&apos;)


class GitValidator:
    @staticmethod
```

```
    def validate_local_path(path: str) -&gt; Path:
        &quot;&quot;&quot;
        Check that path exists and is a directory.
        Raises LocalPathNotFoundError or LocalPathNotDirectoryError.
        Returns resolved Path.
        &quot;&quot;&quot;

    @staticmethod
    def validate_remote_url(url: str) -&gt; str:
        &quot;&quot;&quot;
        Check that url matches SSH or HTTPS git remote patterns.
        Raises RemoteURLInvalidError.
        Returns url unchanged.
        &quot;&quot;&quot;

    @staticmethod
    def is_git_repo(path: str | Path) -&gt; bool:
        &quot;&quot;&quot;Return True if path contains a .git directory or is a bare repo (has HEAD file).&quot;&quot

    @staticmethod
    def validate_branch_name(branch: str) -&gt; str:
        &quot;&quot;&quot;
        Check branch name is a valid git ref name.
        Raises ValueError if invalid.
        Rules: non-empty, no spaces, no .., no leading/trailing slash, no @{
        &quot;&quot;&quot;


### app/schemas/student.py

&quot;&quot;&quot;Pydantic input schemas for Student operations.&quot;&quot;&quot;
from pydantic import BaseModel, Field, field_validator
from typing import Optional
from app.db.enums import StudentStatus


class StudentCreate(BaseModel):
    full_name: str = Field(min_length=1, max_length=200)
    status: StudentStatus = StudentStatus.active
    queue_position: int = Field(ge=1, le=999, default=999)
    notes: Optional[str] = Field(default=None, max_length=1000)

    @field_validator(&apos;full_name&apos;)
    @classmethod
    def strip_whitespace(cls, v: str) -&gt; str:
        return v.strip()


class StudentUpdate(BaseModel):
    full_name: Optional[str] = Field(default=None, min_length=1, max_length=200)
    status: Optional[StudentStatus] = None
    queue_position: Optional[int] = Field(default=None, ge=1, le=999)
    notes: Optional[str] = Field(default=None, max_length=1000)


### app/schemas/account.py

&quot;&quot;&quot;Pydantic input schemas for Account operations.&quot;&quot;&quot;
from pydantic import BaseModel, Field, field_validator
from typing import Optional
from app.db.enums import AuthType, AccountStatus


class AccountCreate(BaseModel):
    student_id: str = Field(min_length=1)
    provider: str = Field(pattern=r&apos;^(github|gitlab|bitbucket|other)$&apos;)
    username: str = Field(min_length=1, max_length=100, pattern=r&apos;^[\w\.\-]+$&apos;)
    auth_type: AuthType
```

```
    status: AccountStatus = AccountStatus.active


class AccountUpdate(BaseModel):
    username: Optional[str] = Field(default=None, min_length=1, max_length=100)
    auth_type: Optional[AuthType] = None
    status: Optional[AccountStatus] = None


### app/schemas/repository.py

&quot;&quot;&quot;Pydantic input schemas for Repository operations.&quot;&quot;&quot;
from pydantic import BaseModel, Field, field_validator
from typing import Optional


class RepositoryCreate(BaseModel):
    account_id: str = Field(min_length=1)
    repo_name: str = Field(min_length=1, max_length=200, pattern=r&apos;^[\w\.\-]+$&apos;)
    remote_url: str = Field(min_length=1)
    local_path: str = Field(min_length=1)
    default_branch: str = Field(default=&quot;main&quot;, min_length=1, max_length=100)
    template_source_id: Optional[str] = None

    @field_validator(&apos;remote_url&apos;)
    @classmethod
    def validate_remote(cls, v: str) -&gt; str:
        from app.git.validators import GitValidator
        return GitValidator.validate_remote_url(v)

    @field_validator(&apos;local_path&apos;)
    @classmethod
    def validate_path_format(cls, v: str) -&gt; str:
        # Only check format (not existence) at schema level
        from pathlib import Path
        if not Path(v).is_absolute():
            raise ValueError(&quot;local_path must be an absolute path&quot;)
        return v
```

Tests

tests/test_validators.py

Test cases:
1. `test_validate_local_path_exists` — valid dir path returns Path object
2. `test_validate_local_path_not_found` — raises `LocalPathNotFoundError`
3. `test_validate_local_path_is_file` — raises `LocalPathNotDirectoryError`
4. `test_validate_remote_url_ssh` — `git@github.com:user/repo.git` passes
5. `test_validate_remote_url_https` — `https://github.com/user/repo.git` passes
6. `test_validate_remote_url_invalid` — `ftp://bad` raises `RemoteURLInvalidError`
7. `test_validate_branch_name_valid` — `main`, `feature/foo` pass
8. `test_validate_branch_name_spaces` — raises `ValueError`
9. `test_is_git_repo_true` — temp dir with `.git/` subdir returns True
10. `test_is_git_repo_false` — empty temp dir returns False

tests/test_schemas.py

Test cases:
1. `test_student_create_valid` — valid data creates `StudentCreate` without error
2. `test_student_create_empty_name` — raises `ValidationError`
3. `test_student_create_strips_whitespace` — `" Alice "` becomes `"Alice"`
4. `test_account_create_invalid_provider` — `provider="twitter"` raises `ValidationError`
5. `test_account_create_invalid_username_chars` — username with spaces raises `ValidationError`
6. `test_repo_create_relative_path` — raises `ValidationError`

Acceptance Criteria

1. `from app.errors import StudentFlowError, GitOperationError, HandoffError` works without error
2. `GitValidator.validate_local_path("/nonexistent")` raises `LocalPathNotFoundError`
3. `GitValidator.validate_remote_url("git@github.com:user/repo.git")` returns the URL unchanged
4. `StudentCreate(full_name="")` raises `pydantic.ValidationError`
5. All 10 + 6 = 16 tests pass: `pytest tests/test_validators.py tests/test_schemas.py -v`
6. Importing `from app.services.handoff_service import HandoffError` still works (re-exported from `app.errors`)

Notes for Codex

- Update `app/services/handoff_service.py` to import exceptions from `app.errors` instead of defining them locally — this is the ONLY change to that service file in this task
- `GitValidator` uses only `pathlib.Path` and `re` — no `subprocess`, no `git` binary
- Pydantic v2 uses `field_validator` not `validator` — check import paths carefully
- Do NOT add validators to `app/db/models.py` — those are SQLAlchemy models, not input DTOs
---
codex-tasks/T08-handoff-service.md
Розмір: 6,239 байт

T08: Implement HandoffService

Goal

Implement `HandoffService` in `app/services/handoff_service.py`. The handoff is the atomic process of: (1) finalising the current active student's work by committing a checkpoint tag on canonical, (2) deactivating the current student, (3) activating the next student in queue, (4) pushing the canonical state into the next student's repo remote, (5) recording a `HandoffEvent` in the DB. If any step fails the service must roll back all state changes made so far and set the active student to `error` status.

Context

- Project root: `/home/vokov/projects/studentflow/`
- Read `docs/handoff-protocol.md` in full before writing any code.
- Read `docs/sync-strategy.md` §"Handoff" section.
- Models: `HandoffEvent`, `Student`, `Repository`, `SyncJob`, `AppSetting` in `app/db/models.py` (T02)
- Enums: `StudentStatus`, `HandoffStatus`, `SyncJobType` in `app/db/enums.py` (T02)
- SyncService: `app/services/sync_service.py` (T07) — inject, call `sync_student()` before handoff
- GitClient: `app/git/client.py` (T06) — inject for tag/push operations
- AuditService: `app/services/audit_service.py` (T09) — inject for audit logging

Constraints

- Handoff MUST be atomic from the operator's perspective: either it fully succeeds or it fully rolls back

- Roll back means: student statuses reverted, HandoffEvent set to `failed`, no orphaned tags pushed to remotes
- The next student MUST be determined by lowest `queue_position` among students with status `active` or `paused` (excluding the current one and `archived`/`error` students)
- If no next student exists, handoff raises `NoNextStudentError` (define this exception in `handoff_service.py`)
- The checkpoint git tag format is: `checkpoint/{student_username}/{YYYY-MM-DD}/{short_commit_hash}` — never include secrets in the tag name
- `HandoffEvent.commit_hash` is the canonical HEAD commit hash AFTER the final sync, not before
- SyncService must be called (to flush current student's work) before any state transition happens
- Never call `keyring` directly — go through `CredentialService` via injection
- All DB writes must happen in a single session passed from caller or managed internally via context manager

Files to Create/Update

```
app/services/handoff_service.py    — full implementation
tests/test_handoff_service.py      — unit tests
```

Implementation Requirements

```
### HandoffService class

class HandoffService:
    def __init__(
        self,
        session,                    # SQLAlchemy Session
        git_client: GitClient,
        sync_service: SyncService,
        credential_service: CredentialService,
        audit_service: AuditService,
    ): ...

    def get_active_student(self) -&gt; Student | None:
        &quot;&quot;&quot;Return the currently active student (status=active), or None.&quot;&quot;&quot;

    def get_next_student(self) -&gt; Student | None:
        &quot;&quot;&quot;Return student with lowest queue_position that is eligible to receive handoff.&quot;&quot

    def handoff_to_next(self, operator: str = &quot;operator&quot;) -&gt; HandoffEvent:
        &quot;&quot;&quot;
        Full handoff pipeline. Steps:
        1. get_active_student() — raise NoActiveStudentError if None
        2. get_next_student()   — raise NoNextStudentError if None
        3. sync_service.sync_student(current) — flush current student&apos;s work
        4. tag checkpoint on canonical repo
        5. push canonical state into next student&apos;s remote
        6. DB: set current student status → exhausted
        7. DB: set next student status → active
        8. DB: create HandoffEvent(status=completed, commit_hash=...)
        9. audit_service.log(...)
        10. return HandoffEvent

        On any exception after step 3:
        - revert student statuses
        - set HandoffEvent.status = failed (if created)
        - audit_service.log(action=&apos;handoff_failed&apos;, ...)
        - re-raise as HandoffError wrapping original exception
```

"""

def resume_handoff(self, handoff_event_id: str) -> HandoffEvent:
"""Retry a failed handoff from step 5 onward, given a failed HandoffEvent ID.""&

Custom exceptions (define in handoff_service.py)
class HandoffError(Exception): ...
class NoActiveStudentError(HandoffError): ...
class NoNextStudentError(HandoffError): ...

Checkpoint tag format
def _build_checkpoint_tag(username: str, commit_hash: str) -> str:
from datetime import date
return f"checkpoint/{username}/{date.today().isoformat()}/{commit_hash[:8]}"

Tests

File: `tests/test_handoff_service.py`

Test cases (all Git operations mocked via `unittest.mock.MagicMock`):

1. `test_handoff_success` — two students in queue, handoff completes, HandoffEvent status=completed, statuses updated
2. `test_handoff_no_active_student` — raises `NoActiveStudentError`
3. `test_handoff_no_next_student` — raises `NoNextStudentError`
4. `test_handoff_rollback_on_push_failure` — git push raises, student statuses reverted, HandoffEvent status=failed
5. `test_get_next_student_skips_archived` — archived/error students excluded from candidates
6. `test_checkpoint_tag_format` — tag matches `checkpoint/{username}/{date}/{hash8}` pattern

Acceptance Criteria

1. `HandoffService.handoff_to_next()` with two valid students in DB completes without error
2. After successful handoff: `from_student.status == StudentStatus.exhausted`, `to_student.status == StudentStatus.active`
3. `HandoffEvent` record exists with `status=completed` and non-empty `commit_hash`
4. If `GitClient.push()` raises, both student statuses remain unchanged (rollback verified)
5. `NoNextStudentError` raised when no eligible next student exists
6. All 6 tests pass: `pytest tests/test_handoff_service.py -v`

Notes for Codex

- `AuditService` (T09) may not be implemented yet — inject a mock or use a stub that accepts `.log()` calls
- Use `unittest.mock.patch` to mock GitClient methods — do NOT create actual git repos in tests
- The `resume_handoff` method can have a minimal implementation for MVP: just raise `NotImplementedError` with a clear message if the event is not in `failed` status
- `queue_position` is an integer — lower = higher priority; ties broken by `created_at` ascending
---
codex-tasks/T04-template-service.md
Розмір: 10,961 байт

T04: Implement TemplateService

Goal

Implement `TemplateService` in `app/services/template_service.py`. This service manages local directory-based project templates used to seed new student repositories. It computes SHA256 checksums for integrity verification and maintains `TemplateSource` DB records.

Context

- Project root: `/home/vokov/projects/studentflow/`
- Model: `TemplateSource` in `app/db/models.py` (from T02)
- Architecture: `docs/architecture.md` — TemplateService section
- Sync strategy: `docs/sync-strategy.md` — "Init from Template" section

Constraints

- `import_from_path()` must validate the path exists and is a directory before proceeding
- `compute_checksum()` must produce a deterministic SHA256: sort files by path, hash content of each, hash the combined result
- `validate_template()` recomputes checksum and compares against stored `TemplateSource.checksum` — it does NOT fail if checksum is None (treat as "not yet validated")
- `set_active()` deactivates ALL other templates first, then activates the specified one
- Templates are never deleted through this service — only deactivated via `is_active=False`
- No Git operations in this service — TemplateService only manages metadata and filesystem reads

Files to Create/Update

app/services/template_service.py — full implementation
tests/test_template_service.py — unit tests using tmp_path fixture

Implementation Requirements

TemplateService class

```
class TemplateService:
    &quot;&quot;&quot;Manages project template sources for bootstrapping student repositories.

    Templates are local directories containing project files. TemplateService
    registers them as TemplateSource DB records and computes checksums for
    integrity verification.
    &quot;&quot;&quot;

    def import_from_path(self, path: str, name: str, description: str = &quot;&quot;) -&gt; TemplateSource:
        &quot;&quot;&quot;Register a local directory as a template source.

        Args:
            path: Absolute path to the template directory
            name: Human-readable name, e.g., &quot;Project v2 — Spring 2024&quot;
            description: Optional description

        Returns:
            The created TemplateSource record

        Raises:
            ValueError: if path does not exist or is not a directory
```

```
        ValueError: if a TemplateSource with the same local_path already exists
    &quot;&quot;&quot;

    def validate_template(self, template_source: TemplateSource) -&gt; bool:
    &quot;&quot;&quot;Check that the template directory still exists and matches its checksum.

    If TemplateSource.checksum is None, only checks that the path exists.
    If checksum is set, recomputes and compares.

    Returns:
        True if valid (path exists and checksum matches or was None)
        False if path missing or checksum mismatch
    &quot;&quot;&quot;

    def list_templates(self, active_only: bool = False) -&gt; list[TemplateSource]:
    &quot;&quot;&quot;Return all TemplateSource records.

    Args:
        active_only: If True, return only records with is_active=True

    Returns:
        List of TemplateSource records ordered by created_at DESC
    &quot;&quot;&quot;

    def set_active(self, template_id: str) -&gt; TemplateSource:
    &quot;&quot;&quot;Set one template as active, deactivating all others.

    Args:
        template_id: UUID of the TemplateSource to activate

    Returns:
        The activated TemplateSource record

    Raises:
        ValueError: if no TemplateSource with template_id exists
    &quot;&quot;&quot;

    def compute_checksum(self, path: str) -&gt; str:
    &quot;&quot;&quot;Compute a deterministic SHA256 checksum of a directory&apos;s contents.

    Algorithm:
    1. Walk directory recursively, collect all file paths
    2. Sort paths lexicographically (deterministic)
    3. For each file: hash its content with SHA256
    4. Combine all file hashes: SHA256(&quot;{relative_path}:{file_sha256}\n&quot; for each)
    5. Return the final combined SHA256 hex digest

    Args:
        path: Absolute path to the directory

    Returns:
        64-character hex SHA256 digest

    Raises:
        ValueError: if path is not a directory
    &quot;&quot;&quot;

    def get_active_template(self) -&gt; TemplateSource | None:
    &quot;&quot;&quot;Return the currently active template, or None if none is set.&quot;&quot;&quot;


### compute_checksum implementation detail

import hashlib
from pathlib import Path

def compute_checksum(self, path: str) -&gt; str:
    root = Path(path)
```

```
    if not root.is_dir():
        raise ValueError(f&quot;Not a directory: {path}&quot;)

    file_hashes = []
    for file_path in sorted(root.rglob(&quot;*&quot;)):
        if file_path.is_file():
            rel_path = str(file_path.relative_to(root))
            content_hash = hashlib.sha256(file_path.read_bytes()).hexdigest()
            file_hashes.append(f&quot;{rel_path}:{content_hash}&quot;)

    combined = &quot;\n&quot;.join(file_hashes)
    return hashlib.sha256(combined.encode(&quot;utf-8&quot;)).hexdigest()
```

import_from_path complete implementation

```
def import_from_path(self, path: str, name: str, description: str = &quot;&quot;) -&gt; TemplateSource:
    from pathlib import Path
    p = Path(path).resolve()

    if not p.exists():
        raise ValueError(f&quot;Path does not exist: {path}&quot;)
    if not p.is_dir():
        raise ValueError(f&quot;Path is not a directory: {path}&quot;)

    with SessionLocal() as session:
        existing = session.query(TemplateSource).filter_by(local_path=str(p)).first()
        if existing is not None:
            raise ValueError(f&quot;Template already registered for path: {path}&quot;)

        checksum = self.compute_checksum(str(p))

        template = TemplateSource(
            name=name,
            local_path=str(p),
            is_active=False,  # Caller must explicitly activate
            checksum=checksum,
            description=description or None,
            created_at=datetime.utcnow(),
            updated_at=datetime.utcnow(),
        )
        session.add(template)
        session.commit()
        return template
```

Tests (tests/test_template_service.py)

Use `pytest`'s `tmp_path` fixture to create test template directories.

test_import_from_path_creates_record:

```
def test_import_from_path_creates_record(tmp_path, db_session):
    # Create a simple template directory
    (tmp_path / &quot;README.md&quot;).write_text(&quot;# Project\n&quot;)
    (tmp_path / &quot;src&quot;).mkdir()
    (tmp_path / &quot;src&quot; / &quot;main.py&quot;).write_text(&quot;print(&apos;hello&apos;)\n&quot;)

    service = TemplateService()
    template = service.import_from_path(str(tmp_path), &quot;Test Template&quot;)

    assert template.id is not None
    assert template.name == &quot;Test Template&quot;
    assert template.local_path == str(tmp_path)
    assert template.checksum is not None
    assert len(template.checksum) == 64  # SHA256 hex digest
```

```
    assert template.is_active is False
```

test_import_from_path_nonexistent_raises:

```
def test_import_from_path_nonexistent_raises(tmp_path):
    service = TemplateService()
    with pytest.raises(ValueError, match=&quot;does not exist&quot;):
        service.import_from_path(str(tmp_path / &quot;nonexistent&quot;), &quot;Bad Template&quot;)
```

test_compute_checksum_deterministic:

```
def test_compute_checksum_deterministic(tmp_path):
    (tmp_path / &quot;a.txt&quot;).write_text(&quot;content a&quot;)
    (tmp_path / &quot;b.txt&quot;).write_text(&quot;content b&quot;)
    service = TemplateService()
    checksum1 = service.compute_checksum(str(tmp_path))
    checksum2 = service.compute_checksum(str(tmp_path))
    assert checksum1 == checksum2
    assert len(checksum1) == 64
```

test_compute_checksum_changes_with_content:

```
def test_compute_checksum_changes_with_content(tmp_path):
    (tmp_path / &quot;file.txt&quot;).write_text(&quot;original&quot;)
    service = TemplateService()
    checksum1 = service.compute_checksum(str(tmp_path))
    (tmp_path / &quot;file.txt&quot;).write_text(&quot;modified&quot;)
    checksum2 = service.compute_checksum(str(tmp_path))
    assert checksum1 != checksum2
```

test_validate_template_ok:

```
def test_validate_template_ok(tmp_path):
    (tmp_path / &quot;file.txt&quot;).write_text(&quot;hello&quot;)
    service = TemplateService()
    template = service.import_from_path(str(tmp_path), &quot;Valid Template&quot;)
    assert service.validate_template(template) is True
```

test_validate_template_fails_if_path_missing:

```
def test_validate_template_fails_if_path_missing(tmp_path):
    import shutil
    (tmp_path / &quot;file.txt&quot;).write_text(&quot;hello&quot;)
    service = TemplateService()
    template = service.import_from_path(str(tmp_path), &quot;Template&quot;)
    shutil.rmtree(str(tmp_path))  # Delete the directory
    assert service.validate_template(template) is False
```

test_validate_template_fails_on_checksum_mismatch:

```
def test_validate_template_fails_on_checksum_mismatch(tmp_path):
    (tmp_path / &quot;file.txt&quot;).write_text(&quot;hello&quot;)
    service = TemplateService()
    template = service.import_from_path(str(tmp_path), &quot;Template&quot;)
    (tmp_path / &quot;file.txt&quot;).write_text(&quot;modified after import&quot;)
    assert service.validate_template(template) is False
```

test_set_active_deactivates_others:

```
def test_set_active_deactivates_others(tmp_path):
    dir1 = tmp_path / &quot;t1&quot;
    dir1.mkdir()
    (dir1 / &quot;f.txt&quot;).write_text(&quot;a&quot;)
    dir2 = tmp_path / &quot;t2&quot;
    dir2.mkdir()
    (dir2 / &quot;f.txt&quot;).write_text(&quot;b&quot;)

    service = TemplateService()
    t1 = service.import_from_path(str(dir1), &quot;Template 1&quot;)
```

```
    t2 = service.import_from_path(str(dir2), &quot;Template 2&quot;)

    service.set_active(t1.id)
    service.set_active(t2.id)

    templates = service.list_templates()
    active = [t for t in templates if t.is_active]
    assert len(active) == 1
    assert active[0].id == t2.id
```

test_list_templates_active_only:

```
def test_list_templates_active_only(tmp_path):
    dir1 = tmp_path / &quot;t1&quot;
    dir1.mkdir()
    (dir1 / &quot;f.txt&quot;).write_text(&quot;a&quot;)
    dir2 = tmp_path / &quot;t2&quot;
    dir2.mkdir()
    (dir2 / &quot;f.txt&quot;).write_text(&quot;b&quot;)

    service = TemplateService()
    t1 = service.import_from_path(str(dir1), &quot;T1&quot;)
    t2 = service.import_from_path(str(dir2), &quot;T2&quot;)
    service.set_active(t1.id)

    all_templates = service.list_templates(active_only=False)
    active_only = service.list_templates(active_only=True)

    assert len(all_templates) == 2
    assert len(active_only) == 1
    assert active_only[0].id == t1.id
```

Acceptance Criteria

1. `from app.services.template_service import TemplateService` — imports without error
2. All 9 tests pass: `pytest tests/test_template_service.py -v` shows 9 passed
3. `compute_checksum()` is deterministic: calling it twice on the same unchanged directory returns the same value
4. `compute_checksum()` is sensitive: changing any file's content changes the checksum
5. `import_from_path()` raises `ValueError` for missing paths (not a generic Exception)
6. `set_active()` ensures exactly one template is active at a time
7. No Git operations appear in this file — TemplateService is filesystem-only

---

tests/test_validators.py

Розмір: 1,671 байт

```
"""Tests for GitValidator (T12)."""
import pytest

from app.errors import LocalPathNotDirectoryError, LocalPathNotFoundError,
RemoteURLInvalidError
from app.git.validators import GitValidator

def test_validate_local_path_exists(tmp_path):
result = GitValidator.validate_local_path(str(tmp_path))
assert result == tmp_path.resolve()

def test_validate_local_path_not_found(tmp_path):
with pytest.raises(LocalPathNotFoundError):
GitValidator.validate_local_path(str(tmp_path / "nonexistent"))

def test_validate_local_path_is_file(tmp_path):
f = tmp_path / "file.txt"
f.write_text("x")
```

```
with pytest.raises(LocalPathNotDirectoryError):
GitValidator.validate_local_path(str(f))

def test_validate_remote_url_ssh():
url = "git@github.com:user/repo.git"
assert GitValidator.validate_remote_url(url) == url

def test_validate_remote_url_https():
url = "https://github.com/user/repo.git"
assert GitValidator.validate_remote_url(url) == url

def test_validate_remote_url_invalid():
with pytest.raises(RemoteURLInvalidError):
GitValidator.validate_remote_url("ftp://bad")

def test_validate_branch_name_valid():
assert GitValidator.validate_branch_name("main") == "main"
assert GitValidator.validate_branch_name("feature/foo") == "feature/foo"

def test_validate_branch_name_spaces():
with pytest.raises(ValueError):
GitValidator.validate_branch_name("bad branch")

def test_is_git_repo_true(tmp_path):
(tmp_path / ".git").mkdir()
assert GitValidator.is_git_repo(tmp_path) is True

def test_is_git_repo_false(tmp_path):
assert GitValidator.is_git_repo(tmp_path) is False
---
### tests/test_repos_viewmodel.py
**Розмір:** 1,905 байт

import pytest
from sqlalchemy.orm import sessionmaker

from app.db.enums import SyncStatus
from app.db.models import Account, Repository, Student
from app.ui.viewmodels.repos_viewmodel import RepoViewModel

def _make_factory(db_session):
return sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)

def _add_repo(db_session, name: str, is_canonical: bool = False) -> Repository:
student = Student(full_name=name)
db_session.add(student)
db_session.flush()
account = Account(student_id=student.id, provider="github", username=name,
auth_type="pat")
db_session.add(account)
db_session.flush()
repo = Repository(
account_id=account.id,
repo_name=name,
remote_url=f"https://github.com/{name}/project.git",
local_path=f"/tmp/{name}",
default_branch="main",
sync_status=SyncStatus.clean.value,
is_canonical=is_canonical,
)
db_session.add(repo)
db_session.commit()
return repo

def test_set_canonical_marks_repo(db_session):
repo = _add_repo(db_session, "alice")
```

```
vm = RepoViewModel(session_factory=_make_factory(db_session))
vm.set_canonical(repo.id)
db_session.expire_all()
assert db_session.get(Repository, repo.id).is_canonical is True

def test_set_canonical_clears_previous(db_session):
old = _add_repo(db_session, "old", is_canonical=True)
new = _add_repo(db_session, "new")
vm = RepoViewModel(session_factory=_make_factory(db_session))
vm.set_canonical(new.id)
db_session.expire_all()
assert db_session.get(Repository, old.id).is_canonical is False
assert db_session.get(Repository, new.id).is_canonical is True

def test_set_canonical_unknown_id_raises(db_session):
vm = RepoViewModel(session_factory=_make_factory(db_session))
with pytest.raises(ValueError, match="not found"):
vm.set_canonical("nonexistent-id")
```
---
tests/test_repository_service.py
Розмір: 4,964 байт

```
from unittest.mock import MagicMock

import pytest
from sqlalchemy.orm import sessionmaker

from app.db.enums import SyncStatus
from app.db.models import Account, AppSetting, Credential, Repository, Student
from app.services.repository_service import RepositoryService

def test_validate_local_path_with_git_dir(tmp_path):
(tmp_path / ".git").mkdir()
service = RepositoryService()
assert service.validate_local_path(str(tmp_path)) is True

def test_validate_local_path_missing_dir(tmp_path):
service = RepositoryService()
assert service.validate_local_path(str(tmp_path / "nonexistent")) is False

def test_validate_local_path_not_git_repo(tmp_path):
(tmp_path / "some_file.txt").write_text("not a repo")
service = RepositoryService()
assert service.validate_local_path(str(tmp_path)) is False

def test_bootstrap_from_canonical_creates_repo_record(tmp_path, db_session):
test_session_factory = sessionmaker(bind=db_session.bind, autocommit=False,
autoflush=False)

student = Student(full_name="Test Student")
db_session.add(student)
db_session.flush()

account = Account(student_id=student.id, provider="github", username="testuser",
auth_type="pat")
db_session.add(account)
db_session.flush()

credential = Credential(
account_id=account.id,
secret_ref="studentflow:github:testuser:xxxxxxxx",
secret_kind="pat",
)
db_session.add(credential)
db_session.add(AppSetting(key="canonical_repo_path", value=str(tmp_path / "canonical")))
db_session.add(AppSetting(key="default_branch", value="main"))
db_session.commit()

canonical_repo = tmp_path / "canonical"
```

```
canonical_repo.mkdir()
(canonical_repo / "HEAD").write_text("ref: refs/heads/main\n")

mock_git = MagicMock()
mock_git.get_commit_hash.return_value = "a" * 40

mock_creds = MagicMock()
mock_creds.retrieve.return_value = "fake_token"

service = RepositoryService(
git_client=mock_git,
credential_service=mock_creds,
session_factory=test_session_factory,
)

repo = service.bootstrap_from_canonical(
student.id,
account.id,
str(tmp_path / "student-repo"),
"https://github.com/test/repo.git",
)

mock_git.clone.assert_called_once_with(str(canonical_repo), str(tmp_path / "student-repo"),
credential=None)
mock_git.add_remote.assert_called_once_with(
str(tmp_path / "student-repo"),
"student",
"https://github.com/test/repo.git",
)
mock_git.push.assert_called_once_with(str(tmp_path / "student-repo"), "student", "main",
"fake_token")
assert repo.sync_status == SyncStatus.clean.value

stored_repo = db_session.get(Repository, repo.id)
assert stored_repo is not None
assert stored_repo.last_commit_hash == "a" * 40
assert stored_repo.account_id == account.id


def test_bootstrap_from_canonical_raises_if_no_credential(db_session):
test_session_factory = sessionmaker(bind=db_session.bind, autocommit=False,
autoflush=False)

student = Student(full_name="No Cred Student")
db_session.add(student)
db_session.flush()

account = Account(student_id=student.id, provider="github", username="nocred",
auth_type="pat")
db_session.add(account)
db_session.commit()

service = RepositoryService(session_factory=test_session_factory)
with pytest.raises(ValueError, match="No credential"):
service.bootstrap_from_canonical(
student.id,
account.id,
"/tmp/test",
"https://github.com/test/repo.git",
)


def test_validate_remote_returns_false_on_git_error(db_session):
test_session_factory = sessionmaker(bind=db_session.bind, autocommit=False,
autoflush=False)

student = Student(full_name="Remote Test Student")
db_session.add(student)
db_session.flush()

account = Account(student_id=student.id, provider="github", username="remoteuser",
auth_type="pat")
db_session.add(account)
db_session.flush()
```

```
credential = Credential(
account_id=account.id,
secret_ref="studentflow:github:remoteuser:xxxxxxxx",
secret_kind="pat",
)
repo = Repository(
account_id=account.id,
repo_name="remoteuser-project",
remote_url="https://github.com/test/repo.git",
local_path="/tmp/remoteuser-project",
default_branch="main",
integration_branch="main",
sync_status=SyncStatus.clean.value,
)
db_session.add_all([credential, repo])
db_session.commit()

mock_git = MagicMock()
mock_git.validate_remote.side_effect = Exception("Connection refused")

mock_creds = MagicMock()
mock_creds.retrieve.return_value = "fake_token"

service = RepositoryService(
git_client=mock_git,
credential_service=mock_creds,
session_factory=test_session_factory,
)

assert service.validate_remote(repo.id) is False
```

---

tests/test_template_service.py

Розмір: 4,228 байт

```
from __future__ import annotations

import shutil

import pytest
from sqlalchemy.orm import sessionmaker

from app.services.template_service import TemplateService

@pytest.fixture
def template_service(db_session):
factory = sessionmaker(
bind=db_session.bind,
autocommit=False,
autoflush=False,
expire_on_commit=False,
)
return TemplateService(session_factory=factory)

def test_import_from_path_creates_record(tmp_path, db_session, template_service):
(tmp_path / "README.md").write_text("# Project\n")
(tmp_path / "src").mkdir()
(tmp_path / "src" / "main.py").write_text("print('hello')\n")

template = template_service.import_from_path(str(tmp_path), "Test Template")

assert template.id is not None
assert template.name == "Test Template"
assert template.local_path == str(tmp_path)
assert template.checksum is not None
assert len(template.checksum) == 64
assert template.is_active is False

def test_import_from_path_nonexistent_raises(tmp_path, template_service):
with pytest.raises(ValueError, match="does not exist"):
template_service.import_from_path(str(tmp_path / "nonexistent"), "Bad Template")
```

```
def test_compute_checksum_deterministic(tmp_path, template_service):
(tmp_path / "a.txt").write_text("content a")
(tmp_path / "b.txt").write_text("content b")

checksum1 = template_service.compute_checksum(str(tmp_path))
checksum2 = template_service.compute_checksum(str(tmp_path))

assert checksum1 == checksum2
assert len(checksum1) == 64

def test_compute_checksum_changes_with_content(tmp_path, template_service):
(tmp_path / "file.txt").write_text("original")

checksum1 = template_service.compute_checksum(str(tmp_path))
(tmp_path / "file.txt").write_text("modified")
checksum2 = template_service.compute_checksum(str(tmp_path))

assert checksum1 != checksum2

def test_validate_template_ok(tmp_path, template_service):
tpl = tmp_path / "tpl"
tpl.mkdir()
(tpl / "file.txt").write_text("hello")

template = template_service.import_from_path(str(tpl), "Valid Template")

assert template_service.validate_template(template) is True

def test_validate_template_fails_if_path_missing(tmp_path, template_service):
tpl = tmp_path / "tpl"
tpl.mkdir()
(tpl / "file.txt").write_text("hello")

template = template_service.import_from_path(str(tpl), "Template")
shutil.rmtree(str(tpl)) # delete only the template subdir, not tmp_path itself

assert template_service.validate_template(template) is False

def test_validate_template_fails_on_checksum_mismatch(tmp_path, template_service):
tpl = tmp_path / "tpl"
tpl.mkdir()
(tpl / "file.txt").write_text("hello")

template = template_service.import_from_path(str(tpl), "Template")
(tpl / "file.txt").write_text("modified after import")

assert template_service.validate_template(template) is False

def test_set_active_deactivates_others(tmp_path, template_service):
dir1 = tmp_path / "t1"
dir1.mkdir()
(dir1 / "f.txt").write_text("a")
dir2 = tmp_path / "t2"
dir2.mkdir()
(dir2 / "f.txt").write_text("b")

t1 = template_service.import_from_path(str(dir1), "Template 1")
t2 = template_service.import_from_path(str(dir2), "Template 2")

template_service.set_active(t1.id)
template_service.set_active(t2.id)

templates = template_service.list_templates()
active = [template for template in templates if template.is_active]
assert len(active) == 1
assert active[0].id == t2.id

def test_list_templates_active_only(tmp_path, template_service):
dir1 = tmp_path / "t1"
dir1.mkdir()
(dir1 / "f.txt").write_text("a")
```

```
dir2 = tmp_path / "t2"
dir2.mkdir()
(dir2 / "f.txt").write_text("b")

t1 = template_service.import_from_path(str(dir1), "T1")
template_service.import_from_path(str(dir2), "T2")
template_service.set_active(t1.id)

all_templates = template_service.list_templates(active_only=False)
active_only = template_service.list_templates(active_only=True)

assert len(all_templates) == 2
assert len(active_only) == 1
assert active_only[0].id == t1.id
```
---
tests/test_prompt_service.py
Розмір: 13,577 байт

```
"""Tests for PromptService."""
from __future__ import annotations

import re
import subprocess
from datetime import datetime
from unittest.mock import MagicMock

import pytest
from sqlalchemy.orm import sessionmaker

from app.db.enums import PromptStatus, StudentStatus, SyncStatus
from app.db.models import Account, Credential, Prompt, Repository, Student

def _factory(db_session):
return sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False,
expire_on_commit=False)

def _make_student(db_session, name="Alice", queue_pos=1, student_number=1):
s = Student(
full_name=name,
status=StudentStatus.active.value,
queue_position=queue_pos,
student_number=student_number,
)
db_session.add(s)
db_session.flush()
a = Account(
student_id=s.id, provider="github", username=name.lower(), auth_type="pat"
)
db_session.add(a)
db_session.flush()
r = Repository(
account_id=a.id,
repo_name=f"{name.lower()}-project",
remote_url=f"https://github.com/{name.lower()}/project.git",
local_path=f"/tmp/{name.lower()}",
default_branch="main",
sync_status=SyncStatus.clean.value,
is_canonical=False,
)
db_session.add(r)
cred = Credential(
account_id=a.id,
secret_ref=f"studentflow:github:{name.lower()}:abc123",
secret_kind="pat",
)
db_session.add(cred)
db_session.commit()
return s, a, r
```

── slug generation ──

```
def test_make_slug_basic():
from app.services.prompt_service import _make_slug
assert _make_slug("Auth Flow Fix") == "auth-flow-fix"

def test_make_slug_empty_returns_prompt():
from app.services.prompt_service import _make_slug
assert _make_slug("") == "prompt"
assert _make_slug(" ") == "prompt"

def test_make_slug_strips_special_chars():
from app.services.prompt_service import _make_slug
slug = _make_slug("Hello, World! (2026)")
assert re.match(r"^[a-z0-9-]+$", slug)

def test_make_slug_truncates_to_40_chars():
from app.services.prompt_service import _make_slug
assert len(_make_slug("a" * 100)) <= 40
```

── file path generation ──

```
def test_make_file_path_format():
from app.services.prompt_service import _make_file_path
assert _make_file_path(3, 1, "auth-fix") == "prompts/student03-001-auth-fix.md"

def test_make_file_path_zero_pads():
from app.services.prompt_service import _make_file_path
assert _make_file_path(7, 14, "test") == "prompts/student07-014-test.md"
assert _make_file_path(12, 100, "big") == "prompts/student12-100-big.md"
```

── file content generation ──

```
def test_make_file_content_contains_metadata():
from app.services.prompt_service import _make_file_content
content = _make_file_content(
title="Auth Flow Fix",
student_number=3,
seq_number=1,
created_by="operator",
repo_name="alice-project",
branch="main",
prompt_text="Fix the login.",
created_at=datetime(2026, 3, 17, 12, 0, 0),
)
assert "# Auth Flow Fix" in content
assert "#03" in content
assert "001" in content
assert "operator" in content
assert "alice-project" in content
assert "main" in content
```

```
assert "Fix the login." in content
```

── service: _get_active_student ──

```
def test_get_active_student_returns_first_by_queue(db_session):
from app.services.prompt_service import PromptService
s1, _, _ = _make_student(db_session, "Alice", queue_pos=2, student_number=1)
s2, _, _ = _make_student(db_session, "Bob", queue_pos=1, student_number=2)
svc = PromptService(session_factory=_factory(db_session))
with _factory(db_session)() as session:
result = svc._get_active_student(session)
assert result.id == s2.id

def test_get_active_student_raises_when_none(db_session):
from app.services.prompt_service import PromptService
svc = PromptService(session_factory=_factory(db_session))
with _factory(db_session)() as session:
with pytest.raises(ValueError, match="No active student"):
svc._get_active_student(session)
```

── service: _next_seq ──

```
def test_next_seq_starts_at_1(db_session):
from app.services.prompt_service import PromptService
s, _, _ = _make_student(db_session)
svc = PromptService(session_factory=_factory(db_session))
with _factory(db_session)() as session:
assert svc._next_seq(session, s.id) == 1

def test_next_seq_increments(db_session):
from app.services.prompt_service import PromptService
s, _, r = _make_student(db_session)
db_session.add(Prompt(
student_id=s.id, repository_id=r.id, student_number=1, seq_number=1,
title="t", slug="t", content="c", file_path="prompts/s.md",
git_branch="main", status=PromptStatus.pushed.value, created_by="operator",
created_at=datetime.utcnow(),
))
db_session.commit()
svc = PromptService(session_factory=_factory(db_session))
with _factory(db_session)() as session:
assert svc._next_seq(session, s.id) == 2
```

── service: create_and_push_prompt ──

```
def test_create_and_push_prompt_success(db_session):
from app.services.prompt_service import PromptService
s, _, r = _make_student(db_session)

mock_git = MagicMock()
mock_git.add_file.return_value = None
mock_git.commit.return_value = "a" * 40
```

```
mock_git.push.return_value = None

mock_creds = MagicMock()
mock_creds.retrieve.return_value = "fake_token"

mock_audit = MagicMock()

svc = PromptService(
git_client=mock_git,
credential_service=mock_creds,
audit_service=mock_audit,
session_factory=_factory(db_session),
)

prompt = svc.create_and_push_prompt(title="Auth Flow Fix", content="Fix the login.")

assert prompt.status == PromptStatus.pushed.value
assert prompt.git_commit_hash == "a" * 40
assert prompt.pushed_at is not None
assert prompt.file_path == "prompts/student01-001-auth-flow-fix.md"
assert prompt.seq_number == 1
assert prompt.student_number == 1
assert prompt.push_error is None
mock_git.add_file.assert_called_once()
mock_git.commit.assert_called_once()
mock_git.push.assert_called_once()
assert mock_audit.log.call_count >= 2


def test_create_and_push_prompt_push_failure_stores_error(db_session):
from app.git.client import GitAuthError
from app.services.prompt_service import PromptService
s, _, r = _make_student(db_session)

mock_git = MagicMock()
mock_git.add_file.return_value = None
mock_git.commit.return_value = "b" * 40
mock_git.push.side_effect = GitAuthError("authentication failed")

mock_creds = MagicMock()
mock_creds.retrieve.return_value = "fake_token"

svc = PromptService(
git_client=mock_git,
credential_service=mock_creds,
audit_service=MagicMock(),
session_factory=_factory(db_session),
)

prompt = svc.create_and_push_prompt(title="Test", content="content")

assert prompt.status == PromptStatus.failed.value
assert prompt.push_error is not None
assert len(prompt.push_error) < 300
assert prompt.git_commit_hash == "b" * 40
assert prompt.pushed_at is None


def test_push_error_never_contains_token(db_session):
from app.git.client import GitOperationError
from app.services.prompt_service import PromptService
s, _, r = _make_student(db_session)

secret = "ghp_supersecrettoken12345678901234567"
mock_git = MagicMock()
mock_git.add_file.return_value = None
mock_git.commit.return_value = "c" * 40
mock_git.push.side_effect = GitOperationError(f"push failed: {secret}")

mock_creds = MagicMock()
mock_creds.retrieve.return_value = secret

svc = PromptService(
git_client=mock_git,
```

```
credential_service=mock_creds,
audit_service=MagicMock(),
session_factory=_factory(db_session),
)

prompt = svc.create_and_push_prompt(title="Test", content="c")
assert secret not in (prompt.push_error or "")

def test_list_prompts_all(db_session):
from app.services.prompt_service import PromptService
s1, _, r1 = _make_student(db_session, "Alice", student_number=1)
s2, _, r2 = _make_student(db_session, "Bob", queue_pos=2, student_number=2)
for s, r, n in [(s1, r1, 1), (s2, r2, 2)]:
db_session.add(Prompt(
student_id=s.id, repository_id=r.id, student_number=n, seq_number=1,
title="t", slug="t", content="c", file_path="prompts/s.md",
git_branch="main", status=PromptStatus.pushed.value, created_by="operator",
created_at=datetime.utcnow(),
))
db_session.commit()
svc = PromptService(session_factory=_factory(db_session))
assert len(svc.list_prompts()) == 2

def test_list_prompts_filtered_by_student(db_session):
from app.services.prompt_service import PromptService
s1, _, r1 = _make_student(db_session, "Alice", student_number=1)
s2, _, r2 = _make_student(db_session, "Bob", queue_pos=2, student_number=2)
db_session.add(Prompt(
student_id=s1.id, repository_id=r1.id, student_number=1, seq_number=1,
title="t", slug="t", content="c", file_path="p.md", git_branch="main",
status=PromptStatus.pushed.value, created_by="op", created_at=datetime.utcnow(),
))
db_session.commit()
svc = PromptService(session_factory=_factory(db_session))
assert len(svc.list_prompts(student_id=s1.id)) == 1
assert len(svc.list_prompts(student_id=s2.id)) == 0
```

── service: retry_push ──

```
def test_retry_push_success(db_session):
from app.git.client import GitOperationError
from app.services.prompt_service import PromptService
s, _, r = _make_student(db_session)
p = Prompt(
student_id=s.id, repository_id=r.id, student_number=1, seq_number=5,
title="retry", slug="retry", content="c", file_path="prompts/s.md",
git_branch="main", status=PromptStatus.committed.value,
git_commit_hash="d" * 40, created_by="operator",
created_at=datetime.utcnow(),
)
db_session.add(p)
db_session.commit()

mock_git = MagicMock()
mock_git.push.return_value = None
mock_creds = MagicMock()
mock_creds.retrieve.return_value = "fake_token"

svc = PromptService(
git_client=mock_git,
credential_service=mock_creds,
audit_service=MagicMock(),
session_factory=_factory(db_session),
```

```
)
result = svc.retry_push(p.id)

assert result.status == PromptStatus.pushed.value
assert result.pushed_at is not None
assert result.push_error is None
assert mock_git.push.call_count == 1

def test_retry_push_failure_stores_error(db_session):
from app.git.client import GitOperationError
from app.services.prompt_service import PromptService
s, _, r = _make_student(db_session)
p = Prompt(
student_id=s.id, repository_id=r.id, student_number=1, seq_number=6,
title="retry", slug="retry", content="c", file_path="prompts/s.md",
git_branch="main", status=PromptStatus.committed.value,
git_commit_hash="e" * 40, created_by="operator",
created_at=datetime.utcnow(),
)
db_session.add(p)
db_session.commit()

mock_git = MagicMock()
mock_git.push.side_effect = GitOperationError("push failed: network")
mock_creds = MagicMock()
mock_creds.retrieve.return_value = "fake_token"

svc = PromptService(
git_client=mock_git,
credential_service=mock_creds,
audit_service=MagicMock(),
session_factory=_factory(db_session),
)
result = svc.retry_push(p.id)

assert result.status == PromptStatus.failed.value
assert result.push_error is not None

def test_retry_push_raises_if_already_pushed(db_session):
from app.services.prompt_service import PromptService
s, _, r = _make_student(db_session)
p = Prompt(
student_id=s.id, repository_id=r.id, student_number=1, seq_number=7,
title="done", slug="done", content="c", file_path="prompts/s.md",
git_branch="main", status=PromptStatus.pushed.value,
git_commit_hash="f" * 40, created_by="operator",
created_at=datetime.utcnow(),
)
db_session.add(p)
db_session.commit()

svc = PromptService(session_factory=_factory(db_session))
with pytest.raises(ValueError, match="not in a retryable state"):
svc.retry_push(p.id)
---
### tests/test_ui_skeleton.py
**Розмір:** 915 байт

"""UI skeleton tests for the StudentFlow main window."""

import os

import pytest

os.environ.setdefault("QT_QPA_PLATFORM", "offscreen")
pytest.importorskip("PySide6.QtWidgets")

from PySide6.QtCore import Qt

from app.ui.windows.main_window import MainWindow
```

```
def test_main_window_opens(qtbot):
window = MainWindow()
qtbot.addWidget(window)

assert not window.isVisible()

def test_navigation_switches_screens(qtbot):
window = MainWindow()
qtbot.addWidget(window)

buttons = window.findChildren(type(window._nav_buttons[0]))
qtbot.mouseClick(buttons[1], Qt.MouseButton.LeftButton)
assert window._stack.currentIndex() == 1

qtbot.mouseClick(buttons[6], Qt.MouseButton.LeftButton)
assert window._stack.currentIndex() == 6

def test_window_title_is_studentflow(qtbot):
window = MainWindow()
qtbot.addWidget(window)

assert window.windowTitle() == "StudentFlow"
```
---
tests/test_credential_service.py
Розмір: 4,624 байт

```
"""Tests for CredentialService and SecretVault."""
import pytest
from contextlib import contextmanager
from unittest.mock import patch, MagicMock

import app.db.models # noqa: F401 — register models
from app.db.models import Student, Account, Credential
from app.db.enums import SecretKind
from app.security.vault import SecretVault
from app.security.masking import mask_secret
from app.services.credential_service import CredentialService

@pytest.fixture
def svc(db_session):
"""CredentialService with a mocked vault and injected test session factory."""
mock_vault = MagicMock(spec=SecretVault)
mock_vault.get.return_value = None
mock_vault.exists.return_value = False

@contextmanager
def test_session_factory():
yield db_session

return CredentialService(vault=mock_vault, session_factory=test_session_factory), mock_vault,
db_session

@pytest.fixture
def student_and_account(db_session):
"""Create a persisted Student + Account for tests."""
student = Student(full_name="Alice Johnson", email="alice@example.com")
db_session.add(student)
db_session.flush()
account = Account(
student_id=student.id,
provider="github",
username="alice-dev",
auth_type="pat",
)
db_session.add(account)
db_session.flush()
return student, account

def test_store_creates_credential_record(svc, student_and_account):
```

```
service, mock_vault, session = svc
_, account = student_and_account

credential = service.store(account.id, SecretKind.pat, "ghp_test123")

expected_ref = f"studentflow:github:alice-dev:{account.id[:8]}"
mock_vault.set.assert_called_once_with(expected_ref, "ghp_test123")

assert credential.secret_ref == expected_ref
assert not hasattr(credential, "value")
assert not hasattr(credential, "token")
assert not hasattr(credential, "password")

queried = session.query(Credential).filter_by(account_id=account.id).first()
assert queried is not None
assert queried.secret_ref == expected_ref

def test_retrieve_returns_value():
with patch("keyring.get_password", return_value="ghp_fake_token"):
vault = SecretVault()
svc = CredentialService(vault=vault)
result = svc.retrieve("studentflow:github:alice:a1b2c3d4")
assert result == "ghp_fake_token"

def test_retrieve_raises_on_missing():
with patch("keyring.get_password", return_value=None):
vault = SecretVault()
svc = CredentialService(vault=vault)
with pytest.raises(KeyError):
svc.retrieve("studentflow:github:missing:xxxxxxxx")

def test_rotate_updates_keyring(svc, student_and_account):
service, mock_vault, session = svc
_, account = student_and_account

mock_vault.get.return_value = None
credential = service.store(account.id, SecretKind.pat, "old_token")

mock_vault.reset_mock()
service.rotate(credential.id, "new_ghp_token")

mock_vault.set.assert_called_once_with(credential.secret_ref, "new_ghp_token")
updated = session.get(Credential, credential.id)
assert updated.last_validated_at is not None

def test_delete_removes_from_keyring_and_db(svc, student_and_account):
service, mock_vault, session = svc
_, account = student_and_account

credential = service.store(account.id, SecretKind.pat, "ghp_delete_me")
secret_ref = credential.secret_ref

mock_vault.reset_mock()
service.delete(secret_ref)

mock_vault.delete.assert_called_once_with(secret_ref)
queried = session.query(Credential).filter_by(secret_ref=secret_ref).first()
assert queried is None

def test_validate_exists_true():
with patch("keyring.get_password", return_value="some_value"):
vault = SecretVault()
svc = CredentialService(vault=vault)
assert svc.validate_exists("studentflow:github:alice:a1b2c3d4") is True

def test_validate_exists_false():
with patch("keyring.get_password", return_value=None):
vault = SecretVault()
svc = CredentialService(vault=vault)
assert svc.validate_exists("studentflow:github:missing:xxxxxxxx") is False
```

```
def test_mask_secret():
assert mask_secret("ghp_abc123xyz789") == "••••••••z789"
assert mask_secret("abcd") == "••••"
assert mask_secret("") == "••••"
assert mask_secret("abc") == "••••"
assert mask_secret("abcde") == "••••••••bcde"

def test_secret_key_naming():
key = SecretVault.make_key("github", "alice-dev", "a1b2c3d4-e5f6-7890-abcd-ef1234567890")
assert key == "studentflow:github:alice-dev:a1b2c3d4"
```
---
tests/test_sync_service.py
Розмір: 10,416 байт

```
from unittest.mock import MagicMock

from sqlalchemy.orm import sessionmaker

from app.db.enums import StudentStatus, SyncStatus
from app.db.models import Account, AppSetting, Credential, Repository, Student
from app.git.client import GitOperationError
from app.services.sync_service import SyncService

def make_session_factory(db_session):
return sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)

def create_student_bundle(db_session, *, name: str, username: str, queue_position: int = 1):
student = Student(
full_name=name,
status=StudentStatus.active.value,
queue_position=queue_position,
)
db_session.add(student)
db_session.flush()

account = Account(
student_id=student.id,
provider="github",
username=username,
auth_type="pat",
is_current=True,
)
db_session.add(account)
db_session.flush()

repo = Repository(
account_id=account.id,
repo_name=f"{username}-project",
remote_url=f"https://github.com/example/{username}.git",
local_path=f"/tmp/{username}-repo",
default_branch="main",
integration_branch="main",
sync_status=SyncStatus.clean.value,
last_commit_hash="a" * 40,
is_canonical=False,
)
credential = Credential(
account_id=account.id,
secret_ref=f"studentflow:github:{username}:xxxxxxxx",
secret_kind="pat",
)
db_session.add_all([repo, credential])
db_session.commit()
return student, account, repo, credential

def test_sync_student_success(db_session):
factory = make_session_factory(db_session)
student, _, repo, _ = create_student_bundle(
```

```
db_session,
name="Alice Example",
username="alice",
)

mock_git = MagicMock()
mock_git.check_divergence.return_value = (5, 0)
mock_git.get_commit_hash.return_value = "b" * 40

mock_creds = MagicMock()
mock_creds.retrieve.return_value = "fake_token"

mock_audit = MagicMock()

service = SyncService(
git_client=mock_git,
credential_service=mock_creds,
audit_service=mock_audit,
session_factory=factory,
)

job = service.sync_student(student.id)

assert job.status == "success"
assert "5" in job.summary

stored_repo = db_session.get(Repository, repo.id)
assert stored_repo.sync_status == SyncStatus.clean.value
assert stored_repo.last_commit_hash == "b" * 40

mock_git.fetch.assert_called_once_with(repo.local_path, "student")
mock_git.merge_ff_only.assert_called_once_with(repo.local_path, "student/main")
mock_git.push.assert_called_once_with(repo.local_path, "origin", "main", credential=None)
mock_audit.log.assert_called_once()

def test_sync_student_diverged(db_session):
factory = make_session_factory(db_session)
student, _, repo, _ = create_student_bundle(
db_session,
name="Bob Example",
username="bob",
)

mock_git = MagicMock()
mock_git.check_divergence.return_value = (3, 2)

mock_creds = MagicMock()
mock_creds.retrieve.return_value = "fake_token"

service = SyncService(
git_client=mock_git,
credential_service=mock_creds,
audit_service=MagicMock(),
session_factory=factory,
)

job = service.sync_student(student.id)

assert job.status == "failed"
assert "Diverged" in job.error_message

stored_repo = db_session.get(Repository, repo.id)
assert stored_repo.sync_status == SyncStatus.diverged.value

def test_sync_student_git_fetch_error(db_session):
factory = make_session_factory(db_session)
student, _, repo, _ = create_student_bundle(
db_session,
name="Carol Example",
username="carol",
)

token = "super-secret-token"
```

```
mock_git = MagicMock()
mock_git.fetch.side_effect = GitOperationError(f"Connection refused for {token}")

mock_creds = MagicMock()
mock_creds.retrieve.return_value = token

service = SyncService(
git_client=mock_git,
credential_service=mock_creds,
audit_service=MagicMock(),
session_factory=factory,
)

job = service.sync_student(student.id)

assert job.status == "failed"
assert "fetch failed" in job.error_message
assert token not in job.error_message
assert len(job.error_message) <= 500

stored_repo = db_session.get(Repository, repo.id)
assert stored_repo.sync_status == SyncStatus.error.value


def test_sync_all_continues_on_failure(db_session):
factory = make_session_factory(db_session)
student_1, _, repo_1, _ = create_student_bundle(
db_session,
name="Student One",
username="student1",
queue_position=1,
)
student_2, _, repo_2, _ = create_student_bundle(
db_session,
name="Student Two",
username="student2",
queue_position=2,
)
student_3, _, repo_3, _ = create_student_bundle(
db_session,
name="Student Three",
username="student3",
queue_position=3,
)

divergence_by_path = {
repo_1.local_path: (2, 0),
repo_2.local_path: (0, 0),
repo_3.local_path: (2, 0),
}

mock_git = MagicMock()

def fetch_side_effect(local_path, remote_name):
assert remote_name == "student"
if local_path == repo_2.local_path:
raise GitOperationError("Connection refused")
return None

mock_git.fetch.side_effect = fetch_side_effect
mock_git.check_divergence.side_effect = lambda local_path, branch:
divergence_by_path[local_path]
mock_git.get_commit_hash.side_effect = lambda local_path: {
repo_1.local_path: "1" * 40,
repo_3.local_path: "3" * 40,
}[local_path]

mock_creds = MagicMock()
mock_creds.retrieve.return_value = "fake_token"

service = SyncService(
git_client=mock_git,
credential_service=mock_creds,
```

```
audit_service=MagicMock(),
session_factory=factory,
)

results = service.sync_all()

assert len(results) == 3
by_student = {job.student_id: job for job in results}
assert by_student[student_1.id].status == "success"
assert by_student[student_2.id].status == "failed"
assert by_student[student_3.id].status == "success"

def test_sync_all_empty(db_session):
factory = make_session_factory(db_session)
service = SyncService(
git_client=MagicMock(),
credential_service=MagicMock(),
audit_service=MagicMock(),
session_factory=factory,
)

results = service.sync_all()

assert results == []
```

— canonical_remote_url tests ————————————

```
))
db_session.add(AppSetting(
key="canonical_repo_path",
value="/tmp/canonical-bare",
description="",
))
db_session.commit()

mock_git = MagicMock()
mock_git.check_divergence.return_value = (2, 0)
mock_git.get_commit_hash.return_value = "d" * 40

service = SyncService(
git_client=mock_git,
credential_service=MagicMock(retrieve=MagicMock(return_value="tok")),
audit_service=MagicMock(),
session_factory=factory,
)

job = service.sync_student(student.id)

assert job.status == "success"
mock_git.push_to_url.assert_called_once_with(
"/tmp/canonical-bare",
"https://github.com/org/canonical.git",
"main",
credential=None,
auth_type=None,
)
assert "canonical remote push failed" not in (job.summary or "")

def test_sync_student_canonical_remote_push_failure_is_nonfatal(db_session):
"""When canonical remote push fails, sync job still succeeds and summary has warning."""
factory = make_session_factory(db_session)
student, _, repo, _ = create_student_bundle(
db_session,
name="Remote Fail",
username="remotefail",
)
db_session.add(AppSetting(
key="canonical_remote_url",
value="https://github.com/org/canonical.git",
description="",
))
db_session.add(AppSetting(
key="canonical_repo_path",
value="/tmp/canonical-bare",
description="",
))
db_session.commit()

from app.git.client import GitOperationError as GitOpError

mock_git = MagicMock()
mock_git.check_divergence.return_value = (3, 0)
mock_git.get_commit_hash.return_value = "e" * 40
mock_git.push_to_url.side_effect = GitOpError("remote: authentication required")

service = SyncService(
git_client=mock_git,
credential_service=MagicMock(retrieve=MagicMock(return_value="tok")),
audit_service=MagicMock(),
session_factory=factory,
)

job = service.sync_student(student.id)

assert job.status == "success"
assert "canonical remote push failed" in (job.summary or "")
assert "authentication required" in (job.summary or "")
---
```

tests/test_account_service.py
Розмір: 14,225 байт

```
"""Tests for AccountService (T13)."""
from contextlib import contextmanager

import pytest

import app.db.models # noqa: F401
from app.db.enums import AccountStatus, StudentStatus
from app.db.models import Account, Student
from app.errors import DuplicateStudentError
from app.services.account_service import AccountService

@pytest.fixture
def svc(db_session):
@contextmanager
def factory():
yield db_session

return AccountService(session_factory=factory), db_session

def test_create_student_success(svc):
service, session = svc
student = service.create_student(full_name="Alice", queue_position=1)
assert student.full_name == "Alice"
assert student.status == StudentStatus.active.value
db_student = session.query(Student).filter_by(id=student.id).first()
assert db_student is not None

def test_create_student_duplicate_name_same_status(svc):
service, _ = svc
service.create_student(full_name="Bob", status=StudentStatus.active.value)
with pytest.raises(DuplicateStudentError):
service.create_student(full_name="Bob", status=StudentStatus.active.value)

def test_list_students_returns_all(svc):
service, _ = svc
service.create_student(full_name="A", queue_position=1)
service.create_student(full_name="B", queue_position=2)
service.create_student(full_name="C", queue_position=3)
students = service.list_students()
assert len(students) == 3

def test_archive_student(svc):
service, session = svc
student = service.create_student(full_name="Dave", queue_position=1)
service.archive_student(student.id)
active = service.list_students()
assert not any(s.id == student.id for s in active)
db_s = session.query(Student).filter_by(id=student.id).first()
assert db_s.status == StudentStatus.archived.value

def test_set_queue_position(svc):
service, session = svc
student = service.create_student(full_name="Eve", queue_position=5)
service.set_queue_position(student.id, 1)
db_s = session.query(Student).filter_by(id=student.id).first()
assert db_s.queue_position == 1

def test_get_next_in_queue(svc):
service, _ = svc
service.create_student(full_name="Z Student", queue_position=3)
service.create_student(full_name="A Student", queue_position=1)
service.create_student(full_name="M Student", queue_position=2)
next_s = service.get_next_in_queue()
assert next_s is not None
```

```
assert next_s.queue_position == 1

def test_create_account_success(svc):
service, session = svc
student = service.create_student(full_name="Frank")
account = service.create_account(
student_id=student.id,
provider="github",
username="frank-dev",
auth_type="pat",
)
assert account.username == "frank-dev"
db_a = session.query(Account).filter_by(id=account.id).first()
assert db_a is not None

def test_deactivate_account(svc):
service, session = svc
student = service.create_student(full_name="Grace")
account = service.create_account(
student_id=student.id,
provider="gitlab",
username="grace",
auth_type="ssh",
)
service.deactivate_account(account.id)
db_a = session.query(Account).filter_by(id=account.id).first()
assert db_a.status == AccountStatus.inactive.value

def test_validate_account_no_credential_returns_error(svc):
"""Account with no Credential record → status=error, validated=False."""
service, session = svc
student = service.create_student(full_name="NoCredUser")
account = service.create_account(
student_id=student.id, provider="github", username="nocred", auth_type="pat"
)
result = service.validate_account(account.id)
assert result["status"] == "error"
assert result["validated"] is False
assert "credential" in result["message"].lower()
db_a = session.query(Account).filter_by(id=account.id).first()
assert db_a.status == AccountStatus.error.value

def test_validate_account_credential_not_in_keyring_returns_unvalidated(svc, mock_keyring):
"""Credential record exists but keyring returns None → status=unvalidated."""
from app.db.models import Credential
service, session = svc
student = service.create_student(full_name="MissingKeyUser")
account = service.create_account(
student_id=student.id, provider="github", username="missingkey", auth_type="pat"
)
# Insert a Credential record with a ref that is NOT in the keyring
cred = Credential(
account_id=account.id,
secret_ref="studentflow:github:missingkey:test0000",
secret_kind="pat",
)
session.add(cred)
session.flush()

mock_keyring["get"].return_value = None # keyring has nothing
result = service.validate_account(account.id)
assert result["status"] == "unvalidated"
assert result["validated"] is False
db_a = session.query(Account).filter_by(id=account.id).first()
assert db_a.status == AccountStatus.unvalidated.value

def test_validate_account_credential_in_keyring_returns_active(svc, mock_keyring):
```

```
"""Credential present in keyring → status=active, validated=True."""
from app.db.models import Credential
service, session = svc
student = service.create_student(full_name="ValidUser")
account = service.create_account(
student_id=student.id, provider="github", username="validuser", auth_type="pat"
)
cred = Credential(
account_id=account.id,
secret_ref="studentflow:github:validuser:test0001",
secret_kind="pat",
)
session.add(cred)
session.flush()

mock_keyring["get"].return_value = "ghp_fakefakefake" # keyring has the value
result = service.validate_account(account.id)
assert result["status"] == "active"
assert result["validated"] is True
assert "network" in result["message"].lower()
db_a = session.query(Account).filter_by(id=account.id).first()
assert db_a.status == AccountStatus.active.value
assert db_a.last_validated_at is not None

def test_create_student_assigns_student_number(svc):
"""First student gets a non-null student_number."""
service, session = svc
student = service.create_student(full_name="Numbered Student", queue_position=1)
assert student.student_number is not None
assert student.student_number >= 1
db_s = session.query(Student).filter_by(id=student.id).first()
assert db_s.student_number == student.student_number

def test_create_student_number_increments(svc):
"""Each new student gets a strictly higher student_number than the previous."""
service, _ = svc
s1 = service.create_student(full_name="First Student")
s2 = service.create_student(full_name="Second Student")
s3 = service.create_student(full_name="Third Student")
assert s2.student_number == s1.student_number + 1
assert s3.student_number == s2.student_number + 1

def test_create_student_number_unique(svc):
"""Two students created sequentially have different student_numbers."""
service, _ = svc
s1 = service.create_student(full_name="Unique A")
s2 = service.create_student(full_name="Unique B")
assert s1.student_number != s2.student_number

def test_move_student_up_swaps_with_above(svc):
"""Moving second student up swaps positions with the first."""
service, session = svc
s1 = service.create_student(full_name="First")
service.set_queue_position(s1.id, 1)
s2 = service.create_student(full_name="Second")
service.set_queue_position(s2.id, 2)

service.move_student_up(s2.id)

db_s1 = session.query(Student).filter_by(id=s1.id).first()
db_s2 = session.query(Student).filter_by(id=s2.id).first()
assert db_s2.queue_position == 1
assert db_s1.queue_position == 2

def test_move_student_up_at_top_does_nothing(svc):
"""Moving the topmost student up has no effect."""
service, session = svc
s1 = service.create_student(full_name="Top")
```

```
service.set_queue_position(s1.id, 1)
s2 = service.create_student(full_name="Second")
service.set_queue_position(s2.id, 2)

service.move_student_up(s1.id)

db_s1 = session.query(Student).filter_by(id=s1.id).first()
assert db_s1.queue_position == 1


def test_move_student_down_swaps_with_below(svc):
"""Moving first student down swaps positions with the second."""
service, session = svc
s1 = service.create_student(full_name="Alpha")
service.set_queue_position(s1.id, 1)
s2 = service.create_student(full_name="Beta")
service.set_queue_position(s2.id, 2)

service.move_student_down(s1.id)

db_s1 = session.query(Student).filter_by(id=s1.id).first()
db_s2 = session.query(Student).filter_by(id=s2.id).first()
assert db_s1.queue_position == 2
assert db_s2.queue_position == 1


def test_move_student_down_at_bottom_does_nothing(svc):
"""Moving the last student down has no effect."""
service, session = svc
s1 = service.create_student(full_name="Only")
service.set_queue_position(s1.id, 1)

service.move_student_down(s1.id)

db_s1 = session.query(Student).filter_by(id=s1.id).first()
assert db_s1.queue_position == 1


def test_move_student_up_three_students(svc):
"""Moving the third student up puts it between first and second."""
service, session = svc
s1 = service.create_student(full_name="P1")
service.set_queue_position(s1.id, 1)
s2 = service.create_student(full_name="P2")
service.set_queue_position(s2.id, 2)
s3 = service.create_student(full_name="P3")
service.set_queue_position(s3.id, 3)

service.move_student_up(s3.id)

db_s2 = session.query(Student).filter_by(id=s2.id).first()
db_s3 = session.query(Student).filter_by(id=s3.id).first()
assert db_s3.queue_position == 2
assert db_s2.queue_position == 3


def test_move_student_ignores_archived(svc):
"""Archived students are not considered neighbours in move operations."""
service, session = svc
s1 = service.create_student(full_name="Active1")
service.set_queue_position(s1.id, 1)
s2 = service.create_student(full_name="Archived")
service.set_queue_position(s2.id, 2)
service.archive_student(s2.id)
s3 = service.create_student(full_name="Active3")
service.set_queue_position(s3.id, 3)

# s3 should move up over s2 (archived) — but since s2 is excluded,
# s3 is now the second in the list and moves up to swap with s1
service.move_student_up(s3.id)

db_s1 = session.query(Student).filter_by(id=s1.id).first()
db_s3 = session.query(Student).filter_by(id=s3.id).first()
# s3 should have swapped with s1 (the only non-archived above it)
assert db_s3.queue_position == 1
```

```
assert db_s1.queue_position == 3

def test_validate_account_network_success(svc, mock_keyring):
"""Keyring present + network reachable → status=active, message shows remote URL."""
from unittest.mock import MagicMock
from app.db.models import Credential, Repository
from app.db.enums import SyncStatus
service, session = svc

student = service.create_student(full_name="NetworkOKUser")
account = service.create_account(
student_id=student.id, provider="github", username="networkokuser", auth_type="pat"
)
cred = Credential(
account_id=account.id,
secret_ref="studentflow:github:networkokuser:test0002",
secret_kind="pat",
)
repo = Repository(
account_id=account.id,
repo_name="network-ok-project",
remote_url="https://github.com/networkokuser/project.git",
local_path="/tmp/network-ok-repo",
default_branch="main",
integration_branch="main",
sync_status=SyncStatus.clean.value,
)
session.add_all([cred, repo])
session.flush()

mock_keyring["get"].return_value = "ghp_networkoktoken"

mock_git = MagicMock()
mock_git.validate_remote.return_value = True
net_service = type(service)(session_factory=service._session_factory, git_client=mock_git)

result = net_service.validate_account(account.id)

assert result["status"] == "active"
assert result["validated"] is True
assert "verified" in result["message"].lower()
mock_git.validate_remote.assert_called_once_with(
"https://github.com/networkokuser/project.git",
"ghp_networkoktoken",
"pat",
)
db_a = session.query(Account).filter_by(id=account.id).first()
assert db_a.status == AccountStatus.active.value
assert db_a.last_validated_at is not None

def test_validate_account_network_failure(svc, mock_keyring):
"""Keyring present + network unreachable → status=error, validated=False."""
from unittest.mock import MagicMock
from app.db.models import Credential, Repository
from app.db.enums import SyncStatus
service, session = svc

student = service.create_student(full_name="NetworkFailUser")
account = service.create_account(
student_id=student.id, provider="github", username="networkfailuser", auth_type="pat"
)
cred = Credential(
account_id=account.id,
secret_ref="studentflow:github:networkfailuser:test0003",
secret_kind="pat",
)
repo = Repository(
account_id=account.id,
repo_name="network-fail-project",
```

```
remote_url="https://github.com/networkfailuser/project.git",
local_path="/tmp/network-fail-repo",
default_branch="main",
integration_branch="main",
sync_status=SyncStatus.clean.value,
)
session.add_all([cred, repo])
session.flush()

mock_keyring["get"].return_value = "ghp_expiredtoken"

mock_git = MagicMock()
mock_git.validate_remote.return_value = False
net_service = type(service)(session_factory=service._session_factory, git_client=mock_git)

result = net_service.validate_account(account.id)

assert result["status"] == "error"
assert result["validated"] is False
assert "expired" in result["message"].lower() or "failed" in result["message"].lower()
mock_git.validate_remote.assert_called_once()
db_a = session.query(Account).filter_by(id=account.id).first()
assert db_a.status == AccountStatus.error.value
---
### tests/__init__.py
**Розмір:** 36 байт

"""Test package for StudentFlow."""
---
### tests/test_settings_viewmodel.py
**Розмір:** 2,139 байт

import pytest
from sqlalchemy.orm import sessionmaker

from app.db.models import AppSetting
from app.ui.viewmodels.settings_viewmodel import SettingsViewModel

def test_get_setting_returns_value(db_session):
db_session.add(AppSetting(key="canonical_repo_path", value="/tmp/canon", description="x"))
db_session.commit()
factory = sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)
vm = SettingsViewModel(session_factory=factory)
assert vm.get_setting("canonical_repo_path") == "/tmp/canon"

def test_get_setting_missing_returns_none(db_session):
factory = sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)
vm = SettingsViewModel(session_factory=factory)
assert vm.get_setting("nonexistent") is None

def test_set_setting_creates_and_updates(db_session):
db_session.add(AppSetting(key="canonical_repo_path", value="/old", description="x"))
db_session.commit()
factory = sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)
vm = SettingsViewModel(session_factory=factory)
vm.set_setting("canonical_repo_path", "/new")
assert vm.get_setting("canonical_repo_path") == "/new"

def test_set_setting_rejects_empty_string(db_session):
factory = sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)
vm = SettingsViewModel(session_factory=factory)
with pytest.raises(ValueError, match="cannot be empty"):
vm.set_setting("canonical_repo_path", "")

def test_list_operator_settings_returns_expected_keys(db_session):
for key, val in [
("canonical_repo_path", "/tmp/c"),
("default_branch", "main"),
```

```
("sync_timeout_seconds", "60"),
]:
db_session.add(AppSetting(key=key, value=val, description=""))
db_session.commit()
factory = sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)
vm = SettingsViewModel(session_factory=factory)
result = vm.list_operator_settings()
keys = [row["key"] for row in result]
assert "canonical_repo_path" in keys
assert "default_branch" in keys
assert "sync_timeout_seconds" in keys
```
---
tests/test_schemas.py
Розмір: 1,404 байт

```
"""Tests for Pydantic input schemas (T12)."""
import pytest
from pydantic import ValidationError

from app.schemas.student import StudentCreate
from app.schemas.account import AccountCreate
from app.schemas.repository import RepositoryCreate

def test_student_create_valid():
s = StudentCreate(full_name="Alice", queue_position=1)
assert s.full_name == "Alice"

def test_student_create_empty_name():
with pytest.raises(ValidationError):
StudentCreate(full_name="")

def test_student_create_strips_whitespace():
s = StudentCreate(full_name=" Alice ", queue_position=1)
assert s.full_name == "Alice"

def test_account_create_invalid_provider():
with pytest.raises(ValidationError):
AccountCreate(
student_id="abc",
provider="twitter",
username="alice",
auth_type="pat",
)

def test_account_create_invalid_username_chars():
with pytest.raises(ValidationError):
AccountCreate(
student_id="abc",
provider="github",
username="alice smith",
auth_type="pat",
)

def test_repo_create_relative_path():
with pytest.raises(ValidationError):
RepositoryCreate(
account_id="abc",
repo_name="myrepo",
remote_url="https://github.com/user/myrepo.git",
local_path="relative/path",
)
```
---
tests/conftest.py
Розмір: 2,510 байт

```
"""Pytest fixtures shared across all test modules."""
```

```
from contextlib import contextmanager
from unittest.mock import MagicMock, patch

import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

@pytest.fixture(scope="function")
def temp_db(tmp_path):
"""Create a temporary in-memory SQLite database for testing.

Returns the engine. Use db_session fixture for a session.
"""
import app.db.models # noqa: F401 — registers all models with Base before create_all
from app.db.base import Base

db_path = tmp_path / "test.db"
test_engine = create_engine(
f"sqlite:///{db_path}",
connect_args={"check_same_thread": False},
)
Base.metadata.create_all(bind=test_engine)
yield test_engine
Base.metadata.drop_all(bind=test_engine)

@pytest.fixture(scope="function")
def db_session(temp_db):
"""Yield a SQLAlchemy session bound to the temp test database.

Automatically rolls back after each test.
"""
TestSessionLocal = sessionmaker(bind=temp_db, autocommit=False, autoflush=False)
session = TestSessionLocal()
try:
yield session
finally:
session.rollback()
session.close()

@pytest.fixture
def session_factory(db_session):
"""Context-manager session factory that yields the shared db_session."""
@contextmanager
def _factory():
yield db_session
return _factory

@pytest.fixture(autouse=True)
def mock_keyring():
"""Mock keyring globally for all tests to prevent touching the OS keyring."""
with patch("keyring.get_password", return_value=None) as mock_get, \
patch("keyring.set_password", return_value=None) as mock_set, \
patch("keyring.delete_password", return_value=None) as mock_del:
yield {"get": mock_get, "set": mock_set, "delete": mock_del}

@pytest.fixture
def mock_git_client():
"""Return a MagicMock with all GitClient methods pre-configured."""
client = MagicMock()
client.clone.return_value = None
client.fetch.return_value = None
client.pull.return_value = None
client.push.return_value = None
client.merge.return_value = None
client.tag.return_value = None
client.get_head_commit.return_value = "abc1234567890000"
client.is_diverged.return_value = False
return client
```

```
@pytest.fixture
def mock_audit():
"""Return a MagicMock AuditService that accepts .log() calls."""
audit = MagicMock()
audit.log.return_value = None
return audit
```
---
tests/test_models.py
Розмір: 4,867 байт

```
def test_student_create(db_session):
from app.db.enums import StudentStatus
from app.db.models import Student

student = Student(full_name="Alice Johnson", email="alice@example.com")
db_session.add(student)
db_session.commit()
queried = db_session.query(Student).filter_by(full_name="Alice Johnson").first()
assert queried is not None
assert queried.email == "alice@example.com"
assert queried.status == StudentStatus.active.value
assert queried.queue_position is None
assert len(queried.id) == 36

def test_account_student_relationship(db_session):
from app.db.models import Account, Student

student = Student(full_name="Bob Chen")
db_session.add(student)
db_session.flush()
account = Account(
student_id=student.id,
provider="github",
username="bob-dev",
auth_type="pat",
)
db_session.add(account)
db_session.commit()
queried_student = db_session.get(Student, student.id)
assert len(queried_student.accounts) == 1
assert queried_student.accounts[0].username == "bob-dev"
assert queried_student.accounts[0].student.full_name == "Bob Chen"

def test_credential_no_secret_field():
from sqlalchemy import inspect

from app.db.models import Credential

mapper = inspect(Credential)
column_names = [col.key for col in mapper.attrs]
forbidden = {"value", "password", "token", "secret", "private_key_contents"}
found = forbidden.intersection(set(column_names))
assert found == set(), f"Credential model has forbidden secret fields: {found}"
assert "secret_ref" in column_names, "secret_ref column must exist"

def test_activity_log_append_only():
from sqlalchemy import inspect

from app.db.models import ActivityLog

mapper = inspect(ActivityLog)
column_names = [col.key for col in mapper.attrs]
assert "updated_at" not in column_names, "ActivityLog must not have updated_at"
assert "modified_at" not in column_names, "ActivityLog must not have modified_at"
assert "created_at" in column_names, "ActivityLog must have created_at"
assert not hasattr(ActivityLog, "update"), "ActivityLog must not have an update() method"

def test_all_enums_importable():
```

```
from app.db.enums import (
AccountProvider,
AccountStatus,
AuthType,
EntityType,
HandoffStatus,
SecretKind,
StudentStatus,
SyncJobStatus,
SyncJobType,
SyncStatus,
)

assert StudentStatus.active.value == "active"
assert AccountProvider.github.value == "github"
assert SyncStatus.diverged.value == "diverged"
assert HandoffStatus.rolled_back.value == "rolled_back"
assert EntityType.handoff.value == "handoff"
assert SecretKind.pat.value == "pat"

def test_prompt_status_enum_values():
from app.db.enums import PromptStatus
assert PromptStatus.draft.value == "draft"
assert PromptStatus.written.value == "written"
assert PromptStatus.committed.value == "committed"
assert PromptStatus.pushed.value == "pushed"
assert PromptStatus.failed.value == "failed"

def test_student_has_student_number(db_session):
from app.db.models import Student
student = Student(full_name="Numbered Student", student_number=42)
db_session.add(student)
db_session.commit()
fetched = db_session.get(Student, student.id)
assert fetched.student_number == 42

def test_prompt_model_creation(db_session):
from app.db.enums import PromptStatus, SyncStatus
from app.db.models import Account, Prompt, Repository, Student

student = Student(full_name="Test Student", student_number=1)
db_session.add(student)
db_session.flush()
account = Account(student_id=student.id, provider="github", username="tester",
auth_type="pat")
db_session.add(account)
db_session.flush()
repo = Repository(
account_id=account.id, repo_name="tester-project",
remote_url="https://github.com/tester/project.git",
local_path="/tmp/tester", default_branch="main",
sync_status=SyncStatus.clean.value,
)
db_session.add(repo)
db_session.flush()

prompt = Prompt(
student_id=student.id, repository_id=repo.id,
student_number=1, seq_number=1,
title="Auth Flow Fix", slug="auth-flow-fix",
content="Please implement authentication.",
file_path="prompts/student01-001-auth-flow-fix.md",
git_branch="main", status=PromptStatus.draft.value,
created_by="operator",
)
db_session.add(prompt)
db_session.commit()
```

```
fetched = db_session.get(Prompt, prompt.id)
assert fetched is not None
assert fetched.seq_number == 1
assert fetched.student_number == 1
assert fetched.status == "draft"
assert fetched.git_commit_hash is None
assert fetched.pushed_at is None
```
---
tests/test_audit_service.py
Розмір: 3,255 байт

```
import json

from sqlalchemy.orm import sessionmaker

from app.db.models import ActivityLog
from app.services.audit_service import AuditService

def make_session_factory(db_session):
return sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)

def test_log_creates_activity_log_record(db_session):
service = AuditService(session_factory=make_session_factory(db_session))

service.log(
"student_created",
"student",
entity_id="student-1",
details={"full_name": "Alice", "status": "active"},
)

records = db_session.query(ActivityLog).all()
assert len(records) == 1
assert records[0].action == "student_created"
assert records[0].entity_type == "student"
assert records[0].entity_id == "student-1"
assert json.loads(records[0].details) == {"full_name": "Alice", "status": "active"}

def test_log_does_not_raise_on_db_error(db_session):
class BrokenSession:
def __enter__(self):
return self

def __exit__(self, exc_type, exc, tb):
return False

def add(self, _record):
raise RuntimeError("db write failed")

def commit(self):
raise AssertionError("commit should not be reached when add fails")

service = AuditService(session_factory=lambda: BrokenSession())

service.log("student_created", "student", entity_id="student-1", details={"full_name": "Alice"})

assert db_session.query(ActivityLog).count() == 0

def test_log_sanitises_secrets_in_details(db_session):
service = AuditService(session_factory=make_session_factory(db_session))
token = "ghp_" + ("A" * 36)

service.log(
"credential_stored",
"credential",
entity_id="credential-1",
details={"secret_preview": token},
)

record = db_session.query(ActivityLog).one()
stored_details = json.loads(record.details)
assert stored_details["secret_preview"] == "[REDACTED]"
```

```
assert token not in record.details

def test_get_logs_filters_by_entity_type(db_session):
service = AuditService(session_factory=make_session_factory(db_session))

service.log("student_created", "student", entity_id="student-1")
service.log("account_created", "account", entity_id="account-1")
service.log("sync_started", "sync_job", entity_id="sync-1")

records = service.get_logs(entity_type="account")

assert len(records) == 1
assert records[0].entity_type == "account"

def test_get_logs_filters_by_action(db_session):
service = AuditService(session_factory=make_session_factory(db_session))

service.log("student_created", "student", entity_id="student-1")
service.log("student_updated", "student", entity_id="student-1")
service.log("student_created", "student", entity_id="student-2")

records = service.get_logs(action="student_updated")

assert len(records) == 1
assert records[0].action == "student_updated"

def test_get_logs_respects_limit(db_session):
service = AuditService(session_factory=make_session_factory(db_session))

for index in range(10):
service.log("student_created", "student", entity_id=f"student-{index}")

records = service.get_logs(limit=3)

assert len(records) == 3
---
### tests/test_dashboard_viewmodel.py
**Розмір:** 11,855 байт

from sqlalchemy.orm import sessionmaker

from app.db.models import Account, AppSetting, Repository, Student
from app.ui.viewmodels.dashboard_viewmodel import DashboardViewModel

def _make_factory(db_session):
return sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)

def test_get_canonical_summary_no_setting(db_session):
vm = DashboardViewModel(session_factory=_make_factory(db_session))
summary = vm.get_canonical_summary()
assert summary["path"] is None or isinstance(summary["path"], str)
assert "path_valid" in summary
assert "canonical_repo" in summary

def test_get_canonical_summary_with_setting(db_session, tmp_path):
db_session.add(AppSetting(key="canonical_repo_path", value=str(tmp_path), description="x"))
db_session.commit()
(tmp_path / "HEAD").write_text("ref: refs/heads/main\n")
vm = DashboardViewModel(session_factory=_make_factory(db_session))
summary = vm.get_canonical_summary()
assert summary["path"] == str(tmp_path)
assert summary["path_valid"] is True

def test_get_canonical_summary_invalid_path(db_session):
db_session.add(
AppSetting(key="canonical_repo_path", value="/nonexistent/path", description="x")
)
db_session.commit()
vm = DashboardViewModel(session_factory=_make_factory(db_session))
summary = vm.get_canonical_summary()
assert summary["path_valid"] is False
```

```
def test_get_canonical_summary_includes_repo_record(db_session, tmp_path):
student = Student(full_name="Test Student")
db_session.add(student)
db_session.flush()

account = Account(student_id=student.id, provider="github", username="op",
auth_type="pat")
db_session.add(account)
db_session.flush()

repo = Repository(
account_id=account.id,
repo_name="canonical-repo",
remote_url="https://github.com/op/project.git",
local_path=str(tmp_path),
default_branch="main",
is_canonical=True,
)
db_session.add(repo)
db_session.add(AppSetting(key="canonical_repo_path", value=str(tmp_path), description="x"))
db_session.commit()

vm = DashboardViewModel(session_factory=_make_factory(db_session))
summary = vm.get_canonical_summary()
assert summary["canonical_repo"] is not None
assert summary["canonical_repo"].repo_name == "canonical-repo"
```

── New dashboard viewmodel tests (added for dashboard enhancement) ──────────

```
from contextlib import contextmanager
from datetime import datetime

from app.db.enums import PromptStatus, StudentStatus, SyncJobStatus, SyncJobType
from app.db.models import Account, Prompt, SyncJob

def _add_student(db_session, name, queue_pos, status="active", student_number=None):
s = Student(
full_name=name,
status=status,
queue_position=queue_pos,
student_number=student_number,
)
db_session.add(s)
db_session.flush()
return s
```

get_active_student_info ──

```
def test_get_active_student_info_returns_none_when_empty(db_session):
vm = DashboardViewModel(session_factory=_make_factory(db_session))
assert vm.get_active_student_info() is None

def test_get_active_student_info_returns_active_student(db_session):
_add_student(db_session, "Alice", queue_pos=1, student_number=1)
db_session.commit()
vm = DashboardViewModel(session_factory=_make_factory(db_session))
info = vm.get_active_student_info()
```

```
assert info is not None
assert info["full_name"] == "Alice"
assert info["student_number"] == 1
assert info["queue_position"] == 1
assert info["status"] == "active"
assert info["prompt_count"] == 0

def test_get_active_student_info_picks_lowest_queue_position(db_session):
_add_student(db_session, "Second", queue_pos=2, student_number=2)
_add_student(db_session, "First", queue_pos=1, student_number=1)
db_session.commit()
vm = DashboardViewModel(session_factory=_make_factory(db_session))
info = vm.get_active_student_info()
assert info["full_name"] == "First"

def test_get_active_student_info_counts_pushed_prompts(db_session):
s = _add_student(db_session, "Bob", queue_pos=1, student_number=2)
account = Account(student_id=s.id, provider="github", username="bob", auth_type="pat")
db_session.add(account)
db_session.flush()
repo = Repository(
account_id=account.id,
repo_name="bob-project",
remote_url="https://github.com/bob/project.git",
local_path="/tmp/bob",
default_branch="main",
)
db_session.add(repo)
db_session.flush()
for seq in (1, 2):
db_session.add(Prompt(
student_id=s.id, repository_id=repo.id,
student_number=2, seq_number=seq,
title="t", slug=f"t{seq}", content="c",
file_path=f"prompts/s{seq}.md", git_branch="main",
status=PromptStatus.pushed.value, created_by="op",
created_at=datetime.utcnow(),
))
db_session.commit()
vm = DashboardViewModel(session_factory=_make_factory(db_session))
info = vm.get_active_student_info()
assert info["prompt_count"] == 2

def test_get_active_student_info_ignores_non_active(db_session):
_add_student(db_session, "Paused", queue_pos=1, status="paused", student_number=1)
db_session.commit()
vm = DashboardViewModel(session_factory=_make_factory(db_session))
assert vm.get_active_student_info() is None
```

get_student_queue ——

```
def test_get_student_queue_empty(db_session):
vm = DashboardViewModel(session_factory=_make_factory(db_session))
assert vm.get_student_queue() == []

def test_get_student_queue_returns_active_and_paused_only(db_session):
_add_student(db_session, "Active", queue_pos=1, status="active", student_number=1)
_add_student(db_session, "Paused", queue_pos=2, status="paused", student_number=2)
_add_student(db_session, "Archived", queue_pos=3, status="archived", student_number=3)
db_session.commit()
vm = DashboardViewModel(session_factory=_make_factory(db_session))
```

```
queue = vm.get_student_queue()
names = [e["full_name"] for e in queue]
assert "Active" in names
assert "Paused" in names
assert "Archived" not in names

def test_get_student_queue_ordered_by_queue_position(db_session):
_add_student(db_session, "Z", queue_pos=3, student_number=3)
_add_student(db_session, "A", queue_pos=1, student_number=1)
_add_student(db_session, "M", queue_pos=2, student_number=2)
db_session.commit()
vm = DashboardViewModel(session_factory=_make_factory(db_session))
queue = vm.get_student_queue()
assert [e["full_name"] for e in queue] == ["A", "M", "Z"]

def test_get_student_queue_respects_limit(db_session):
for i in range(10):
_add_student(db_session, f"S{i}", queue_pos=i + 1, student_number=i + 1)
db_session.commit()
vm = DashboardViewModel(session_factory=_make_factory(db_session))
assert len(vm.get_student_queue(limit=3)) == 3
```

get_recent_sync_activity

```
def test_get_recent_sync_activity_empty(db_session):
vm = DashboardViewModel(session_factory=_make_factory(db_session))
assert vm.get_recent_sync_activity() == []

def test_get_recent_sync_activity_returns_jobs(db_session):
s = _add_student(db_session, "Alice", queue_pos=1, student_number=1)
account = Account(student_id=s.id, provider="github", username="alice", auth_type="pat")
db_session.add(account)
db_session.flush()
repo = Repository(
account_id=account.id,
repo_name="alice-project",
remote_url="https://github.com/alice/project.git",
local_path="/tmp/alice",
default_branch="main",
)
db_session.add(repo)
db_session.flush()
job = SyncJob(
repo_id=repo.id,
student_id=s.id,
job_type=SyncJobType.sync_current.value,
status=SyncJobStatus.success.value,
started_at=datetime.utcnow(),
)
db_session.add(job)
db_session.commit()
vm = DashboardViewModel(session_factory=_make_factory(db_session))
jobs = vm.get_recent_sync_activity()
assert len(jobs) == 1
assert jobs[0]["student_name"] == "Alice"
assert jobs[0]["student_number"] == 1
assert jobs[0]["status"] == SyncJobStatus.success.value
assert jobs[0]["job_type"] == SyncJobType.sync_current.value

def test_get_recent_sync_activity_respects_limit(db_session):
s = _add_student(db_session, "Alice", queue_pos=1, student_number=1)
```

```
account = Account(student_id=s.id, provider="github", username="alice2", auth_type="pat")
db_session.add(account)
db_session.flush()
repo = Repository(
account_id=account.id,
repo_name="alice-project2",
remote_url="https://github.com/alice/project2.git",
local_path="/tmp/alice2",
default_branch="main",
)
db_session.add(repo)
db_session.flush()
for _ in range(8):
db_session.add(SyncJob(
repo_id=repo.id, student_id=s.id,
job_type=SyncJobType.sync_current.value,
status=SyncJobStatus.success.value,
))
db_session.commit()
vm = DashboardViewModel(session_factory=_make_factory(db_session))
assert len(vm.get_recent_sync_activity(limit=3)) == 3

def test_sync_active_student_no_active_raises(db_session):
"""sync_active_student raises ValueError when no active student exists."""
import pytest
vm = DashboardViewModel(session_factory=_make_factory(db_session))
with pytest.raises(ValueError, match="No active student"):
vm.sync_active_student()

def test_sync_active_student_delegates_to_service(db_session):
"""sync_active_student calls SyncService.sync_student and returns result dict."""
from unittest.mock import MagicMock, patch
from app.db.enums import SyncJobStatus
_add_student(db_session, "Alice", queue_pos=1, student_number=1, status="active")
db_session.commit()
vm = DashboardViewModel(session_factory=_make_factory(db_session))
mock_job = MagicMock()
mock_job.status = SyncJobStatus.success.value
mock_job.summary = "3 commits synced"
mock_job.error_message = None
with patch("app.services.sync_service.SyncService.sync_student", return_value=mock_job):
result = vm.sync_active_student()
assert result["status"] == SyncJobStatus.success.value
assert result["student_name"] == "Alice"
assert result["summary"] == "3 commits synced"
assert result["error_message"] is None

def test_handoff_to_next_delegates_to_service(db_session):
"""handoff_to_next wires HandoffService and returns result dict."""
from unittest.mock import MagicMock, patch
_add_student(db_session, "Alice", queue_pos=1, student_number=1, status="active")
_add_student(db_session, "Bob", queue_pos=2, student_number=2, status="paused")
db_session.commit()
vm = DashboardViewModel(session_factory=_make_factory(db_session))
mock_event = MagicMock()
mock_event.checkpoint_tag = "handoff/2026-01-01-120000/from-alice-to-bob"
with patch("app.services.handoff_service.HandoffService.handoff_to_next",
return_value=mock_event):
result = vm.handoff_to_next()
assert result["checkpoint_tag"] == "handoff/2026-01-01-120000/from-alice-to-bob"
assert result["from_student"] == "Alice"
assert result["to_student"] == "Bob"
---
### tests/test_git_client.py
**Розмір:** 8,064 байт
```

```
from __future__ import annotations

import subprocess
from pathlib import Path
from unittest.mock import patch

import pytest

from app.git.client import GitClient, GitOperationError

def git(*args: str, cwd: Path | None = None) -> subprocess.CompletedProcess[str]:
cmd = ["git"]
if cwd is not None:
cmd.extend(["-C", str(cwd)])
cmd.extend(args)
return subprocess.run(
cmd,
check=True,
capture_output=True,
text=True,
timeout=10,
)

def configure_repo(repo_path: Path) -> None:
git("config", "user.email", "test@example.com", cwd=repo_path)
git("config", "user.name", "Test User", cwd=repo_path)

def commit_file(repo_path: Path, name: str, content: str, message: str) -> None:
(repo_path / name).write_text(content, encoding="utf-8")
git("add", name, cwd=repo_path)
git("commit", "-m", message, cwd=repo_path)

def make_remote_and_clones(tmp_path: Path) -> tuple[Path, Path, Path]:
bare = tmp_path / "bare.git"
clone1 = tmp_path / "clone1"
clone2 = tmp_path / "clone2"

git("init", "--bare", str(bare))
git("clone", str(bare), str(clone1))
configure_repo(clone1)
commit_file(clone1, "f.txt", "initial\n", "init")
git("branch", "-M", "main", cwd=clone1)
git("push", "origin", "main", cwd=clone1)
git("symbolic-ref", "HEAD", "refs/heads/main", cwd=bare)

git("clone", str(bare), str(clone2))
git("remote", "rename", "origin", "student", cwd=clone2)
configure_repo(clone2)

return bare, clone1, clone2

def test_init_from_dir_creates_repo(tmp_path: Path) -> None:
src = tmp_path / "template"
src.mkdir()
(src / "README.md").write_text("# Project\n", encoding="utf-8")
(src / "main.py").write_text("print('hello')\n", encoding="utf-8")

dest = tmp_path / "student_repo"
client = GitClient()
client.init_from_dir(str(src), str(dest))

assert (dest / ".git").exists()
assert (dest / "README.md").exists()
assert (dest / "main.py").exists()

def test_get_commit_hash_returns_40_chars(tmp_path: Path) -> None:
src = tmp_path / "src"
src.mkdir()
(src / "f.txt").write_text("content", encoding="utf-8")
```

```
dest = tmp_path / "repo"
client = GitClient()
client.init_from_dir(str(src), str(dest))

commit_hash = client.get_commit_hash(str(dest))
assert len(commit_hash) == 40
assert all(char in "0123456789abcdef" for char in commit_hash)

def test_check_divergence_clean(tmp_path: Path) -> None:
_, _, clone2 = make_remote_and_clones(tmp_path)

client = GitClient()
ahead, behind = client.check_divergence(str(clone2), "student/main")

assert ahead == 0
assert behind == 0

def test_check_divergence_student_ahead(tmp_path: Path) -> None:
_, clone1, clone2 = make_remote_and_clones(tmp_path)
commit_file(clone1, "f.txt", "initial\nsecond\n", "second")
commit_file(clone1, "f.txt", "initial\nsecond\nthird\n", "third")
git("push", "origin", "main", cwd=clone1)
git("fetch", "student", cwd=clone2)

client = GitClient()
ahead, behind = client.check_divergence(str(clone2), "student/main")

assert ahead == 2
assert behind == 0

def test_create_tag(tmp_path: Path) -> None:
src = tmp_path / "src"
src.mkdir()
(src / "f.txt").write_text("content", encoding="utf-8")

repo_path = tmp_path / "repo"
client = GitClient()
client.init_from_dir(str(src), str(repo_path))
client.create_tag(str(repo_path), "handoff/2024-01-15-143022/from-alice-to-bob")

result = git("tag", "-l", cwd=repo_path)
assert "handoff/2024-01-15-143022/from-alice-to-bob" in result.stdout

def test_check_divergence_uses_rev_list_and_timeout() -> None:
client = GitClient(timeout=17)

with patch("app.git.client.subprocess.run") as run_mock:
run_mock.side_effect = [
subprocess.CompletedProcess(args=[], returncode=0, stdout="2\n", stderr=""),
subprocess.CompletedProcess(args=[], returncode=0, stdout="1\n", stderr=""),
]

ahead, behind = client.check_divergence("/tmp/repo", "student/main")

assert (ahead, behind) == (2, 1)
first_call = run_mock.call_args_list[0]
second_call = run_mock.call_args_list[1]
assert first_call.args[0] == ["git", "-C", "/tmp/repo", "rev-list", "--count", "HEAD..student/main"]
assert second_call.args[0] == ["git", "-C", "/tmp/repo", "rev-list", "--count",
"student/main..HEAD"]
assert first_call.kwargs["timeout"] == 17
assert second_call.kwargs["timeout"] == 17

def test_netrc_file_cleaned_up_after_push_failure(tmp_path: Path) -> None:
repo_path = tmp_path / "repo"
git("init", str(repo_path))
configure_repo(repo_path)
git("remote", "add", "student", "https://github.com/test/repo.git", cwd=repo_path)

client = GitClient()

with patch.object(client, "_get_remote_url", return_value="https://github.com/test/repo.git"):
```

```
with patch.object(client, "_make_netrc_env", return_value={"TEST_ENV": "1"},
"/tmp/test-cleanup.netrc")):
with patch("app.git.client.subprocess.run") as run_mock:
run_mock.side_effect = subprocess.CalledProcessError(
returncode=1,
cmd=["git", "push"],
stderr="Authentication failed",
)
with patch("app.git.client.os.unlink") as unlink_mock:
with pytest.raises(GitOperationError):
client.push(
str(repo_path),
"student",
"main",
credential="secret-token",
auth_type="pat",
)

unlink_mock.assert_called_once_with("/tmp/test-cleanup.netrc")

def _init_repo(tmp_path):
"""Helper: init a git repo with one commit."""
subprocess.run(["git", "init", str(tmp_path)], check=True, capture_output=True)
subprocess.run(["git", "-C", str(tmp_path), "config", "user.email", "t@t.com"], check=True,
capture_output=True)
subprocess.run(["git", "-C", str(tmp_path), "config", "user.name", "T"], check=True,
capture_output=True)
(tmp_path / "readme.txt").write_text("hello")
subprocess.run(["git", "-C", str(tmp_path), "add", "."], check=True, capture_output=True)
subprocess.run(["git", "-C", str(tmp_path), "commit", "-m", "init"], check=True,
capture_output=True)

def test_add_file_stages_specific_file(tmp_path):
_init_repo(tmp_path)
(tmp_path / "prompts").mkdir()
(tmp_path / "prompts" / "student01-001-test.md").write_text("# Test")
(tmp_path / "dirty.txt").write_text("should not be staged")

GitClient().add_file(str(tmp_path), "prompts/student01-001-test.md")

staged = subprocess.run(
["git", "-C", str(tmp_path), "diff", "--cached", "--name-only"],
capture_output=True, text=True, check=True,
).stdout.strip().splitlines()
assert staged == ["prompts/student01-001-test.md"]

status = subprocess.run(
["git", "-C", str(tmp_path), "status", "--porcelain"],
capture_output=True, text=True, check=True,
).stdout
assert "dirty.txt" in status

def test_commit_returns_hash_and_creates_commit(tmp_path):
_init_repo(tmp_path)
(tmp_path / "prompts").mkdir()
(tmp_path / "prompts" / "f.md").write_text("# Hello")
subprocess.run(["git", "-C", str(tmp_path), "add", "prompts/f.md"], check=True,
capture_output=True)

commit_hash = GitClient().commit(str(tmp_path), "prompt: student01-001-test")

assert len(commit_hash) == 40
assert all(c in "0123456789abcdef" for c in commit_hash)
log = subprocess.run(
["git", "-C", str(tmp_path), "log", "--oneline", "-1"],
capture_output=True, text=True, check=True,
).stdout
assert "student01-001-test" in log
```

```
def test_add_file_raises_on_missing_file(tmp_path):
_init_repo(tmp_path)
with pytest.raises(GitOperationError):
GitClient().add_file(str(tmp_path), "nonexistent/file.md")
```
---
tests/test_prompts_viewmodel.py
Розмір: 3,235 байт

```
"""Tests for PromptsViewModel."""
from __future__ import annotations

from datetime import datetime
from unittest.mock import MagicMock

import pytest
from sqlalchemy.orm import sessionmaker

from app.db.enums import PromptStatus, StudentStatus, SyncStatus
from app.db.models import Account, Credential, Prompt, Repository, Student

def _factory(db_session):
return sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False,
expire_on_commit=False)

def _make_student(db_session, name="Alice", queue_pos=1, student_number=1,
status=StudentStatus.active.value):
s = Student(full_name=name, status=status, queue_position=queue_pos,
student_number=student_number)
db_session.add(s)
db_session.commit()
return s

def test_get_active_student_returns_dict(db_session):
from app.ui.viewmodels.prompts_viewmodel import PromptsViewModel
s = _make_student(db_session)
vm = PromptsViewModel(session_factory=_factory(db_session))
result = vm.get_active_student()
assert result is not None
assert result["id"] == s.id
assert result["full_name"] == "Alice"
assert result["student_number"] == 1

def test_get_active_student_returns_none_when_empty(db_session):
from app.ui.viewmodels.prompts_viewmodel import PromptsViewModel
vm = PromptsViewModel(session_factory=_factory(db_session))
assert vm.get_active_student() is None

def test_get_active_student_picks_lowest_queue_position(db_session):
from app.ui.viewmodels.prompts_viewmodel import PromptsViewModel
s1 = _make_student(db_session, "Alice", queue_pos=2, student_number=1)
s2 = _make_student(db_session, "Bob", queue_pos=1, student_number=2)
vm = PromptsViewModel(session_factory=_factory(db_session))
result = vm.get_active_student()
assert result["id"] == s2.id

def test_create_prompt_delegates_to_service(db_session):
from app.ui.viewmodels.prompts_viewmodel import PromptsViewModel
mock_service = MagicMock()
fake_prompt = MagicMock()
mock_service.create_and_push_prompt.return_value = fake_prompt
vm = PromptsViewModel(session_factory=_factory(db_session), prompt_service=mock_service)
result = vm.create_prompt(title="T", content="C")
mock_service.create_and_push_prompt.assert_called_once_with(title="T", content="C",
created_by="operator")
assert result is fake_prompt

def test_retry_push_delegates_to_service(db_session):
```

```
from app.ui.viewmodels.prompts_viewmodel import PromptsViewModel
mock_service = MagicMock()
fake_prompt = MagicMock()
mock_service.retry_push.return_value = fake_prompt
vm = PromptsViewModel(session_factory=_factory(db_session), prompt_service=mock_service)
result = vm.retry_push("some-id")
mock_service.retry_push.assert_called_once_with("some-id")
assert result is fake_prompt

def test_list_prompts_delegates_to_service(db_session):
from app.ui.viewmodels.prompts_viewmodel import PromptsViewModel
mock_service = MagicMock()
mock_service.list_prompts.return_value = []
vm = PromptsViewModel(session_factory=_factory(db_session), prompt_service=mock_service)
vm.list_prompts(student_id="abc")
mock_service.list_prompts.assert_called_once_with(student_id="abc")
```

---

tests/test_handoff_service.py

Розмір: 10,092 байт

```
from datetime import date
from unittest.mock import MagicMock

import pytest
from sqlalchemy.orm import sessionmaker

from app.db.enums import HandoffStatus, StudentStatus, SyncStatus
from app.db.models import Account, AppSetting, Credential, HandoffEvent, Repository, Student
from app.services.handoff_service import (
HandoffError,
HandoffService,
NoActiveStudentError,
NoNextStudentError,
_build_checkpoint_tag,
)

def make_session_factory(db_session):
return sessionmaker(bind=db_session.bind, autocommit=False, autoflush=False)

def create_student_bundle(
db_session,
*,
name: str,
username: str,
queue_position: int,
status: str,
is_current: bool,
repo_status: str = SyncStatus.clean.value,
):
student = Student(
full_name=name,
status=status,
queue_position=queue_position,
)
db_session.add(student)
db_session.flush()

account = Account(
student_id=student.id,
provider="github",
username=username,
auth_type="pat",
is_current=is_current,
)
db_session.add(account)
db_session.flush()

credential = Credential(
```

```
account_id=account.id,
secret_ref=f"studentflow:github:{username}:secret",
secret_kind="pat",
)
repo = Repository(
account_id=account.id,
repo_name=f"{username}-project",
remote_url=f"https://github.com/example/{username}.git",
local_path=f"/tmp/{username}-repo",
default_branch="main",
integration_branch="main",
sync_status=repo_status,
last_commit_hash="a" * 40,
is_canonical=False,
)
db_session.add_all([credential, repo])
db_session.commit()
return student, account, repo, credential

def create_canonical_repo(db_session):
student = Student(
full_name="Canonical Operator",
status=StudentStatus.archived.value,
queue_position=999,
)
db_session.add(student)
db_session.flush()

account = Account(
student_id=student.id,
provider="github",
username="canonical",
auth_type="pat",
is_current=False,
)
db_session.add(account)
db_session.flush()

repo = Repository(
account_id=account.id,
repo_name="canonical",
remote_url="file:///tmp/canonical.git",
local_path="/tmp/canonical",
default_branch="main",
integration_branch="main",
sync_status=SyncStatus.clean.value,
last_commit_hash="c" * 40,
is_canonical=True,
)
db_session.add(repo)
db_session.add(AppSetting(key="canonical_repo_path", value=repo.local_path))
db_session.commit()
return repo

def build_service(db_session, **overrides):
factory = make_session_factory(db_session)
sync_service = overrides.get("sync_service", MagicMock())
credential_service = overrides.get("credential_service", MagicMock())
audit_service = overrides.get("audit_service", MagicMock())
git_client = overrides.get("git_client", MagicMock())
return HandoffService(
git_client=git_client,
sync_service=sync_service,
credential_service=credential_service,
audit_service=audit_service,
session_factory=factory,
)
```

```
def test_handoff_success(db_session):
canonical_repo = create_canonical_repo(db_session)
current, current_account, _, _ = create_student_bundle(
db_session,
name="Alice Example",
username="alice",
queue_position=1,
status=StudentStatus.active.value,
is_current=True,
)
next_student, next_account, next_repo, _ = create_student_bundle(
db_session,
name="Bob Example",
username="bob",
queue_position=2,
status=StudentStatus.paused.value,
is_current=False,
)

git_client = MagicMock()
git_client.get_commit_hash.return_value = "b" * 40

sync_service = MagicMock()
credential_service = MagicMock()
credential_service.retrieve.return_value = "token"
audit_service = MagicMock()

service = build_service(
db_session,
git_client=git_client,
sync_service=sync_service,
credential_service=credential_service,
audit_service=audit_service,
)

event = service.handoff_to_next()

db_session.expire_all()
stored_current = db_session.get(Student, current.id)
stored_next = db_session.get(Student, next_student.id)
stored_current_account = db_session.get(Account, current_account.id)
stored_next_account = db_session.get(Account, next_account.id)
stored_event = db_session.get(HandoffEvent, event.id)

assert stored_current.status == StudentStatus.exhausted.value
assert stored_next.status == StudentStatus.active.value
assert stored_current_account.is_current is False
assert stored_next_account.is_current is True
assert stored_event.status == HandoffStatus.completed.value
assert stored_event.commit_hash == "b" * 40
assert stored_event.repo_id == canonical_repo.id

sync_service.sync_student.assert_called_once_with(current.id)
git_client.create_tag.assert_called_once_with(
canonical_repo.local_path,
_build_checkpoint_tag(current_account.username, "b" * 40),
)
git_client.fetch.assert_called_once_with(next_repo.local_path, "origin")
git_client.merge_ff_only.assert_called_once_with(next_repo.local_path, "origin/main")
git_client.push.assert_called_once_with(
next_repo.local_path,
"student",
"main",
"token",
auth_type="pat",
)
audit_service.log.assert_called_once()

def test_handoff_no_active_student(db_session):
```

```
create_canonical_repo(db_session)
create_student_bundle(
db_session,
name="Paused Student",
username="paused",
queue_position=1,
status=StudentStatus.paused.value,
is_current=False,
)
service = build_service(db_session)

with pytest.raises(NoActiveStudentError):
service.handoff_to_next()

def test_handoff_no_next_student(db_session):
create_canonical_repo(db_session)
current, _, _, _ = create_student_bundle(
db_session,
name="Active Student",
username="active",
queue_position=1,
status=StudentStatus.active.value,
is_current=True,
)
create_student_bundle(
db_session,
name="Archived Student",
username="archived",
queue_position=2,
status=StudentStatus.archived.value,
is_current=False,
)
create_student_bundle(
db_session,
name="Error Student",
username="errored",
queue_position=3,
status=StudentStatus.error.value,
is_current=False,
)

service = build_service(db_session)

with pytest.raises(NoNextStudentError):
service.handoff_to_next()

service._sync_service.sync_student.assert_not_called()
db_session.expire_all()
assert db_session.get(Student, current.id).status == StudentStatus.active.value

def test_handoff_rollback_on_push_failure(db_session):
create_canonical_repo(db_session)
current, _, _, _ = create_student_bundle(
db_session,
name="Alice Example",
username="alice",
queue_position=1,
status=StudentStatus.active.value,
is_current=True,
)
next_student, _, _, _ = create_student_bundle(
db_session,
name="Bob Example",
username="bob",
queue_position=2,
status=StudentStatus.paused.value,
is_current=False,
)
```

```
git_client = MagicMock()
git_client.get_commit_hash.return_value = "d" * 40
git_client.push.side_effect = RuntimeError("push failed")

sync_service = MagicMock()
credential_service = MagicMock()
credential_service.retrieve.return_value = "token"
audit_service = MagicMock()

service = build_service(
db_session,
git_client=git_client,
sync_service=sync_service,
credential_service=credential_service,
audit_service=audit_service,
)

with pytest.raises(HandoffError, match="push failed"):
service.handoff_to_next()

db_session.expire_all()
stored_current = db_session.get(Student, current.id)
stored_next = db_session.get(Student, next_student.id)
stored_events = db_session.query(HandoffEvent).all()

assert stored_current.status == StudentStatus.active.value
assert stored_next.status == StudentStatus.paused.value
assert len(stored_events) == 1
assert stored_events[0].status == HandoffStatus.failed.value
audit_service.log.assert_called_once()


def test_get_next_student_skips_archived(db_session):
create_canonical_repo(db_session)
current, _, _, _ = create_student_bundle(
db_session,
name="Current Student",
username="current",
queue_position=1,
status=StudentStatus.active.value,
is_current=True,
)
create_student_bundle(
db_session,
name="Archived Student",
username="archived",
queue_position=2,
status=StudentStatus.archived.value,
is_current=False,
)
create_student_bundle(
db_session,
name="Error Student",
username="erroruser",
queue_position=3,
status=StudentStatus.error.value,
is_current=False,
)
paused_next, _, _, _ = create_student_bundle(
db_session,
name="Paused Student",
username="paused",
queue_position=4,
status=StudentStatus.paused.value,
is_current=False,
)

service = build_service(db_session)

next_student = service.get_next_student()
```

```
assert next_student is not None
assert next_student.id == paused_next.id
assert next_student.id != current.id

def test_checkpoint_tag_format():
tag = _build_checkpoint_tag("alice", "1234567890abcdef")

assert tag == f"checkpoint/alice/{date.today().isoformat()}/12345678"
```